

MATH 3190 – Fundamentals of Data Science

Course Notes

Spring 2026

Table of Contents

Notes 1: Intro, Unix, and GitHub	1
Notes 2: R Guide	13
Notes 3: Quarto	42
Notes 4: Databases and SQL	48
Notes 5: Verification Methods	59
Notes 6: Optimization	88
Notes 7: Advanced Regression	107
Notes 8: Dimension Reduction	149
Notes 9: Bayesian Methods	163
Notes 10: More Classification	185
Notes 11: Time Series	216
Reference Sheets	243

Notes 1 - Intro and GitHub

Math 3190 - Fundamentals of Data Science

Rick Brown
Southern Utah University

1

- 1 Introduction to Data Science
- 2 Installation Details
- 3 Introduction to Positron
- 4 Introduction to Unix
- 5 Unix Organization and Commands
- 6 Git and GitHub

2

Introduction to Data Science

Section 1

Introduction to Data Science

3

Introduction to Data Science

BIG DATA

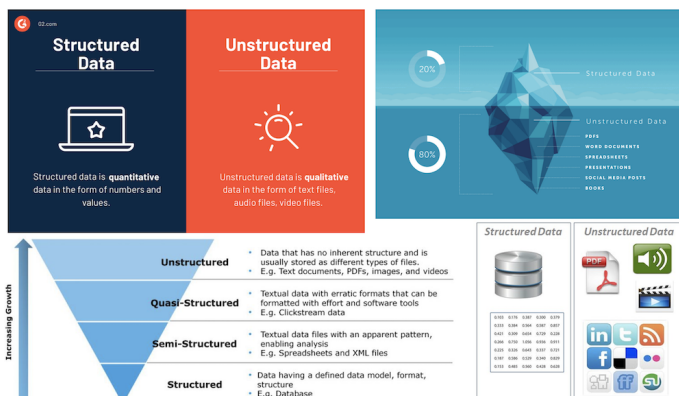


Big Data has fundamentally changed how we look at science and business. Along with advances in analytic methods, they are providing unparalleled insights into our physical world and society

4

Introduction to Data Science

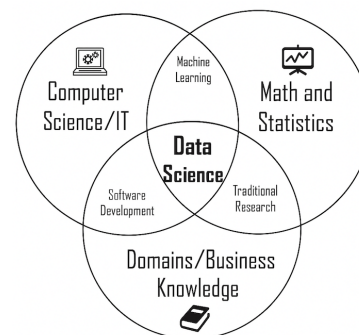
Structured vs. Unstructured Data



5

Introduction to Data Science

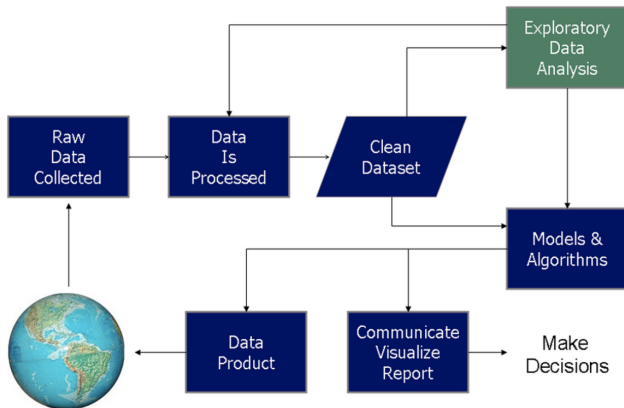
Data Science Revolution



- Few have all the skills
- Flexibility in area (business, strategy, health care) and conditions
- Data science makes companies and data better!

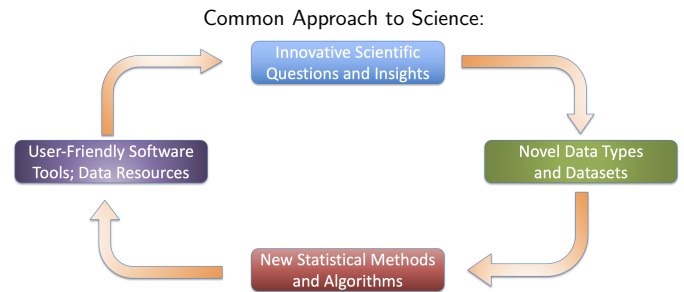
6

Data Science Process



7

Scientific Cycle for Data Science



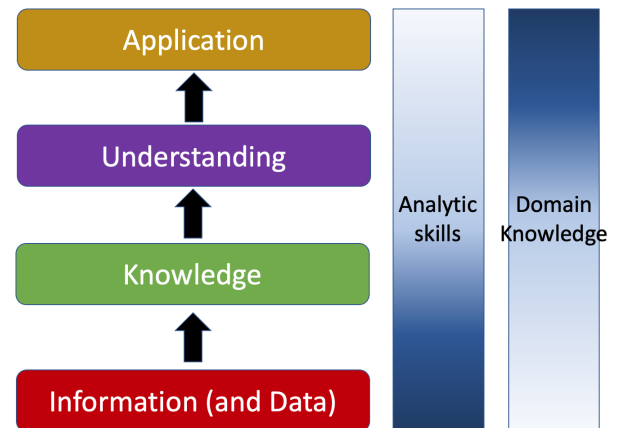
8

Domain Knowledge

Domain knowledge is knowledge of a specific, specialized discipline or field, in contrast to general (or domain-independent) knowledge. For example, in describing a software engineer may have general knowledge of computer programming as well as domain knowledge about developing programs for a particular industry. People with domain knowledge are often regarded as specialists or experts in their field. (Wikipedia!)

9

Analytics Hierarchy



10

Section 2

Installation Details

11

Important installations

You will need to install the following:

Mac Users

- R and R Studio or Positron (a new data science IDE!)
- Know how to access a terminal (RStudio, Positron or Terminal)
- Git (type `git --version` in the terminal)

Windows Users:

- R and Positron
- A terminal app (Git Bash [recommended], MobaXterm, Putty)
- Git for Windows

12

Installation Details

Important Installations

For installing **R** and RStudio: <https://cloud.r-project.org> and <https://posit.co/download/rstudio-desktop>

Install Positron here (optional): <https://positron.posit.co/download.html>

Install Git here: <https://git-scm.com/install> and if on Windows, that will also install Git Bash.

Note: When installing on Windows, make sure to chose the option to call the default branch “main” so it aligns better with GitHub. If you forget to do this, you can always change the default branch by typing

```
git config --global init.defaultBranch main
```

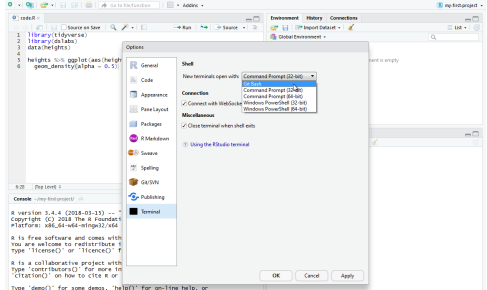
in the Git Bash terminal after installation.

13

Installation Details

For Windows: link Git Bash and RStudio

We can access the terminal either through RStudio or by opening Git Bash directly. For RStudio, set Git Bash as the default Unix shell: go to Tools then Global Options. . . , then select Terminal, then select Git Bash. It might have this already selected by default.



14

Introduction to Positron

Section 3

Introduction to Positron

15

Introduction to Positron

Positron

Positron is a next-generation IDE (Integrated Development Environment) that is designed specifically for data science. It is like a mix between VS Code and RStudio and runs both **R** and Python natively.

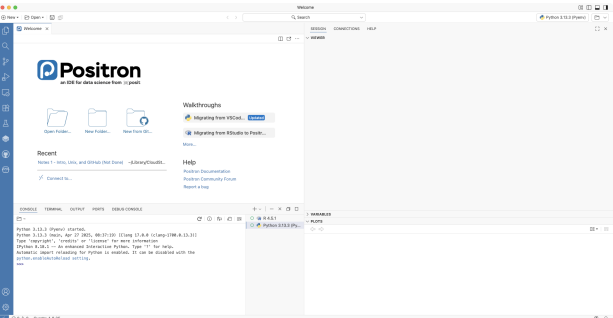
It is developed by Posit, which also created RStudio.

In this class, you can use RStudio or Positron. If you consider yourself primarily a Python programmer, then Positron may be a better fit for you than RStudio. However, I will give instructions in subsequety notes in RStudio, not Positron.

16

Introduction to Positron

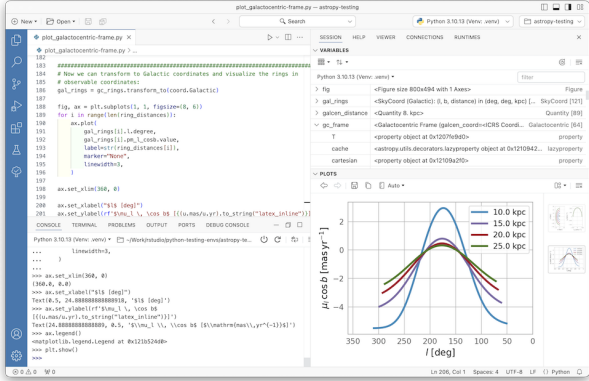
Positron First Impressions



17

Introduction to Positron

Positron Typical Workflow



18

Getting Started with Positron

When you first open Positron, it will want you to update the extensions. Click on the Extensions pane on the left side (the one with 4 squares and a red number on it). Then choose to update all extensions.

We will also need to make sure it connects to R (and Python if you have that installed). Click on "Start Session" in the top right. You should be able to click on the R version you have installed on your computer. If you have Python installed, you can easily switch back and forth between R and Python.

19

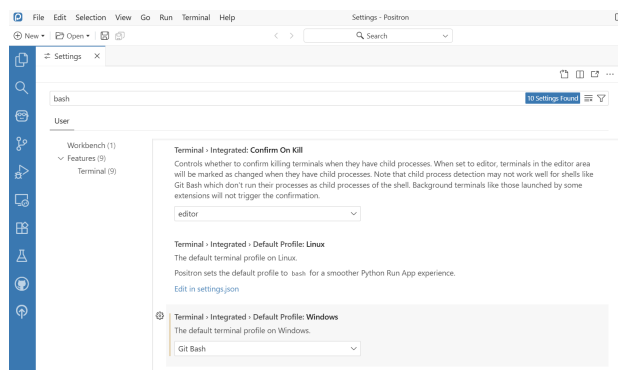
For Windows: link Git Bash and Positron

We can access the terminal either through Positron or by opening Git Bash directly. For Positron, set Git Bash as the default Unix shell:

- Go to settings in the lower left.
- Search for Bash
- Change the Terminal > Integrated > Default Profile: Windows to Git Bash. (See image on next slide)
- Type Shift-Control-P and search for "Terminal: Create New Terminal".
- Git Bash terminal should now appear in the TERMINAL tab in Positron.

20

For Windows: link Git Bash and Positron



21

Section 4

Introduction to Unix

22

Unix Introduction

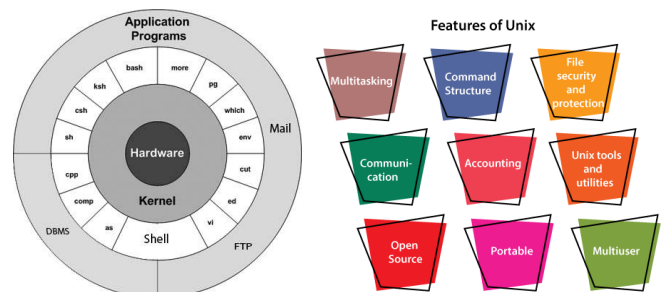
Unix is a family of multitasking, multiuser computer operating systems that derive from the original AT&T Unix. Originally developed in 1969 at Bell Labs. (Thanks Dennis Ritchie and Ken Thompson!)



23

Unix Introduction

The Unix operating system consists of many libraries and utilities along with the master control program, the kernel.



24

History of Operating Systems

Most operating systems can be grouped into two lineages:

- Microsoft's Windows NT-based operating systems
- "Unix-like" operating systems (e.g., Linux, macOS, Android, iOS, Chrome OS, Orbis OS used on the PlayStation 4, firmware on your router)

UNIX

VS

Linux



Darwin



25

Unix and Data Science

Unix is the operating system of choice in data science.

We will introduce you to the Unix way of thinking using an example: how to keep a data analysis project organized, and later how to do things more efficiently.

We will learn some of the most commonly used commands along the way.

26

Unix Resources

Here are some resources for more information:

- <https://www.codecademy.com/learn/learn-the-command-line>
- <https://www.edx.org/course/introduction-linux-linuxfoundationx-lfs101x-1>
- <https://www.coursera.org/learn/unix>

There are many reference books as well. Here are some particularly clear, succinct, and complete examples:

- <https://www.quora.com/Which-are-the-best-Unix-Linux-reference-books>
- <https://gumroad.com/l/bite-size-linux>
- <https://jvns.ca/blog/2018/08/05/new-zine--bite-size-command-line>

27

Naming Conventions

Before you start, pick a name convention that you will use to systematically name your files and directories. The Smithsonian Data Management Best Practices has "five precepts of file naming and organization":

- Have a distinctive, human-readable name that gives an indication of the content.
- Follow a consistent pattern that is machine-friendly.
- Organize files into directories (when necessary) that follow a consistent pattern.
- Avoid repetition of semantic elements among file and directory names.
- Have a file extension that matches the file format (no changing extensions!)

The Tidyverse Style Guide¹ is highly recommended!

¹<https://style.tidyverse.org/>

28

Section 5

Unix Organization and Commands

29

Basic Unix Commands

In addition to clicking, dragging, and dropping files and folders, we will be typing Unix commands into the terminal. (Similar typing commands into the **R** console!)

You will need access to a terminal (see installation instructions in Installation Details section). Once you have a terminal open, you can start typing into the *command line*, usually after a \$ or % symbol. Once you hit enter, Unix will try to execute this command. For example:

```
echo "hello world"
```

30

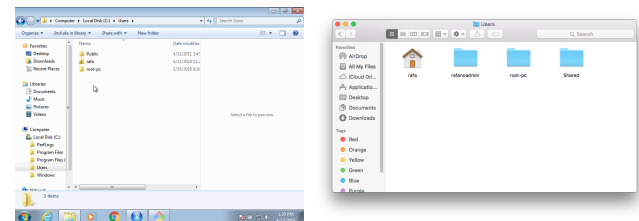
The Filesystem

We refer to all the files, folders, and programs on your computer as the **filesystem**. Keep in mind that folders and programs are also files (for later discussion). We will focus on files and folders for now and discuss programs, or **executables**, in a later section.

The first concept to understand is how your filesystem is organized; a series of nested folders, each containing files, folders, and executables.

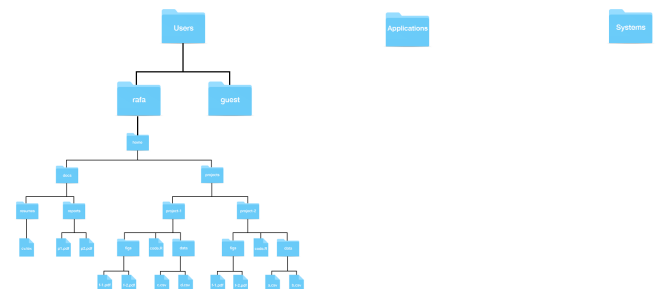
The Home Directory

The **home directory** is where your stuff is kept, as opposed to the system files, which are kept elsewhere. The name of your home directory is likely the same as your username. For example:



Mac Filesystem Structure

The Mac filesystem structure looks like this:



Working Directory

In Unix, the concept of a current location is indispensable. We refer to this as the **working directory**. Each terminal window you have open has a working directory.

How do we know our working directory? We use the Unix command: `pwd`, which stands for *print working directory*.

```
pwd
```

37

Paths

The string returned by `pwd` is the **full path** of the working directory. Every directory has a full path. We can also reference **relative paths**, which tell us how to get to a directory from the working directory.

In Unix, the shorthand `~` is a nickname for your home directory. So, for example, if `docs` is a directory in your home directory, the full path can be written `~/docs`.

38

Some Basic Unix Commands

To see the content of your home directory, open a terminal and type:

```
ls
```

You can also add **options** (`-` and `--`), **arguments**, and **wild cards** (`*`) to change function behavior:

```
ls -l
ls -a
ls -t
ls -r
ls -lart
ls -l *.txt
```

39

Some Basic Unix Commands

To create and remove directories, use the `mkdir` and `rmdir` functions, for example (use `ls` in between to see the directory come and go):

```
mkdir projects
mkdir junk
rmdir projects
```

Note: `rmdir` does not work if the directory is not empty.

Pro tip 1: The up arrow retrieves your previous commands.

Pro tip 2: You can auto-complete by hitting tab.

Pro tip 3: Type "Control-C" to clear a command.

40

Some Basic Unix Commands

To change the working directory use the `cd` command, which stands for *change directory*. For example:

```
cd projects
```

To check that it worked, use `pwd`. Now try:

```
cd .
cd ..
cd ../../
cd /
cd ~
cd ~/projects
cd ../junk
```

41

Some Basic Unix Commands

In Unix, we move files from one directory using the `mv` command:

```
mv path-to-file path-to-destination-directory
```

Warning: `mv` will not ask "are you sure?" if your move results in overwriting a file. Some `mv` examples:

```
mkdir ~/junk ~/projects/resumes
mv ~/projects/resumes/cv.tex ../../
mv ~/cv.tex ~/mycv.tex
mv ~/projects/mycv.tex junk/
mv ~/junk/mycv.tex ../projects/resumes/
```

42

Some Basic Unix Commands

The command `cp` behaves similar to `mv` except instead of moving, we copy the file. We first list the file we want to copy and then we list where we want to copy it and what its new name should be.

```
cp ~/projects/resumes/mycv.tex ~/mycv.tex
```

So in all the `mv` examples in the prior slide, you can switch `mv` to `cp` and they will copy instead of move. However to copy entire directories, add the recursive (`-r`) option:

```
cp -r ~/projects/resumes ~/junk/
```

43

Some Basic Unix Commands

In Unix, we can create a new blank file using the `touch` command.

```
touch filename
```

If that command is run again, then the file does not change except the modified date gets updated.

We remove files (and directories) with the `rm` command.

```
rm filename
```

Warning: Unlike throwing files into the trash or recycle bin, `rm` is permanent. Be careful! Note the following:

```
rm mycv.tex
rm junk/*.tex
rm -r projects junk
```

44

Some Basic Unix Commands

Unix uses an extreme version of abbreviations, which makes it hard to guess how to call commands. However, Unix includes complete help files or **man pages** (man is short for manual). In most systems, you can type `man` followed by the command name to get help. For example:

```
$ man ls
```

This command is not available in some of compact implementations of Unix (e.g., Git Bash). Alternatively, we can type the command followed by `-help`:

```
ls --help
```

Pro tip 4: Type `q` to quit out of a manual listing.

45

Advanced Unix functions

Most Unix implementations include a large number of powerful tools and utilities. It will take time to become comfortable with Unix, but as you struggle (if you do), you will find yourself learning just by looking up solutions on the internet.

46

Section 6

Git and GitHub

47

Git and GitHub Introduction

Many software developers and data scientists rely on Git and GitHub everyday.

There are three main reasons to use Git and GitHub.

- **Sharing:** GitHub allows us to easily share code with or without the advanced version control functionality.
- **Collaborating:** Multiple people make changes to code and keep versions synched. GitHub also has a special utility, called a **pull request**, that can be used by anybody to suggest changes to your code.
- **Version control:** The version control capabilities of Git permit us to keep track of changes, revert back to previous versions, and create **branches** in to test out ideas, then decide if we **merge** with the original.

See the Installation Details section for installing Git on your system.

48

GitHub Accounts

After installing Git, you need to create a GitHub account. To do this, go to <https://github.com> where you will see a link to sign up in the top right corner.

Pick a name carefully! Choose something short, somehow related to your name, and professional. Remember that you will use this to share code with others and you might be sharing this with potential collaborators or future employers!

49

GitHub Repositories

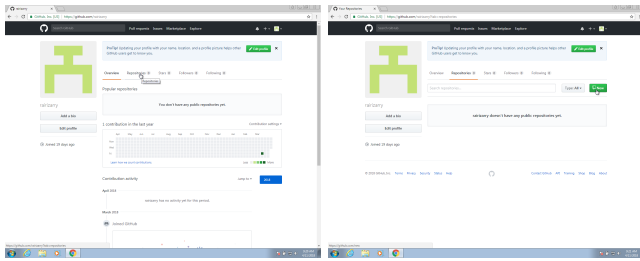
A GitHub **repository** (repo for short) is like a folder on your computer and allows you to have two copies of your code: one on your computer and one on GitHub. If you add collaborators, then each will have a copy on their computer.

The GitHub copy is usually considered the **main** copy (previously called the **master** and still referred to as master in many tutorials and pages). Git will help you keep all the different copies synced.

50

GitHub Repositories

The first step is to initialize the repository on GitHub. You will have a page on GitHub with the URL: <http://github.com/username>. On your account, you can click on *Repositories* and then click on *New* to create a new repo:

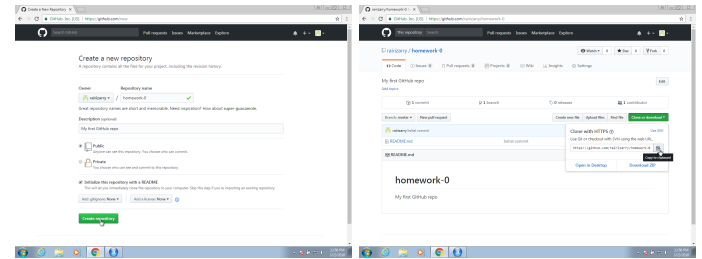


51

GitHub Repositories

Choose a good descriptive name. In your first lab, you will create one called "Math_3190_Assignments". You don't need to add a README.md, a .gitignore or choose a license at this time.

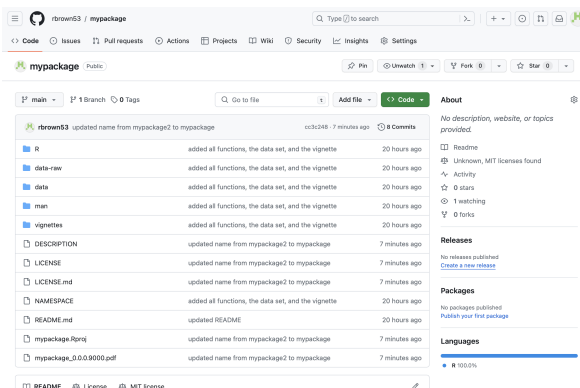
You will then be able to **clone** it on your computer using the terminal. Copy the link to connect to this repo for the next step.



52

Forking

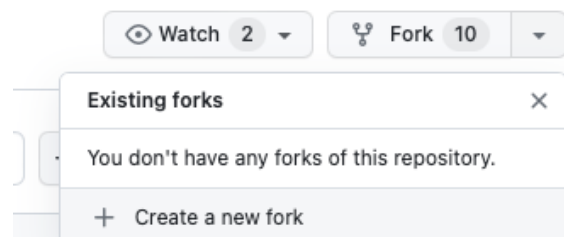
GitHub also allows you to **fork** others' repos. Go to <https://github.com/rbrown53/mypackage>



53

Forking

Click *Fork* in the top right and this will create a fork of the repository in your GitHub account. This will allow you to alter the code of other people (assuming you are allowed to do so by their license).



54

Git Setup - Tokens

Instead of using passwords, to connect local folders with GitHub repositories, we need to generate a GitHub **token**. Here are steps for obtaining one:

- ❶ Go to your GitHub home page: <https://github.com/username>
- ❷ Click on your account icon on the top right and go down to "Settings".
- ❸ On the left side, at the bottom, click on "Developer Settings".
- ❹ On the left, click on "Personal access tokens".
- ❺ Click on Tokens (classic). We will be using the classic tokens, not the fine-grained tokens.
- ❻ Click on "Generate new token" and the "Generate new token (classic)".
- ❼ Select an expiration time. 90 days is good.
- ❽ Select "repo", "workflow", "gist", and "user" under scopes.
- ❾ Click on "Generate token".

More info can be found here:

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>

55

Git Setup

Now we will work a bit in the Terminal. We need to let Git know who we are. This will make it clear who is making changes to files and directories. In a terminal window use the `git config` command:

```
git config --global user.name "My Name"
git config --global user.mail "my@email.com"
```

56

Git Setup

The main actions in Git are to:

- ❶ **pull** changes from the remote GitHub repo.
- ❷ **add** files to the staging area, or as we say in the Git lingo: **stage** files.
- ❸ **commit** changes to the local repository.
- ❹ **push** changes to the **remote** GitHub repo.

57

Git Setup

To effectively permit version control and collaboration in Git, files move across four different areas:



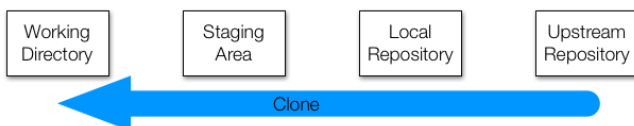
But how does it all get started? We can clone an existing repo or initialize one. We will explore cloning first.

58

Cloning Git Repositories

We will **clone** your existing my.package **upstream repository** to your local computer.

What does clone mean? We are going to actually copy the entire Git structure, files and directories to all stages: working directory, staging area, and local repository.



59

Cloning Git Repositories

Eventually, you will fork the mypackage repo from my GitHub. Then you will want to clone it to work on it on your local machine.

We can open a terminal and type:

```
cd ~
pwd
mkdir git-example
cd git-example
git clone https://github.com/yourusername/mypackage.git
cd mypackage
ls
```

60

Cloning Git Repositories

Note: the **working directory** is the same as your Unix working directory. When you edit files (e.g. in RStudio), you change the files in this directory. Git can tell you how these files relate to the upstream directory:

```
git status
```



Now let's add some changes to the local repository. Open the DESCRIPTION file in the mypackage directory, and make a change.

61

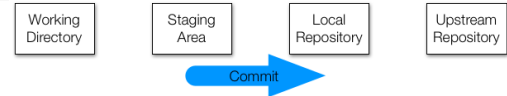
Working with Git Repositories

Now let's **add** the changes to the staging area and **commit** the changes to the local Git directory:

```
git add DESCRIPTION
```



```
git commit -m "updated DESCRIPTION with 'hello' function"
```



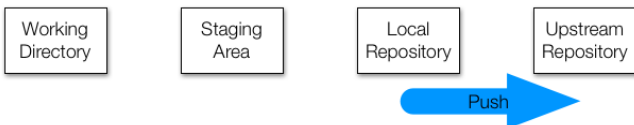
62

Working with Git Repositories

Now we can **push** the changes to the remote repo:

```
git push
```

The system will ask for your username and password. The username is your GitHub username, but the password is your personal token that we generated earlier.

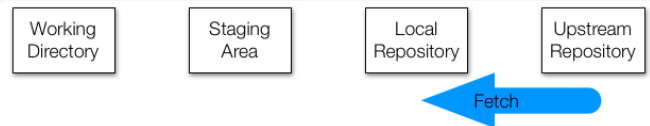


63

Working with Git Repositories

We can also **fetch** any changes on the remote repo (how do you think this is different from clone?):

```
git fetch
```

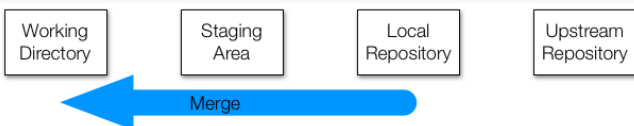


64

Working with Git Repositories

And then we need to **merge** these changes to our staging and working areas:

```
git merge
```

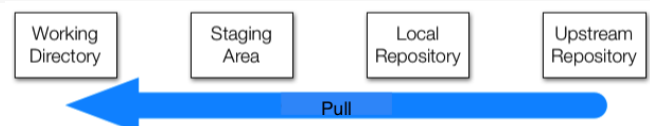


65

Working with Git Repositories

However, we often just want to change both with one command. For this, we use:

```
git pull
```



Important note: it is often a good idea to pull any changes when you start each day, so as to avoid **conflicts**.

66

Initializing a Git Directory

What if we already have a local directory and want to move it to a GitHub repository? See the following:

- 1 Create a new GitHub repository (e.g., Math_3190_Assignments)
- 2 Initialize the local repository

```
git init
```

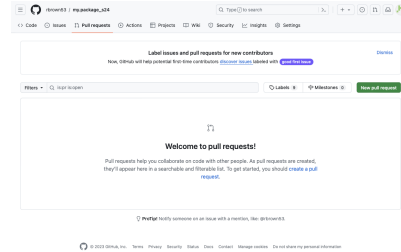
- 3 Use the **add** and **commit** commands to add to the local repository
- 4 Connect the local and remote repos and push:

```
git remote add origin \
https://github.com/username/Math_3190_Assignments.git
git push -u origin main
```

67

Pull Requests

Pull requests enable sharing of changes from other branches/forks of a repo. Potential changes can be reviewed before they are merged into the base branch. More details on how to do these will be provided in the lab.



68

Extra - .gitignore File

When we initialize Git in a folder, a hidden folder called `.git` is created. Another hidden file that is useful is `.gitignore`, which lets Git know what files it should NOT keep track of. We can add this when creating a repo on GitHub or we can add this on the local machine by navigating to the directory in the terminal and typing

```
touch .gitignore
```

69

Extra - Hidden Files

These file and folders that begin with a period (.) are hidden by default, but sometime we want to edit them, especially the `.gitignore`. We can see hidden files by doing the following:

Mac: In Finder, type Shift-Command-`.` (period). Type that again to hide the files again.

Windows 10:

- 1 Open File Explorer from the taskbar.
- 2 Select View > Options > Change folder and search options.
- 3 Select the View tab and, in Advanced settings, select Show hidden files, folders, and drives and OK.

Windows 11:

- 1 Open File Explorer from the taskbar.
- 2 Select View > Show > Hidden items.

70

Conclusion

Positron also has a more visual version control that uses Git built in. It is on the left side and its symbol looks like a branching path.

On Canvas, you'll find a file called "Unix and Git Cheat Sheet.pdf" that contains all of the Git and Unix commands that are contained in these notes plus more.

71

Session info

```
sessionInfo()

R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7.2

Matrix products: default
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib; /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Denver
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base

loaded via a namespace (and not attached):
[1] compiler_4.5.2    fastmap_1.2.0     cli_3.6.5         tools_4.5.2
[5] httools_0.5.9     rstudioapi_0.17.1 yaml_2.3.12       rmarkdown_2.30
[9] knitr_1.51        jsonlite_2.0.0    xfun_0.55         digest_0.6.39
[13] rlang_1.1.6       evaluate_1.0.5
```

72

Notes 2 - R Guide

Math 3190 - Fundamentals of Data Science

Rick Brown
Southern Utah University

1

- 1 Introduction to **R** and RStudio
- 2 Data Structures
- 3 R Functions
- 4 Data Import and Export
- 5 Tidyverse
- 6 Graphing in Base **R** (not a focus for us)
- 7 Graphing with ggplot2 (focus on this)

2

Section 1

Introduction to **R** and RStudio

3

Introduction

R is a powerful and popular programming language for statistical computing and data analysis. It provides a wide range of tools and packages for data manipulation, visualization, and modeling. **R** has a rich ecosystem with a vast community contributing to its growth, making it an excellent choice for data scientists, statisticians, and researchers.

4

Introduction

While **R** was developed by statisticians for statistical analysis, it has evolved to be a full-fledged programming language comparable to C, Python, Java, etc., though it is arguably still best used for data analysis tasks since many tasks are much more efficient in other languages. In this class, we will explore the syntax of **R** and the most commonly used functionality of the language.

5

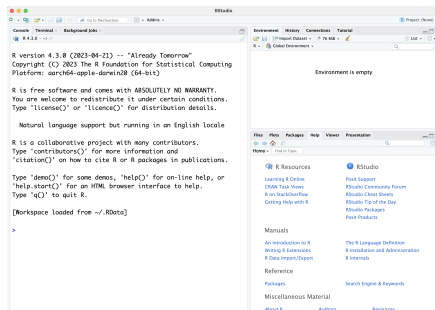
Installing R and RStudio

In the Introduction module on Canvas, there are some documents about installing **R** and **RStudio**. Be sure to follow those instructions to get the programs installed on your system. In this class, we will use **RStudio** exclusively once it is installed.

6

R First Impressions

When you first open **RStudio**, you will be presented with several windows, the main one being the **R console**, as shown below.



If yours does not look like this, then **something is wrong!**

7

R Console

You can interact with the console directly by typing in commands. For instance, if you type

```
1+1
```

into the console and it will give

```
[1] 2
```

as an output. The [1] on the left indicates that the output has only 1 value. I encourage you to play around with the console for a few minutes.

8

R Operations

You can perform all of the usual arithmetic operations in **R**. This includes

- **+** for addition.
- **-** for subtraction.
- ***** for multiplication.
- **/** for division.
- **^** for exponents.

R will do the order of operations: PEMDAS, so be sure to use parentheses properly.

```
2-6/2+5
```

```
[1] 4
```

```
(2-6)/2+5
```

```
[1] 3
```

9

R Operations

You can also use **<**, **>**, **<=**, **>=**, or **==** for **R** to check if one expression is less than, greater than, at most, at least, or exactly equal to another expression.

R will return either **TRUE** or **FALSE**, which is known as a Boolean data type.

```
3 > 1
```

```
[1] TRUE
```

```
-5 < -4
```

```
[1] TRUE
```

```
5 <= 5
```

```
[1] TRUE
```

```
5 == 5
```

```
[1] TRUE
```

10

R Data Types

Everything that you enter in **R** will have a data type. The primary data types in **R** are:

- **numeric**: data that are numbers. This will be the default for any number entered.
- **integer**: data that are numbers without decimal points. You can force a number to be stored as an integer by typing a capital "L" after the number. Example: 14L.
- **character**: data that are strings, which can include numbers and letters. This will be the default for anything entered in quotes (either single quotes or double quotes) like 'yellow' or "yellow".

11

R Data Types

- **factor**: data that are categorical. **R** thinks of these just in terms of categories.
- **logical**: true or false. Also known as Boolean. **R** will convert to logical if you are making a comparison like we did a couple slides ago.

The type of variable is important when working in **R**. If you have the wrong data type, then you may get errors. The code below will give an error.

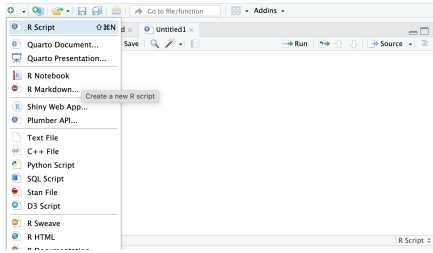
```
"1" + "1"
```

```
Error in "1" + "1" : non-numeric argument to binary operator
```

12

R Scripts

When programming in **R**, it is good practice to type all of your code in an **R** script. You can access a new script by clicking on the green plus icon at the top of the **RStudio** menu and then clicking on "R Script".



This way, you can save all your code to easily rerun it later.

13

R Commenting

You can comment out code or other words in **R** scripts with the **#** key. Anything on the same line after **#** will not be run.

```
x <- 5
# print(x)
x + 2
[1] 7
```

In the output above, the value of **x** was not printed since the `print(x)` command was commented out.

14

Built-In R Functions

There are many functions that **R** comes pre-loaded with. To use a function in **R**, you must type the name of the function, then put the arguments of the function in parentheses. Some examples are

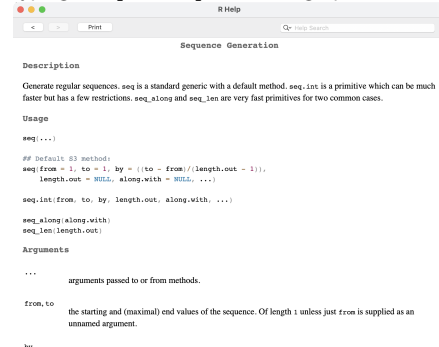
- `c()` to create a data vector.
- `mean()` to compute the average of a vector.
- `sd()` to compute the standard deviation of a vector.
- `length()` to compute the number of elements in a vector.
- `rep()` to repeat a value, string, or vector.
- `seq()` to create a sequence of values.
- `summary()` to compute some summary statistics for a vector.
- `cos()` to compute the cosine of a value.
- `exp()` to compute the number *e* raised to a power.
- `log()` to compute the logarithm (natural log is default) of a number.

There are hundreds of built-in **R** functions.

15

More on Functions

If you are ever in doubt about what a function does or what inputs it takes, you can always type a question mark in front of the function. For instance, typing `?seq` or `?seq()` will bring up this documentation:



16

Creating a Numeric Variable

It is remarkably useful to create variables for use in **R**. For example, if I have a small data set containing the values [1, 5, 3, 6], then I can input that into **R** and give that data set a name.

```
x <- c(1, 5, 3, 6)
```

The code above will create a new variable called **x** and then if I type **x**, it will display what is contained in **x**.

```
x
[1] 1 5 3 6
```

Then I can tell **R** to report how many elements are in **x** using the `length()` function, for instance.

```
length(x)
[1] 4
```

17

Creating a Character Variable

Variables do not have to be numbers. They can also be words (called "strings"). Just be sure to put strings in quotes.

```
y <- c("This", "Is", "A", "String", "Variable")
y
[1] "This"      "Is"        "A"         "String"
[5] "Variable"
```

There are many functions that work with character variables.

```
y <- c("This", "Is", "A", "String", "Variable")
grep("A", y, ignore.case = T)
[1] 3 5
```

This `grep()` function tells you which element(s) of a vector contain a pattern that you give it.

18

A Note on Assignment

When we created the variable `x`, we used an *assignment operator*. There are two primary assignment operators in **R**:

- ① The arrow: `<-`
- ② The equal sign: `=`

Either one works. `x <- c(1, 5, 3, 6)` and `x = c(1, 5, 3, 6)` will both define a new vector `x` the same way.

Note: There is a somewhat fierce debate about assignment operators in **R** with some arguing the arrow is the only way it should be done to avoid confusion in functions (and for tidiness and tradition). Others argue that the arrow is outdated since it takes more key strokes and relies too much on white space since `x <- 5` and `x < -5` are different commands. The first assigns the value 5 to `x` and the second checks if `x` is less than `-5`.

19

R Libraries

Often times, **R** will not have a built-in function that does what you want to do. In that case, you can usually find a package (or library) you can install that will have other functions. You can install **R** packages by issuing the `install.packages()` function. You will need to put the name of the package in quotes. For example, to install the `knitr` package, you can type

```
install.packages("knitr")
```

It may prompt you to choose an install source. You can always select the Cloud for this. Once it is installed, you do not need to install it again. You will, however, need to load it each time you reopen **R**. You can load a package with the `library()` function. The name of the package does not need to be in quotes anymore, but it can be.

```
library(knitr)
```

20

R Libraries

The `knitr` library has many functions that base **R** does not. For example, the `kable()` function in the `knitr` packages will convert a data frame to a nice table

```
kable(mtcars[1:5, 1:5], format = "latex")
```

	mpg	cyl	disp	hp	drat
Mazda RX4	21.0	6	160	110	3.90
Mazda RX4 Wag	21.0	6	160	110	3.90
Datsun 710	22.8	4	108	93	3.85
Hornet 4 Drive	21.4	6	258	110	3.08
Hornet Sportabout	18.7	8	360	175	3.15

There are many different packages that we will be installing and using this semester.

21

Section 2

Data Structures

22

Introduction

A data structure is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks. Data structures in **R** programming are tools for holding multiple values.

R's base data structures are often organized by their dimensionality (1D, 2D, or nD) and whether they're homogeneous (all elements must be of the identical type) or heterogeneous (the elements are often of various types). This gives rise to the six data types which are most frequently utilized in data analysis.

23

Data Structure Types

The most essential data structures used in **R** include:

- Vectors
- Data Frames
- Factors

24

Vectors

A vector is an ordered collection of basic data types of a given length. The only key thing here is all the elements of a vector must be of the identical data type e.g homogeneous data structures. Vectors are one-dimensional data structures.

We can create a vector in **R** using the `c()` function. Separate values with commas. Note: other functions, like the `rep()` function, can be used inside the `c()` function.

```
x <- c(10, 12, 3, 4, 5, 5) # Define vector x
x <- c(10, 11 + cos(0), 3:4, rep(5, 2)) # Operations work in c()
x # Print vector x to screen. print(x) also works
[1] 10 12 3 4 5 5
```

25

Functions with Vectors

R makes it very easy to work with vectors. If you put a vector in a function that works on a single number, it will apply that function to every element of the vector.

```
x = c(10, 12, 3, 4, 5, 5) # Define x differently
rep(x, 2) # Repeat x twice
[1] 10 12 3 4 5 5 10 12 3 4 5 5
```

Many functions expect a vector to be entered.

```
length(x) # Report how many elements are in x
[1] 6

# Takes the difference between
diff(x) # values in x that are one apart
[1] 2 -9 1 1 0
```

26

Operations with Vectors

If we perform an arithmetic operation of a number with a vector, then it will do the operation element-wise (to each element in the vector).

```
x + 2 # Add 2 to every element in x
[1] 12 14 5 6 7 7

2*x # Multiply each element in x by 2
[1] 20 24 6 8 10 10

x^2 # Square each element in x
[1] 100 144 9 16 25 25

x > 3 # Check whether each element is greater than 3
[1] TRUE TRUE FALSE TRUE TRUE TRUE
```

27

Subsetting Vectors

If we want to work with only certain elements in a vector, we can subset it using square brackets `[]`.

```
x[1:2] # Takes the first two elements in x
[1] 10 12

x[c(1,2)] # Another way to do that
[1] 10 12
```

We can exclude certain elements in `x` by using negative values in the brackets.

```
x[-c(2,5)] # Excludes the 2nd and 5th elements of x
[1] 10 3 4 5
```

28

Subsetting Vectors

We can also put a logical statement in the `[]` when subsetting.

```
x
[1] 10 12 3 4 5 5

x[x > 5]
[1] 10 12

x[x == 5 | x == 10] # The | means "or"
[1] 10 5 5

x[x > 5 & x < 11] # The & means "and"
[1] 10
```

29

Working with Multiple Vectors

We can also add, subtract, multiply, or divide one vector by another as long as the sizes are the same. These operations will be performed pairwise.

```
y = c(10, 5, 3, 2, 7, 9)
x + y
[1] 20 17 6 6 12 14
```

This next line will give an unexpected result since the sizes are not the same.

```
x + y[1:5]
[1] 20 17 6 6 12 15
```

30

Data Frames

Data frames are generic data objects of **R** which are used to store the tabular data. Data frames are the foremost popular data objects in **R** programming because we are comfortable in seeing the data within the tabular form. They are two-dimensional, heterogeneous data structures. These are lists of vectors of equal lengths.

Data frames have the following constraints placed upon them:

- A data-frame must have column names and every row should have a unique name. If you don't specify the row name, **R** will give it a number by default.
- Each column must have the identical number of items.
- Each item in a single column must be of the same data type.
- Different columns may have different data types.
- To create a data frame we use the `data.frame()` function.

31

Data Frame Example

Let's load a data frame that is already in **R** using the `data()` function. We can check the object we load into memory is a data frame by using the `class()` function.

```
data(USArrests)
class(USArrests)
[1] "data.frame"
```

We can observe the first rows of a data frame using the `head()` function. By default, `head()` will display the first 6 rows, but we can adjust that.

```
head(USArrests, 3) # Displays the first three rows.
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
Alaska	10.0	263	48	44.5
Arizona	8.1	294	80	31.0

32

Data Frame Example

This `USArrests` data frame has 4 columns (variables) and 50 rows. We can check that using the `ncol()` and `nrow()` functions or by using the `dim()` function.

```
ncol(USArrests); nrow(USArrests); dim(USArrests)
[1] 4
[1] 50
[1] 50 4
```

This data frame's columns are named `Murder`, `Assault`, `UrbanPop`, and `Rape`. The `UrbanPop` variable gives the percentage of the state's population that lives in urban areas. The other variables give the number of arrests for that crime per 100,000 people.

There is no state variable. Instead, the rows are named after the states. The command `rownames(USArrests)` will give the names of each row.

33

Subsetting Data Frames

VERY IMPORTANT

We can access a specific column of the data frame using the `$` operator and then typing the name of the variable (or column) that we are interested in.

```
USArrests$Murder
```

```
[1] 13.2 10.0 8.1 8.8 9.0 7.9 3.3 5.9 15.4 17.4
[11] 5.3 2.6 10.4 7.2 2.2 6.0 9.7 15.4 2.1 11.3
[21] 4.4 12.1 2.7 16.1 9.0 6.0 4.3 12.2 2.1 7.4
[31] 11.4 11.1 13.0 0.8 7.3 6.6 4.9 6.3 3.4 14.4
[41] 3.8 13.2 12.7 3.2 2.2 8.5 4.0 5.7 2.6 6.8
```

We can achieve the same thing by using square brackets after the name of the data frame. Since the murder values are in the first column, we can type

```
USArrests[,1] # Will also give the Murder values
```

34

Subsetting Data Frames

If we only want specific rows, we can put those before the comma in the square bracket

```
USArrests[c(1,5),] # Gives the 1st and 5th rows
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
California	9.0	276	91	40.6

We can also select a specific value of a specific column by putting the row number first and the column number second. Or we can use the `$` operator.

```
USArrests[10, 2] # Gives Assault rate for the 10th state
```

```
[1] 211
```

```
USArrests$Assault[10] # Another way to get the same value
```

```
[1] 211
```

35

Creating Data Frames

We can create a data frame using the `data.frame()` function. It is best to give a name to each column and then list what we want for each column.

```
df1 <- data.frame(Name=c("Juan", "Carlos"), Age = c(22, 31))
df1
```

	Name	Age
1	Juan	22
2	Carlos	31

We can add another column to the data frame using the `cbind()` function.

```
vote <- c(TRUE, FALSE) # Create a new vector
df2 <- cbind(df1, vote)
df2
```

	Name	Age	vote
1	Juan	22	TRUE
2	Carlos	31	FALSE

36

Changing Variable Names

You can also use the `rbind()` function to add more rows.

We can change the name of a variable using the `names()` function. We can change the names of the rows too. Suppose I want to change all the variable names to be lowercase and add row names. I can type the following.

```
names(df2) <- c("name", "age", "vote") # Change column names
rownames(df2) <- c("person1", "person2") # Change row names
df2
      name age vote
person1  Juan  22  TRUE
person2 Carlos  31 FALSE
```

37

Factors

Factors are the data objects which are used to categorize the data and store it as levels. They are useful for storing categorical data. They can store both strings and integers. They are useful to categorize unique values in columns like "TRUE" or "FALSE", or "MALE" or "FEMALE", etc.. They are useful in data analysis for statistical modeling.

To create a factor in **R** you need to use the function called `factor()`. The argument to this `factor()` is the vector.

```
fac = factor(c("Male", "Female", "Male",
               "Male", "Female", "Male", "Female"))
fac
[1] Male  Female Male   Male  Female Male   Female
Levels: Female Male
```

38

Working with Factors

We can rearrange the levels of the factor (which are alphabetical by default) also using the `factor()` function.

```
fac = factor(fac, levels = c("Male", "Female"))
fac
[1] Male  Female Male   Male  Female Male   Female
Levels: Male Female
```

We can also force a variable to be a factor using the `as.factor()` function.

```
department = c("Math", "CS", "Other")
class(department)
[1] "character"

dep_factor = as.factor(department)
class(dep_factor)
[1] "factor"
```

39

Section 3

R Functions

40

R Functions

We have already used functions in **R**, but this topic will expand our knowledge of functions and introduce writing our own functions.

First, consider the function `seq()`. If we look at the documentation of the sequence function (by typing `?seq`), it will tell us the syntax is

```
seq(from = 1, to = 1, by = ((to - from)/(length.out - 1)),
    length.out = NULL, along.with = NULL, ...)
```

So, if we type `seq(1, 5, 2)`, then it will output the numbers from 1 to 5 counting by 2s since those are the first three arguments.

```
seq(1, 5, 2)
[1] 1 3 5
```

41

Working with Functions

If I want my third number to instead be the length of the vector, then I need to specify that.

```
seq(1, 5, length.out = 2)
[1] 1 5
```

In fact, it is not required to type the entire name of that `length.out` argument. You just need to type enough letters for **R** to know what argument you are referring to.

```
seq(1, 5, length = 2)
[1] 1 5

seq(1, 5, l = 2)
[1] 1 5
```

Even just typing `l` was enough since no other argument starts with that letter. It is good practice, however, to type enough for clarity.

42

Default Function Inputs

Additionally, many functions will have a default entry for an argument. For example, the `seq()` function, by default, has a lower and upper bound of 1. So, if you just type

```
seq()
[1] 1
```

It gives an output.

Another example: the `sample()` function randomly selects values of an input vector. The syntax is

```
sample(x, size, replace = FALSE, prob = NULL)
```

So, by default it will sample without replacement. Typing `replace = T` or `replace = TRUE` will have it sample with replacement.

Note: we should always put an equal sign = not an arrow `<-` when telling R what our function inputs are in the function.

43

Writing Functions

It is possible to write your own function in R. Writing a function begins with the `function()` function. The syntax is

```
function_name <- function(all inputs) { function_body }
```

Example: if we want to write a function that takes an input, divides it by 2, and then squares it, we will type

```
halfsquare <- function(x) {
  result <- (x/2)^2
  return(result)
}
halfsquare(10)
[1] 25
```

In this function, we specified that `x` was the input, so we can refer to `x` in the function even if `x` was never defined globally.

44

Writing Functions

It is important to make sure the variable we want the function to output is listed on the last line of the function body. Some will argue that should be put in a `return()` function, but it doesn't matter.

We can also give our function default inputs. When defining the function, just let the input equal something. That way, it will still return something even when no input is given.

```
halfsquare <- function(x = 2) {
  result <- (x/2)^2
  result
}
halfsquare(10)
[1] 25
halfsquare()
[1] 1
```

45

Writing Functions

The functions we write can have any number of inputs.

```
diff_func <- function(x, y, z = 0) {
  d <- x - y - z
  d
}
diff_func(100, 10, 1)
[1] 89
```

When writing your own functions, make sure you do not name it anything that is already a built-in R function. For example, there is already a `diff()` function, which is why I named this `diff_func`. This is good practice for all variable names as well.

46

Documenting Functions

When we write a function that is a bit complicated, it is good practice to document the function to explain how it works. This can be done in comments in the function body.

```
diff_func <- function(x, y, z = 0) {
  # Takes the first entered value and subtracts the others.
  # A third number is optional.
  d <- x - y - z
  d
}
```

47

Function Code

If you are curious about the code used to define a function, you can simply type the function name without parentheses.

```
diff_func
function (x, y, z = 0)
{
  d <- x - y - z
  d
}
```

48

Function Code

Many built-in functions, however, simply call a version of the function written more efficiently in the C programming language. Therefore, we cannot see the exact code of the function. It will tell you what environment, or package, the function is in.

```
rnorm
function (n, mean = 0, sd = 1)
.Call(C_rnorm, n, mean, sd)
<bytecode: 0x1176cd870>
<environment: namespace:stats>
```

49

Functions with Same Name

Sometime you will encounter different functions that have the same name. This can occur when you load different **R** packages that each have a function with the same name or when you name your function a name that already exists. **R** can know which function you want to use if you type the name of the packages followed by two colons (:) with no spaces between them.

So, the syntax is

```
package_name::function_name()
```

50

Functions with Same Name

For example, suppose I wrote function to calculate the weighted average of a vector and called that function mean.

```
mean <- function(x, weight) {
  sum(x * weight) / sum(weight)
}
x <- c(1, 2, 3, 4, 5)
w <- c(5, 1, 5, 1, 3) # Vector of weights
mean(x, w)
[1] 2.733333
```

Of course, there already is a built-in mean function that now we cannot use by typing mean(x), since it will only use the new function I wrote. To use the built-in mean() function, which is in the base environment, I can type

```
base::mean(x) # The syntax is package::function()
[1] 3
```

51

Removing Functions

To remove any object that you create, you can use the rm() function. In this case, it would be better if I remove the function called mean() that I wrote and name it something else so there is no confusion. It is not possible to remove a function that is built-in or loaded in with an **R** package.

```
rm(mean)
weighted_mean <- function(x, weight) {
  sum(x * weight) / sum(weight)
}
x <- c(1, 2, 3, 4, 5)
w <- c(5, 1, 5, 1, 3) # Vector of weights
weighted_mean(x, w)
[1] 2.733333
mean(x) # The unweighted, built-in mean function
[1] 3
```

52

Section 4

Data Import and Export

53

Introduction

When working with **R** it is not typical that we will use data sets that are loaded with packages or that we type the data in ourselves. We will often want to read in data files that are stored on our computers.

We can read in data files using a couple **R** functions. But first, it is important to know common ways that data sets are stored.

54

Types of Data Files

There are a couple very common types of data files

- .txt files. These are the standard text file.
- .csv files. These are comma separated value files. Microsoft Excel can save files this way.
- .xlsx files. These are Microsoft Excel files.
- .xls files. These are an older type of Microsoft Excel file.
- .json files. These are files that have data stored in key-value pairs and arrays
- .dat files. These are another type of file that store data.

In this class, we will primarily be using .txt, .csv, and .xlsx files. So we will focus on those.

55

How Data Are Stored

In a data file, we will usually want the data stored in **tidy format**. This indicates that the data file is set up similar to a data frame with one observation per row. Then each row can have the information from all of the variables.

However, it is important to know how the data values are separated from each other. One way to do this is with a space. These are called **space delimited files**. In this case, the data may look like this:

```
Age Weight
31 220
15 94
52 131
```

This is also an example of a data file that has a header. That is, the first row contains the column names, not data values.

56

How Data Are Stored

Another way is to use a comma. If commas are used, it is called a **comma separated value** or **comma delimited** file (even if the file name does not end in .csv).

```
Age,Weight
31,220
15, 94
52,131
```

Spaces in these files are ignored unless they are in quotes.

There are other ways to separate data values like using tab delimiters or new line delimiters. Comma and space are the most common, though.

57

Reading in Data Files

There are two primary functions we can use to read in data files.

- read.csv()
- read.table()

The read.csv() function is simpler, but not as powerful or adaptable as the read.table() function.

58

The read.csv() Function

This function works best for csv files or text files with comma delimiters. The syntax is

```
read.csv(file, header = TRUE, sep = ",", quote = "\"",
         dec = ".", fill = TRUE, comment.char = "#", ...)
```

59

The read.table() Function

This is the more complicated, but more customizable, function. The syntax is given below, but many of these arguments will not be used.

```
read.table(file, header = FALSE, sep = "", quote = "\"",
           dec = ".", numerals = c("allow.loss",
                                   "warn.loss", "no.loss"),
           row.names, col.names, as.is = !stringsAsFactors,
           tryLogical = TRUE,
           na.strings = "NA", colClasses = NA, nrows = -1,
           skip = 0, check.names = TRUE,
           fill = !blank.lines.skip,
           strip.white = FALSE, blank.lines.skip = TRUE,
           comment.char = "#",
           allowEscapes = FALSE, flush = FALSE,
           stringsAsFactors = FALSE, fileEncoding = "",
           encoding = "unknown", text, skipNul = FALSE)
```

60

Using the `read.csv()` and `read.table()` Functions

To use these functions, we need to let **R** know exactly where the file is. We can do this in one of two ways.

- 1 Set the working directory as the folder that contains the file.
- 2 Put the file path in the `read.csv()` or `read.table()` function we are using.

61

Setting the Working Directory

Here are the steps to set a new working directory in **RStudio**.

- 1 Click on "Session" in the menu bar.
- 2 Go down to "Set Working Directory" and then "Choose Directory".
- 3 Navigate to the folder (not file) that contains the file you want to read in.
- 4 Click "Open".

Now the directory will be set and you can put the file name into the `read.csv()` or `read.table()` function.

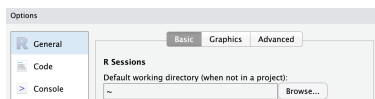
Note: you will need to set the working directory every time you quit and reopen **RStudio**. You can avoid this by changing the default directory to a folder than will contain all of your data files for this class if you'd like.

62

Set the Default Directory

The steps for changing the default directory are:

- 1 Open the **RStudio** settings by clicking on "Tools" from the menu bar and the going down to "Global Options".
- 2 The settings will open (that look like the image below). Click on the Browse... button under the option for default directory.
- 3 Navigate to the folder (not file) that contains the file you want to read in and click "Open".



- 4 Click "Apply" at the bottom right side of the settings pane to save and close the settings.

63

Read Data File using `read.csv()`

Once we have changed our directory, we are ready to read in a file. Suppose I want to read in the file called `USTempLocation.csv`.

Here is what the file looks like if we open it in a text editor.

```
City,JanTemp,Lat,Long
"Mobile,AL",44,31.2,88.5
"Montgomery,AL",38,32.9,86.8
"Phoenix,AZ",35,33.6,112.5
"Little_Rock,AR",31,35.4,92.8
"Los_Angeles,CA",47,34.3,118.7
"San_Francisco,CA",42,38.4,123
"Denver,CO",15,48.7,105.3
"New_Haven,CT",22,41.7,73.4
"Wilmington,DE",26,48.5,76.3
"Washington,DC",38,39.7,77.5
```

64

Read Data File using `read.csv()`

Since this is a csv file, we will begin using the `read.csv()` function.

```
dat <- read.csv("USTempLocation.csv")
head(dat)
```

Notice that by default, the `read.csv()` function will use the first row in the file as column names. We can change this by using the `header=F` argument, but we usually want the first row to be names.

65

Read Data File using `read.table()`

We can also use the `read.table()` function to read in that data file, but a few more options need to be added.

```
dat <- read.table("USTempLocation.csv", sep = ",",
                  header=T)
head(dat)
```

	City	JanTemp	Lat	Long
1	Mobile,AL	44	31.2	88.5
2	Montgomery,AL	38	32.9	86.8
3	Phoenix,AZ	35	33.6	112.5
4	Little_Rock,AR	31	35.4	92.8
5	Los_Angeles,CA	47	34.3	118.7
6	San_Francisco,CA	42	38.4	123.0

66

Read Data File using read.table()

By default, `read.table()` will use a space delimiter, not a comma delimiter, and it will not count the first row as column names. If we had used the default options, then it would have looked like this:

```
dat <- read.table("USTempLocation.csv")
head(dat)
```

```
      City.JanTemp.Lat.Long
Mobile,AL      ,44,31.2,88.5
Montgomery,AL  ,38,32.9,86.8
Phoenix,AZ     ,35,33.6,112.5
Little_Rock,AR ,31,35.4,92.8
Los_Angeles,CA ,47,34.3,118.7
San_Francisco,CA,42,38.4,123
```

Not good!

67

Read Data File using read.table()

The `read.table()` function is also useful when the data file is not comma delimited. For instance, suppose I want to read the file `turkey.txt`, which contains info on the bone lengths of turkey and whether they are wild or not. The data file looks like this:

```
id humerus tibiotarsus type
B093 151 152 0
B102 153 156 0
B103 149 151 0
L641 150 141 0
L783 154 157 0
L985 169 163 0
B710 153 151 0
B790 156 155 0
B819 158 152 0
```

By default, `read.table()` will assume the data is space delimited, like it is in this case.

So we can read it in using this code:

```
turkey <- read.table("turkey.txt", header = T)
head(turkey, 2)
```

```
  id humerus tibiotarsus type
1 B093      151         152    0
2 B102      153         156    0
```

68

Read Data File Without Setting Directory

We can also read a data file without setting the directory to be where the file is beforehand as long as we put the path to the file in the function we are using.

For example, if my data file is on the desktop of my computer, then the file path would be `"~/Desktop/"` and then I would put the file name after that.

For Mac users, the `"~"` character represents the home folder of the logged-in user. You can determine the path to a file by clicking on it and typing `"option-command-c"`.

On Windows, if you click the Start button and then click Computer, click to open the location of the desired file, hold down the Shift key and right-click the file. Then on that menu, you can select `"Copy as path"`.

```
dat <- read.table("~/Desktop/USTempLocation.csv")
head(dat)
```

69

Reading Excel Files

In the `readxl` package is the function `read_excel()`, which can read in files that end in `.xls` or `.xlsx`.

```
install.packages("readxl")
library(readxl)
dat <- read_excel("~/Desktop/USTempLocation.xlsx")
head(dat)
```

If there is more than one sheet in the Excel notebook, you can specify which one using the `sheet` option in the `read_excel` function.

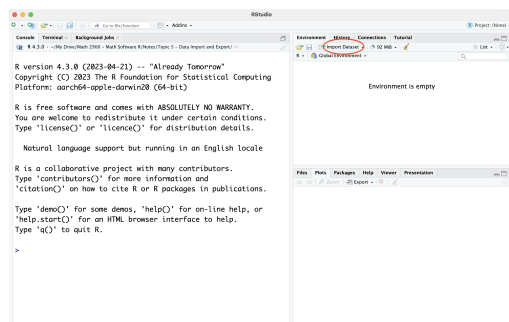
Note: the `read_excel()` function will read in the data file as a *tibble*, not as a data frame. We will discuss tibbles more in Section 5. Tibbles work just like data frames for nearly all practical things we do with them.

70

Reading Data Files Using RStudio Import Dataset

While I recommend using the `read.csv()` or `read.table()` functions because then it is easy to run the code again to read in the file, you can also read in data files using the `"Import Dataset"` button in **RStudio**.

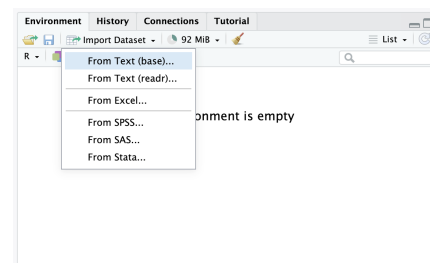
In the Environment tab on the top right, there is an Import Dataset button.



71

Reading Data Files Using RStudio Import Dataset

Clicking on the Import Dataset button will give these options:



Click on `"From Text (base)..."`, navigate to the data file on your computer, and then click `"Open"`.

72

Reading Data Files Using RStudio Import Dataset

That will bring up this window. We then can change the options as needed.

73

Exporting Data

We can export a data frame into a file using the `write.csv()` or `write.table()` function. The syntax of `write.table()` is

```
write.table(x, file = "", append = FALSE, quote = TRUE,
  sep = " ", eol = "\n", na = "NA", dec = ".",
  row.names = TRUE, col.names = TRUE,
  qmethod = c("escape", "double"),
  fileEncoding = "")
```

`x` is the data frame we want to export and then we need to put the file name (and a file path if we don't want it saved in our working directory)

Like the read functions, `write.csv()` is just a simpler version of `write.table()`.

74

Exporting Data

We can make a data frame and then export it as a text file using the following code

```
Xdf = data.frame(age = c(10, 24, 51, 14),
  state = c("UT", "UT", "CA", "CA"))
```

```
Xdf
  age state
1  10    UT
2  24    UT
3  51    CA
4  14    CA
```

```
write.table(Xdf, "age_state.txt", sep = ",")
```

We won't need to do this often, but it is good to be able to save data frames easily into data files.

75

Section 5

Tidyverse

76

Introduction

Modern **R** users are migrating away from many base **R** packages and functions to instead work in the **tidyverse**.

The tidyverse is both a philosophy for coding and organizing data as well as a collection of packages in **R**.

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
```

```
✓ dplyr      1.1.4    ✓ readr      2.1.4
✓ forcats    1.0.0    ✓ stringr    1.5.1
✓ ggplot2    3.4.4    ✓ tibble     3.2.1
✓ lubridate  1.9.3    ✓ tidyr      1.3.0
✓ purrr      1.0.2
```

```
— Conflicts — tidyverse_conflicts() —
```

```
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()     masks stats::lag()
ℹ Use the conflicted package to force all conflicts to become errors
```

77

Tidy Format (murders data)

We say that a data table is in **tidy** format if each row represents one observation and columns represent the different variables available for each of these observations. For example, the following data set is in tidy format:

```
library(dslabs)
data(murders)
head(murders)
```

	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65

78

Not Tidy Format (fertility)

The following dataset is organized, but not tidy. Why?

```
##      country 1960 1961 1962
## 1    Germany 2.41 2.44 2.47
## 2 South Korea 6.16 5.99 5.79
```

79

Tidy Format (fertility)

Here is what the data would look like in tidy format:

	country	year	fertility
1	Germany	1960	2.41
2	South Korea	1960	6.16
3	Germany	1961	2.44
4	South Korea	1961	5.99
5	Germany	1962	2.47
6	South Korea	1962	5.79

The same information is provided, but there are important differences in the format. For the **tidyverse** packages to be optimally used, data need to be reshaped into 'tidy' format. The advantage of working in tidy format allows the data analyst to focus on more important aspects of the analysis rather than the format of the data.

80

Tibbles

A **tibble** is a modern version of a data.frame.

```
library(tidyverse)
dat1 <- tibble(x = 1:4, y = 5:8, z = c("A", "B", "C", "D"))
```

Or convert a data frame to a tibble

```
dat <- data.frame(x=1:4, y = 5:8, z = c("A", "B", "C", "D"))
dat1 <- as_tibble(dat)
```

```
dat1
# A tibble: 4 x 3
      x     y z
  <int> <int> <chr>
1     1     5 A
2     2     6 B
3     3     7 C
4     4     8 D
```

81

Tibbles

Important characteristics that make tibbles unique:

- 1 Tibbles are primary data structure for the tidyverse
- 2 Tibbles display better and printing is more readable
- 3 Tibbles can be grouped
- 4 Subsets of tibbles are tibbles
- 5 Tibbles can have complex entries—numbers, strings, logicals, lists, functions, other tibbles, etc.

82

Subsetting Tibbles

Note: tibbles work just like data frames in just about every way except one. With data frames, using brackets `[]` will give vectors and with tibbles, using `[]` will give tibbles.

```
class(dat[,1]) # Class of first column from a data frame
```

```
[1] "integer"
```

```
class(dat1[,1]) # Class of first column from a tibble
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

```
mean(dat[,1])
```

```
[1] 2.5
```

```
mean(dat1[,1])
```

```
[1] NA
```

83

Subsetting Tibbles

We can choose the columns of a tibble as a vector using the `$` operator or by putting double brackets `[[ ]]`.

```
dat1$x # Gives the first column (whose name is x)
```

```
[1] 1 2 3 4
```

```
dat1[[1]] # Gives the first column
```

```
[1] 1 2 3 4
```

```
mean(dat1[[1]])
```

```
[1] 2.5
```

This subsetting is not a problem for rows. Rows of data frames are data frames and rows of tibbles are tibbles, so nothing of note changes there.

84

Data Import in the Tidyverse

The tidyverse has its own functions that will read in data sets as tibbles. They are the functions

- `read_csv()`
- `read_table()`

These work very much like the `read.csv()` and `read.table()` functions. The primary difference is that `read.csv()` and `read.table()` read in the data as a data frame whereas `read_csv()` and `read_table()` read in the data as a tibble.

The `read_excel()` function in the `readxl` library also reads in data files as tibbles even though the `readxl` library is not technically part of the tidyverse.

85

dplyr Functions

One of the most useful packages in the tidyverse is the **dplyr** package that is used for data wrangling. dplyr is called that since it is a tool (like a set of pliers) for data frames (or tibbles).

The dplyr package has the following useful functions:

- `mutate()` adds new variables that are functions of existing variables.
- `filter()` picks cases based on their values. Selects rows.
- `select()` picks variables based on their names. Selects columns.
- `summarize()` or `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.
- `group_by()` allows you to perform any operation “by group”

Note an important point: most dplyr functions (and most functions in the tidyverse) input a tibble and then output a modified tibble, although many can also work with data frames.

86

Mutate

The function **mutate** takes the data frame or tibble, the instructions for the new columns in next arguments, and returns a modified data frame. For example:

```
murders <- as_tibble(murders)
head(murders)
```

```
# A tibble: 6 x 5
  state   abb region population total
  <chr>   <chr> <fct>      <dbl> <dbl>
1 Alabama AL    South    4779736  135
2 Alaska  AK    West      710231   19
3 Arizona AZ     West    6392017  232
4 Arkansas AR    South    2915918   93
5 California CA    West   37253956 1257
6 Colorado CO    West    5029196   65
```

87

Mutate

To add murder rates, we mutate as follows:

```
murdersRate <- mutate(murders,
  rate = total / population * 100000
)
head(murdersRate)
```

```
# A tibble: 6 x 6
  state   abb region population total rate
  <chr>   <chr> <fct>      <dbl> <dbl> <dbl>
1 Alabama AL    South    4779736  135  2.82
2 Alaska  AK    West      710231   19  2.68
3 Arizona AZ     West    6392017  232  3.63
4 Arkansas AR    South    2915918   93  3.19
5 California CA    West   37253956 1257  3.37
6 Colorado CO    West    5029196   65  1.29
```

88

Filter

Now suppose that we want to filter the data table to only show the entries for which the murder rate is lower than 0.71. We do this as follows:

```
filter(murdersRate, rate <= 0.71)
murdersRate[murdersRate$rate <= 0.71,] # How do get the same
# result without
# filter function
```

```
# A tibble: 5 x 6
  state   abb region population total rate
  <chr>   <chr> <fct>      <dbl> <dbl> <dbl>
1 Hawaii  HI     West    1360301   7  0.515
2 Iowa    IA    North Central 3046355  21  0.689
3 New Hampshire NH    Northeast 1316470   5  0.380
4 North Dakota ND    North Central 672591   4  0.595
5 Vermont VT     Northeast 625741   2  0.320
```

89

Select

If we want to view just a few of our columns, we can use the following:

```
murdersRate <- mutate(murders, rate = total / population * 100000)
murdersRateSelect <- select(murdersRate, state, rate)
filter(murdersRateSelect, rate <= 0.71)
# The above is the same as the following
murdersRate[murdersRate$rate <= 0.71, c("state", "rate")]
```

```
# A tibble: 5 x 2
  state   rate
  <chr>   <dbl>
1 Hawaii  0.515
2 Iowa    0.689
3 New Hampshire 0.380
4 North Dakota 0.595
5 Vermont  0.320
```

90

Nesting Functions

Instead of defining new objects along the way, we could do everything in one complex nested function:

```
filter(select(mutate(murders, rate = total / population * 100000),
  state, rate), rate <= 0.71)
```

```
# A tibble: 5 x 2
  state      rate
  <chr>     <dbl>
1 Hawaii    0.515
2 Iowa      0.689
3 New Hampshire 0.380
4 North Dakota 0.595
5 Vermont   0.320
```

This is fairly concise but a little confusing. Is there a better, clearer way?

91

Pipes

In the previous example, we performed the following wrangling operations:

original data → mutate → select → filter

We can perform a series of operations in **R** by sending the results of one function to another using the **pipe operator**: `|>` that was added in **R** version 4.1.

There is also a pipe (that was actually added first) in the **magrittr** package that is loaded with the tidyverse with the syntax `%>%`. This **magrittr** pipe can do a few things the native pipe cannot¹, but for the vast majority of cases they work the same, so it is recommended to use the native pipe since it is built-in and runs slightly faster.

¹<https://magrittr.tidyverse.org>

92

Pipes

The pipe is a combination of characters that when used properly does two things: *It shortens and simplifies the code* and it makes the code more intuitive to read.

There is a keyboard shortcut in RStudio for inserting the pipe. While you can always just type `|>`, you can also type:

Mac: Command-Shift-M

Windows: Control-Shift-M

However, this will not give you the default pipe unless you change a setting in RStudio. Go to

Tools → Global Options → Code → Select “Use native pipe operator”.

93

Pipes

All the pipe does is provide **forward application** of an object to the first argument of a function. The pipe sends left side of the input to the function to the right of the pipe. For example, if we wanted to calculate

$$\log_2(\sqrt{16})$$

We could use:

```
16 |> sqrt() |> log2()
[1] 2
```

Since the pipe sends values to the first argument, we can define other arguments as follows:

```
16 |> sqrt() |> log(base = 2)
[1] 2
```

While piping works the way it is formatted above, it is better practice to use a new line after each pipe.

94

Pipes (murders)

Creating the prior tibble operation using pipes:

```
murders |>
  mutate(rate = total / population * 100000) |>
  select(state, rate) |>
  filter(rate <= 0.71)
```

```
# A tibble: 5 x 2
  state      rate
  <chr>     <dbl>
1 Hawaii    0.515
2 Iowa      0.689
3 New Hampshire 0.380
4 North Dakota 0.595
5 Vermont   0.320
```

95

Piping into Other Arguments

By default, pipes will send the object being piped to the first argument of the next command, but we can send it to another argument by using the underscore (`_`) placeholder and specifying the argument.

```
murdersRate |>
  lm(rate ~ population, data = _)
```

Call:

```
lm(formula = rate ~ population, data = murdersRate)
```

Coefficients:

```
(Intercept)  population
2.575e+00    3.363e-08
```

We can also use the pipe placeholder along with `$`, `[]`, and `[[]]`:

```
murdersRate |> _$rate |> head(5)
[1] 2.824424 2.675186 3.629527 3.189390 3.374138
```

96

Arrange

We know about the **order** and **sort** functions, but for ordering entire tables, the **arrange** function is much more useful. For example, here we order the tibble by the state's murder rate:

```
murdersRate |>
  arrange(rate) |>
  head()

# A tibble: 6 x 6
  state      abb region      population total rate
  <chr>      <chr> <fct>      <dbl> <dbl> <dbl>
1 Vermont    VT  Northeast    625741     2 0.320
2 New Hampshire NH Northeast    1316470     5 0.380
3 Hawaii     HI   West      1360301     7 0.515
4 North Dakota ND North Central 672591     4 0.595
5 Iowa       IA  North Central 3046355    21 0.689
6 Idaho      ID   West      1567582    12 0.766
```

97

Arrange (descending order)

Note that the default behavior is to order in ascending order. The function **desc** transforms a vector so that it is in descending order. To sort the table in descending order, we can type:

```
murdersRate |>
  arrange(desc(rate)) |>
  head()

# A tibble: 6 x 6
  state      abb region      population total rate
  <chr>      <chr> <fct>      <dbl> <dbl> <dbl>
1 District of Columbia DC South      601723     99 16.5
2 Louisiana      LA   South     4533372    351  7.74
3 Missouri       MO  North Central 5988927    321  5.36
4 Maryland       MD   South     5773552    293  5.07
5 South Carolina SC   South     4625364    207  4.48
6 Delaware       DE   South      897934     38  4.23
```

98

Nested sorting

If we are ordering by a column with ties, we can use a second (or third) column to break the tie. For example:

```
murdersRate |>
  arrange(region, rate) |>
  head()

# A tibble: 6 x 6
  state      abb region      population total rate
  <chr>      <chr> <fct>      <dbl> <dbl> <dbl>
1 Vermont    VT  Northeast    625741     2 0.320
2 New Hampshire NH Northeast    1316470     5 0.380
3 Maine      ME  Northeast    1328361    11 0.828
4 Rhode Island RI Northeast    1052567    16 1.52
5 Massachusetts MA Northeast     6547629   118 1.80
6 New York   NY  Northeast    19378102   517 2.67
```

99

Summarize

The **summarize** function computes summary statistics in an intuitive way. The 'heights' dataset includes heights and sex reported by students in an in-class survey.

```
data(heights)
heights |>
  filter(sex == "Female") |>
  summarize(
    avg = mean(height),
    std_dev = sd(height)
  )

      avg std_dev
1 64.93942 3.760656
```

100

Group then summarize with group_by()

A common operation in data exploration is to first split data into groups and then compute summaries for each group. For example, we may want to compute the average and standard deviation for men's and women's heights separately. We can do the following

```
heights |>
  group_by(sex) |>
  summarize(
    average = mean(height),
    standard_deviation = sd(height)
  )

# A tibble: 2 x 3
  sex      average standard_deviation
  <fct>      <dbl>          <dbl>
1 Female    64.9            3.76
2 Male     69.3            3.61
```

101

pivot_longer() Function

Sometimes it is the case that the data need to be manually put into tidy format. This is where the **pivot_longer()** function can help.

```
prices <- read.csv("data/houseprice.txt")
head(prices, 10)

      gainesville orlando tampa
1          173.0    243.9 230.7
2          145.5    201.1 115.7
3          190.6    185.3 211.0
4          186.3    187.5 203.5
5          248.7    207.9 149.9
6          206.4    234.8 166.8
7           86.8    253.2 134.1
8          204.6    144.7 214.2
9          174.5      NA 105.5
10         220.0      NA 216.2
```

102

pivot_longer() Function

```
new_prices <- pivot_longer(prices, cols = everything())
head(new_prices)

# A tibble: 6 x 2
  name      value
  <chr>    <dbl>
1 gainesville 173
2 orlando    244.
3 tampa      231.
4 gainesville 146.
5 orlando    201.
6 tampa      116.
```

This looks much better! Except the column names are just set to the default of name and value. Also, there are some NA values as we saw in the original data set.

103

pivot_longer() Function

```
house_prices <- pivot_longer(prices, cols = everything()) |>
  na.omit() |>
  rename(city = name, price = value) |>
  arrange(city)
head(house_prices)

# A tibble: 6 x 2
  city      price
  <chr>    <dbl>
1 gainesville 173
2 gainesville 146.
3 gainesville 191.
4 gainesville 186.
5 gainesville 249.
6 gainesville 206.
```

104

pivot_wider() Function

It's relatively rare to need `pivot_wider()` to make tidy data, but it can be useful for creating summary tables for presentation, or data in a format needed by other tools. We won't focus too much on `pivot_wider()`, but here is an example. We do need an "id" or a "row number" variable to make this work.

```
price_wide <- house_prices |>
  mutate(row_num = c(1:11, 1:8, 1:10)) |>
  pivot_wider(names_from = city, values_from = price)
head(price_wide, 3)

# A tibble: 3 x 4
  row_num gainesville orlando tampa
  <int>    <dbl>    <dbl> <dbl>
1     1      173     244.  231.
2     2     146.     201.  116.
3     3     191.     185.  211
```

105

More on the tidyverse

There are some other tidyverse operations, including the `inner_join()`, `left_join()`, `right_join()`, `full_join()`, `pull()`, `dot()`, `reframe()`, `nest_by()`, and `pick()` functions. We will work with a few of these throughout the course.

106

Section 6

Graphing in Base R (not a focus for us)

107

Introduction

One of the primary reasons **R** became so popular is its graphics capabilities. In this topic, we discuss how to create and customize plots.

R has many built-in functions for graph creation:

- `plot()`: used for making scatter plots for two quantitative variables.
- `hist()`: used for making histograms for a quantitative variable.
- `boxplot()`: used for making box plots for a quantitative variable.
- `barplot()`: used for making bar plots for a categorical variable.
- `pie()`: used for making pie charts for a categorical variable (not recommended).

There are many other plots, but we will start here.

108

The plot() Function

The `plot()` function is probably the most versatile and complicated of all the functions listed on the previous slide. The syntax is

```
plot(x, y = NULL, type = "p", xlim = NULL, ylim = NULL,
     log = "", main = NULL, sub = NULL, xlab = NULL,
     ylab = NULL, ann = par("ann"), axes = TRUE,
     frame.plot = axes, panel.first = NULL,
     panel.last = NULL, asp = NA,
     xgap.axis = NA, ygap.axis = NA,
     ...)
```

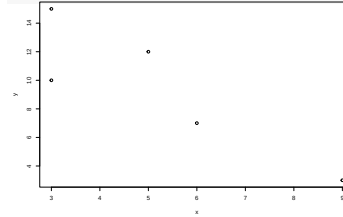
There are many arguments we can use to adjust and customize the output of the plot.

109

The plot() Function

At its most basic, plotting in R is very easy.

```
x <- c(3, 3, 5, 6, 9)
y <- c(15, 10, 12, 7, 3)
plot(x, y) # You can also use plot(y~x)
```



This plot, however, is very boring and possibly difficult to read.

110

Customizing Plots

We can customize plots by using the arguments of the plot function. Here are some commonly arguments and what they do:

- `type`: Changes the type of graph. By default, the graph will put points for each (x,y) pair. This can be changed to "l" for lines, "b" for both lines and points, "h" for vertical lines up to the height of the point, "n" for no output, etc.
- `xlim`: Adjusts the bounds on the x axis. Requires a vector `c(lower, upper)`.
- `ylim`: Adjusts the bounds on the y axis. Requires a vector `c(lower, upper)`.
- `xlab`: Gives the label on the x axis. Requires a string in quotes.
- `ylab`: Gives the label on the y axis. Requires a string in quotes.
- `main`: Gives the title of the plot. Requires a string in quotes.

111

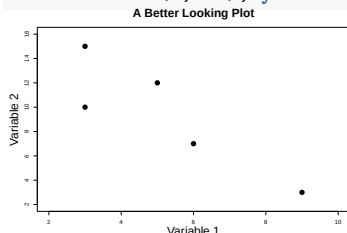
Customizing Plots (Continued)

- `cex`: Adjusts the size of what is plotted. Requires a number. The default size is 1.
- `cex.lab`: Adjusts the font size of the axis labels. Requires a number. The default size is 1.
- `cex.main`: Adjusts the font size of the main title. Requires a number. The default size is 1.
- `pch`: Changes the look of what is plotted. Requires a number or a symbol in quotes.
- `lty`: This changes the type of the line that is plotted if `type = "l"`. Requires a number. The default type is 1 (solid).
- `lwd`: This changes the width of the line that is plotted if `type = "l"`. Requires a number. The default width is 1.
- `col`: Adjusts the color of the plot. Requires a number or a string with the color name in quotes.
- `mgp`: Adjusts where the labels, axis numbers, and axes are on the plot. Requires a vector of length 3. The default is `c(3,1,0)`.

112

The plot() Function Customized

```
x <- c(3, 3, 5, 6, 9)
y <- c(15, 10, 12, 7, 3)
plot(x, y, xlab = "Variable 1", ylab = "Variable 2",
     main = "A Better Looking Plot", pch = 16, cex = 2,
     cex.lab = 2, cex.main = 2, mgp = c(2.5,1,0),
     xlim = c(2, 10), ylim = c(2, 16))
```



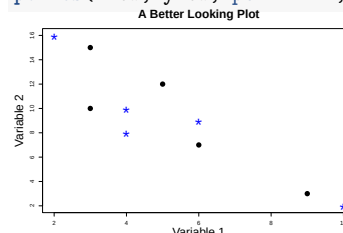
113

The points() and lines() Functions

If we ever want to add more points or additional lines to our graph, we can do so using the `points()` and `lines()` functions, respectively.

These functions do not create a new plot. They take the existing plot and simply add to it. For example, adding on to the previous graph.

```
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
points(xnew, ynew, pch = "*", cex = 3, col = "blue")
```

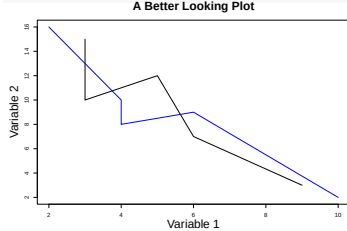


114

The points() and lines() Functions

If we had done a line plot and used the `lines()` function, it would look like this:

```
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
lines(xnew, ynew, pch = "*", cex = 3, col = "blue")
```



115

Creating a Legend

When making a more complicated plot with multiple lines or types of points, it is good practice to include a legend. We can do this with the `legend()` function.

There are lots of arguments for the legend function, but at its most basic, we should just match all the options we used when plotting. We can specify an exact position of the legend using x and y values, or you can just type a position like "topright" or "bottom". Then you need some text as the next argument. An example is on the next slide.

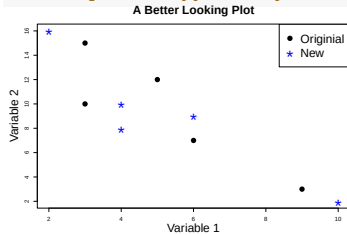
116

Creating a Legend Example

Again, adding to the last plot, we get:

```
legend("topright", c("Original", "New"), pch = c(16, 42),
      cex = c(2, 2), col = c("black", "blue"),
      pt.cex = c(2, 3))
```

In the legend, cex affects text size
and pt.cex affects symbol size.

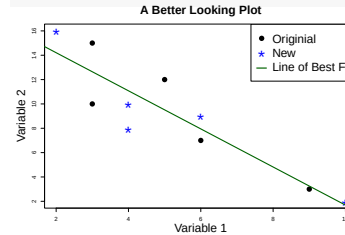


117

Adding a Line to the Plot

We can add a line of best fit through those data using the `abline()` function.

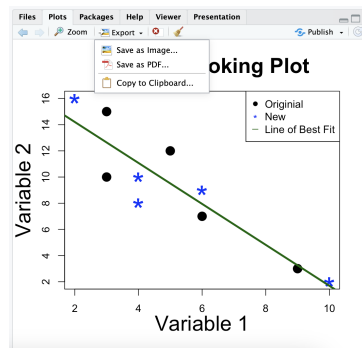
```
abline(17.3377, -1.5649, col = "darkgreen", lwd = 3)
legend("topright", c("Original", "New", "Line of Best Fit"),
      pch=c(16, 42, 95), cex=c(2, 2, 2), pt.cex=c(2, 3, 3),
      col = c("black", "blue", "darkgreen"))
```



118

Saving Plots

You can save a plot by clicking on the Export button in the Plots viewer in **RStudio**.



119

Saving Plots

You can also save plots by using the `pdf()`, `png()`, or `jpeg()` functions. For any of those functions, we can use the following syntax (shown with the `pdf()` function):

```
pdf("file_path/file_name.pdf", width, height, ...)
```

All plotting code should go here

```
dev.off()
```

We end with the `dev.off()` function so **R** knows when the plotting is done.

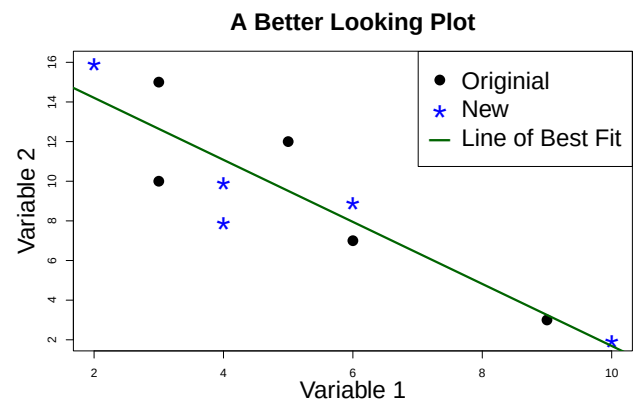
120

Saving Plots Example

```
x <- c(3, 3, 5, 6, 9); y <- c(15, 10, 12, 7, 3)
pdf("better_plot.pdf", width=9, height=6)
plot(x, y, xlab = "Variable 1", ylab = "Variable 2",
     main = "A Better Looking Plot", pch = 16, cex = 2,
     cex.lab = 2, cex.main = 2, mgp = c(2.5,1,0),
     xlim = c(2, 10), ylim = c(2, 16))
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
points(xnew, ynew, pch = "*", cex = 3, col = "blue")
abline(17.3377, -1.5649, col = "darkgreen", lwd = 3)
legend("topright", c("Original", "New", "Line of Best Fit"),
     pch=c(16, 42, 95), cex=c(2, 2, 2), pt.cex=c(2, 3, 3),
     col = c("black", "blue", "darkgreen"))
dev.off()
pdf
2
```

121

Saving Plots Example



122

Other Plots

We mentioned that there are lots of other types of plots:

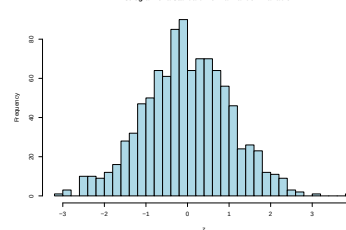
- `hist()`: used for making histograms for a quantitative variable.
- `boxplot()`: used for making box plots for a quantitative variable.
- `barplot()`: used for making bar plots for a categorical variable.
- `pie()`: used for making pie charts for a categorical variable (not recommended).

Many of these have similar syntax to the `plot()` function. As always, you can type "?" before the name of the function to pull up documentation on these functions.

123

Histogram Example

```
set.seed(1) # Sets the random seed so
             # results are reproducible
z <- rnorm(1000) # Generates random values
hist(z, breaks = 30, col = "lightblue",
     main = "Histogram of a Standard Normal Random Variable")
```

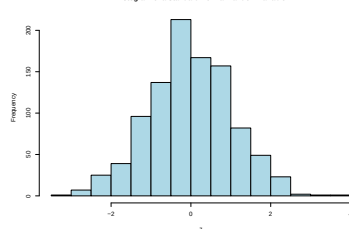


124

Histogram Example

This is a histogram of the same data, but with a different binwidth.

```
hist(z, breaks = 15, col = "lightblue",
     main = "Histogram of a Standard Normal Random Variable")
```

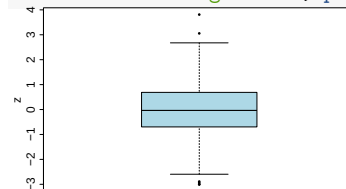


125

Boxplot Example

```
set.seed(1)
z <- rnorm(1000)

boxplot(z, cex.lab = 2, cex.axis = 2, ylab = "z",
        col = "lightblue", pch = 16)
```



126

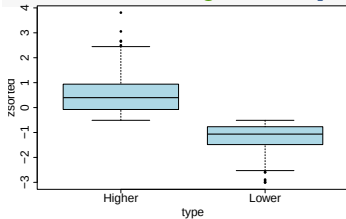
Boxplot Example

Boxplots are most useful for making comparisons.

```
zsorted <- sort(z) # Arrange from least to greatest
type <- c(rep("Lower", 300), rep("Higher", 700))
```

Splits z by type and make two boxplots

```
boxplot(zsorted ~ type, cex.lab = 2, cex.axis = 2,
        col = "lightblue", pch = 16)
```

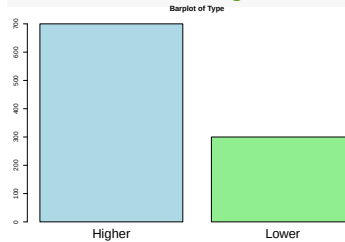


127

Barplot Example

For barplots, we need to use the `table()` function to count the frequencies of each category for a categorical variable.

```
type <- c(rep("Lower", 300), rep("Higher", 700))
barplot(table(type), names = c("Higher", "Lower"),
        cex.names = 2, main = "Barplot of Type",
        col = c("lightblue", "lightgreen"))
```

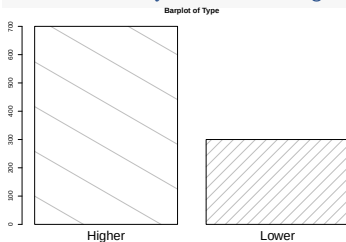


128

Barplot Example

There is lots of customization you can do with barplots. For example the density option determines how close the lines in the barplot are to each other while the angle option will determine the angle of those lines.

```
barplot(table(type), names = c("Higher", "Lower"),
        cex.names = 2, main = "Barplot of Type",
        density = c(1, 5), angle = c(-30, 45))
```



129

Section 7

Graphing with ggplot2 (focus on this)

130

ggplot2 Introduction

While knowing how to plot using the base **R** packages is important, many **R** users are using the ggplot2 package (which is part of the tidyverse) more and more for making better-looking plots.

Advantages of ggplot2

- It's consistent! gg = "grammar of graphics"; easy base system for adding/removing plot elements, with room for being fancy too
- Very flexible
- Themes available to polish plot appearance
- Active maintenance/development = getting better all the time!
- It can do quick-and-dirty and complex, so you only need one system
- Plots, or whole parts of plots, can be saved as objects
- Easy to add complexity or revert to earlier plot

131

Introduction

Disadvantages of ggplot2

- Sometimes more complicated than base **R** plotting
- Difficult to work with in iterated functions
- No 3-D graphics
- ggplot is often slower than base graphics
- The default colors can be difficult to change
- You might need to change the structure of your data frame to make certain plots (use `tidyr::pivot_longer()`)

132

ggplot Basics

There are three primary components to plotting with ggplot2:

- The **data** component. This is what data set and variables we are actually plotting.
- The **geometry** component. This describes what it is we are plotting. Examples include barplots, scatter plots, histograms, smooth densities, qqplots, boxplots, etc.
- The **aesthetic mapping** or just the **mapping**. The two most important cues in this plot are the point positions on the x-axis and y-axis. Each point represents a different observation, and we map data about these observations to visual cues like x- and y-scale. Color is another visual cue that we map to region. How this is defined depends on what type of geometry we are using.

133

Example dataset: Diamonds

```
install.packages("ggplot2")
library(ggplot2)
```

Variable	Description	Values
price	price in US dollars	\$326-\$18,823
carat	weight of the diamond	0.2-5.01
cut	quality of the cut	Fair, Good, Very Good, Premium, Ideal
color	diamond color	J (worst) to D (best)
clarity	measurement of how clear the diamond is	I1 (worst), SI2, SI1, VS2, VS1, VVS2, VVS1, IF (best)
x	length in mm	0-10.74
y	width in mm	0-58.9
z	depth in mm	0-31.8
depth	total depth percentage	43-79
table	width of top of diamond relative to widest point	43-95

Figure 1: Diamonds

134

Example dataset: Diamonds

```
head(diamonds)
# A tibble: 6 x 10
  carat cut      color clarity depth table price     x
  <dbl> <ord>    <ord> <ord> <dbl> <dbl> <int> <dbl>
1  0.23 Ideal      E    SI2     61.5     55   326   3.95
2  0.21 Premium    E    SI1     59.8     61   326   3.89
3  0.23 Good       E    VS1     56.9     65   327   4.05
4  0.29 Premium    I    VS2     62.4     58   334   4.2
5  0.31 Good       J    SI2     63.3     58   335   4.34
6  0.24 Very Good J    VVS2     62.8     57   336   3.94
# i 2 more variables: y <dbl>, z <dbl>
```

135

Example dataset: Diamonds

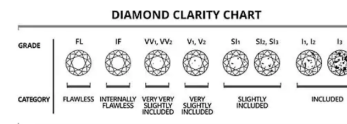


Figure 2: Diamond clarity is a measure of the purity and rarity of the stone, graded by the visibility of these characteristics under 10-power magnification. A stone is graded as flawless if, under 10-power magnification, no inclusions (internal flaws) and no blemishes (external imperfections) are visible.

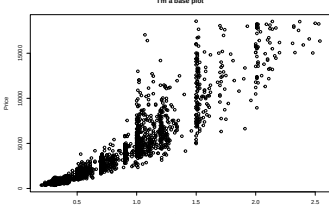
- The dataset contains information about 53,940 round-cut diamonds
- There are 10 variables measuring various pieces of information about the diamonds.
- There are 3 variables with an ordered factor structure: cut, color, & clarity

136

Example in Base Plotting

There is essentially just one primary function to know: `ggplot()`. However, `ggplot()` needs lots of other basic functions.

```
# load the diamonds dataset and take a sample of 2000
data(diamonds); set.seed(2023)
diam <- diamonds[sample(1:53940, 2000),]
plot(diam$carat, diam$price, main = "I'm a base plot",
     xlab = "Carat", ylab = "Price")
```

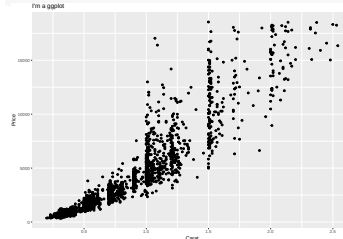


137

Example in ggplot2

In a ggplot, we need to begin with the `ggplot()` function and then add on (literally with a + sign) to that plot using other commands. In this case, I put `geom_point()` to add those solid dots.

```
ggplot(data = diam) +
  geom_point(aes(x = carat, y = price)) +
  labs(x = "Carat", y = "Price", title = "I'm a ggplot")
```

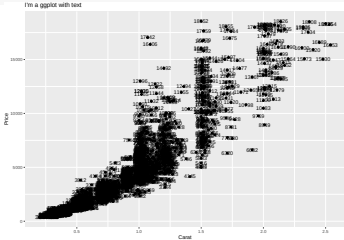


138

Example in ggplot2

Here is an example adding text to the plot.

```
ggplot(data = diam) +
  geom_point(aes(x = carat, y = price)) +
  geom_text(aes(x = carat, y = price, label = price)) +
  labs(x = "Carat", y = "Price",
       title = "I'm a ggplot with text")
```



139

Global vs Local Aesthetic Mapping

Instead of putting the x and y in the `geom_()` function, we can put it in the `ggplot()` and it will apply everywhere.

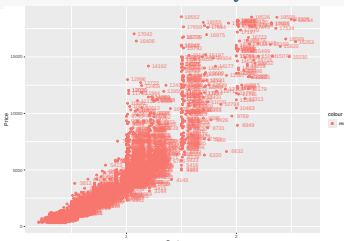
Anything put into the `ggplot()` function will apply globally to the entire plot whereas anything put into the geometry will only apply to that geometry. Some options, like size, can only be put into the geometry.

140

Example in ggplot2

This will make both the points and text red. The `nudge_x` option will move the text to the right 0.1 units.

```
ggplot(data = diam, aes(x = carat, y = price, color = "red")) +
  geom_point() +
  geom_text(aes(label = price), nudge_x = 0.1) +
  labs(x = "Carat", y = "Price")
```

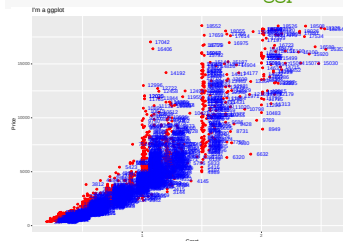


141

Example in ggplot2

This will make the points red, but the text blue.

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point(color = "red") +
  geom_text(aes(label = price), nudge_x = 0.1, color = "blue") +
  labs(x = "Carat", y = "Price",
       title = "I'm a ggplot")
```

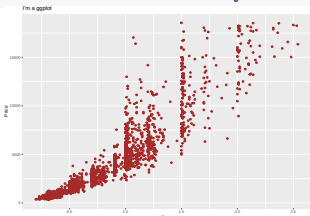


142

Piping in ggplot2

Pipes work very well with ggplot also. Remember that pipes are a part of the tidyverse (in the dplyr package) and by default, piping puts the thing being piped into the first argument of the function.

```
library(tidyverse)
diam |>
  ggplot(aes(carat, price)) + geom_point(col = "brown") +
  labs(x = "Carat", y = "Price", title = "I'm a ggplot")
```

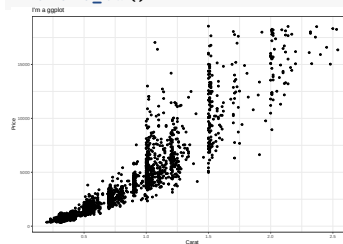


143

Example in ggplot2

We can change the background using `theme_...()`. There are `theme_bw()`, `theme_dark()`, `theme_classic()`, and more.

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point() +
  labs(x = "Carat", y = "Price", title = "I'm a ggplot") +
  theme_bw()
```

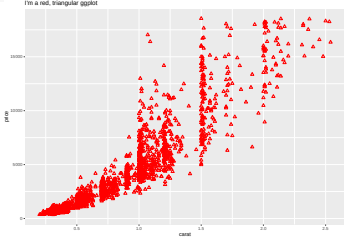


144

Changing the Graph Options in ggplot2

We can change the type of points added in the `geom_point()` function.

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point(size = 2, color = "red", shape = 2) +
  labs(title = "I'm a red, triangular ggplot")
```



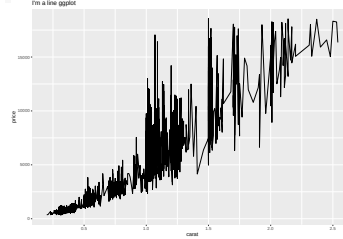
The shape argument works just like pch in base plotting.

145

Changing the Graph Type in ggplot2

Instead of adding `geom_point()`, we can add something else, like `geom_line()`

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_line() + labs(title = "I'm a line ggplot")
```



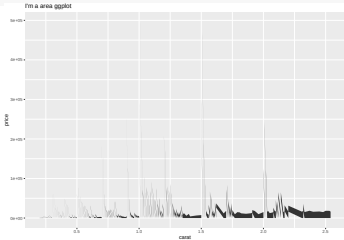
This looks strange for this plot, though.

146

Changing the Graph Type in ggplot2

Or even something like shading the area under the points.

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_area() + labs(title = "I'm a area ggplot")
```



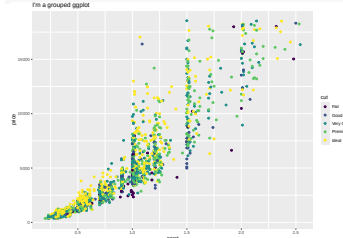
This looks even stranger in this case.

147

Grouping by Another Variable

ggplot makes it easy to split the data using another variable. Simply put the `col` argument in the `aes()` function in ggplot. This will automatically add a legend as well. We can change the shape based on another variable too.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +
  geom_point() + labs(title = "I'm a grouped ggplot")
```



148

Grouping by Another Variable

We can manually change the colors using the `values` option in the `scale_color_manual()` add on function.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +
  geom_point() + labs(title = "I'm a grouped ggplot") +
  scale_color_manual(values = c("red", "orange", "yellow",
                                "green", "lightblue"))
```

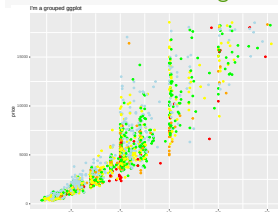


149

Grouping by Another Variable

We can manually change the order of the categories in the legend using the `breaks` option in the `scale_color_manual()` add on function.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +
  geom_point() + labs(title = "I'm a grouped ggplot") +
  scale_color_manual(
    breaks = c("Ideal", "Premium", "Very Good", "Good", "Fair"),
    values = c("lightblue", "green", "yellow", "orange", "red"))
```

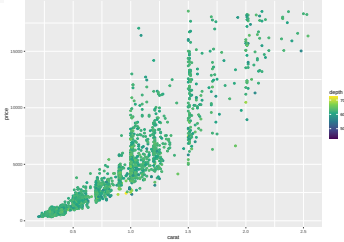


150

Grouping by Another Variable

We can also group by a continuous variable. In this case, we can change the color based on the depth variable.

```
# color by a continuous variable
ggplot(data = diam, aes(x = carat,
  y = price, color = depth)) + geom_point() +
  scale_colour_continuous(type = "viridis")
```

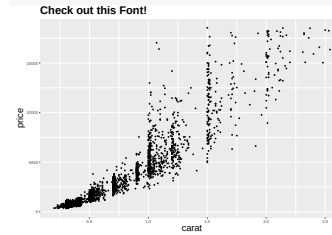


151

Changing Font Size and Type

We can change font size and type in the theme() function:

```
# color by a continuous variable
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point(shape = 16, size = 1.5) +
  labs(title = "Check out this Font!") +
  theme(axis.title = element_text(size = 20),
    plot.title = element_text(size = 24, face = "bold"))
```

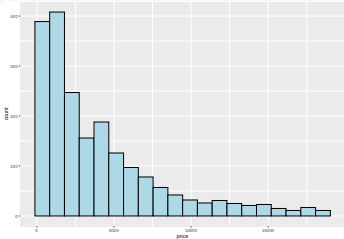


152

ggplot2 for a Single Quantitative Variable: Histogram

Of course, we can also use ggplot() for plotting a single variable. We can make histograms, boxplots, dotplots, etc.

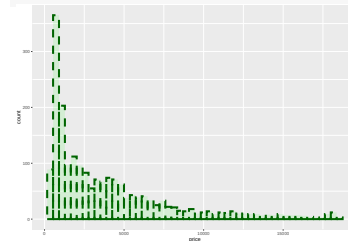
```
# Save the base plot as an object p. Then add to p.
p <- ggplot(data = diam, aes(x = price))
p + geom_histogram(color = "black", fill = "lightblue",
  bins = 20)
```



153

ggplot2 for a Single Quantitative Variable: Histogram

```
# We already created p, so we can add other options to it.
# Smaller alpha makes the plot more transparent.
p + geom_histogram(color = "darkgreen", fill = "green",
  bins = 50, alpha = 0.1, lty = 2, lwd = 2)
```

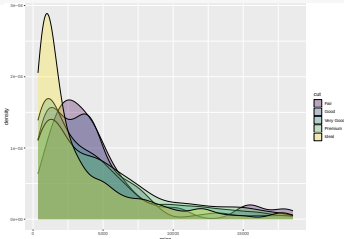


154

ggplot2 for a Single Quant. Variable: Layered Histograms

A layered histogram is a good way to compare the distribution of a variable across groups. geom_histogram works well for two groups, but geom_density is easier to look at for several groups.

```
ggplot(data = diam, aes(x = price, fill = cut)) +
  geom_density(alpha = 0.3)
```

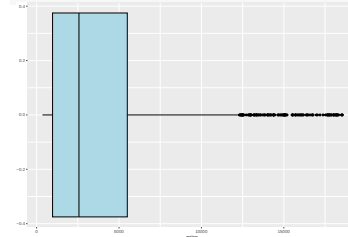


155

ggplot2 for a Single Quantitative Variable: Boxplot

Creating a basic boxplot. We can also make it vertical by putting y = price in the aes() function instead of x = price.

```
ggplot(data = diam, aes(x = price)) +
  geom_boxplot(color = "black", fill = "lightblue")
```

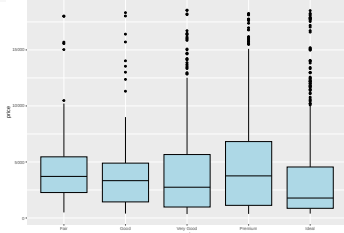


156

ggplot2 for a Single Quant. Variable: Side-by-Side Boxplots

We can make side-by-side boxplots grouped by a categorical variable as x (or as y if you want side-by-side horizontal boxplots).

```
ggplot(data = diam, aes(x = cut, y = price)) +
  geom_boxplot(color = "black", fill = "lightblue")
```

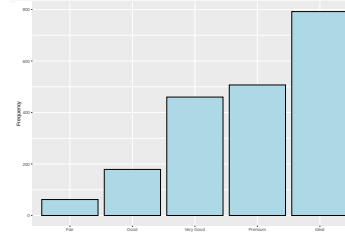


157

ggplot2 for a Single Categorical Variable: Barplot

We can make plots for categorical variables as well.

```
ggplot(data = diam, aes(x = cut)) +
  geom_bar(color = "black", fill = "lightblue") +
  labs(x = "Diamond Cut", y = "Frequency")
```

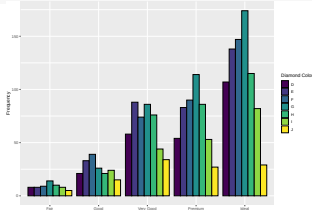


158

ggplot2 for a Single Categ. Variable: Side-by-Side Barplots

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color and put the bars side by side.

```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "dodge") +
  labs(x = "Diamond Cut", y = "Frequency",
       fill = "Diamond Color")
```

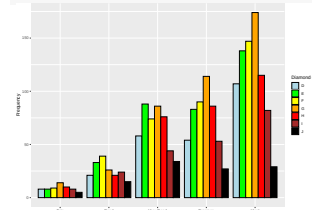


159

ggplot2 for a Single Categ. Variable: Side-by-Side Barplots

We can change the fill colors using `scale_fill_manual()`.

```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "dodge") +
  labs(x = "Diamond Cut", y = "Frequency",
       fill = "Diamond Color") +
  scale_fill_manual(values = c("lightblue", "green", "yellow",
                              "orange", "red", "brown", "black"))
```

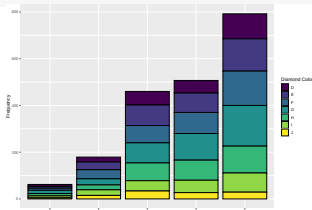


160

ggplot2 for a Single Categorical Variable: Stacked Barplot

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color and stack the bars.

```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "stack") +
  labs(x = "Diamond Cut", y = "Frequency",
       fill = "Diamond Color")
```

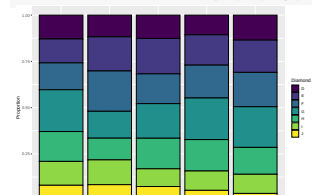


161

ggplot2 for a Single Categorical Variable: Stacked Barplot

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color, stack the bars, and adjust them so each bar totals 100%.

```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "fill") +
  labs(x = "Diamond Cut", y = "Proportion",
       fill = "Diamond Color")
```

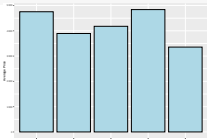


162

ggplot2 Barplot Identity

We often want to use a column of a data frame or tibble as the heights of our bar plot instead of having ggplot tabulate them for us. For this, we need to put `stat = identity` in the `geom_bar()` function.

```
diam |> group_by(cut) |>
  summarize(avg_price = mean(price)) |>
  ggplot(aes(x = cut, y = avg_price)) +
  geom_bar(color = "black", stat = "identity",
           fill = "lightblue") +
  labs(x = "Diamond Cut", y = "Average Price")
```



163

Using ggplot2 for more complex plots

Use `facet_grid()` or `facet_wrap()` to create a separate plot for each value of a factor variable. We don't have to change any of the original plotting code, just add the facet command to it. Faceting can also be done on more than one categorical variable to create a grid of plots.

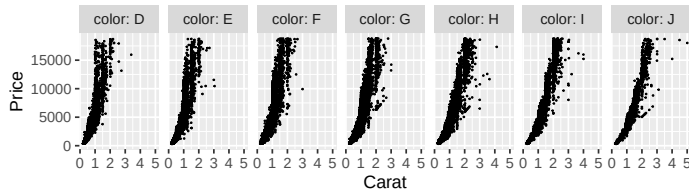
Additionally, it is sometimes helpful to save a simpler version of a plot, and then add onto it later with additional layers (for example, an if/else statement that plots different layers dependent on if a criterion is met or not).

We might want to summarize the data in the previous plot with a smoother on top of the points. With ggplot, we can simply add the `geom_smooth()` command. Each geom just adds another layer to the plot.

164

Using ggplot2 for more complex plots

```
# make the basis for a plot using ggplot save it as p
p <- ggplot(data = diamonds, aes(x = carat, y = price))
# add a geom (points) and display the plot
p + geom_point(size=0.1) + facet_grid(cols = vars(color),
  labeller = label_both) + labs(x = "Carat", y = "Price",
  title = "Carat versus price, separated by color")
Carat versus price, separated by color
```



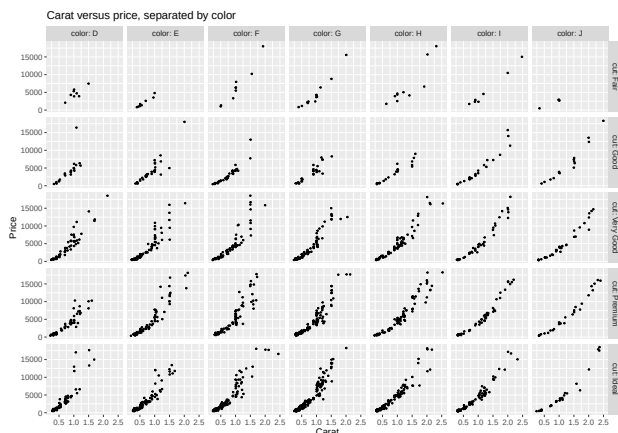
165

Using ggplot2 for more complex plots

```
# make the basis for a plot using ggplot save it as p
p <- ggplot(data = diam, aes(x = carat, y = price))
# add a geom (points) and display the plot
p + geom_point(size=0.1) + facet_grid(rows = vars(cut),
  cols = vars(color), labeller = label_both) +
  labs(x = "Carat", y = "Price",
  title = "Carat versus price, separated by color")
```

166

Using ggplot2 for more complex plots



167

Summary

The syntax of a ggplot is `ggplot(data, aes(x, y))` and you add on to the plot with `+` at the end of each line.

The most useful functions to add onto a ggplot are:

- `geom_point()`, `geom_line()`, `geom_histogram()`, `geom_boxplot()`, `geom_text()`, etc.
- `labs()` for labels including a plot title.
- `lims()` for limits.
- `theme()` for text size and visually changing other things.
- `scale_color_manual()` or `scale_fill_manual()` for changing the color or fill of the plot manually.

168

Further Resources & Assistance

- Cheat sheet for data visualization with ggplot2 (accessible in Rstudio by going to Help -> Cheat Sheets -> Data visualization with ggplot2)
- ggplot2 documentation
- Google
- Stack overflow
- Hadley Wickham's book: <https://ggplot2-book.org/>
- Rafael Irizarry's book: <http://rafalab.dfci.harvard.edu/dsbook-part-1/dataviz/ggplot2.html>.
- Note: these slides were adapted from slides created by Aubrey Odom.

169

Session info

```
sessionInfo()

R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7.2

Matrix products: default
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/liblapack.dylib; LAPACK version 3.12.1

locale:
 [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Denver
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets
[6] methods   base

other attached packages:
[1] knitr_1.51      dslabs_0.9.1    lubridate_1.9.4
[4] forcats_1.0.1   stringr_1.6.0   dplyr_1.1.4
[7] purrr_1.2.0     readr_2.1.6     tidyr_1.3.2
[10] tibble_3.3.0    ggplot2_4.0.1   tidyverse_2.0.0

loaded via a namespace (and not attached):
[1] gtable_0.3.6      jsonlite_2.0.0
[3] compiler_4.5.2    tinytex_0.58
[5] tidyselect_1.2.1  scales_1.4.0
[7] yaml_2.3.12       fastmap_1.2.0
[9] R6_2.6.1          labeling_0.4.3
[11] generics_0.1.4    pillar_1.11.1
[13] RColorBrewer_1.1-3 tzdb_0.5.0
```

170

Notes 3 - Quarto

Math 3190 - Fundamentals of Data Science

Rick Brown
Southern Utah University

1

Table of Contents

- 1 Quarto
- 2 Python in Quarto

2

Quarto

Section 1

Quarto

3

Quarto

Reproducible Reports

The final product of a data analysis project is often a report: scientific publications, news articles, an analysis report for your company, or lecture notes for a class.

Now imagine after you are done you realize you:

- had the wrong data set
- have a new data set for the same analysis
- made a mistake and fix the error, or
- your boss or someone you are training wants to see the code and be able to reproduce the results

Situations like these are common for a data scientist.

4

Quarto

Quarto

Quarto is a format for **literate programming** documents.

It is based on **markdown**, a markup language that is widely used to generate html pages.

You can learn more about markdown here:
<https://www.markdowntutorial.com/>

Quarto is similar to environments like Jupyter Notebooks, Google Colab, and R Markdown (R Markdown is very much like Quarto, but is dependent on **R**). However, Quarto is language agnostic. This means it natively works in both **R** and Python (and Julia and Observable JavaScript).

You can download Quarto here: <https://quarto.org/docs/download/>.

5

Quarto

Quarto Setup

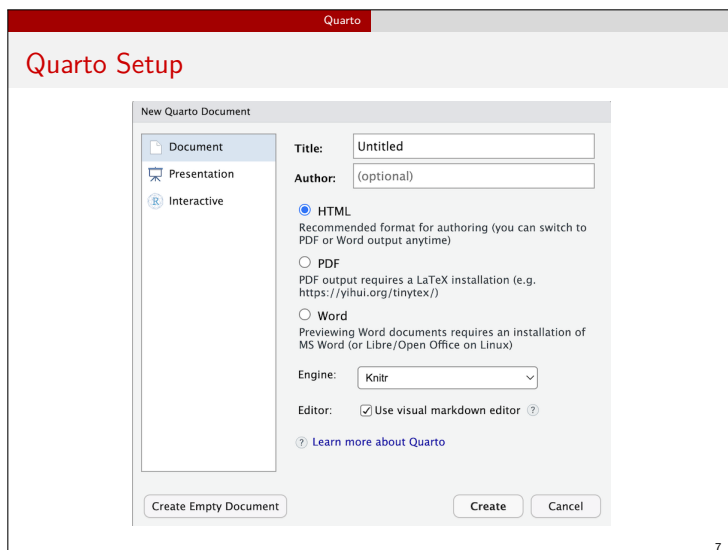
Literate programming weaves instructions, documentation, and detailed comments in between machine executable code.

With Quarto, you need to **compile** the document into the final report. This allows the code to run in a clean workspace each time, so it is reproducible and gives the same results each time.

You can start a Quarto document in Positron by clicking on **New** then **Quarto Document**. You will then be asked for a title and author.

Final reports can be to be in: HTML, PDF, Microsoft Word, or presentation formats like PowerPoint, Beamer, or HTML slides.

6



7

Quarto Setup

As a convention, we use the .qmd extension for these files.

Once you gain experience with Quarto, you will be able to do this without the template and can simply start from a blank template.

In the template, you will see several things to note (in the following slides).

First, the default when opening a Quarto document is that it is in **Visual** editing mode. This will likely be a little more comfortable for you if you are not used to coding in something like LaTeX.

8

Quarto Visual

9

Quarto Source

However, we will focus on the source editor since it is much more customizable (although you are not prohibited from using the visual editor).

10

The Header

At the top of either the visual or source editor, you will see the following:

```
---
title: "Untitled"
format: html
editor: visual
---
```

This is known as the YAML header. YAML stands for YAML Ain't Markup Language.

One parameter that we will highlight is format. By changing this to, say, pdf or docx, we can control the type of output that is produced.

11

The Header

There are many, many options that can be adjusted in the YAML.

```
---
title: "Project Title"
author: "Your Name"
date: "1/1/2026"
format:
  html:
    code_folding: hide
    toc: true
    theme: "flatly"
editor: source # Makes the source editor the default
               # when file is opened.
# Like R, you can put comments in the YAML.
---
```

12

R Code Chunks

In various places in the document, we see something like this:

```
```{r}
1 + 1
```
```

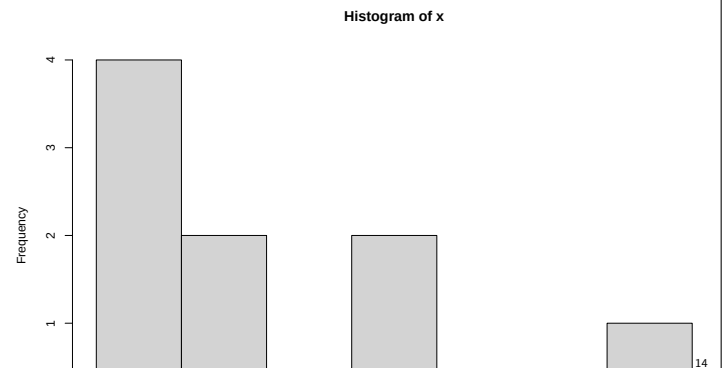
These are the code chunks. When you compile the document, the **R** code inside the chunk, in this case `1+1`, will be evaluated and the result included in that position in the final document.

To add your own **R** chunks, you can type the characters above quickly with the key binding `Command-Option-I` (that is a capital i) on the Mac and `Ctrl-Alt-I` on Windows. You can also click on the green code chunk button in the toolbar at the top.

13

R Code Chunks

These code chunks can be used to show code, but that code will also run. Even plots can be made, although this plot is too big to fit on this slide. We will see soon how to adjust the size of plots.



14

R Code Chunks

By default, both the code and the output will show up in your document. To avoid having the code show up, you can use an argument. To avoid this, you can set options for that code chunk using a special `#|` comment in the chunk. For example:

```
```{r}
#| echo: false
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
```

These comments need to be put at the beginning of the code chunk. Notice that this is YAML, not **R** code. That is why `false` is lowercase while in **R** it would be all capitalized.

15

R Code Chunks

It's a good habit to add a label to the **R** code chunks. This can be useful when debugging, among other situations. You do this by using the `#| label: comments:`

```
```{r}
#| label: x_hist
#| echo: false
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
```

16

Global Options and knitr

If there are certain options you want to apply to all code chunks, you can specify them in the YAML header with `execute:`. Be aware that white space matters in the YAML. The `echo` and `eval` options need to be indented, need to have a colon, and need to have one space after that colon.

```
---
title: "Untitled"
format: html
execute:
  echo: false
  eval: true
---
```

We use the `knitr` package to compile Quarto documents based on **R**, so we need to install that package. The specific function used to compile is the `knit()` function, which takes a filename as input. RStudio provides a Render button that does that for you, though.

17

Adding Equations

You can add formatted equations using between dollar signs `$`. Example, if you type

```
$x^2+\frac{1}{2}$
```

It will look like $x^2 + \frac{1}{2}$. In fact, nearly all LaTeX code works in Quarto.

If you put two dollar signs, the equation will appear on its own line.

```
$$\sum_{i=1}^{10}\left(\lambda_i+\frac{10}{\alpha}\right)$$
```

$$\sum_{i=1}^{10} \left(\lambda_i + \frac{10}{\alpha} \right)$$

18

Adding and Adjusting Images

We can add images to our Quarto file by using

```
! [image_caption_goes_here] (image_name.ext) {width=50%}
```

if the image is already in the same directory. Otherwise, you can put the path to the file in those parentheses. We can adjust the width of the image as well by changing the number in the width=50%.

19

Adding and Adjusting Images

If the image is generated by **R** code, we can adjust the size by using `fig-width`, `fig-height`, `out-width` and/or `out-height` options in the code chunk.

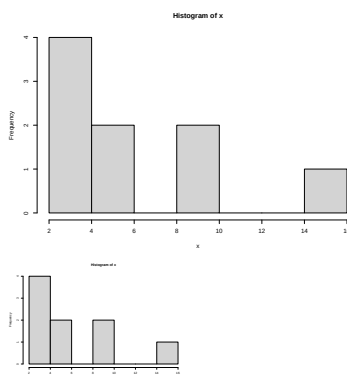
```
```{r}
#| label: image_code
#| label: image_code
#| eval: true
#| echo: true
#| out-width: "25%"
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
```

The result is shown on the next slide.

20

Adding and Adjusting Images

The first image has `out-width: "50%"` and the second has `out-width: "25%"`



21

Other Quarto Options

We will explore other Quarto options: headers, tabsets, and latex equations in class and in your homework.

Note: From now on, all homework should be completed using a Quarto document (uploaded to GitHub). Your document should include headers, descriptive text, **R** code, and plots/figures!

22

More on Quarto

There is a lot more you can do with Quarto. I highly recommend you continue learning as you gain more experience writing reports in **R**. There are many free resources on the internet including:

- 1 R Studio's tutorial: <https://quarto.org>
- 2 The Quarto guide: <https://quarto.org/docs/guide/>
- 3 The Quarto definitive guide: <https://bookdown.org/yihui/rmarkdown>
- 4 The knitr book: <https://yihui.name/knitr/>
- 5 L^AT_EX command guide: <https://www.bu.edu/math/files/2013/08/LongTeX1.pdf>

23

Extra - Tips

- Make sure you Render your document often. That way you will find any errors quickly without having to search for them.
- Comments in **R** chunks still are the familiar `#`. But comments in Quarto documents outside of **R** chunks are `<!-- comment here -->`.
- You can bulk comment by highlighting all lines you want commented and then typing `Cmd-Shift-C` on a Mac or `Ctrl-Shift-C` on Windows.

24

Extra - Mac Quick Look Previewing

If you use the preview feature in macOS (pressing the space bar when highlighting a file in preview), you'll notice that there is no preview available for .md files, .Rmd files, or files without an extension. You can install [QLMarkdown](#) and [Syntax Highlight](#) to enable quick viewing of those files.

25

Section 2

Python in Quarto

26

Python Intro

One useful feature in Quarto is the ability to run code chunks from other languages. One of the most useful is the ability to run Python inside of Quarto.

There are two ways to do this:

- 1 Run Python natively.
- 2 Run Python through **R**.

27

Run Python in Quarto Natively

If you have Python installed, you can put `engine: jupyter` or `engine: jupyter3` in the YAML. Then you can put Python code chunks like this:

```
```{python}
import numpy as np
np.array([[1, 2, 3], [4, 5, 6]])
```
```

Note that **R** code chunks won't work when using the `jupyter` engine unless other options are specified.

28

Run Python in Quarto through **R**

We can run Python in Quarto through **R** using the `reticulate` package. This will make it a bit slower than running Python natively.

```
install.packages("reticulate")
library(reticulate)
```

29

Installing Python and Python Packages

To install Python, we can run the `install_python()` function from the `reticulate` package. Just indicate which Python version you want to install. As of the time these notes were created, the latest version is 3.12.1, but you can look up the latest version or go with the default.

```
install_python("3.14.0")
```

This can take a while to install. Please be patient!

To install Python packages, we need to use an **R** code chunk using the `py_install()` function.

```
py_install(packages = "matplotlib")
py_install(packages = "numpy")
py_install(packages = "pandas")
```

30

Running Python

Once we have installed Python, we can insert a Python code chunk just like an **R** code chunk:

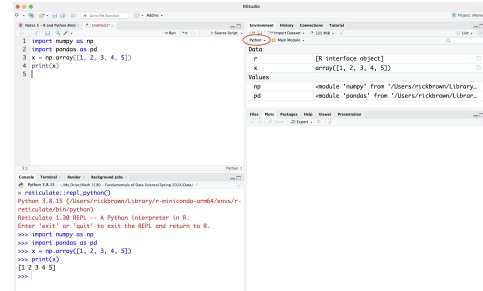
```
```{python}
import numpy as np
import pandas as pd
x_python = np.array([1, 2, 3, 4, 5])
print(x_python)
```

[1 2 3 4 5]
```

31

Python Environment

When running Python in RStudio, the arrows in the console change and the environment switched to a Python environment. We can view what objects are stored in the **R** and Python environments by clicking on the dropdown button.



32

Switch Between Python and R

We can switch from an **R** environment (indicated by a single >) to a Python one (indicated by three arrows >>>) by

- Running a Python code chunk.
- Running the line `reticulate::repl_python()`.
- Double clicking on a Python object in the Python environment list.
- Clicking on the **R** icon at the top left of the Console to bring up a menu and then select Python.

We can switch from a Python environment back to **R** by

- Running an **R** code chunk.
- Typing `exit`.
- Clicking on the Python icon at the top left of the Console to bring up a menu and then select **R**.

33

Using Python Objects in R and Vice-Versa

We can send objects from Python to **R** and vice-versa. Everything defined in Python is stored in the `py` object, which can be called from **R** using `py$<name_of_object>`. To use an **R** object in Python, we can use preface it with `r.`, so it would be `r.<name_of_object>`.

R Code. x_python was defined earlier.

```
library(reticulate)
```

```
py$x_python
```

```
[1] 1 2 3 4 5
```

```
y = 2 * py$x_python # Still R code
```

Python Code

```
print(r.y)
```

```
[ 2.  4.  6.  8. 10.]
```

34

Type Conversions

While many objects are stored similarly in **R** and Python, it is good to know how things are stored when they are passed back and forth. This table shows the conversion:

| R | Python |
|------------------------|-------------------|
| Single-element vector | Scalar |
| Multi-element vector | List |
| List of multiple types | Tuple |
| Named list | Dict |
| Matrix/Array | NumPy ndarray |
| Data Frame | Pandas DataFrame |
| Function | Python function |
| Raw | Python bytearray |
| NULL, TRUE, FALSE | None, True, False |

35

Session info

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7.2

Matrix products: default
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBL
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12

locale:
 [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Denver
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base

other attached packages:
[1] reticulate_1.44.1

loaded via a namespace (and not attached):
 [1] digest_0.6.39    fastmap_1.2.0    xfun_0.55       Matrix_1.7-4
 [5] lattice_0.22-7   knitr_1.51       httools_0.5.9   png_0.1-8
 [9] rmarkdown_2.30   tinytex_0.58     cli_3.6.5       grid_4.5.2
[13] compiler_4.5.2   rprojroot_2.1.1  here_1.0.2      ratstudioapi_0.17.1
[17] tools_4.5.2      evaluate_1.0.5   Rcpp_1.1.0      yaml_2.3.12
[21] xlang_1.1.6      jsonlite_2.0.0
```

36

Notes 4 - Databases and SQL

Math 3190 - Fundamentals of Data Science

Rick Brown
Southern Utah University

1

1 Databases and SQL

2 SQL in Quarto

3 Local Databases

2

Databases and SQL

Section 1

Databases and SQL

3

Databases and SQL

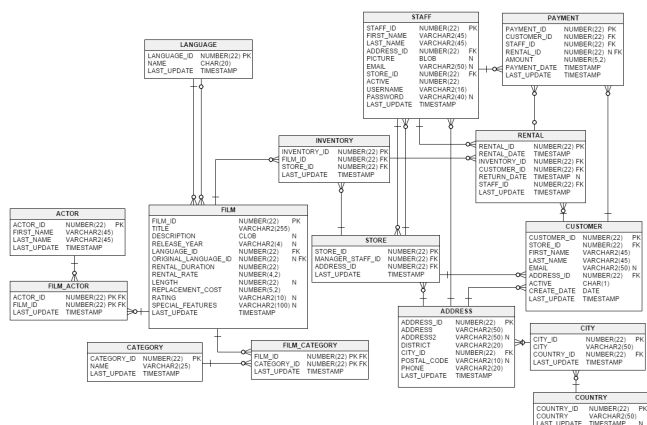
Intro to Database Design

In many businesses and government agencies, data files are not directly accessible. Instead, data are stored in databases that are usually in the cloud. The most common type of database is a *relational database*. These types of databases are made up of **tables**, which are very similar to data frames in **R**. These tables usually have at least one variable in common with at least one other table that can be used to connect them.

4

Databases and SQL

Relational Database Example



5

Databases and SQL

SQL Intro

The most common way to access information stored in relational databases is with SQL.

SQL stands for structured query language and is ubiquitous in business, government, and academia where large data sets are used. SQL is usually either pronounced like "sequel" or the letters are individually said "S-Q-L".

We are going to go over the common SQL commands and then look at how to access SQL databases and use SQL within **R**.

SQL is one of the most basic "coding languages" there is because of how similar the statements are to spoken English.

6

SELECT, FROM, and WHERE Statements

The backbone of SQL are the SELECT, FROM, and WHERE statements.

- SELECT is very much like the `select()` function in the tidyverse (dplyr package). It selects the columns or variables from the database. You can use "*" to select all columns.
- FROM is the statement that specifies the table in the database from which we are taking the data.
- WHERE is similar to the `filter()` function in the tidyverse. This will subset the table based on some criterion or criteria.

Note: these words do not *need* to be written in all capital letters, but it is the convention to type them in all caps for readability.

7

SQL Syntax

The basic syntax for SQL statements is as follows:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

8

SQL Example

Suppose we have a database named sakila that models a DVD rental store, featuring things like films, actors, film-actor relationships. In this database are many tables, one of which is "film". Here is some information about the film table:

Let's run some SQL queries with this table.

| film | |
|----------------------|-----------|
| film_id | smallint |
| title | varchar |
| description | text |
| release_year | year |
| language_id | tinyint |
| original_language_id | tinyint |
| rental_duration | tinyint |
| rental_rate | decimal |
| length | smallint |
| replacement_cost | decimal |
| rating | enum |
| special_features | set |
| last_update | timestamp |

9

SQL Query 1

```
SELECT title, release_year, rating
FROM film
WHERE rating = 'PG';
```

Table 1: My Caption

| title | release_year | rating |
|------------------|--------------|--------|
| ACADEMY DINOSAUR | 2006 | PG |
| AGENT TRUMAN | 2006 | PG |
| ALASKA PHANTOM | 2006 | PG |
| ALI FOREVER | 2006 | PG |
| AMADEUS HOLY | 2006 | PG |

10

WHERE Statement Commands

The following can be used in a WHERE statement:

| Operator | Description |
|----------|---|
| = | Equal |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal |
| <= | Less than or equal |
| <> or != | Not equal |
| AND | Restricts the output based on another criterion. |
| OR | Expands the output based on another criterion. |
| NOT | Gives the opposite of the criterion. |
| BETWEEN | Between a certain range. |
| IN | To specify multiple possible values for a column. |
| LIKE | Search for a pattern. |

11

SQL Query 2: BETWEEN

The syntax for the BETWEEN statement is

```
WHERE column_name BETWEEN value1 AND value2
SELECT film_id, title, length, rental_duration
FROM film
WHERE film_id BETWEEN 13 AND 14
OR film_id BETWEEN 21 AND 23;
```

Table 2: Displaying records 1 - 5

| film_id | title | length | rental_duration |
|---------|----------------------|--------|-----------------|
| 13 | ALI FOREVER | 150 | 4 |
| 14 | ALICE FANTASIA | 94 | 6 |
| 21 | AMERICAN CIRCUS | 129 | 3 |
| 22 | AMISTAD MIDSUMMER | 85 | 6 |
| 23 | ANACONDA CONFESSIONS | 92 | 3 |

12

SQL Query 3: IN

The IN operator is a shorthand for multiple OR conditions.

```
SELECT film_id, title, length, rental_duration
FROM film
WHERE film_id IN ("1", "50", "100", "613");
```

Table 3: Displaying records 1 - 4

| film_id | title | length | rental_duration |
|---------|------------------|--------|-----------------|
| 1 | ACADEMY DINOSAUR | 86 | 6 |
| 50 | BAKED CLEOPATRA | 182 | 3 |
| 100 | BROOKLYN DESERT | 161 | 7 |
| 613 | MYSTIC TRUMAN | 92 | 5 |

Note: the quotes aren't needed for numbers, but can still be used.

13

SQL Query 4: LIKE

There are two wildcards often used in conjunction with the LIKE operator:

- The percent sign % represents zero, one, or multiple characters.
- The underscore sign _ represents one single character. This can be repeated to stand for multiple characters.

```
SELECT film_id, title, length, rental_duration
FROM film
WHERE title LIKE "b%" AND length >= 175;
```

Table 4: Displaying records 1 - 4

| film_id | title | length | rental_duration |
|---------|------------------|--------|-----------------|
| 50 | BAKED CLEOPATRA | 182 | 3 |
| 61 | BEAUTY GREASE | 175 | 5 |
| 88 | BORN SPINAL | 179 | 7 |
| 94 | BRAVEHEART HUMAN | 176 | 7 |

14

SQL Query 5: LIKE

```
SELECT film_id, title, length, rental_duration
FROM film
WHERE title LIKE "__ab%";
```

Table 5: 3 records

| film_id | title | length | rental_duration |
|---------|-----------------|--------|-----------------|
| 9 | ALABAMA DEVIL | 114 | 3 |
| 34 | ARABIA DOGMA | 62 | 6 |
| 773 | SEABISCUIT PUNK | 112 | 6 |

15

More on SELECT Statments

In the SELECT statement, instead of simply putting column names, we can also use the following functions:

- MIN() returns the minimum value of a numeric column.
- MAX() returns the maximum value of a numeric column.
- COUNT() returns the number of rows that matches a specified criterion.
- SUM() returns the total sum of a numeric column.
- AVG() returns the average value of a numeric column.

Additionally, we can use aliases to rename columns with the AS statement.

16

SQL Query 6: SELECT Functions

```
SELECT AVG(length) AS "Average Length"
FROM film;
```

Table 6: 1 records

| Average Length |
|----------------|
| 115.272 |

17

SQL Query 7: SELECT Functions

```
SELECT COUNT(title) AS "Number of A or B Movies"
FROM film
WHERE title LIKE "a%" OR title LIKE "b%";
```

Table 7: 1 records

| Number of A or B Movies |
|-------------------------|
| 109 |

18

SQL Query 8: SELECT Functions

The following will return all the films with a length higher than the average length.

```
SELECT title, length
FROM film
WHERE length > (SELECT AVG(length) FROM film);
```

Table 8: Displaying records 1 - 5

| title | length |
|------------------|--------|
| AFFAIR PREJUDICE | 117 |
| AFRICAN EGG | 130 |
| AGENT TRUMAN | 169 |
| ALAMO VIDEOTAPE | 126 |
| ALASKA PHANTOM | 136 |

19

ORDER BY Statement

We can also use the ORDER BY statement to change the order in which the data table is given.

The syntax for ORDER BY is

```
ORDER BY column1, column2, ... columnN
```

We can put a ASC for ascending or DESC for descending values after any one of the columns we are ordering by.

20

SQL Query 9: ORDER BY Statement

```
SELECT title, length
FROM film
WHERE length > (SELECT AVG(length) FROM film)
ORDER BY length ASC, title DESC;
```

Table 9: Displaying records 1 - 5

| title | length |
|------------------------|--------|
| WORDS HUNTER | 116 |
| MADIGAN DORADO | 116 |
| INSTINCT AIRPORT | 116 |
| DUCK RACER | 116 |
| RESURRECTION SILVERADO | 117 |

21

GROUP BY Statement

The GROUP BY statement groups rows that have the same values into summary rows. It is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

22

SQL Query 10: GROUP BY Statement

```
SELECT AVG(length), rating
FROM film
WHERE release_year = 2006
GROUP BY rating
ORDER BY rating;
```

Table 10: Displaying records 1 - 5

| AVG(length) | rating |
|-------------|--------|
| 111.0506 | G |
| 113.2286 | NC-17 |
| 112.0052 | PG |
| 120.4439 | PG-13 |
| 118.6615 | R |

23

SELECT DISTINCT Statement

The SELECT DISTINCT statement is used to return only distinct (different) values.

Syntax:

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

24

SQL Query 11: DISTINCT Statement

```
SELECT DISTINCT release_year
FROM film;
```

Table 11: 1 records

| release_year |
|--------------|
| 2006 |

So this database only has films from the year 2006.

25

UNION Statement

The UNION operator is used to combine the result-set of two or more SELECT statements.

Syntax:

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

26

SQL Query 12: UNION Statement

```
SELECT AVG(length) AS "Average Length and Payment Amount"
FROM film
UNION
SELECT AVG(amount)
FROM payment;
```

Table 12: 2 records

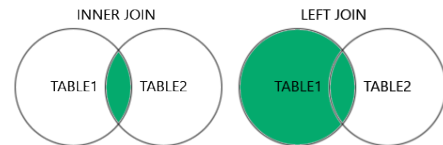
| Average Length and Payment Amount |
|-----------------------------------|
| 4.200667 |
| 115.272000 |

27

JOIN Statements

A JOIN clause is used to combine rows from two or more tables, based on a related column between them. There are four types of JOIN statements:

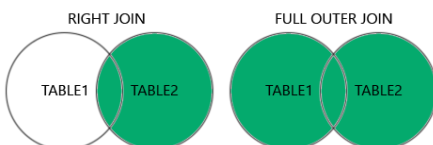
- (INNER) JOIN: Returns records that have matching values in both tables.
- LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table.



28

JOIN Statements

- RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table.
- FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table.



29

JOIN Statements

The syntax for JOIN statements is as follows:

```
SELECT column_name(s)
FROM table1
(JOIN STATEMENT) table2
ON table1.column_name = table2.column_name;
```

Replace the (JOIN STATEMENT) with INNER JOIN, LEFT JOIN, RIGHT JOIN, or FULL JOIN.

You can select columns from either table, so it is common to denote which table we are selecting from by using table.column_name.

30

JOIN Examples

Consider the following two tables called Student and StudentCourse, respectively.

| ROLL_NO | NAME | ADDRESS | PHONE | Age |
|---------|----------|-----------|------------|-----|
| 1 | HARSH | DELHI | XXXXXXXXXX | 18 |
| 2 | PRATIK | BIHAR | XXXXXXXXXX | 19 |
| 3 | RIYANKA | SILIGURI | XXXXXXXXXX | 20 |
| 4 | DEEP | RAMNAGAR | XXXXXXXXXX | 18 |
| 5 | SAPTARHI | KOLKATA | XXXXXXXXXX | 19 |
| 6 | DHANRAJ | BARABAJAR | XXXXXXXXXX | 20 |
| 7 | ROHIT | BALURGHAT | XXXXXXXXXX | 18 |
| 8 | NIRAJ | ALIPUR | XXXXXXXXXX | 19 |

| COURSE_ID | ROLL_NO |
|-----------|---------|
| 1 | 1 |
| 2 | 2 |
| 2 | 3 |
| 3 | 4 |
| 1 | 5 |
| 4 | 9 |
| 5 | 10 |
| 4 | 11 |

31

JOIN Example: INNER JOIN

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE
FROM Student
INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

| COURSE_ID | NAME | Age |
|-----------|----------|-----|
| 1 | HARSH | 18 |
| 2 | PRATIK | 19 |
| 2 | RIYANKA | 20 |
| 3 | DEEP | 18 |
| 1 | SAPTARHI | 19 |

32

JOIN Example: LEFT JOIN

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

| NAME | COURSE_ID |
|----------|-----------|
| HARSH | 1 |
| PRATIK | 2 |
| RIYANKA | 2 |
| DEEP | 3 |
| SAPTARHI | 1 |
| DHANRAJ | NULL |
| ROHIT | NULL |
| NIRAJ | NULL |

33

JOIN Example: RIGHT JOIN

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
RIGHT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

| NAME | COURSE_ID |
|----------|-----------|
| HARSH | 1 |
| PRATIK | 2 |
| RIYANKA | 2 |
| DEEP | 3 |
| SAPTARHI | 1 |
| NULL | 4 |
| NULL | 5 |
| NULL | 4 |

34

JOIN Example: FULL JOIN

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
FULL JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

| NAME | COURSE_ID |
|----------|-----------|
| HARSH | 1 |
| PRATIK | 2 |
| RIYANKA | 2 |
| DEEP | 3 |
| SAPTARHI | 1 |
| DHANRAJ | NULL |
| ROHIT | NULL |
| NIRAJ | NULL |
| NULL | 4 |
| NULL | 5 |
| NULL | 4 |

35

Section 2

SQL in Quarto

36

SQL in Quarto

In Quarto, you can insert an SQL code chunk.

```
```{sql}
--| connection: connection_name
```
```

Be aware that the setup comments for SQL code chunks are `--|` instead of `#|` because an SQL comment is `--` whereas **R** and Python comments are both `#`.

We will need to establish a connection before using the SQL code chunk, though. This connection must be to a database, either remote or local.

37

Define an SQL Connection in R

To connect to a remote database, we can use the following code chunk in Quarto. We will need a few SQL and database packages, though. Some common ones are DBI, RMySQL, and RSQL, and RSQLite. This database is a MySQL database, so we will need the RMySQL library.

```
```{r}
#| echo: true
#| eval: true
library(DBI)
library(RMySQL)
con <- dbConnect(RMySQL::MySQL(),
 dbname = "sakila",
 host = "relational.fit.cvut.cz",
 port = 3306,
 user = "guest",
 password = "relational")
```
```

38

Define an SQL Connection in R

Note: this website for the sakila database has since gone down since these notes were created. A remote database I found that is still up (as of January 31, 2024) can be accessed using the code on the following slide:

```
```{r}
#| echo: true
#| eval: true
con = dbConnect(RMySQL::MySQL(),
 dbname='Rfam',
 host='mysql-rfam-public.ebi.ac.uk',
 port=4497,
 user='rfamro',
 password='')
```
```

The following slides will still use the sakila database as an example.

39

Define an SQL Connection in R

Then we can use that connection in our SQL code chunk.

```
```{sql}
--| connection: con
--| echo: true
--| eval: true
--| max-print: 3
SELECT title, release_year, rating
FROM film
WHERE rating = 'PG';
```
```

Table 13: Displaying records 1 - 3

| title | release_year | rating |
|------------------|--------------|--------|
| ACADEMY DINOSAUR | 2006 | PG |
| AGENT TRUMAN | 2006 | PG |
| ALASKA PHANTOM | 2006 | PG |

40

Creating a Data Frame in R from SQL

Instead of printing the table, we can store the table we created from our SQL command as a data frame using the `output.var` argument in the SQL code chunk:

```
```{sql}
--| connection: con
--| output.var: pg_movies
SELECT title, release_year, rating
FROM film
WHERE rating = 'PG';
```
```

41

Using SQL Table in R

Now we can access the `pg_movies` variable in **R**.

```
pg_movies <- tibble(pg_movies)
head(pg_movies, 5)
# A tibble: 5 x 3
  title          release_year rating
<chr>          <dbl> <chr>
1 ACADEMY DINOSAUR      2006 PG
2 AGENT TRUMAN          2006 PG
3 ALASKA PHANTOM        2006 PG
4 ALI FOREVER           2006 PG
5 AMADEUS HOLY          2006 PG

dim(pg_movies)
[1] 194 3
```

42

Using Tidyverse Functions

The same thing can be accomplished if we had read in the film table as a data frame or tibble.

```
movies <- read.csv("film.csv")
pg_movies <- movies |>
  filter(rating == "PG") |>
  select(title, release_year, rating) |> tibble()
head(pg_movies, 5)

# A tibble: 5 x 3
  title          release_year rating
  <chr>          <int> <chr>
1 ACADEMY DINOSAUR      2006 PG
2 AGENT TRUMAN         2006 PG
3 ALASKA PHANTOM        2006 PG
4 ALI FOREVER           2006 PG
5 AMADEUS HOLY          2006 PG
```

43

Using Tidyverse Functions

The following statements and functions are equivalent:

| SQL | R Tidyverse |
|-------------------|--------------|
| SELECT | select() |
| WHERE | filter() |
| ORDER BY | arrange() |
| GROUP BY | group_by() |
| INNER JOIN | inner_join() |
| LEFT JOIN | left_join() |
| RIGHT JOIN | right_join() |
| FULL (OUTER) JOIN | full_join() |

44

Using R Objects in SQL Chunks

If you need to use the values of an **R** variables into SQL queries, you can do so by prefacing **R** variable references with a `?`. For example:

```
```{r}
movie_length <- 130
```

```{sql}
--| connection: con
--| max-print: 5
SELECT title, length
FROM film
WHERE length >= ?movie_length
```
```

45

Using R Objects in SQL Chunks

Table 14: Displaying records 1 - 5

| title | length |
|-----------------|--------|
| AFRICAN EGG | 130 |
| AGENT TRUMAN | 169 |
| ALASKA PHANTOM | 136 |
| ALI FOREVER | 150 |
| ALLEY EVOLUTION | 180 |

46

Close Session

When we are finished with connecting to the database, it is best to close the connection using the following line:

```
dbDisconnect(con) # Disconnects from server.
```

47

Other SQL Uses

There are lots of other things we can do using SQL. We can create tables, create databases, drop tables, insert rows and columns into tables, and more.

For this class, however, we will leave it there since most of the time, we will simply be accessing databases for data analysis rather than editing them.

48

Section 3

Local Databases

49

Simulating SQL Database with Local Files in Memory

To simulate a connection to a database, we can use the following:

```
con <- dbConnect(RSQLite::SQLite(), ":memory:")
sakila_files <- list.files("../Data/Sakila DB Data")
for (file in sakila_files) {
  data_file <- read.csv(paste0("../Data/Sakila DB Data/", file))
  # Add data frame as a table
  dbWriteTable(con, substr(file, 1, nchar(file) - 4), data_file)
  rm(data_file)
}
dbListTables(con)
```

| | | | |
|---------------------|-------------|-------------|--------------|
| [1] "actor" | "address" | "category" | "city" |
| [5] "country" | "customer" | "film" | "film_actor" |
| [9] "film_category" | "film_text" | "inventory" | "language" |
| [13] "payment" | "rental" | "staff" | "store" |

These tables are not stored in R's memory, though!

50

Simulating SQL Database with Local Files

Table 15: Displaying records 1 - 5

| title | length |
|------------------|--------|
| AFFAIR PREJUDICE | 117 |
| AFRICAN EGG | 130 |
| AGENT TRUMAN | 169 |
| ALAMO VIDEOTAPE | 126 |
| ALASKA PHANTOM | 136 |

51

Connect to Local Data Databases

If we have an SQL-type file (like one that ends in .sqlite), then we can connect to it without reading it into R's memory.

For example, I have a database called "portal_mammals.sqlite" (downloadable from [here](#)). If it is in my working directory, I can connect to this database using

```
mammals_con <- DBI::dbConnect(RSQLite::SQLite(),
                              "portal_mammals.sqlite")
DBI::dbListTables(mammals_con) # List all database tables
```

| | | |
|-------------|-----------|-----------|
| [1] "plots" | "species" | "surveys" |
|-------------|-----------|-----------|

52

Connect to Local Data Databases

Then I can run SQL queries using the dbGetQuery() function:

```
dbGetQuery(mammals_con, "SELECT * FROM surveys;") |> head()
```

| | record_id | month | day | year | plot_id | species_id | sex | hindfoot_length |
|---|-----------|-------|-----|------|---------|------------|-----|-----------------|
| 1 | 1 | 7 | 16 | 1977 | 2 | NL | M | |
| 2 | 2 | 7 | 16 | 1977 | 3 | NL | M | |
| 3 | 3 | 7 | 16 | 1977 | 2 | DM | F | |
| 4 | 4 | 7 | 16 | 1977 | 7 | DM | M | |
| 5 | 5 | 7 | 16 | 1977 | 3 | DM | M | |
| 6 | 6 | 7 | 16 | 1977 | 1 | PF | M | |

or I can connect to the database in Quarto using

```
```{sql}
--| connection: mammals_con
--| max-print: 5
SELECT * FROM surveys;
```
```

53

Connect to Local Data Databases

Select from December 2000:

```
SELECT record_id, month, day, year, species_id, sex, weight
FROM surveys
WHERE month = 12 AND year = 2000;
```

Table 16: Displaying records 1 - 5

| record_id | month | day | year | species_id | sex | weight |
|-----------|-------|-----|------|------------|-----|--------|
| 31616 | 12 | 22 | 2000 | DO | M | 52 |
| 31617 | 12 | 22 | 2000 | PB | F | 26 |
| 31618 | 12 | 22 | 2000 | OT | M | 24 |
| 31619 | 12 | 22 | 2000 | PB | F | 25 |
| 31620 | 12 | 22 | 2000 | DM | M | 38 |

54

Create a Local Database

We can create a .sqlite file from a .csv using the `csv_to_sqlite()` function from the `inborutils` package. This is not available on the CRAN, but can be installed using `devtools::install_github("inbo/inborutils")`.

This function reads a csv in smaller chunks (50,000 lines at a time by default) and writes it to a .sqlite file so we can connect to it as a database.

Example:

```
library(inborutils)
csv_to_sqlite(csv_file = "combined.csv",
              sqlite_file = "combined.sqlite",
              table_name = "combined")

combined_con <- DBI::dbConnect(RSQLite::SQLite(),
                              "combined.sqlite")
dbGetQuery(combined_con, "SELECT * FROM combined") |> head()
```

55

Create a Local Database

| record_id | month | day | year | plot_id | species_id | sex |
|-----------|-------|-----|------|---------|------------|---------|
| 1 | 1 | 7 | 16 | 1977 | 2 | NL M |
| 2 | 72 | 8 | 19 | 1977 | 2 | NL M |
| 3 | 224 | 9 | 13 | 1977 | 2 | NL <NA> |
| 4 | 266 | 10 | 16 | 1977 | 2 | NL <NA> |
| 5 | 349 | 11 | 12 | 1977 | 2 | NL <NA> |
| 6 | 363 | 11 | 12 | 1977 | 2 | NL <NA> |

| hindfoot_length | weight | genus | species | taxa | plot_type |
|-----------------|--------|-------|------------------|--------|-----------|
| 1 | 32 | NA | Neotoma albigula | Rodent | Control |
| 2 | 31 | NA | Neotoma albigula | Rodent | Control |
| 3 | NA | NA | Neotoma albigula | Rodent | Control |
| 4 | NA | NA | Neotoma albigula | Rodent | Control |
| 5 | NA | NA | Neotoma albigula | Rodent | Control |
| 6 | NA | NA | Neotoma albigula | Rodent | Control |

56

Create a Local Database

You can add another table to the same .sqlite file using `csv_to_sqlite()` as long as a different table name is specified.

```
csv_to_sqlite(csv_file = "actor.csv",
              sqlite_file = "sakila.sqlite",
              table_name = "actor")
csv_to_sqlite(csv_file = "film.csv",
              sqlite_file = "sakila.sqlite",
              table_name = "film")
```

The above code will create the `sakila.sqlite` file in the first function call with the table "actor". The second function call will then add the "film" table to that same file.

57

Convert from dplyr Functions to SQL

The `dbplyr` package is an add-on to the `dplyr` package that allows you to use remote database tables as if they are in-memory data frames by automatically converting dplyr code into SQL. We can install it in the normal way: `install.packages("dbplyr")`. If you don't install it, you will be prompted to when you do something that requires it.

Example:

```
# Access the SQL table "surveys" from the mammals_con connection
mammal_surveys <- tbl(mammals_con, "surveys")
mammal_surveys
```

```
# Source:   table<'surveys'> [?? x 9]
# Database: sqlite 3.51.0 [/Users/rickbrown/My Drive/Math 3190 - Fun
  record_id month   day   year plot_id species_id sex
    <int> <int> <int> <int>   <int>   <chr>   <chr>
1         1     7    16  1977      2    NL      M
2         2     7    16  1977      3    NL      M
3         3     7    16  1977      2    DM      F
4         4     7    16  1977      7    DM      M
```

58

Convert from dplyr Functions to SQL

```
mammal_summary <- mammal_surveys |>
  filter(month == 7 & year == 1992) |>
  group_by(species_id) |>
  summarize(avg_weight = mean(weight, na.rm = TRUE)) |>
  arrange(desc(avg_weight))
head(mammal_summary)

# Source:   SQL [?? x 2]
# Database: sqlite 3.51.0 [/Users/rickbrown/My Drive/Math 3190 - Fun
# Ordered by: desc(avg_weight)
  species_id avg_weight
    <chr>      <dbl>
1 NL          131.
2 DS          120
3 SO          57.5
4 DO          44.6
5 DM          43.4
6 OL          32.4
```

59

Convert from dplyr Functions to SQL

The `show_query()` function will show an SQL query that will accomplish the same thing the dplyr functions did to create the object. Sometimes these queries look a bit strange, but they work!

```
show_query(mammal_summary)

<SQL>
SELECT `species_id`, AVG(`weight`) AS `avg_weight`
FROM (
  SELECT `surveys`.*
  FROM `surveys`
  WHERE (`month` = 7.0 AND `year` = 1992.0)
) AS `q01`
GROUP BY `species_id`
ORDER BY `avg_weight` DESC
```

60

Same thing with SQL

```
SELECT `species_id`, AVG(`weight`) AS `avg_weight`
FROM (
  SELECT `surveys`.*
  FROM `surveys`
  WHERE (`month` = 7.0 AND `year` = 1992.0)
) AS `q01`
GROUP BY `species_id`
ORDER BY `avg_weight` DESC
```

Table 17: Displaying records 1 - 10

species_id	avg_weight
NL	130.66667
DS	120.00000
SO	57.50000
DO	44.60000
DM	43.42500

61

Object Sizes

What is especially amazing about this is even though we are working with an object called `mammal_surveys`, it is actually not read into memory. We can see that here using the `obj_size()` function in the `lobstr` package:

```
lobstr::obj_size(mammal_surveys)
```

5.98 kB

```
# collect function brings the object into memory
```

```
mammal_surveys2 <- collect(mammal_surveys)
```

```
lobstr::obj_size(mammal_surveys2)
```

1.57 MB

So, being an **R** wizard also makes you an SQL wizard (by proxy)!

62

Close Connection

As always, we should not forget to disconnect when we are done.

```
dbDisconnect(con)
dbDisconnect(combined_con)
dbDisconnect(mammals_con)
```

63

Conclusion

- Knowing SQL is a great skill to not only access databases remotely, but also to be able to work with large files locally without reading them into **R**.
- Even though tools exist to convert **R** code to SQL queries, it is still important to know both to be able to handle working with any database or data set.

64

Session info

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7.2

Matrix products: default
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.0

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Denver
tzcode source: internal

attached base packages:
[1] stats graphics grDevices utils datasets
[6] methods base

other attached packages:
[1] inborutils_0.4.0 lubridate_1.9.4 forcats_1.0.1
[4] stringr_1.6.0 dplyr_1.1.4 purrr_1.2.0
[7] readr_2.1.6 tidyr_1.3.2 tibble_3.3.0
[10] ggplot2_4.0.1 tidyverse_2.0.0 RMySQL_0.11.1
[13] DBI_1.2.3

loaded via a namespace (and not attached):
[1] gtable_0.3.6 xfun_0.55
[3] htmlwidgets_1.6.4 tzdb_0.5.0
[5] vctrs_0.6.6 tools_4.5.2
[7] crosstalk_1.2.2 generics_0.1.4
[9] parallel_4.5.2 curl_7.0.0
[11] proxy_0.4-29 RSQLite_2.4.4
[13] blob_1.2.4 pkgconfig_2.0.3
```

65

Notes 5: Verification and Resampling Methods

Table of contents

Evaluation Metrics and the Confusion Matrix	60
Evaluation Metrics	60
Training and Test Sets	60
Overall Accuracy	61
The Confusion Matrix	63
Sensitivity and Specificity	64
Cross Validation with kNN Example	66
Cross Validation	66
K-Nearest Neighbors (kNN)	66
K-Nearest Neighbors (kNN): Example	67
Over-Training	69
Over-Smoothing	69
Picking the k in kNN	70
Mathematical Description of Cross Validation	71
K-Fold Cross Validation	72
Bootstrap	75
More on Bootstrapping	77
Bootstrap Example Using R:	80
More on the caret Package	82
The <code>caret train()</code> function	83
Cross Validation	83
Example: Fitting with Loess	86
Conclusion	87

Evaluation Metrics and the Confusion Matrix

Evaluation Metrics

Before we start describing approaches to optimize the way we build algorithms, we first need to define what we mean when we say one approach is better than another.

We use the `caret` package, which has several useful functions for building and assessing machine learning methods.

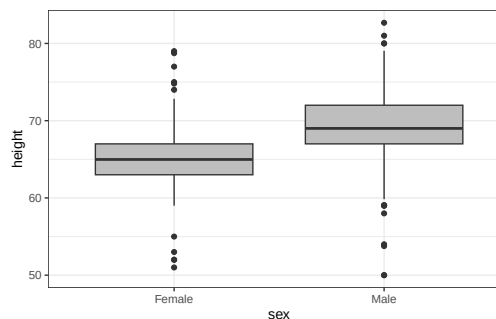
```
library(tidyverse)
library(caret)
```

For a first example, we use the height data in `dslabs`:

```
library(dslabs)
data(heights)
```

To summarize the data, consider the following boxplot:

```
ggplot(heights, aes(x = sex, y = height)) +
  geom_boxplot(fill = "gray") + theme_bw()
```



We will start with a simple example: suppose we want to predict sex using height. We start by defining the outcome and predictors.

```
y <- heights$sex
x <- heights$height
```

In this case, we have only one predictor, height, and `y` is clearly a categorical outcome since observed values are either `Male` or `Female`.

We know that we will not be able to predict Y very accurately based on X because male and female average heights are not that different relative to within group variability. But can we do better than guessing? To answer this question, we need a quantitative definition of better.

Training and Test Sets

Ultimately, a machine learning algorithm is evaluated on how it performs in the real world with completely new datasets. However, we usually have a dataset for which we know the outcomes, as we do with the heights: we know the sex of every student in our dataset. Therefore, to mimic the ultimate evaluation process, we typically split the data into two parts and act as if we don't know the outcome for one of these.

We refer to the group for which we know the outcome, and use to develop the algorithm, as the **training** set. We refer to the group for which we pretend we don't know the outcome as the **test** set.

`caret` includes the function `createDataPartition()` that helps us generate training and test sets:

```
set.seed(2026)
test_index <- createDataPartition(y, times = 1, p = 0.5, list = FALSE)
```

The argument `times` is used to define how many random samples of indexes to return, `p` is used to define what proportion of the data is represented by the index, and `list` is used to decide if we want the indexes returned as a list or not.

If `times = 1`, this is essentially the same as doing

```
set.seed(2026)
test_index <- sample(1:nrow(heights), size = round(nrow(heights) * 0.5))
```

We can use the result of the `createDataPartition()` function call to define the training and test sets like this:

```
test_set <- heights[test_index, ]
train_set <- heights[-test_index, ]
```

We develop an algorithm using **only** the training set. Once we are done developing the algorithm, we will **freeze** it and evaluate it using the test set. The simplest way to evaluate the algorithm when the outcomes are categorical is by simply reporting the proportion of cases that were correctly predicted in the test set. This metric is usually referred to as **overall accuracy**.

Overall Accuracy

To demonstrate the use of overall accuracy, we will build two competing algorithms and compare them.

Let's start by developing the simplest possible machine algorithm: guessing the outcome.

```
y_hat <- sample(c("Male", "Female"), nrow(test_set), replace = TRUE)
```

We are completely ignoring the predictor and simply guessing the sex.

In machine learning applications, it is useful to use factors to represent the categorical outcomes because **R** functions developed for machine learning, such as those in the `caret` package, require or recommend that categorical outcomes be coded as factors. So convert `y_hat` to factors using the `factor` function:

```
set.seed(2026)
y_hat <- sample(c("Male", "Female"), nrow(test_set), replace = TRUE) |>
  factor(levels = levels(test_set$sex))
```

The *overall accuracy* is simply defined as the overall proportion that is predicted correctly:

```
mean(y_hat == test_set$sex)
[1] 0.487619
```

Not surprisingly, our accuracy is about 50%. We are guessing!

Can we do better? Exploratory data analysis suggests we can because, on average, males are slightly taller than females:

```
heights |>
  group_by(sex) |>
  summarize(means = mean(height), sds = sd(height))
# A tibble: 2 x 3
  sex      means    sds
<fct> <dbl> <dbl>
1 Female  64.9   3.76
2 Male    69.3   3.61
```

But how do we make use of this insight? Let's try another simple approach: predict Male if height is within two standard deviations from the average male:

```
y_hat <- ifelse(x > 62, "Male", "Female") |>
  factor(levels = levels(test_set$sex))
```

The accuracy goes up from 0.50 to about 0.80:

```
mean(y == y_hat)
[1] 0.7933333
```

But can we do even better? In the example above, we used a cutoff of 62, but we can examine the accuracy obtained for other cutoffs and then pick the value that provides the best results.

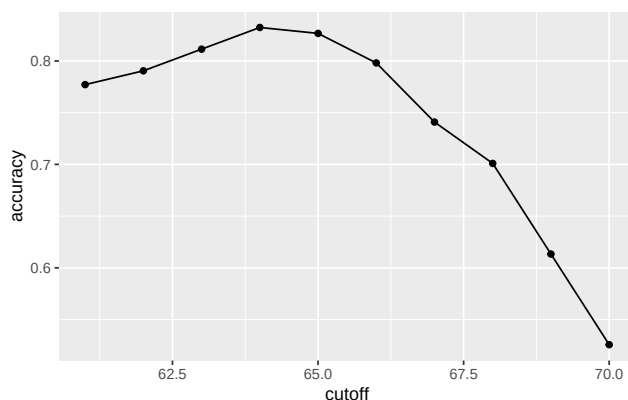
But remember, **it is important that we optimize the cutoff using only the training set:** the test set is only for evaluation.

Although for this simplistic example it is not much of a problem, later we will learn that evaluating an algorithm on the training set can lead to **overfitting**, which often results in dangerously over-optimistic assessments.

Here we examine the accuracy of 10 different cutoffs and pick the one yielding the best result:

```
cutoff <- seq(61, 70)
accuracy <- map_dbl(cutoff, function(x){
  y_hat <- ifelse(train_set$height > x, "Male", "Female") |>
    factor(levels = levels(test_set$sex))
  mean(y_hat == train_set$sex)
})
```

We can make a plot showing the accuracy obtained on the training set for males and females:



We see that the maximum value is:

```
max(accuracy)
[1] 0.832381
```

which is much higher than 0.5. The cutoff resulting in this accuracy is:

```
best_cutoff <- cutoff[which.max(accuracy)]
best_cutoff
[1] 64
```

We can now test this cutoff on our test set to make sure our accuracy is not overly optimistic:

```
y_hat <- ifelse(test_set$height > best_cutoff, "Male", "Female") |>
  factor(levels = levels(test_set$sex))
mean(y_hat == test_set$sex)
[1] 0.8209524
```

We see that it is similar to (actually slightly better than) the accuracy observed for the training set, but it is still better than guessing. And by testing on a dataset that we did not train on, we know our result is not due to cherry-picking a good result.

The prediction rule we developed in the previous section predicts **Male** if the student is taller than 64 inches. Given that the average female is about 64 inches, this prediction rule seems wrong. What happened? If a student is the height of the average female, shouldn't we predict **Female**?

The Confusion Matrix

Generally speaking, overall accuracy can be a deceptive measure. To see this, we will start by constructing what is referred to as the *confusion matrix*, which basically tabulates each combination of prediction and actual value. We can do this in **R** using the function `table`:

```
table(predicted = y_hat, actual = test_set$sex)
```

	actual	
predicted	Female	Male
Female	53	28
Male	66	378

If we study this table closely, it reveals a problem. If we compute the accuracy separately for each sex, we get:

	Actually Positive	Actually Negative
Predicted positive	True positives (TP)	False positives (FP)
Predicted negative	False negatives (FN)	True negatives (TN)

```
test_set |>
  mutate(y_hat = y_hat) |>
  group_by(sex) |>
  summarize(accuracy = mean(y_hat == sex))
```

```
# A tibble: 2 x 2
  sex      accuracy
<fct>    <dbl>
1 Female  0.445
2 Male    0.931
```

There is an imbalance in the accuracy for males and females: too many females are predicted to be male. We are calling almost half of the females male! How can our overall accuracy be so high then? This is because the **prevalence** of males in this dataset is high. These heights were collected from three data sciences courses, two of which had more males enrolled:

```
prev <- mean(y == "Male")
prev
```

```
[1] 0.7733333
```

So when computing overall accuracy, the high percentage of mistakes made for females is outweighed by the gains in correct calls for men. **This can actually be a big problem in machine learning.** If your training data is biased in some way, you are likely to develop algorithms that are biased as well. The fact that we used a test set does not matter because it is also derived from the original biased dataset. This is one of the reasons we look at metrics other than overall accuracy when evaluating a machine learning algorithm.

There are several metrics that we can use to evaluate an algorithm in a way that prevalence does not cloud our assessment, and these can all be derived from the confusion matrix. A general improvement to using overall accuracy is to study **sensitivity** and **specificity** separately.

Sensitivity and Specificity

In general, **sensitivity** is defined as the ability of an algorithm to predict a positive outcome when the actual outcome is positive: $\hat{Y} = 1$ when $Y = 1$. Stated in terms of probability, this is $P(\text{Predicted Positive} | \text{Actually Positive})$.

Because an algorithm that calls everything positive ($\hat{Y} = 1$ no matter what) has perfect sensitivity, this metric on its own is not enough to judge an algorithm.

For this reason, we also examine **specificity**, which is generally defined as the ability of an algorithm to not predict a positive $\hat{Y} = 0$ when the actual outcome is not a positive $Y = 0$. Stated in terms of probability, this is $P(\text{Predicted Negative} | \text{Actually Negative})$.

We name the four entries of the **confusion matrix**:

Sensitivity is typically quantified by $TP / (TP + FN)$, the proportion of actual positives (the first

column = $TP + FN$) that are called positives (TP). This quantity is also referred to as the **true positive rate** (TPR) or **recall**.

Specificity is defined as $TN/(TN + FP)$ or the proportion of negatives (the second column = $FP + TN$) that are called negatives (TN). This quantity is also called the **true negative rate** (TNR).

There is another way of quantifying accuracy which is $TP/(TP + FP)$ or the proportion of outcomes called positives (the first row or $TP + FP$) that are actually positives (TP). This quantity is referred to as **positive predictive value (PPV)** and also as **precision**. Note that, unlike TPR and TNR, precision depends on prevalence since higher prevalence implies you can get higher precision even when guessing.

The multiple names can be confusing, so we include a table to help us remember the terms. The table includes a column that shows the definition if we think of the proportions as probabilities.

Measure of	Name 1	Name 2	Definition	Prob. Representation
Sensitivity	TPR	Recall	$\frac{TP}{TP + FN}$	$P(\hat{Y} = 1 Y = 1)$
Specificity	TNR	$1 - \text{FPR}$	$\frac{TN}{TN + FP}$	$P(\hat{Y} = 0 Y = 0)$
Precision	PPV		$\frac{TP}{TP + FP}$	$P(Y = 1 \hat{Y} = 1)$
	NPV		$\frac{TN}{TN + FN}$	$P(Y = 0 \hat{Y} = 0)$

Here TPR is True Positive Rate, FPR is False Positive Rate, PPV is Positive Predictive Value, and NPV is Negative Predictive Value.

The `caret` package has the function `confusionMatrix()` that computes the confusion matrix for us. The function expects factors as input, and the first level is considered the positive outcome or $Y = 1$. In our example, `Female` is the first level because it comes before `Male` alphabetically. It can be helpful and avoid issues to explicitly tell the function which category we are predicting using the `positive` argument.

```
cm <- confusionMatrix(data = y_hat, reference = test_set$sex,
                      positive = "Female")
```

```
cm$table
```

```

      Reference
Prediction Female Male
      Female    53   28
      Male     66  378
```

You can access these directly, for example, like this:

```
cm$overall["Accuracy"]
```

```

Accuracy
0.8209524
```

```
cm$byClass[c("Sensitivity", "Specificity", "Prevalence")]
```

```

Sensitivity Specificity Prevalence
0.4453782   0.9310345   0.2266667
```

```
cm$byClass["Balanced Accuracy"]
```

```
Balanced Accuracy
0.6882063
```

The balanced accuracy is calculated by taking the average of sensitivity and specificity.

We can see that the high overall accuracy is possible despite relatively low sensitivity. As we hinted at above, the reason this happens is because of the low prevalence (0.23): the proportion of females is low. Because prevalence is low, failing to predict actual females as females (low sensitivity) does not lower the accuracy as much as failing to predict actual males as males (low specificity).

Let's increase the cutoff to 67 inches.

```
y_hat2 <- ifelse(test_set$height > 67, "Male", "Female") |>
  factor()
```

```
cm2 <- confusionMatrix(data=y_hat2, reference=test_set$sex)
cm2$overall["Accuracy"]
```

```
Accuracy
0.7333333
```

```
cm2$byClass[c("Sensitivity", "Specificity", "Prevalence")]
```

```
Sensitivity Specificity Prevalence
0.7899160    0.7167488    0.2266667
```

```
cm2$byClass["Balanced Accuracy"]
```

```
Balanced Accuracy
0.7533324
```

Cross Validation with kNN Example

Cross Validation

In the previous example, we split our data into training and testing groups so as not to overfit and to get an idea of how the model works for new data.

Cross-validation takes this one step forward. Cross-validation is one of the most important ideas in machine learning. Here we focus on the conceptual and mathematical aspects. We will describe how to implement cross validation in practice in later examples.

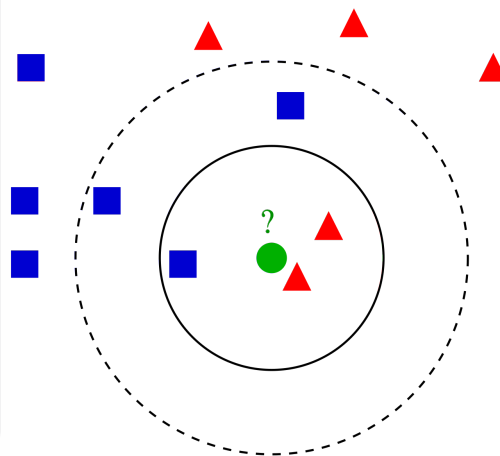
K-Nearest Neighbors (kNN)

To motivate the concept, let's consider an actual machine learning algorithm: **k-nearest neighbors (kNN)**.

K-nearest neighbor algorithm pseudocode

Programming languages like Python and R are used to implement the KNN algorithm. The following is the pseudocode for KNN:

1. Load the data
2. Choose K value
3. For each data point in the data:
 - Find the Euclidean distance to all training data samples
 - Store the distances on an ordered list and sort it
 - Choose the top K entries from the sorted list
 - Label the test point based on the majority of classes present in the selected points
4. End



With k-nearest neighbors (kNN) we estimate the conditional probability function:

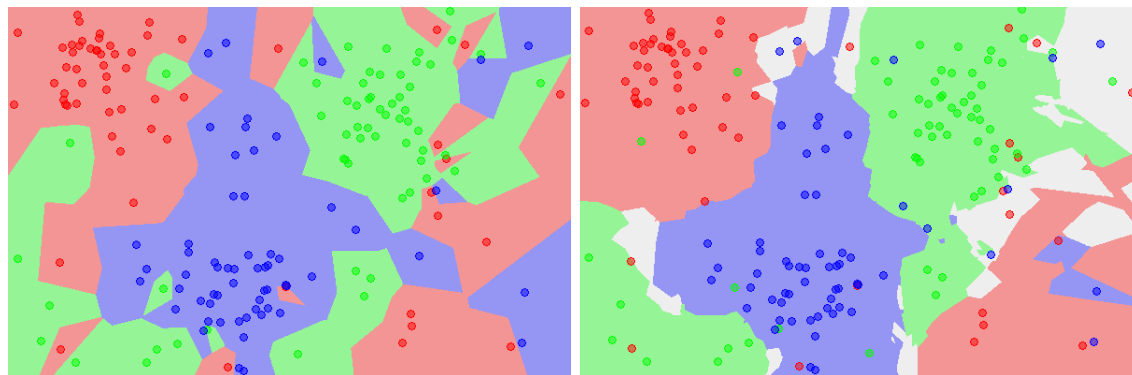
$$p(x_1, x_2) = P(Y = 1 \mid X_1 = x_1, X_2 = x_2).$$

First we define the distance between each of the observations. Then, for any point (x_1, x_2) for which we want an estimate of $p(x_1, x_2)$, we look for the k nearest points to (x_1, x_2) and then take an average of the 0s and 1s associated with these points. We refer to the set of points used to compute the average as the *neighborhood*. This gives us an estimate $\hat{p}(x_1, x_2)$.

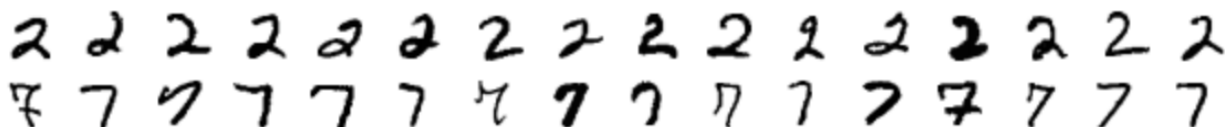
Using $k = 1$:

and

$k = 5$:

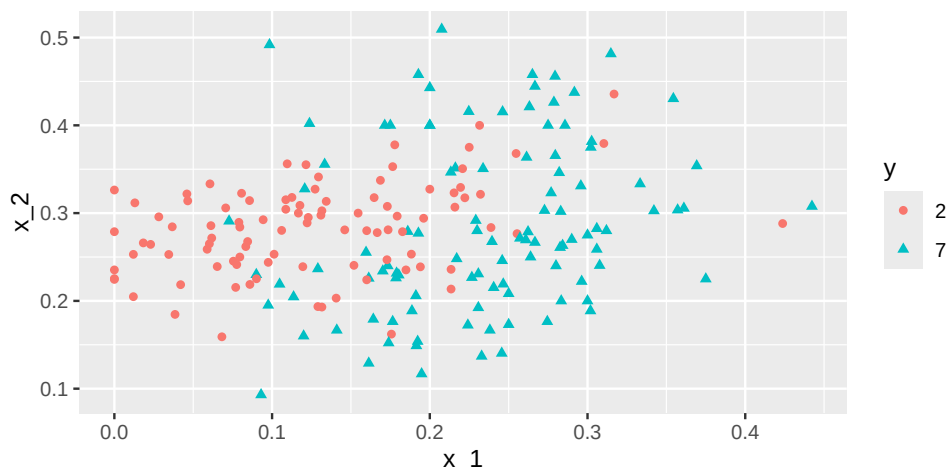
**K-Nearest Neighbors (kNN): Example**

We will use the `mnist_27` data set in the `dslabs` package. This dataset contains information about predicting hand-written numbers. In this dataset, only 2's and 7's are considered. The data have already been split into a testing and training set. Each of these have three columns: `y`, `x_1`, and `x_2`. `y` is the true number, `x_1` is the proportion of dark pixels in the upper-left quadrant, and `x_2` is the proportion of dark pixels in the lower-right quadrant.



Using this data set:

```
library(dslabs)
mnist_27$test |>
  ggplot(aes(x_1, x_2, color = y, shape = y)) +
    geom_point()
```



We can use the `knn3` function from the `caret` package as follows, with the number of neighbors equal to $k = 5$:

```
library(caret) |> suppressPackageStartupMessages()
knn_fit <- knn3(y ~ ., data = mnist_27$train, k = 5)
```

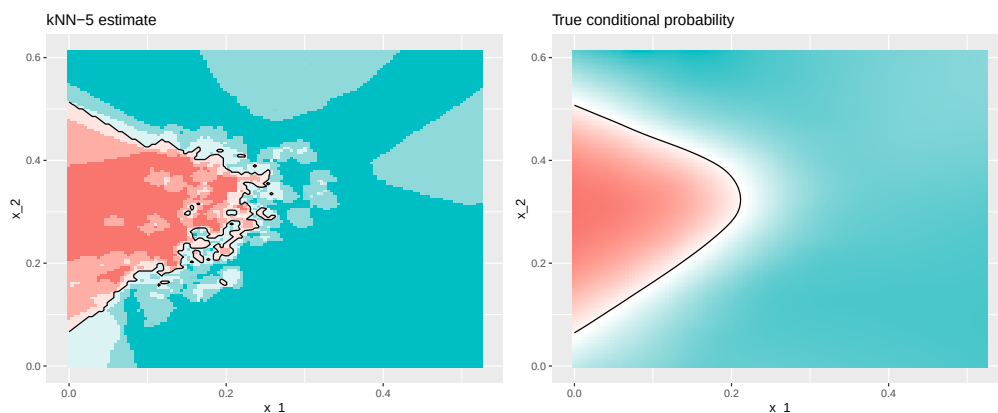
Note that we have two classes here: a 2 or a 7. Therefore, we should use an odd number for k to avoid ties. If we had an odd number of classes, then an even number for k should be used.

Since our data set is balanced (the prevalence is 0.53) and we care just as much about sensitivity as we do about specificity, we will use accuracy to quantify performance.

The `predict` function for `knn` produces a probability for each class.

```
y_hat_knn <- predict(knn_fit, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall["Accuracy"]
```

Accuracy
0.815



Over-Training

Over-training or **over-fitting** results in having higher accuracy in the train set compared to the test set:

```
y_hat_knn <- predict(knn_fit, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn, mnist_27$train$y)$overall["Accuracy"]
```

Accuracy
0.86

```
y_hat_knn <- predict(knn_fit, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall["Accuracy"]
```

Accuracy
0.815

Over-training is at its worst when we set $k = 1$:

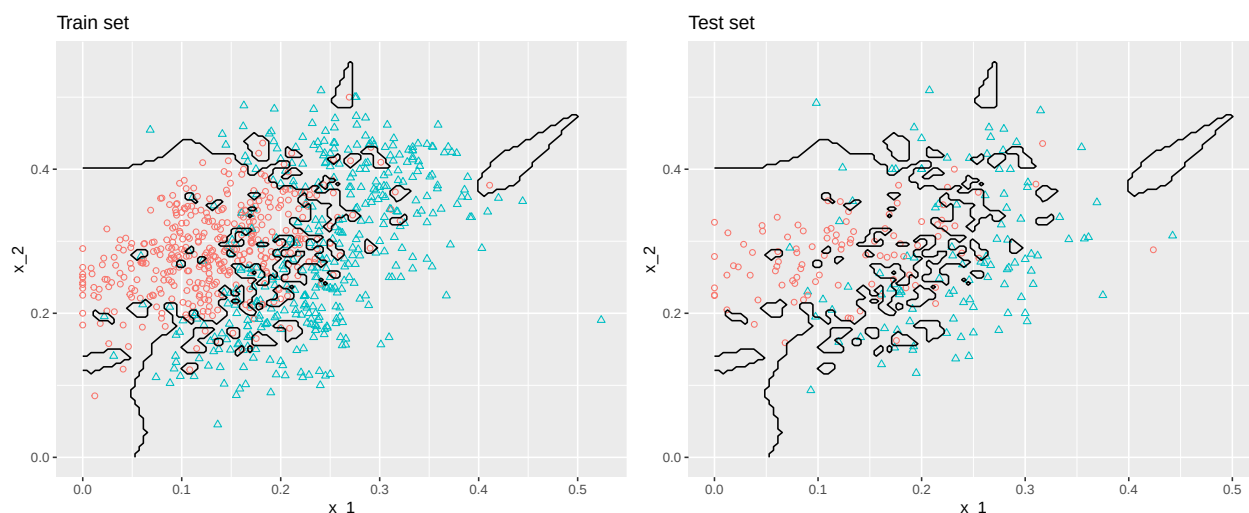
```
knn_fit_1 <- knn3(y ~ ., data = mnist_27$train, k = 1)
y_hat_knn_1 <- predict(knn_fit_1, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn_1, mnist_27$train$y)$overall["Accuracy"]
```

Accuracy
0.99375

```
y_hat_knn_1 <- predict(knn_fit_1, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn_1, mnist_27$test$y)$overall["Accuracy"]
```

Accuracy
0.81

We can see the over-fitting problem in this figure.



Over-Smoothing

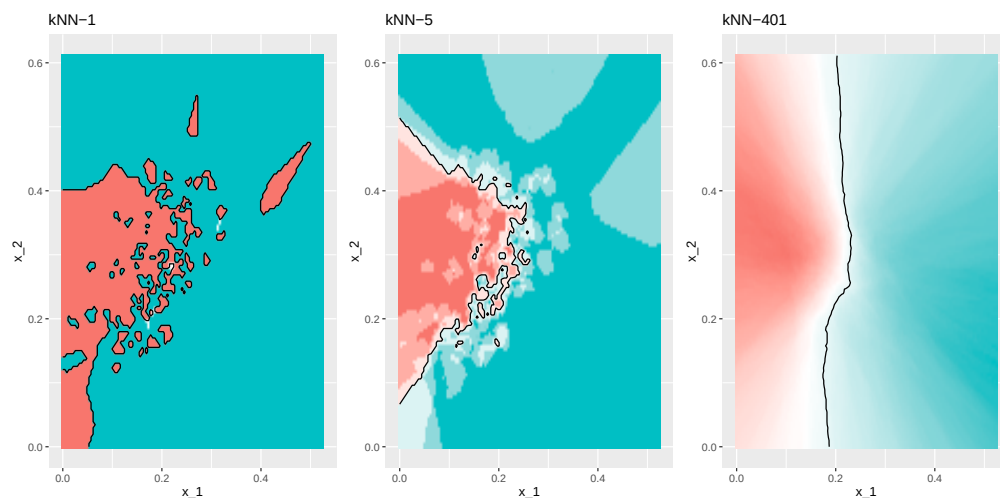
Although not as badly as with $k = 1$, we saw that $k = 5$ is also over-trained. Hence, we should consider a larger k . Let's try, as an example, a much larger number: $k = 401$.

```
knn_fit_401 <- knn3(y ~ ., data = mnist_27$train, k = 401)
y_hat_knn_401 <- predict(knn_fit_401, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn_401, mnist_27$train$y)$overall["Accuracy"]
```

```
Accuracy
0.78625
```

```
y_hat_knn_401 <- predict(knn_fit_401,mnist_27$test,type = "class")
confusionMatrix(y_hat_knn_401, mnist_27$test$y)$overall["Accuracy"]
```

```
Accuracy
0.76
```



Picking the k in kNN

So how do we pick k ? In principle, we want to pick the k that maximizes accuracy, or minimizes the expected MSE (defined later).

The goal of **cross validation** is to estimate these quantities for any given algorithm and set of tuning parameters such as k . To understand why we need a special method to do this let's repeat what we did above but for different values of k :

```
ks <- seq(3, 251, 2)
```

We do this using `map_df` function to repeat the function for each element of the vector.

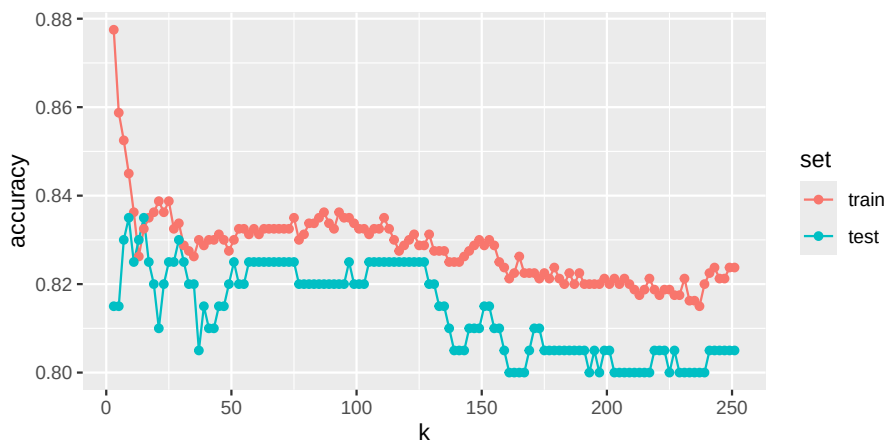
```
library(purrr)
accuracy <- map_df(ks, function(k){
  fit <- knn3(y ~ ., data = mnist_27$train, k = k)

  y_hat <- predict(fit, mnist_27$train, type = "class")
  cm_train <- confusionMatrix(y_hat, mnist_27$train$y)
  train_acc <- cm_train$overall["Accuracy"]

  y_hat <- predict(fit, mnist_27$test, type = "class")
  cm_test <- confusionMatrix(y_hat, mnist_27$test$y)
  test_acc <- cm_test$overall["Accuracy"]

  tibble(train = train_acc, test = test_acc)
})
```

Note that we estimate accuracy by using both the training set and the test set. We can now plot the accuracy estimates for each value of k :



If we were to use these estimates to pick the k that maximizes accuracy, we would use the estimates built on the test data:

```
ks[which.max(accuracy$test)]
```

```
[1] 9
```

```
max(accuracy$test)
```

```
[1] 0.835
```

First, note that the “best” estimate for k obtained for the training set is generally lower than the estimate obtained using the test set, with the difference larger for smaller values of k . This is due to over-training.

Also note that the accuracy versus k plot is quite jagged. We do not expect this because small changes in k should not affect the algorithm’s performance too much. The jaggedness is explained by the fact that the accuracy is computed on a sample and therefore is a random variable. This demonstrates why we prefer to minimize the expected, or average, loss (defined later) rather than the loss we observe with one dataset.

Another reason we need a better estimate of accuracy is that if we use the test set to pick this k , we should not expect the accompanying accuracy estimate to extrapolate to the real world. This is because even here we broke a golden rule of machine learning: we selected the k using the test set. Cross validation also provides an estimate that takes this into account.

Mathematical Description of Cross Validation

A common goal of machine learning is to find an algorithm that produces predictors $\hat{Y}$ for an outcome Y that minimizes the mean squared error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2$$

When all we have at our disposal is one data set, we can estimate the MSE with the observed MSE like this:

$$\hat{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

These two are often referred to as the *true error* and *apparent error*, respectively.

There are two important characteristics of the apparent error we should always keep in mind:

1. Because our data is random, the apparent error is a random variable. For example, the dataset we have may be a random sample from a larger population. An algorithm may have a lower apparent error than another algorithm due to luck.
2. If we train an algorithm on the same dataset that we use to compute the apparent error, we might be overtraining. In general, when we do this, the apparent error will be an underestimate of the true error. We will see an extreme example of this with k-nearest neighbors.

Cross validation is a technique that permits us to alleviate both these problems. To understand cross validation, it helps to think of the true error, a theoretical quantity, as the average of many apparent errors obtained by applying the algorithm to B new random samples of the data, none of them used to train the algorithm:

$$\frac{1}{B} \sum_{b=1}^B \frac{1}{n} \sum_{i=1}^n (y_i^b - \hat{y}_i^b)^2$$

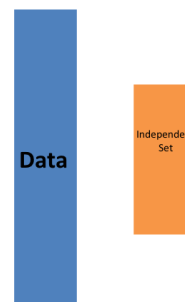
with B a large number that can be thought of as practically infinite.

As already mentioned, this is a theoretical quantity because we only have available one set of outcomes: $y_1, \dots, y_n$. Cross validation is based on the idea of imitating the theoretical setup above as best we can with the data we have.

To do this, we have to generate a series of different random samples. There are several approaches we can use, but the general idea for all of them is to randomly generate smaller datasets that are not used for training, and instead used to estimate the true error.

K-Fold Cross Validation

The first one we describe is **K-fold cross validation**. Note that this K is different than the k -nearest neighbors k value. A machine learning challenge starts with a data set (blue). We need to use this to build an algorithm that will be used in an independent validation data set (orange).



But we don't get to see these independent datasets.

So to imitate this situation, we carve out a piece of our dataset and pretend it is an independent dataset: we divide the dataset into a **training set** (blue) and a **test set** (red). We will train our algorithm exclusively on the training set and use the test set only for evaluation purposes.



- We usually try to select a small piece of the data set so that we have as much data as possible to train. However, we also want the test set to be large enough so that we obtain a stable estimate of the loss without fitting an impractical number of models. Typical choices are to use 10%-20% of the data for testing.
- Now this presents a new problem because for most machine learning algorithms we need to select parameters, for example the number of neighbors k in k-nearest neighbors. Here, we will refer to the set of parameters as λ .
- We need to optimize algorithm parameters without using our test set and we know that if we optimize and evaluate on the same dataset, we will overtrain.
- For each set of algorithm parameters being considered, we want an estimate of the MSE and then we will choose the parameters with the smallest MSE. Cross validation provides this estimate.

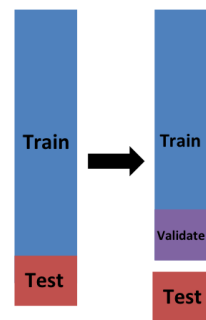
First, before we start the cross validation procedure, it is important to fix all the algorithm parameters. We will use $\hat{y}_i(\lambda)$ to denote the predictors obtained when we use parameters λ .

So, if we are going to imitate this definition:

$$\text{MSE}(\lambda) = \frac{1}{B} \sum_{b=1}^B \frac{1}{n} \sum_{i=1}^n \left(y_i^b - \hat{y}_i^b(\lambda) \right)^2$$

We want to consider datasets that can be thought of as an independent random sample and we want to do this several times. With K-fold cross validation, we do it K times. We are showing an example that uses $K = 5$.

We will eventually end up with K samples, but let's start by describing how to construct the first: we simply pick $m = n/K$ observations at random (we round if m is not a round number) and think of these as a random sample $y_1^b, \dots, y_m^b$, with $b = 1$. We call this the validation set:

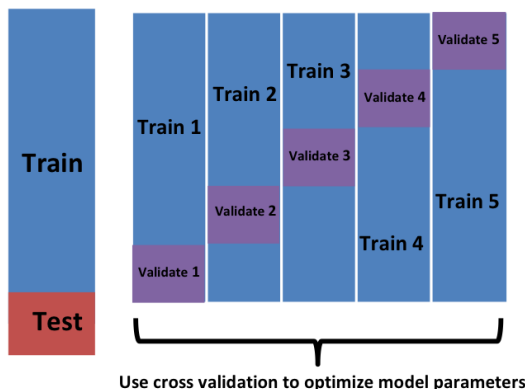


Now we can fit the model in the training set, then compute the apparent error on the validation set:

$$\text{MSE}_b(\lambda) = \frac{1}{m} \sum_{i=1}^m \left(y_i^b - \hat{y}_i^b(\lambda) \right)^2$$

Note: n = sample size of training and testing data sets combined while m = sample size of validation data set.

Note that this is just one sample and will therefore return a noisy estimate of the true error. This is why we take K samples, not just one. In K -fold cross validation, we randomly split the observations into K non-overlapping sets:



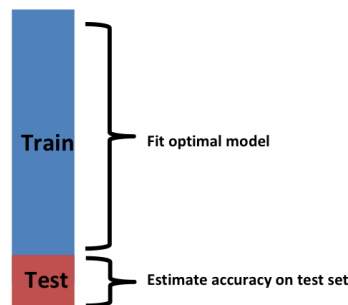
Now we repeat the calculation above for each of these sets $b = 1, \dots, K$ and obtain $\text{MSE}_1(\lambda), \dots, \text{MSE}_K(\lambda)$. Then, for our final estimate, we compute the average:

$$\text{MSE}(\lambda) = \frac{1}{K} \sum_{b=1}^K \text{MSE}_b(\lambda) = \frac{1}{K} \sum_{b=1}^K \left[\frac{1}{m} \sum_{i=1}^m \left(y_i^b - \hat{y}_i^b(\lambda) \right)^2 \right]$$

and obtain an estimate of our loss.

The final step would be to select the λ that minimizes the MSE.

We have described how to use cross validation to optimize parameters. However, we now have to take into account the fact that the optimization occurred on the training data. Here is where we use the test set we separated early on:



Now how do we pick the cross validation K (not the k for kNN)? Large values of K are preferable because the training data better imitates the original dataset. However, larger values of K will have much slower computation time: for example, 100-fold cross validation will be 10 times slower than 10-fold cross validation. For this reason, the choices of $K = 5$ and $K = 10$ are popular.

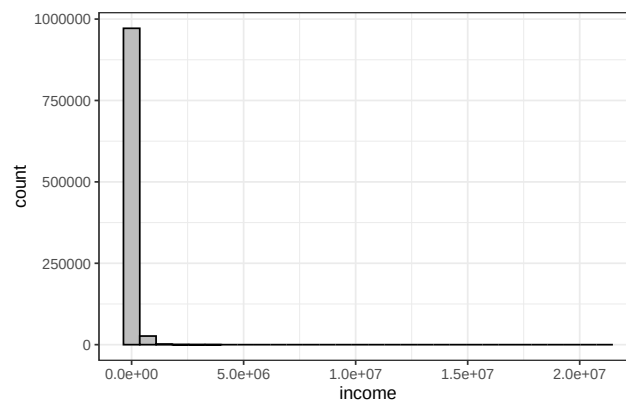
One way we can improve the variance of our final estimate is to take more samples. To do this, we would no longer require the training set to be partitioned into non-overlapping sets. Instead, we would just pick K sets of some size at random.

One popular variation of this technique, at each fold, picks observations at random with replacement (which means the same observation can appear twice). This approach has some advantages and is generally referred to as the *bootstrap*.

Bootstrap

Suppose the income distribution of your population is as follows:

```
set.seed(2026)
N <- 10^6 # Population size
income = 10^(rnorm(N, log10(45000), log10(3)))
ggplot(data.frame(income = income), aes(x = income)) +
  geom_histogram(bins = 30, color = "black", fill = "gray") + theme_bw()
```



The population median is:

```
M <- median(income) # True median
M
[1] 44946.33
```

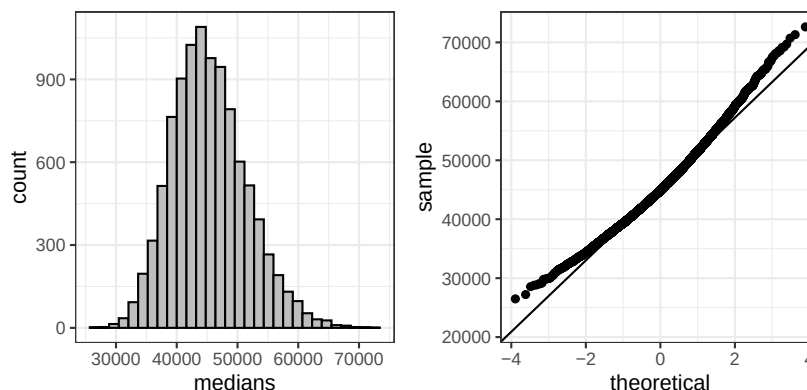
Suppose we don't have access to the entire population, but want to estimate the true median M . We take a sample of 100 and estimate the population median M with the sample median m :

```
n <- 100 # Sample size
samp <- sample(income, n)
median(samp)
[1] 41893.87
```

Can we construct a confidence interval? What is the distribution of m ?

We can use a **Monte Carlo** simulation to learn the distribution of m . A Monte Carlo simulation consists of simulating many samples from the population, computing the statistic of interest on each sample, and then using the resulting values to approximate the distribution of the statistic. Here is how we can do this for the median:

```
library(patchwork)
nsims <- 10^4 # Number of simulated samples
medians <- replicate(nsims, {X <- sample(income, n); median(X)})
p1 <- ggplot(data.frame(medians = medians), aes(x = medians)) +
  geom_histogram(bins = 30, color = "black", fill = "gray") +
  theme_bw()
p2 <- ggplot(data.frame(medians = medians), aes(sample = medians)) +
  geom_qq_line() +
  geom_qq() +
  theme_bw()
p1 + p2
```



If we know this distribution, we can construct a confidence interval. The problem here is that, as we have already described, in practice we do not have access to the distribution. We can see that the 95% confidence interval based on CLT, given here:

```
lower <- median(samp) - 1.96 * sd(samp) / sqrt(n)
upper <- median(samp) + 1.96 * sd(samp) / sqrt(n)
c(lower, upper)
[1] 19631.66 64156.09
```

is quite different from the confidence interval we would generate if we know the actual distribution of m :

```
quantile(medians, c(0.025, 0.975))
      2.5%      97.5%
34539.07 58794.24
```

Why are the intervals so different?

The bootstrap permits us to approximate a Monte Carlo simulation without access to the entire distribution. The general idea is relatively simple. We act as if the observed sample is the population. We then sample (with replacement) datasets, of the same sample size as the original dataset. Then we compute the summary statistic, in this case the median, on these *bootstrap samples*.

Theory tells us that, in many situations, the distribution of the statistics obtained with bootstrap samples approximate the distribution of our actual statistic.

This is how we construct bootstrap samples and an approximate distribution:

```
B <- 10^4 # Number of bootstrap samples
m_star <- replicate(
  B,
  {
    X_star <- sample(income, n, replace = TRUE)
    median(X_star)
  }
)
```

Note a confidence interval constructed with the bootstrap is much closer to one constructed with the theoretical distribution:

```
quantile(m_star, c(0.025, 0.975))
      2.5%      97.5%
34558.60 58612.31
```

More on Bootstrapping

- When did bootstrapping originate? The first paper on the theory of bootstrapping is attributed to Bradley Efron (1979), but it wasn't until the early 1990's with the *Science* paper by Efron & Tibshirani (1991), their subsequent book (1993), and advances in modern computing that the method gained widespread attention and use. It is now one of the most common methods of obtaining measures of uncertainty in numerous statistical applications where no closed-form expressions and/or approximations are available.
- The idea of resampling your data to generate more data is akin to “pulling yourself up by your bootstraps” to do analysis without extra outside help. Statisticians were very suspicious of bootstrapping when it first gained attention, but are now convinced of its power. If it seems like you are getting something for nothing (or downright cheating) with bootstrapping, I tend to agree, but the method has proven useful in an incredible variety of problems (censored data, survival data, time series, classification trees, goodness of fit statistics, linear and nonlinear regression, etc.).

Suppose $x_1, \dots, x_n$ is a random sample from some large population and we wish to estimate some population parameter θ . If $\hat{\theta}$ is an estimate of θ with some estimated standard error $SE(\hat{\theta})$, then under certain conditions a $100(1 - \alpha)\%$ confidence interval for θ can be formed as:

$$\hat{\theta} \pm t_{\alpha/2} SE(\hat{\theta}) \quad (z_{\alpha/2} \text{ may sometimes be used instead of } t_{\alpha/2}).$$

- For example, if $\theta = \mu$ is the population mean, then a $100(1 - \alpha)\%$ confidence interval for μ is given by:

$$\bar{x}_n \pm t_{\alpha/2} \cdot \frac{s}{\sqrt{n}} \Leftrightarrow (\text{estimate}) \pm \left(\begin{array}{c} \text{critical} \\ \text{value} \end{array} \right) \cdot \left(\begin{array}{c} \text{standard error} \\ \text{of the estimate} \end{array} \right).$$

The validity and accuracy of this CI depends on three basic assumptions:

1. $SE(\hat{\theta})$ is a good estimate of the standard deviation of the sampling distribution of $\hat{\theta}$.
2. $\hat{\theta}$ is an unbiased or nearly unbiased estimator for the parameter θ .

3. The sampling distribution of $\hat{\theta}$ is approximately normal. (The Central Limit Theorem can often be used here.)

The method of bootstrapping can address all of these issues:

1. It can provide an estimate of $SE(\hat{\theta})$ when no theoretical closed-form expression for $SE(\hat{\theta})$ exists, or provide an alternative if we are uncertain about the accuracy of an existing estimate. Can you think of some examples where the SE cannot be estimated or must be approximated?
2. It can provide an estimate of the bias of $\hat{\theta}$ as an estimator of θ .
3. It can provide information on the shape of the sampling distribution of $\hat{\theta}$. This can be used to calculate improved confidence intervals over the normal-based ones if the sampling distribution is not normal.

So what is bootstrapping? The theory behind bootstrapping can best be explained with a simple example.

- Suppose we have a simple random sample (SRS) and wish to estimate the standard error of the sample 10% trimmed mean, say $\bar{x}_{TM}$, as an estimate of the population mean, say μ . Unlike the sample mean, whose variance depends only on the population variance ($\sigma/\sqrt{n}$), $Var(\bar{x}_{TM})$ depends on the exact population distribution, which of course we don't know.
- The idea behind the (nonparametric) bootstrap is this: if we knew the x -values for the entire population ($x_1, \dots, x_N$), then we could estimate the sampling distribution of $\bar{x}_{TM}$ by simulation. How?

- 1.

- 2.

- 3.

We could estimate the $SE(\bar{x}_{TM})$ from the standard deviation of the several thousand trimmed means.

- Unfortunately, we do not know the x -values for the population; we only know the x -values for our sample. Bootstrapping says to assume that the population looks exactly like our sample, only many times larger. This is really our best nonparametric guess as to what the population looks like.
- For example, suppose the following SRS of x -values ($n = 20$) represents guesses of my age when I taught a stats class in 2017:

26 25 26 29 29 28 27 29 28 26 41 33 29 34 27 33 27 30 29 30

then we assume that 5% of the population x -values are 26, 5% are 25, etc. In this way, this “bootstrap population” represents our best guess as to what the actual population looks like.

- To perform bootstrapping then, we simulate drawing samples of size $n = 20$ from this “bootstrap population.” If the population size N is large relative to n , this is equivalent to

drawing random samples of size 20 with replacement from the original sample of size 20. Since we are sampling with replacement, we will not necessarily get the same sample every time. For example, if `ages` is the vector of 20 age guesses, issuing the **R** command `sample(ages, 20, replace = T)` two times gave the following two samples:

```
29 27 34 25 27 29 33 28 25 28 26 29 26 26 27 26 27 28 29 28
28 29 33 27 26 27 26 29 33 33 29 29 26 30 26 30 30 27 25 26
```

- Finally, generate a large number of these bootstrap samples (say 10,000 or more), calculate the sample 10% trimmed mean for each sample, and then calculate the sample standard deviation of the 10,000 sample trimmed means as an estimate of $SE(\bar{x}_{TM})$.

Example: Finding the $SE(\bar{x}_{TM})$ using R: Suppose as discussed above that an SRS of size 20 yielded the age guesses 26, 25, ..., 30, and we would like to estimate $SE(\bar{x}_{TM})$. To do this in **R**, we could issue the commands below, and consider the output which follows. There is a command in **R** called `boot()` that will do this automatically, but to understand how to use it, it's useful to see how we might write our own program to do bootstrapping in **R**. To generate one bootstrap sample, we could do the following:

```
set.seed(2026)
ages <- c(26,25,26,29,29,28,27,29,28,26,41,33,29,34,27,33,27,30,29,30)
i <- sample(1:20, 20, replace = T)
i
[1] 1 6 13 15 12 4 16 5 12 2 19 15 14 17 12 10 18 2 5 12
ages[i]
[1] 26 28 29 27 33 29 33 29 33 25 29 27 34 27 33 26 30 25 29 33
mean(ages[i], trim = 0.10)
[1] 29.25
```

We would want to store the value of the trimmed mean for the bootstrap sample (the “29.25”) and then repeat the process (with a loop) several thousand times, getting a new vector `i` each time we called “sample”. Notice that rather than sampling with replacement from the data vector (e.g., `sample(ages, 20, replace = T)`), I sampled from the integers 1 to n and then got the data values for the bootstrap sample by using the expression `x[i]`. There is no difference between the two ways of doing it, but this way illustrates how the `bootstrapping()` function does it in our `math3190package` and will help in understanding how to use `bootstrapping`. The commands to carry out the bootstrapping look like this.

```
library(math3190package) |> suppressPackageStartupMessages()
set.seed(2026)
ages <- c(26,25,26,29,29,28,27,29,28,26,41,33,29,34,27,33,27,30,29,30)
trmean <- function(x,i) {mean(x[i], trim = 0.10)}
(b1 = bootstrapping(ages, trmean, 10000))
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>          <dbl>          <dbl> <dbl>    <dbl>
1 stat1          28.8          28.8 0.0769  0.733
```

- From this output, we have the average of the 10,000 bootstrapped trimmed means is 28.83, the trimmed mean value from our original sample is 28.75, so estimated bias = sample_average - original_value = 0.08. Then the bootstrap estimated standard error: $SE_B(\bar{x}_{TM}) = 0.733$.
- The estimated bias is thus the difference between the average of the 10,000 bootstrap trimmed means and the actual sample trimmed mean.
- **If** (not usually the case) the sampling distribution of the sample trimmed mean is approximately normal, we could establish an approximate 95% confidence interval for μ_{TM} as:

$$\bar{x}_{TM} \pm z_{.025} \cdot SE(\bar{x}_{TM}) = 28.75 \pm 1.96(0.733) = 28.75 \pm 1.44 \implies 27.3 < \mu_{TM} < 30.2.$$

In the bootstrap procedure outlined above, it is important to note the following assumptions and related issues:

1. It assumes an infinite population since it uses sampling with replacement from the observed sample. This is not a problem if the population size N is in fact large relative to the sample size n . However, if N is small, it is unclear what to do!
2. It assumes simple random sampling. Since the bootstrap samples are generated by simple random sampling (from an infinite population), it is estimating the sampling distribution of $\hat{\theta}$ for simple random sampling from an infinite population.

If the original sample was acquired based on some other type of sampling scheme, such as stratified random sampling, then we want to estimate the sampling distribution of $\hat{\theta}$ for a *stratified random sample*. Hence, the bootstrap samples must be taken in exactly the same way as the original sample. It is sometimes very tricky to figure out how to do this correctly and is still an active area of research for many types of sampling.

Bootstrap Example Using R:

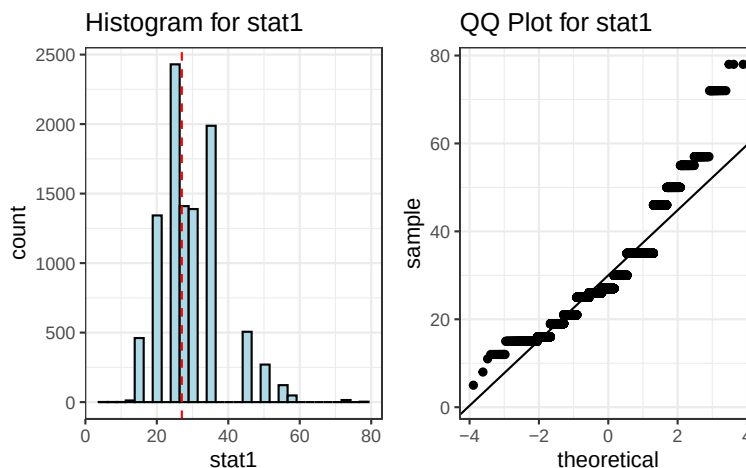
Consider the data on grocery costs at a particular grocery, given below. In class, we learned how to develop a confidence interval for M using percentiles from a binomial distribution with the `SignTest()` function in **R**. Here, we use bootstrapping to estimate the SE of the sample median $m = \widehat{M}$, find a confidence interval for M , and examine the sampling distribution of m .

1	3	4	5	5	8	11	12	15	15	16	19	21	25	26	27
30	35	35	46	50	55	57	72	78	85	93	137	158	212	269	

Taking 10,000 bootstrap samples from our sample of 31 costs is performed as follows:

```
set.seed(2026)
spent <- c(1,3,4,5,5,8,11,12,15,15,16,19, # Defines the 31 grocery amounts
          21,25,26,27,30,35,35,46,50,55, # spent, as given in the notes
          57,72,78,85,93,137,158,212,269)
med <- function(x,i) median(x[i]) # Defines "med" as the median function
(b2 <- bootstrapping(spent, med, 10000)) # Bootstraps the median of spent data
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>          <dbl>          <dbl> <dbl>    <dbl>
1 stat1          29.3            27    2.35    8.48
```

From this output, we have the sample median $m = 27$, the estimated bias $= 2.35$, the mean of the bootstrap medians is 29.3, and the bootstrap estimated standard error: $SE_B(m) = 8.48$. To use a normal-based confidence interval, we need to assess whether the sampling distribution of the sample median is approximately normal. We can do this by examining the distribution of the 10,000 bootstrap medians, by issuing the command `plot(b2)`. This command creates the following two displays. Do the medians appear normal?



Instead of relying on the approximate normality of the bootstrap median distribution to construct a 95% confidence interval, we could compute the 2.5 and 97.5 percentiles for the bootstrap sample, and use these as the limits of the CI. This can adjust for any bias or skewness that might be present in the distribution. One of the methods built into **R** is the “bias-corrected & adjusted” (BCa) method, whereby relevant percentiles of the bootstrap sample are estimated. To illustrate this:

```
boot_ci(b2)      # Constructs bootstrap CIs for M
BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 10000 bootstrap replicates
```

```
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  8.02  41.3
2 basic           0.95    4    38
3 percent         0.95  16    50
4 bca             0.95  15    46
```

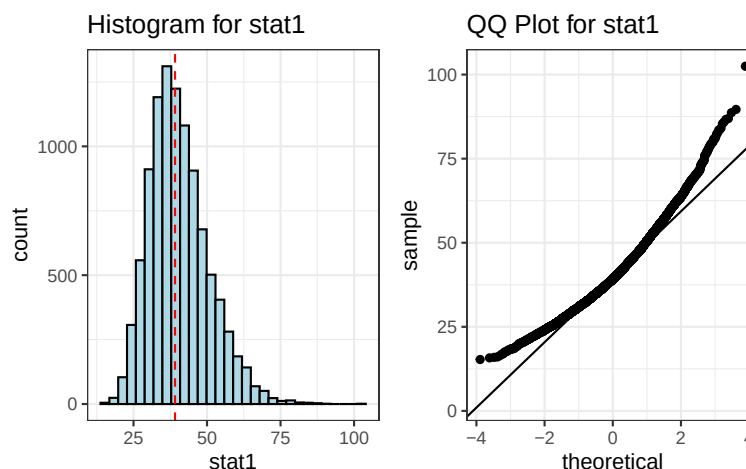
Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

Hence, a 95% nonparametric CI based on this method is (\$15, \$46), which is asymmetric about the estimate of $m = \$27$. One can change the confidence level using the optional argument `conf=` for `boot_ci()`; see the **R** help menu. The BCa method is based on the actual distribution of the bootstrap estimates (rather than just their standard deviation) and requires a large number of bootstrap replications to be reliable (say 5000 or more for a 95% CI). In general, more bootstrap replications are needed to reliably estimate percentiles further in the tails (for a 99% CI, for example, where one needs the 0.5% and 99.5% percentiles).

What if we wanted a 95% confidence interval for the 10% trimmed mean grocery amount? We would carry out the same steps, but would bootstrap the 10% trimmed mean rather than the median. [A 10% trimmed mean in this case drops the three highest and three lowest values, thus avoiding the three outliers.] Code for doing this appears below with the output following. Below that are a histogram of the bootstrap trimmed means and corresponding normal quantile plot are shown above. Also, a 95% confidence interval for the true 10% trimmed mean grocery cost is reported.

```
tpct <- 10
trmean <- function(x, i, trimpct) {mean(x[i], trim = trimpct/100)}
(b3 <- bootstrapping(spent, trmean, 10000, trimpct = tpct))
# A tibble: 1 x 5
  statistic sample_average original_value bias std.error
  <chr>          <dbl>          <dbl> <dbl>    <dbl>
1 stat1          40.5            39.1  1.40     10.1
```

plot(b3)



For more on the Bootstrap, including corrections one can apply to improve these confidence intervals, please consult the book *An introduction to the bootstrap* by Efron, B., & Tibshirani, R. J.

Note that we can use ideas similar to those used in the bootstrap in cross validation: instead of dividing the data into equal partitions, we simply bootstrap many times.

More on the caret Package

We have discussed the kNN machine learning algorithm. This is just one of many algorithms out there. Many of these algorithms are implemented in **R**. However, they are distributed via different packages, developed by different authors, and often use different syntax.

The **caret** (short for **C**lassification **A**nd **R**egression **T**raining) package tries to consolidate these differences and provide consistency. It currently includes 238 different methods which are summarized in the **caret** package manual¹.

¹<https://topepo.github.io/caret/available-models.html>

Keep in mind that **caret** does not include the needed packages and, to implement a package through **caret**, you still need to install the library. The required packages for each method are described in the package manual.

The **caret train()** function

The **caret** package also provides a function that performs cross validation for us. Here we provide some examples showing how we use this incredibly helpful package. We will use the 2 or 7 example to illustrate:

```
library(tidyverse)
library(dslabs)
data("mnist_27")
```

The **caret train** function lets us train different algorithms using similar syntax. So, for example, we can type:

```
library(caret)
train_glm <- train(y ~ ., method = "glm", data = mnist_27$train)
train_knn <- train(y ~ ., method = "knn", data = mnist_27$train)
```

To make predictions, we can use the output of this function directly without needing to look at the specifics of **predict.glm** and **predict.knn**. Instead, we can learn how to obtain predictions from **predict.train**.

The code looks the same for both methods:

```
y_hat_glm <- predict(train_glm, mnist_27$test, type = "raw")
y_hat_knn <- predict(train_knn, mnist_27$test, type = "raw")
```

This permits us to quickly compare the algorithms. For example, we can compare the accuracy like this:

```
confusionMatrix(y_hat_glm, mnist_27$test$y)$overall[["Accuracy"]]
[1] 0.775
```

```
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall[["Accuracy"]]
[1] 0.835
```

Cross Validation

When an algorithm includes a tuning parameter, **train** automatically uses cross validation to decide among a few default values. To find out what parameter or parameters are optimized, you can read the manual ² or study the output of:

```
getModelInfo("knn")
```

We can also use a quick lookup like this:

```
modelLookup("knn")
```

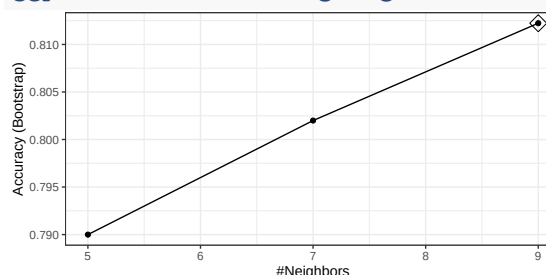
²<http://topepo.github.io/caret/available-models.html>

If we run it with default values:

```
train_knn <- train(y ~ ., method = "knn", data = mnist_27$train)
```

You can quickly see the results of the cross validation using the `ggplot` function. The argument `highlight` highlights the max (thought it may give a warning about a function being deprecated):

```
ggplot(train_knn, highlight=TRUE)+theme_bw()
```



By default, the cross validation is performed by taking 25 bootstrap samples comprised of 25% of the observations. We can change this to K-fold cross validation using the `trControl` option:

```
trControl = trainControl(method = "cv", number = 10).
```

It does 10-fold cross validation by default if `number` is not specified.

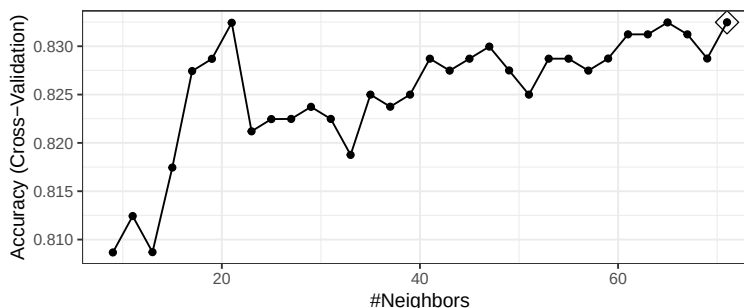
For the `kNN` method, the default is to try $k = 5, 7, 9$. We change this using the `tuneGrid` parameter. The grid of values must be supplied by a data frame with the parameter names as specified in the `modelLookup` output.

Here, we present an example where we try out 30 values between 9 and 67. To do this with `caret`, we need to define a column named `k`, so we use this: `data.frame(k = seq(9, 67, 2))`.

Note that when running this code, we are fitting 30 versions of `kNN` to 25 bootstrapped samples. Since we are fitting $30 \times 25 = 750$ `kNN` models.

We will set the seed because cross validation is a random procedure and we want to make sure the result here is reproducible.

```
set.seed(2026)
train_knn <- train(y ~ ., method = "knn",
  data = mnist_27$train,
  trControl = trainControl(method = "cv"),
  tuneGrid = data.frame(k = seq(9, 71, 2)))
ggplot(train_knn, highlight = TRUE) + theme_bw()
```



```
train_knn$bestTune
```

```
  k
32 71
```

To access the parameter that maximized the accuracy, you can use this:

```
train_knn$bestTune
```

```
  k
32 71
```

and the best performing model like this:

```
train_knn$finalModel
```

```
71-nearest neighbor model
```

```
Training set outcome distribution:
```

```
  2    7
401 399
```

The function `predict` will use this best performing model. Here is the accuracy of the best model when applied to the test set, which we have not used at all yet because the cross validation was done on the training set:

```
confusionMatrix(predict(train_knn, mnist_27$test, type = "raw"),
  mnist_27$test$y)$overall["Accuracy"]
```

```
Accuracy
0.825
```

The `results` component of the `train` output includes several summary statistics related to the variability of the cross validation estimates:

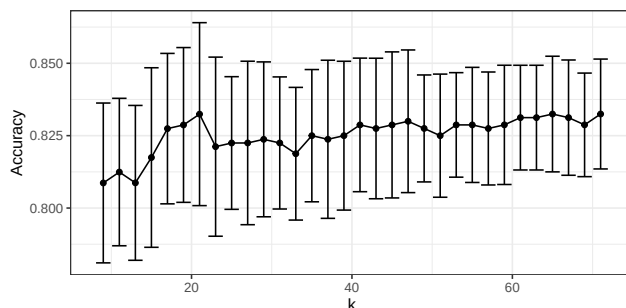
```
names(train_knn$results)
```

```
[1] "k"          "Accuracy"   "Kappa"      "AccuracySD"
[5] "KappaSD"
```

Note: `Kappa` is [Cohen's Kappa](#) and it is like classification accuracy, except that it is normalized at the baseline of random chance on your dataset. It is a more useful measure to use on problems that have an imbalance in the classes

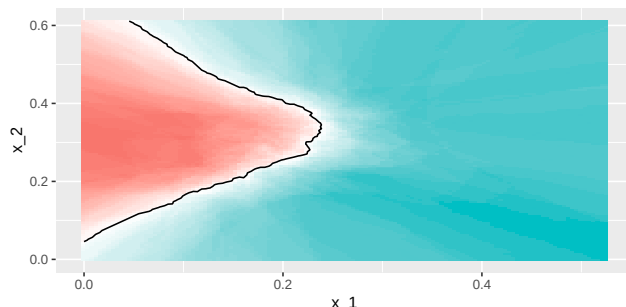
We can also see the standard deviation bars obtained from the cross validation samples. These are not confidence intervals.

```
train_knn$results |>
  ggplot(aes(x = k, y = Accuracy)) +
  geom_line() + geom_point() + theme_bw() +
  geom_errorbar(aes(x = k,
    ymin = Accuracy - AccuracySD,
    ymax = Accuracy + AccuracySD))
```



Example: Fitting with Loess

The best fitting kNN model approximates the true conditional probability:



However, we do see that the boundary is somewhat wiggly. This is because kNN, like the basic bin smoother, does not use a kernel. Kernels can be used to make a smoother line since they are based on models instead of just the empirical data.

To improve this we could try using a loess kernel. By reading through the available models part of the manual³ we see that we can use the `gamLoess` method. In the manual⁴ we also see that we need to install the `gam` package if we have not done so already with `install.packages("gam")`.

Then we see that we have two parameters to optimize:

```
modelLookup("gamLoess")
```

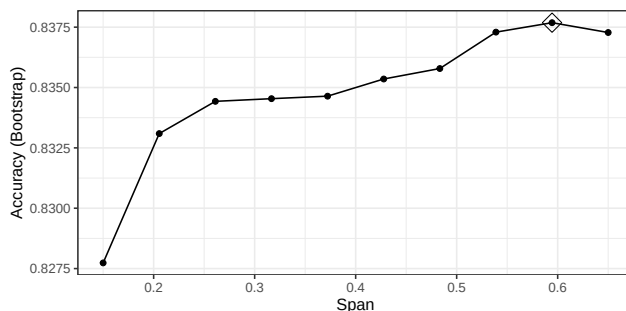
	model	parameter	label	forReg	forClass	probModel
1	gamLoess	span	Span	TRUE	TRUE	TRUE
2	gamLoess	degree	Degree	TRUE	TRUE	TRUE

We will stick to a degree of 1. But to try out different values for the span, we still have to include a column in the table with the name `degree` so we can do this:

```
grid <- expand.grid(span = seq(0.15, 0.65, length = 10),
  degree = 1)
```

We will use the default cross validation control parameters.

```
set.seed(2026)
train_loess <- train(y ~ ., method = "gamLoess",
  tuneGrid = grid, data = mnist_27$train)
ggplot(train_loess, highlight = TRUE) + theme_bw()
```



³<https://topepo.github.io/caret/available-models.html>

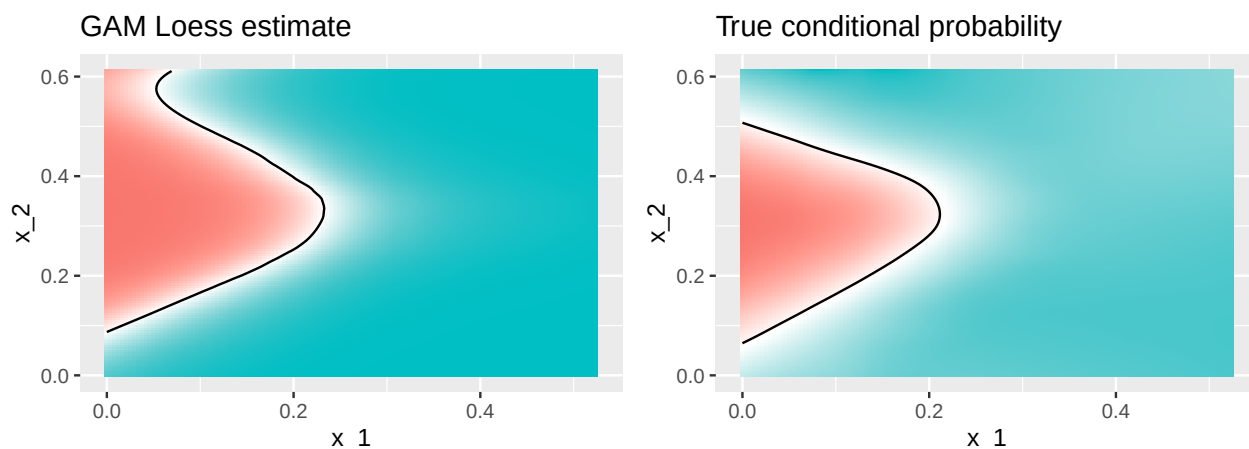
⁴<https://topepo.github.io/caret/train-models-by-tag.html>

We can see that the method performs similar to kNN:

```
confusionMatrix(data = predict(train_loess, mnist_27$test),  
                 reference = mnist_27$test$y)$overall["Accuracy"]
```

Accuracy
0.83

It produces a smoother estimate of the conditional probability, which, although not at all necessary, is a bit more aesthetically pleasing:



Conclusion

Many algorithms in statistical and machine learning rely on cross validation and/or other similar verification methods. We will continue to see this used as we continue in the course.

Notes 6: Optimization

Table of contents

Intro	88
Calculus Optimization Review	88
Newton's Method	89
Gradient Descent Method	92
Line Search	93
Optimization in Higher Dimensions	94
Gradient Descent	94
Newton's Method	97
R optimize() and optim() Functions	99
Maximum Likelihood Estimation	100
Bonus: Conjugate Gradient Method	103
The Method	103
Preconditioning	105

Intro

Optimization problems come up quite frequently in statistics and data science problems. We have encountered some optimization problems already in this class in the form of maximum likelihood maximization and Bayesian estimation.

Calculus Optimization Review

One way to optimize a function is to do so directly using calculus. Suppose we want to optimize $f(x)$ by finding either a global maximum or minimum. The steps to do this include:

1. Find $f'(x)$.
2. Set $f'(x) = 0$ and solve for x to find the critical number(s).
3. Plug the critical number(s) and endpoint(s) into $f(x)$. If there are no endpoints, take a limit as x approaches the end of the domain.
4. Take the x value that maximizes or minimizes the function, if it exists.

Example 1: If $f(x) = \frac{x}{x^2 + 1}$, find the value of x that maximizes and minimizes the function.

So, we can say that $\operatorname{argmax}_x \left\{ \frac{x}{x^2 + 1} \right\}$ is _____ and the $\operatorname{argmin}_x \left\{ \frac{x}{x^2 + 1} \right\}$ is _____.

Example 2: Now let's try to maximize this function: $g(x) = x^2 + xe^x$.

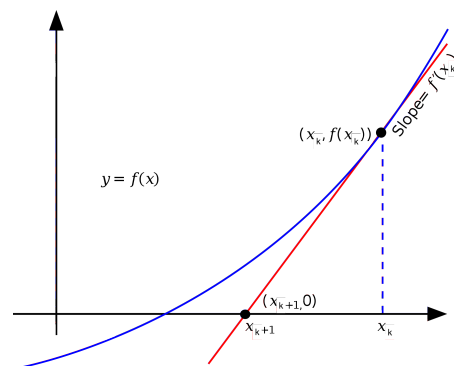
This function has a minimizer, but it is not possible to directly solve for it. We would need a numerical method to get an approximation for x .

Newton's Method

One fairly straight forward and powerful method for finding roots (or zeros) of a function is **Newton's Method**. This method is iterative, which means that it typically needs to be run multiple times with each iteration giving a better approximation of the root.

The idea here is to first start with a simple guess of which x is the root. Call this first guess x_k (where $k = 0$ for the initial guess). Then use a tangent line to the curve at $x = x_k$ that intercepts the x -axis at x_{k+1} where x_k is the k th iteration of x .

Since any tangent line that does not have a slope of zero will intercept the x -axis somewhere, we will call the x value where it does x_{k+1} . Then it will be true that $f'(x_k) = \frac{f(x_k) - 0}{x_k - x_{k+1}}$. Then we can simply solve this for the x_{k+1} :



Therefore, we have

Example 3: Trying this out for finding the root of $g'(x) = 2x + e^x(x + 1)$ from Example 2 above. First, let $f(x) = g'(x)$.

To program this in **R**, it is common to use a tolerance for stopping. That is, if x_{k+1} is very close to x_k , then we should stop.

It only took 5 iterations to converge to within 10^{-10} . If we wanted to make it even more precise, we could set the tolerance even lower, like $1e-16$. This would make x_k and x_{k+1} equal to 15 decimal places.

Now, notice that by finding the root of $f(x)$ in this example, we were really finding the critical number (the x value where the derivative is zero) of $g(x)$ since $g'(x) = f(x)$. That means that the iteration sequence for finding the critical number of $g(x)$ is

$$x_{k+1} = x_k - \frac{g'(x_k)}{g''(x_k)}.$$

That gives us our first optimization method. This is sometimes referred to as **Newton's Method for Optimization**.

```
f <- function(x) {
  2*x + exp(x)*(x+1)
}
f_prime <- function(x) {
  2 + exp(x)*(x+1) + exp(x)
}

xk <- 0 # Initialize our first guess
maxit <- 100 # Set a max iteration number
for(k in 1:maxit) {
  xnew <- xk - f(xk)/f_prime(xk)
  if(abs(xk - xnew) < 1e-10) {
    break
  }
  xk <- xnew
}
xk # Estimate for the zero
[1] -0.2752084

k # Number of iterations to converge
[1] 5
```


Example 4: Find the x values that minimizes $f(x) = x^4 - 3x^2 + x$ using Newton's method.

```
f <- function(x) {
  x^4 - 3*x^2 + x
}
f_prime <- function(x) {
  4*x^3 - 6*x + 1
}
f_double_prime <- function(x) {
  12*x^2 - 6
}

xk <- 0 # Initialize our first guess
maxit <- 100 # Set a max number of iterations
for(k in 1:maxit) {
  xnew <- xk -
    f_prime(xk)/f_double_prime(xk)
  if(abs(xk - xnew) < 1e-10) {
    break
  }
  xk <- xnew
}
xk # Estimate for the zero
[1] 0.1699384

k # Number of iterations to converge
[1] 4
```

Does $x = 0.1699384$ minimize the function? What if we started at a value different than 0:

```
xk <- 1 # Initialize our first guess
for(k in 1:maxit) {
  xnew <- xk -
    f_prime(xk)/f_double_prime(xk)
  if(abs(xk - xnew) < 1e-10) {
    break
  }
  xk <- xnew
}
xk # Estimate for the zero
[1] 1.130901
```

```
xk <- -1 # Initialize our first guess
for(k in 1:maxit) {
  xnew <- xk -
    f_prime(xk)/f_double_prime(xk)
  if(abs(xk - xnew) < 1e-10) {
    break
  }
  xk <- xnew
}
xk # Estimate for the zero
[1] -1.30084
```

Now we are getting three answers at three different starting points! What is happening?

```
c(f(0.1699384), f(1.130901), f(-1.30084))
[1] 0.08413522 -1.07023018 -3.51390504
```

This is illustrating one of the most common problems with optimization algorithms: they can converge to local extrema instead of global extrema. In addition, some methods can also converge to the wrong type of extrema. That is, they could find a maximum when a minimum is desired and vice-versa.

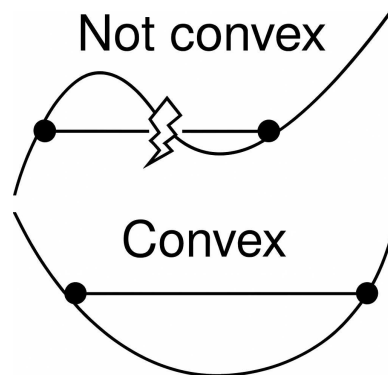
This first issue, converging to the wrong point, occurs if the function is not **convex**. A function is convex if the line segment between any two distinct points on the graph of the function lies above the graph between the two points. There are different types of convex functions.

- f is convex if and only if $f''(x) \geq 0$ for all x .
- f is called **strictly convex** if $f''(x) > 0$ for all x .
- f is called **strongly convex** if $f''(x) \geq m > 0$ for all x .

A convex function is shaped like a cup $\cup$. A **concave** function is shaped like a cap $\cap$. If $f(x)$ is convex, then $-f(x)$ would be concave and vice versa. Many functions are neither convex nor concave.

We can alleviate these problems (to an extent) by

1. Forcing our method to always find a minimum.
 - This will make it so we do not find type of extrema we don't want.
 - We can always find a maximum of $f(x)$ by minimizing $-f(x)$.
2. Allowing a change in the step size of our method.
 - This can (sometimes) get us “unstuck” from a local minimum.
 - Truthfully, getting stuck at a local minimum is the biggest problem for any optimization technique and not one we will be able to tackle here. The easiest way to get around this is to choose different starting points.



Gradient Descent Method

One way to remove the potential to converge to a maximum is to remove the second derivative from the equation and instead multiply by a value that is always positive. If we have $x_{k+1} = x_k - \gamma \cdot f'(x_k)$, then if $f'(x_k) > 0$, we will move to the left (toward a minimum) and if $f'(x_k) < 0$, we will move to the right (again, toward a minimum).

So, all we need to tell us which direction to move is $f'(x_k)$. This does not, however, tell us how far to move. One possible value for γ is $\gamma = 1/|f''(x_k)|$. This usually works, and when it does, it often works well, but if the function has a minimum where the second derivative is 0 or if the function is not twice differential, then this will fail. Also, in higher dimensions, the second derivative matrix, called the Hessian, is also much more difficult and time consuming to compute.

A method that does not rely on the second derivative or Hessian is called a first-order optimization method. One that does rely on the second derivative is a second-order method.

One thing we would like to be true about our step size, γ is that it should get smaller as we approach the minimizer. Thus, γ should change at each iteration. So let's call it γ_k .

Line Search

One way to choose γ_k is to make sure we achieve the maximum amount of decrease of the objective function at each individual step. That means we want to minimize $f(x_k - \gamma_k f'(x_k))$. That is, we want

$$\gamma_k = \operatorname{argmin}_{\gamma \geq 0} \left\{ f\left(x_k - \gamma f'(x_k)\right) \right\}.$$

So, we have to find a minimizer (γ) while we are trying to find another minimizer (x)!

One way to find γ_k is to use a line search. One option is a **backtracking line search**. In this method, we fix a line-search parameter β for $0 < \beta < 1$. Then start with $\gamma = 1$ and while $f(x_k - \gamma f'(x_k)) > f(x_k) - \frac{\gamma}{4} f'(x_k)^2$, update $\gamma = \beta \cdot \gamma$ and try again. Once we have a value of γ when that inequality is not true, we set $\gamma_k = \gamma$.

The code below shows the backtracking line search in action!

```
xk <- 0 # Initialize our first guess
for(k in 1:maxit) {
  gamma <- 1
  beta <- 0.5
  while(f(xk - gamma * f_prime(xk)) > f(xk) - gamma/4 * f_prime(xk)^2) {
    gamma <- beta * gamma
  }
  xnew <- xk - gamma * f_prime(xk)
  if(abs(xk - xnew) < 1e-10) {
    break
  }
  xk <- xnew
}
xk # Estimate for the zero
[1] -1.30084

k
[1] 10
```

Using $1/|f''(x_k)|$ as our step size (Newton's method), $k = 9$ for convergence. So, very close! The time for this method is 0.005 seconds on my computer while Newton's method took 0.003 seconds.

So where does the $f(x_k) - \frac{\gamma}{4} f'(x_k)^2$ come from in the line search inequality? We want to minimize $g(\gamma) = f(x_k - \gamma f'(x_k))$ with respect to γ . Take the derivative with respect to γ :

$$g'(\gamma) = -f'(x_k) f'(x_k - \gamma f'(x_k)).$$

When $\gamma = 0$, the equation of the tangent line for $f(x_k - \gamma f'(x_k))$ is $y = g(0) + g'(0)(\gamma - 0)$, which is

$$\begin{aligned} y &= f(x_k - 0 f'(x_k)) - f'(x_k) \cdot f'(x_k - 0 f'(x_k))(\gamma - 0) \\ &= f(x_k) - \gamma f'(x_k)^2. \end{aligned}$$

So, we are sure that $f(x_k - \gamma f'(x_k)) = f(x_k) - \gamma f'(x_k)^2$ when $\gamma = 0$. When γ is small we have,

$$f(x_k - \gamma f'(x_k)) \approx f(x_k) - \gamma f'(x_k)^2$$

due to a linear approximation. Since $\gamma > 0$ and $f'(x_k)^2 > 0$, we know that when γ is small, for $0 < \alpha < 1$,

$$f(x_k) - \gamma f'(x_k)^2 < f(x_k) - \alpha \cdot \gamma f'(x_k)^2.$$

Then that means

$$f(x_k - \gamma f'(x_k)) < f(x_k) - \alpha \cdot \gamma f'(x_k)^2$$

as well since $f(x_k - \gamma f'(x_k)) \approx f(x_k) - \gamma f'(x_k)^2 < f(x_k) - \alpha \cdot \gamma f'(x_k)^2$.

Therefore, specifying a larger γ value (like $\gamma = 1$) and then gradually making it smaller will eventually get it down to a point where

$$f(x_k - \gamma f'(x_k)) < f(x_k) - \alpha \cdot \gamma f'(x_k)^2.$$

That will help us find one of the largest γ values that will work as a step size. It has been found that α between 0.01 and 0.3 works well, so we will use $\alpha = 0.25$. For more info on this, see *Convex Optimization* by Boyd and Vandenberghe page 465.

If we are able to perfectly optimize γ_k at each iteration to take the largest possible step, we would be performing the **steepest descent** optimization method. Steepest descent is a special case of **gradient descent**. Gradient descent is any method whose step direction is equal to $-f'(x_k)$ regardless of step size while steepest descent takes the largest step possible in that direction at each iteration. So while we are probably not actually performing **steepest descent** (since we are estimating the step size), we are attempting to do so. Gradient descent is by definition a first-order optimization method. So even though Newton's method involves $f'(x_k)$, it is not a gradient descent method since it relies on the second derivative.

Optimization in Higher Dimensions

Gradient Descent

Now that we have some intuition about these optimization algorithms, we should discuss them in higher dimensions. A multi-dimensional derivative is known as a **gradient** (hence the name of gradient descent). A gradient gives the direction of fastest change (increase or decrease). These gradients can be computed using **partial derivatives**. This is a Calculus III topic, but they are pretty simple. When taking the partial derivatives of a function with respect to a variable, we just treat all other variables as constants.

Example 5 Let $f(x, y, z) = 12x^2y + y^2z + z^2$.

$$\frac{\partial f}{\partial x} =$$

$$\frac{\partial f}{\partial y} =$$

$$\frac{\partial f}{\partial z} =$$

The gradient of $f(x_1, x_2, \dots, x_n) = f(\mathbf{x})$ is $\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$. So the gradient of a function of n

variables is an n directional vector. The gradient of the function in Example 5 is

$$f(x, y, z) \text{ is } \nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \vdots \\ \frac{\partial f}{\partial z} \end{bmatrix} = \begin{bmatrix} 24x \\ 12x^2 + 2yz \\ \vdots \\ y^2 + 2z \end{bmatrix}. \text{ That means the slope at a given point } (x, y, z) \text{ is}$$

greatest when we go $24x$ in the x -direction, $12x^2 + 2yz$ in the y -direction, and $y^2 + 2x$ in the z -direction.

For using the gradient descent method in higher dimensions, we need to replace $f'(x)$ with $\nabla f(\mathbf{x})$. So, we have

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \gamma_k \nabla f(\mathbf{x}).$$

We can find γ_k again using a line search. We still fix a line-search parameter β for $0 < \beta < 1$. Then start with $\gamma = 1$ and while $f(\mathbf{x}_k - \gamma \nabla f(\mathbf{x}_k)) > f(\mathbf{x}_k) - \frac{\gamma}{4} \|\nabla f(\mathbf{x}_k)\|^2$, update $\gamma = \beta\gamma$ and try again. Once we have a value of γ when that inequality is not true, we set $\gamma_k = \gamma$.

Notice that $f'(x_k)^2$ has been replaced with $\|\nabla f(\mathbf{x}_k)\|^2$, which is the squared **Euclidean norm** of the gradient vector. That is, $\|\nabla f(\mathbf{x}_k)\|^2 = \sum_{i=1}^n \left(\frac{\partial f}{\partial x_i} \Big|_{\mathbf{x}_k} \right)^2$. We can find this in **R** using either `norm(gradient, type = "2")^2` or `sum(gradient^2)`.

Also, when we check for convergence, the norm of the difference in the vectors can be used.

Example 6: Recall in Notes 7 when we discussed the Poisson regression, we used maximum likelihood to find the estimates for the intercept and slope. In that example we had the log likelihood was

$$\ell(\beta_0, \beta_1) = \sum_{i=1}^n y_i(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \ln(y_i!)$$

However, we want to maximize this function and our algorithm only works to minimize functions. Easy fix! Maximizing a function is the same as minimizing its negative. So, we want to minimize

$$n\ell(\beta_0, \beta_1) = - \left(\sum_{i=1}^n y_i(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \ln(y_i!) \right)$$

Now obtain the gradient:

$$\nabla n\ell(\beta_0, \beta_1) = \begin{bmatrix} \frac{\partial n\ell}{\partial \beta_0} \\ \frac{\partial n\ell}{\partial \beta_1} \end{bmatrix} = \begin{bmatrix} -\left(\sum_{i=1}^n y_i - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i)\right) \\ -\left(\sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \exp(\beta_0 + \beta_1 x_i)\right) \end{bmatrix}$$

In this case, the variables we are trying to find are β_0 and β_1 . Let's code an implementation of this in **R**.

```
set.seed(2024)
x <- rnorm(5) # Simulate 5 random values for x
# Simulate 5 random values for y (with a couple different means)
y <- c(rpois(3, 2), rpois(2, 6))
mod <- glm(y ~ x, family = "poisson") # Fit the Poisson model
b0 <- coef(mod)[1] # Extract the intercept
b1 <- coef(mod)[2] # Extract the slope
b0; b1
(Intercept)
  1.527908
      x
0.08645586

neg_likelihood <- function(beta) {
  # Make it negative since we are trying to maximize it.
  # That is the same as minimizing the negative.
  -(sum(y*(beta[1] + beta[2]*x)) - sum(exp(beta[1] + beta[2]*x)) -
    sum(log(factorial(y))))
}

gradient <- function(beta) {
  # These should be the partial derivatives of the function we are
  # minimizing. So, their signs are also opposites.
  dl_db0 <- -(sum(y) - sum(exp(beta[1] + beta[2]*x)))
  dl_db1 <- -(sum(x*y) - sum(x*exp(beta[1] + beta[2]*x)))
  c(dl_db0, dl_db1) # Return the gradient vector
}

maxit <- 100
betak <- c(0, 0) # Initialize our first guess
for(k in 1:maxit) {
  gamma <- 1
  line_beta <- 0.5 # Line search parameter, not a beta from the model!
  while(neg_likelihood(betak - gamma * gradient(betak)) >
    neg_likelihood(betak) - gamma/4 * sum(gradient(betak)^2)) {
    gamma <- line_beta * gamma
  }
  newbeta <- betak - gamma * gradient(betak)
  if(norm(betak - newbeta, type = "2") < 1e-10) {
```

```

    break
  }
  betak <- newbeta
}
betak # Estimate for the zero
[1] 1.52790762 0.08645587

k
[1] 42

```

Newton's Method

Now, for one input variable, Newton's method looks like

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

We already saw that a multi-dimensional version of $f'(x_k)$ is the gradient. A multi-dimensional version of a second derivative is called a Hessian. Here is how the Hessian of a function $f(x_1, x_2, \dots, x_n) = f(\mathbf{x})$ is defined:

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

Note that when we take the second partial derivatives with more than one variable, $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial}{\partial x_i} \left[\frac{\partial f}{\partial x_j} \right]$. So, we take the partial derivative with respect to the variable listed second in the denominator first, and then take the other partial derivative. However, this usually does not matter since $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$ in almost every practical case.

Notice also that these operations are getting larger. $f(\mathbf{x})$ is a scalar, $\nabla f(\mathbf{x})$ is an $n \times 1$ vector, and $\mathbf{H}_f$ is an $n \times n$ matrix.

Getting back to the equation, we are dividing by $f''(x_k)$ in Newton's method. We don't divide matrices, instead, we take inverses and multiply. So, Newton's method becomes

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\mathbf{H}_f]^{-1} \nabla f(\mathbf{x}).$$

Because an inverse of a matrix exists here, this method can become **very** computational expensive when n is large.

Example 6: Let's try Newton's method on the Poisson regression problem. Recall the gradient:

$$\nabla n\ell(\beta_0, \beta_1) = \begin{bmatrix} \frac{\partial n\ell}{\partial \beta_0} \\ \frac{\partial n\ell}{\partial \beta_1} \end{bmatrix} = \begin{bmatrix} -\left(\sum_{i=1}^n y_i - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i)\right) \\ -\left(\sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \exp(\beta_0 + \beta_1 x_i)\right) \end{bmatrix}$$

To complete the Hessian, we need to get more partial derivatives:

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 n\ell}{\partial \beta_0^2} & \frac{\partial^2 n\ell}{\partial \beta_0 \partial \beta_1} \\ \frac{\partial^2 n\ell}{\partial \beta_1 \partial \beta_0} & \frac{\partial^2 n\ell}{\partial \beta_1^2} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i) & \sum_{i=1}^n x_i \exp(\beta_0 + \beta_1 x_i) \\ \sum_{i=1}^n x_i \exp(\beta_0 + \beta_1 x_i) & \sum_{i=1}^n x_i^2 \exp(\beta_0 + \beta_1 x_i) \end{bmatrix}$$

Let's code it up!

```
neg_likelihood <- function(beta) {
  # Make it negative since we are trying to maximize it.
  # That is the same as minimizing the negative.
  -(sum(y*(beta[1] + beta[2]*x)) - sum(exp(beta[1] + beta[2]*x)) -
    sum(log(factorial(y))))
}

gradient <- function(beta) {
  # These should be the partial derivatives of the function we are
  # minimizing. So, their signs are also opposites.
  dl_db0 <- -(sum(y) - sum(exp(beta[1] + beta[2]*x)))
  dl_db1 <- -(sum(x*y) - sum(x*exp(beta[1] + beta[2]*x)))
  c(dl_db0, dl_db1) # Return the gradient vector
}

hessian <- function(beta) {
  matrix(
    c(sum(exp(beta[1] + beta[2]*x)), sum(x*exp(beta[1] + beta[2]*x)),
      sum(x*exp(beta[1] + beta[2]*x)), sum(x^2*exp(beta[1] + beta[2]*x))),
    nrow = 2
  )
}

maxit <- 100
betak <- c(0, 0) # Initialize our first guess
for(k in 1:maxit) {
  newbeta <- betak - solve(hessian(betak)) %>% gradient(betak)
  if(norm(betak - newbeta, type = "2") < 1e-10) {
    break
  }
  betak <- newbeta
}
betak # Estimate for the zero
```



```

      [,1]
[1,] 1.52790761
[2,] 0.08645586

```

```

k
[1] 9

```

This converges in far fewer iterations than the gradient descent method and a line search is not needed.

The bottleneck here is usually finding the Hessian, since that needs to be done by hand first, and then inverting the Hessian if n is big. However, we don't actually need to compute and store the Hessian matrix. We just need to know what $[\mathbf{H}_f]^{-1} \nabla f(\mathbf{x})$ is equal to. If we say $[\mathbf{H}_f]^{-1} \nabla f(\mathbf{x}) = \mathbf{r}$, it can often be much quicker to use numerical methods to determine what $\mathbf{r}$ is by setting $\mathbf{H}_f \mathbf{r} = \nabla f(\mathbf{x})$ and then applying a method like **conjugate gradient** (if applicable) to solve for $\mathbf{r}$ numerically at each iteration. We won't go over the conjugate gradient method, but I have included information about it on the last few pages of these notes in case you are interested.

R optimize() and optim() Functions

R has a couple of nice optimization functions built into the `stats` library that is a part of base R. For one-dimensional optimization, we can use the `optimize()` function. For general purpose optimization, including in multi-dimensional space, the `optim()` function is the one that should be used. These functions use algorithms that are more advanced than the ones we discussed here, but they are similar.

Let's see a couple examples:

```

f <- function(x) {
  x^4 - 3*x^2 + x
}
optimize(f, c(-10, 10)) # Finds the minimum
$minimum
[1] -1.30082

$objective
[1] -3.513905

optimize(f, c(-10, 10), maximum = T) # Finds the maximum (not helpful here)
$maximum
[1] 9.999926

$objective
[1] 9709.71

```

```

set.seed(2024)
x <- rnorm(5)
y <- c(rpois(3, 2), rpois(2, 6))
likelihood <- function(beta) {
  # Define the positive likelihood
  sum(y*(beta[1] + beta[2]*x)) - sum(exp(beta[1] + beta[2]*x)) -
    sum(log(factorial(y)))
}
starting_values <- c(0, 0)
# Putting "fnscale" to a negative number makes us maximize instead of minimize
optim(starting_values, likelihood, control = list(fnscale = -1))
$par
[1] 1.52786041 0.08639149

$value
[1] -9.241506

$counts
function gradient
      65      NA

$convergence
[1] 0

$message
NULL

```

Maximum Likelihood Estimation

Many times in probability, we study distributions that have known parameters. For example, $X_i \sim \text{Normal}(\mu = 10, \sigma^2 = 25)$ or $X_i \sim \text{Exponential}(\theta = 4)$ or $X_i \sim \text{Binomial}(n = 10, p = 0.4)$. But what if we don't know the parameters?

Example: Suppose we have a random sample of size 10 from a distribution that we know is exponential, so $f(x) = \frac{1}{\theta}e^{-x/\theta}$, but we are not sure what θ is. How can we estimate θ ? Here is the sample: 0.2886, 0.0017, 12.70, 12.02, 2.78, 3.23, 21.39, 7.55, 1.28, 2.44.

Note that θ is the mean and the SD. $\text{mean}(\mathbf{x}) = 6.36803$ and $\text{sd}(\mathbf{x}) = 6.997012$. Is one better?

One of the most common ways to estimate a parameter of a distribution is with maximum likelihood estimation. The idea behind maximum likelihood estimation (MLE) is to use knowledge of the pdf (or pmf) of a distribution to find a “good” estimator for the parameter(s). We are trying to maximize something, so ideas from earlier in this set of notes will come in handy here.

It is good to know the pdfs or pmfs of some of the more common distributions:

Distribution Name	pdf or pmf	Support
Normal	$f(x \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{1}{2\sigma^2}(x - \mu)^2\right\}$	$-\infty < x < \infty$
Exponential	$f(x \theta) = \frac{1}{\theta} e^{-x/\theta}$	$x > 0$
Binomial	$f(x n, p) = \binom{n}{x} p^x (1-p)^{n-x}$	$x = 0, 1, \dots, n$
Poisson	$f(x \lambda) = \frac{e^{-\lambda} \lambda^x}{x!}$	$x = 0, 1, \dots$

Here are the steps for MLE:

1. Write down the pdf (or pmf) for the distribution your data came from.
2. Find the likelihood function by multiplying the pdfs (or pmfs) together.
3. Take the natural log of the likelihood function.
4. Take the derivative with respect to the parameter you want to estimate.
5. Set the derivative equal to 0, solve for the parameter, and put a hat (^) over it.
6. Repeat steps 4 and 5 if there are more parameters that need estimating.

Note that the **likelihood function** is identical to the joint pdf (or pmf) except we think of the likelihood as a function of the parameter rather than a function of the x values.

For example, if the joint pdf is $f(x_1, x_2, \dots, x_n; \theta)$ then the likelihood is $L(\theta; x_1, x_2, \dots, x_n)$.

Why do we take the log of the pdf or pmf?

Example: Find the maximum likelihood estimator for θ if we know our random sample came from a distribution with pdf $f(x) = \frac{1}{\theta}e^{-x/\theta}$. The sample is 0.2886, 0.0017, 12.70, 12.02, 2.78, 3.23, 21.39, 7.55, 1.28, 2.44.

Example: Find the MLEs for μ and σ^2 if $X_i \sim N(\mu, \sigma^2)$ for $i = 1, \dots, n$. Are $\hat{\mu}$ and $\hat{\sigma}^2$ unbiased?

Bonus: Conjugate Gradient Method

The conjugate gradient (CG) and preconditioned conjugate gradient (PCG) methods are algorithms for finding numerical solutions for systems of linear equations like those in the form

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \quad (1)$$

where $\mathbf{A}$ and $\mathbf{b}$ are known and we are interested in finding the solution $\mathbf{x}$. There are some restrictions on $\mathbf{A}$ in that it must be symmetric (i.e. $\mathbf{A}^T = \mathbf{A}$) and positive definite (i.e. $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$ for all nonzero $\mathbf{x} \in \mathbb{R}^n$). A matrix with both of these properties is known as symmetric positive definite (SPD). If these are satisfied, CG is one of the most useful numerical techniques for solving large linear systems of equations.

The Method

We will first explain the CG method before discussing the PCG method. The key to CG is the fact that solving the expression (1) is equivalent to the following minimization problem:

$$\arg \min_{\mathbf{x}} J(\mathbf{x}) := \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x},$$

since both give the same solution, which we will denote $\mathbf{x}_*$. Using vector derivatives, the gradient of J is equal to

$$\nabla J(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} := \mathbf{r}(\mathbf{x}),$$

which is also known as the residual of the linear system. The only use of $\mathbf{r}$ in this section will refer to the residual as opposed to a vector of distances, as it is elsewhere in the thesis. Since CG is an iterative method, when $\mathbf{x} = \mathbf{x}_k$, the k th iteration of $\mathbf{x}$, the k th iteration of the residual is given by

$$\mathbf{r}_k = \mathbf{A}\mathbf{x}_k - \mathbf{b}. \quad (2)$$

Therefore, the direction of steepest descent of the function $J(\mathbf{x})$ at iteration k is $-\nabla J(\mathbf{x}) = -\mathbf{r}_k$.

Now, the conjugacy in the name of the CG method comes from the fact that it uses the property that two vectors, $\mathbf{u}$ and $\mathbf{v}$, are called *conjugate* with respect to a SPD matrix $\mathbf{A}$ if $\mathbf{u}^T \mathbf{A} \mathbf{v} = 0$ for $\mathbf{u} \neq \mathbf{v}$. Take a set of nonzero, conjugate vectors $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$. Since they are conjugate, it is simple to show they are also linearly independent, and hence, form a basis in $\mathbb{R}^n$. This means the solution can be written as a linear combination of these vectors,

$$\mathbf{x}_* = \mathbf{x}_0 + \sum_{i=1}^n \alpha_i \mathbf{p}_i.$$

Therefore, if we let

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k, \quad (3)$$

where the subscript denotes iteration number. We will obtain the solution $\mathbf{x}_*$ in at most n iterations since it is certain that $\mathbf{x}_n = \mathbf{x}_*$. Ideally, we will choose these $\mathbf{p}_k$ vectors in such a way that we can approximate $\mathbf{x}_*$ in fewer than n iterations, which is necessary for large n .

We need an expression for α_k and $\mathbf{p}_k$. To determine the α_k values, we will minimize $J(\mathbf{x}_{k+1})$ with respect to α_k :

$$\begin{aligned} J(\mathbf{x}_{k+1}) &= J(\mathbf{x}_k + \alpha_k \mathbf{p}_k) = \frac{1}{2} (\mathbf{x}_k + \alpha_k \mathbf{p}_k)^T \mathbf{A} (\mathbf{x}_k + \alpha_k \mathbf{p}_k) - \mathbf{b}^T (\mathbf{x}_k + \alpha_k \mathbf{p}_k) \\ &= \frac{1}{2} \mathbf{x}_k^T \mathbf{A} \mathbf{x}_k + \alpha_k \mathbf{x}_k^T \mathbf{A} \mathbf{p}_k + \frac{1}{2} \alpha_k^2 \mathbf{p}_k^T \mathbf{A} \mathbf{p}_k - \mathbf{b}^T \mathbf{x}_k - \alpha_k \mathbf{b}^T \mathbf{p}_k, \end{aligned}$$

and so

$$\frac{d}{d\alpha_k} J(\mathbf{x}_{k+1}) = 0 \implies \mathbf{x}_k^T \mathbf{A} \mathbf{p}_k + \alpha_k \mathbf{p}_k^T \mathbf{A} \mathbf{p}_k - \mathbf{b}^T \mathbf{p}_k = 0.$$

Solving this for α_k gives

$$\begin{aligned} \alpha_k &= \frac{\mathbf{b}^T \mathbf{p}_k - \mathbf{x}_k^T \mathbf{A} \mathbf{p}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = \frac{(\mathbf{b}^T - \mathbf{x}_k^T \mathbf{A}) \mathbf{p}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} \\ &= -\frac{\mathbf{r}_k^T \mathbf{p}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}. \end{aligned} \quad (4)$$

Since $\mathbf{p}_k$ can be thought of as the step direction, and we showed $-\mathbf{r}_k$ is the steepest descent direction, we will update the $\mathbf{p}_k$ by using the residuals and the previous $\mathbf{p}$ vector:

$$\mathbf{p}_{k+1} = -\mathbf{r}_{k+1} + \beta_k \mathbf{p}_k. \quad (5)$$

The choice of β_k will be discussed below and $\mathbf{r}_{k+1}$ is updated by using (2) and (3). Specifically,

$$\begin{aligned} \mathbf{r}_{k+1} &= \mathbf{A} \mathbf{x}_{k+1} - \mathbf{b} \quad \text{by (2)} \\ &= \mathbf{A}(\mathbf{x}_k + \alpha_k \mathbf{p}_k) - \mathbf{b} \quad \text{by (3)} \\ &= \mathbf{A} \mathbf{x}_k - \mathbf{b} + \alpha_k \mathbf{A} \mathbf{p}_k \\ &= \mathbf{r}_k + \alpha_k \mathbf{A} \mathbf{p}_k. \end{aligned} \quad (6)$$

β_k must be chosen so that the conjugacy of the $\mathbf{p}_i$ vectors with respect to $\mathbf{A}$ is retained, i.e. $\mathbf{p}_k^T \mathbf{A} \mathbf{p}_{k+1} = 0$. Left-multiplying (5) by $\mathbf{p}_k^T \mathbf{A}$ yields

$$\begin{aligned} \mathbf{p}_k^T \mathbf{A} \mathbf{p}_{k+1} &= \mathbf{p}_k^T \mathbf{A} (-\mathbf{r}_{k+1} + \beta_k \mathbf{p}_k) = 0 \\ \implies -\mathbf{p}_k^T \mathbf{A} \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k^T \mathbf{A} \mathbf{p}_k &= 0. \end{aligned}$$

Solving this for β_k gives us

$$\beta_k = \frac{\mathbf{p}_k^T \mathbf{A} \mathbf{r}_{k+1}}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}.$$

To slightly increase the efficiency of this algorithm, we will now make a few adjustments. Firstly, it is possible to show that $\mathbf{r}_i^T \mathbf{r}_j = 0$ for $i \neq j$ and $\mathbf{r}_i^T \mathbf{p}_j = 0$ for $i > j$. Using this fact and (5), we can change the expression for α_k in (4) to be

$$\alpha_k = -\frac{\mathbf{r}_k^T \mathbf{p}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = -\frac{\mathbf{r}_k^T (-\mathbf{r}_k + \beta_k \mathbf{p}_{k-1})}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = \frac{\mathbf{r}_k^T \mathbf{r}_k - \beta_k \mathbf{r}_k^T \mathbf{p}_{k-1}}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}.$$

Now, using the orthogonality of the $\mathbf{r}_i$ vectors and the fact in (6) that $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$ and thus $\mathbf{A} \mathbf{p}_k = (\mathbf{r}_k - \mathbf{r}_{k+1})/\alpha_k$, β_k can be written

$$\begin{aligned} \beta_k &= \frac{\mathbf{p}_k^T \mathbf{A} \mathbf{r}_{k+1}}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = \frac{(\mathbf{r}_k - \mathbf{r}_{k+1})^T \mathbf{r}_{k+1} / \alpha_k}{\mathbf{p}_k^T (\mathbf{r}_k - \mathbf{r}_{k+1}) / \alpha_k} = \frac{\mathbf{r}_k^T \mathbf{r}_{k+1} - \mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{(-\mathbf{r}_k + \beta_{k-1} \mathbf{p}_{k-1})^T (\mathbf{r}_k - \mathbf{r}_{k+1})} \\ &= \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}. \end{aligned}$$

Algorithm 1 Conjugate Gradient Method

-
0. Set $k = 0$, $\mathbf{r}_0 = \mathbf{A}\mathbf{x}_0 - \mathbf{b}$, and $\mathbf{p}_0 = -\mathbf{r}$;
 1. Set $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}$;
 2. Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$;
 3. Set $\mathbf{r}_{k+1} = \mathbf{r}_k + \alpha_k \mathbf{A} \mathbf{p}_k$;
 4. Set $\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$;
 5. Set $\mathbf{p}_{k+1} = -\mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$;
 6. Return to step 1 and repeat until $\mathbf{r}_{k+1}$ is sufficiently small.
-

Now that we have more economical representations of α_k and β_k , we can write the CG algorithm in full as Algorithm 1.

We have stated that it will take at most n iterations for CG to reach the solution, $\mathbf{x}_*$. While that is true, it often reaches an acceptable solution much more quickly. For instance, if $\mathbf{A}$ has r distinct eigenvectors, the CG algorithm will obtain the solution in at most r iterations. Additionally, there are two error bounds that can be useful in assessing convergence. If we denote the eigenvalues of $\mathbf{A}$ as $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$, then the following two bounds hold:

$$(1) \quad (\mathbf{x}_{k+1} - \mathbf{x}_*)^T \mathbf{A} (\mathbf{x}_{k+1} - \mathbf{x}_*) \leq \left(\frac{\lambda_{n-k} - \lambda_1}{\lambda_{n-k} + \lambda_1} \right)^2 (\mathbf{x}_0 - \mathbf{x}_*)^T \mathbf{A} (\mathbf{x}_0 - \mathbf{x}_*)$$

$$(2) \quad (\mathbf{x}_{k+1} - \mathbf{x}_*)^T \mathbf{A} (\mathbf{x}_{k+1} - \mathbf{x}_*) \leq \left(\frac{\sqrt{\lambda_n/\lambda_1} - 1}{\sqrt{\lambda_n/\lambda_1} + 1} \right)^k (\mathbf{x}_0 - \mathbf{x}_*)^T \mathbf{A} (\mathbf{x}_0 - \mathbf{x}_*).$$

Therefore, it is clear that the speed of the CG convergence is dependent on the eigenvalues of $\mathbf{A}$.

Preconditioning

Since the eigenvalues of $\mathbf{A}$ are important in the convergence speed of the CG algorithm, we can increase the convergence rate by using a preconditioner matrix, $\mathbf{G} = \mathbf{E}\mathbf{E}^T$, so that the CG depends on the eigenvalues of $\mathbf{E}^{-T}\mathbf{A}\mathbf{E}^{-1}$ instead of $\mathbf{A}$. Using a preconditioner will yield the preconditioned conjugate gradient (PCG) method. The best preconditioner is one in which $\mathbf{E}^{-T}\mathbf{A}\mathbf{E}^{-1} \approx \mathbf{I}$, the identity matrix, and $\mathbf{G}$ is efficient to invert. Using a preconditioner is equivalent to making the change of variables $\mathbf{y} = \mathbf{E}\mathbf{x}$ and then solving the linear system

$$(\mathbf{E}^{-T}\mathbf{A}\mathbf{E}^{-1})\mathbf{y} = \mathbf{E}^{-T}\mathbf{b},$$

which is equivalent to (1). The resulting PCG algorithm is given as Algorithm 2. Note that $\mathbf{E}$ is not used in the implementation of PCG; $\mathbf{G}$ is the only new matrix needed. Although each iteration of PCG is more costly than each iteration of CG, the total number of iterations needed for PCG is significantly lower with a good choice of a preconditioner. PCG is the method we will use in this thesis to solve for our MAP estimator.

An illustration of the CG and PCG methods is presented in Figure 1. The left side shows the way $\mathbf{x}_k$ converges to the solution in a small-scale example when using CG. For this example,

$$\mathbf{A} = \begin{bmatrix} 3.5 & -1 \\ -1 & 1.5 \end{bmatrix} \text{ and } \mathbf{b} = \begin{bmatrix} 2.75 \\ 2.25 \end{bmatrix}, \text{ which means } \mathbf{x}_* = \begin{bmatrix} 1.5 \\ 2.5 \end{bmatrix}.$$

Algorithm 2 Preconditioned Conjugate Gradient Method

-
0. Set $k = 0$, $\mathbf{r}_0 = \mathbf{A}\mathbf{x}_0 - \mathbf{b}$, $\mathbf{z}_0 = \mathbf{G}^{-1}\mathbf{r}_0$, and $\mathbf{p}_0 = -\mathbf{z}_0$;
 1. Set $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}$;
 2. Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$;
 3. Set $\mathbf{r}_{k+1} = \mathbf{r}_k + \alpha_k \mathbf{A} \mathbf{p}_k$;
 4. Set $\mathbf{z}_{k+1} = \mathbf{G}^{-1} \mathbf{r}_{k+1}$;
 5. Set $\beta_k = \frac{\mathbf{z}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{z}_k^T \mathbf{r}_k}$;
 6. Set $\mathbf{p}_{k+1} = -\mathbf{z}_{k+1} + \beta_k \mathbf{p}_k$;
 7. Return to step 1 and repeat until $\mathbf{r}_{k+1}$ is sufficiently small.
-

Beginning with $\mathbf{x}_0 = (0, 0)^T$, the first iteration of CG yields $\mathbf{x}_1 = (1.60, 1.30)^T$ and the second iteration $\mathbf{x}_2 = \mathbf{x}_*$ arrives at the true solution, as it should since $n = 2$. The advantage PCG has over CG is illustrated on the right side of Figure 1. We now use the preconditioner

$$\mathbf{G} = \begin{bmatrix} 3.5 & 0 \\ 0 & 1.5 \end{bmatrix}$$

that has just the diagonal entries of $\mathbf{A}$ and is therefore easier to invert. This time, $\mathbf{x}_1 = (1.37, 2.61)^T$, which is substantially closer to $\mathbf{x}_*$ than $\mathbf{x}_1$ was in the CG case and $\mathbf{x}_2 = \mathbf{x}_* = (1.5, 2.5)^T$. For this example, both the CG and PCG converged to the true solution in two iterations, but it is clear that PCG has the potential to converge in fewer iterations for a problem with much larger n .

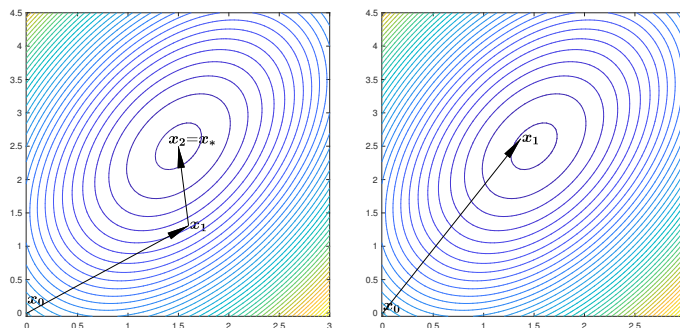


Figure 1: The image on the left tracks the convergence of the CG algorithm and the image on the right shows PCG. Both methods converged in two iterations.

Notes 7: Advanced Regression

Table of contents

Introduction	108
Quick Regression Overview/Review	108
Linear Regression Assumptions and Diagnostics	111
Regression Remedial Measures	113
Transform	113
Bootstrapping	113
Weighted Least Squares	116
Regularization – Ridge, LASSO, and Elastic Net	122
Bias-Variance Trade Off	122
Ridge Regression	123
LASSO Regression	124
LASSO vs Ridge Regression	124
Elastic Net Regularization	124
Regression Regularization in R: glmnet	125
Fitting Model with Selected λ	128
LASSO vs Stepwise Selection	128
Ridge Regression Example	129
glmnet in the caret package	132
Example: Logistic Regression Elastic Net	133
Poisson Regression	135
Poisson Link Function	136
Poisson Regression Assumptions	137
Finding Parameter Estimates	137
Fitting the Poisson Model in R	139
Illustrative Example - Lumber	139
Interpreting the Model Coefficients	141
Confidence Intervals for the Model Coefficients	142
Prediction Intervals	143
Poisson Residuals	144
Poisson Regression Diagnostics	144
Quasi-Poisson Regression	147
Quasi-Poisson Example - Quasi-Poisson Output	148

Introduction

In MATH 2140 or 3150 - Applied Statistics, we discussed many topics in regression. We'll discuss a few more advanced regression techniques:

- Weighted least squares
- Regularization
 - Ridge regression
 - LASSO
 - Elastic net
- Poisson regression
- Negative binomial regression

Quick Regression Overview/Review

A linear regression model is one in which we fit a line (in simple linear regression) or a hyperplane (in multiple linear regression) with two primary goals in mind:

- To get predictions of the response variable, Y , using the explanatory variable(s), X (sometimes $X_1, X_2, \dots, X_p$).
- To quantify the nature and the magnitude of the association between Y and the predictor(s). That is, understand what the slope values are telling us.

In general, the regression model can be written

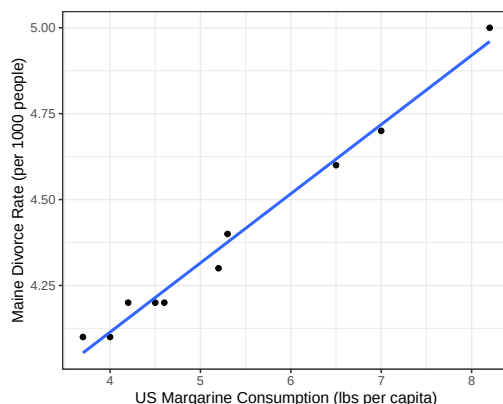
$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \varepsilon_i$$

We can have p predictor variables, each of which has its own slope parameter: the β_j in front of it. In addition, there is an intercept β_0 . So, all together, there are $p + 1$ parameters in this model. The ε is an error term. If we had data for the entire population, then we could compute these β parameters exactly, but we usually do not. That means we have to estimate them. The estimated parameters are $\hat{\beta}_j$, which are usually written as b_j . So, our predicted model becomes

$$\hat{y}_i = b_0 + b_1 x_{i1} + b_2 x_{i2} + \dots + b_p x_{ip}$$

The y has a hat on it to indicate this a predicted value of y rather than an actual y value. We compute these estimated parameters by finding the values that minimize the sum of the squared residuals. A residual is the difference between the actual y value and the predicted one. The residuals are written $e_i = y_i - \hat{y}_i$.

Consider an example of attempting to predict the divorce rate in Maine (in divorces per 1000 Mainers) using the margarine consumption of the United States (in pounds per person per year). This data set is called `divorce_margarine` and can be found in the `dslabs` R library.



This plot shows (perhaps unexpectedly!) a strong, positive, linear association between the two variables. So these variables would be good candidates for a regression model.

We can fit a regression model in **R** using the `lm()` function. Then we can display lots of useful output using the `summary()` function. Like so:

```
library(dslabs)
maine_mod <- lm(divorce_rate_maine ~ margarine_consumption_per_capita,
                data = divorce_margarine)
summary(maine_mod)
```

Call:

```
lm(formula = divorce_rate_maine ~ margarine_consumption_per_capita,
    data = divorce_margarine)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.05583	-0.01816	-0.01452	0.03601	0.04625

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	3.308626	0.048032	68.88	2.20e-12
margarine_consumption_per_capita	0.201386	0.008735	23.05	1.33e-08

(Intercept) ***

margarine_consumption_per_capita ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.03841 on 8 degrees of freedom

Multiple R-squared: 0.9852, Adjusted R-squared: 0.9833

F-statistic: 531.5 on 1 and 8 DF, p-value: 1.33e-08

This output tells us that the intercept of the model is 3.308626 and the slope is 0.201386. So, we can write the regression model

$$\hat{y} = 0.2014x + 3.309 \quad \text{or} \quad \hat{y} = 3.309 + 0.2014x$$

or, in the context of the problem:

$$\widehat{\text{divorce rate}} = 0.2014(\text{margarine}) + 3.309$$

We can then get a prediction for the divorce rate by plugging in a value for the margarine variable. For example, the predicted divorce rate in Maine when the margarine consumption in the US is 5 lbs per capita is $\text{divorce rate} = 0.2014(5) + 3.309 = 4.316$ per 1000 people. In **R**, we can do this using the `predict()` function.

```
predict(maine_mod, data.frame(margarine_consumption_per_capita = 5))
1
4.315556
```

We can even make a **confidence interval** for the average Y value using the `interval = "confidence"` argument. Or we can make a **prediction interval** for any Y value using the `interval = "prediction"` argument:

```
predict(maine_mod, data.frame(margarine_consumption_per_capita = 5),
        interval = "confidence")
      fit      lwr      upr
1 4.315556 4.286815 4.344298
```

```
predict(maine_mod, data.frame(margarine_consumption_per_capita = 5),
        interval = "prediction")
      fit      lwr      upr
1 4.315556 4.222435 4.408678
```

These intervals tell us each of the following:

- We are 95% confident that the average divorce rate in Maine for all years when the margarine consumption in the US is 5 lbs is between 4.29 and 4.34 per 1000 people.
- We are 95% confident that the divorce rate in Maine for any year when the margarine consumption in the US is 5 lbs is between 4.22 and 4.41 per 1000 people.

Intercept interpretation: The intercept tells us the predicted value of Y when X is equal to zero. In this case, we predict that the divorce rate in Maine will be 0.2014 per 1000 people when the per capita consumption of margarine in the US is zero lbs.

Slope interpretation: The slope is the predicted change in the Y variable when the X variable increases by 1 unit. In this case, as the per capita consumption of margarine in the US increases by 1 lb, the divorce rate in Maine is predicted to increase by 0.2014 per 1000 people. In multiple linear regression, there are more variables in the model, so we need to add the phrase “holding all other variables constant” at the end.

Some other important notes: This output is showing the results of two hypothesis tests, each with their own p-value. The p-values are shown in the `Pr(>|t|)` column. The one in the row for the intercept is testing the hypotheses

$$H_0 : \beta_0 = 0$$

$$H_A : \beta_0 \neq 0$$

In other words, it is testing if the true intercept is different than zero. The one in the row for the variable, `margarine_consumption_per_capita` in this case, is testing the hypotheses

$$H_0 : \beta_1 = 0$$

$$H_A : \beta_1 \neq 0$$

In other words, it is testing if the true slope is different than zero. Since the p-values are very small for both (notice the scientific notation: $2.20\text{e-}12 = 2.20 \times 10^{-12} = 0.0000000000022$), we have extremely strong evidence each parameter is not equal to zero.

Another statistic to pay attention to is the **Multiple R-squared**. This is the **coefficient of determination** and it tells us the proportion of the variation in the Y variable that is being explained by this model. Here, we can say that 98.52% of the variation in the divorce rate in Maine is explained by the per capita consumption of margarine in the US. This is a very high R^2 value and indicates this model fits the data extremely well.

Linear Regression Assumptions and Diagnostics

There are some assumptions that must be satisfied for the model to be fit and then for inference to be performed:

1. The means of the errors at a given value of X must be 0: $E(\varepsilon|x_i) = 0$ for all i . This indicates that the variables have a linear association. This assumption is the only one that is needed for prediction.
2. The variances of the errors at a given value of X must be equal: $\text{Var}(\varepsilon|x_i) = \sigma_\varepsilon^2$ for all i . This assumption is needed for inference.
3. The errors must follow a normal distribution: $\varepsilon \sim N$. This assumption is needed for inference.
4. The errors must be independent of each other. Having a random sample from the population ensures this. This assumption is needed for inference.

These assumptions are all about the true, population errors, which are unknown. So we never know for sure if these are met, but the reality is that these regression models work well if these are *reasonably* met. The way we assess them is by looking at some **diagnostic plots** like a **residual plot** and a **QQ plot**.

Residual Plots

Residual plots plot the residuals against the fitted values (the $\hat{y}$ values). These can be used to check the first and second assumptions listed above: the linearity and equal variance assumptions. We will want to plot the *externally studentized*, also known as the **jackknife**, residuals in this class since those are residuals that have been altered to be independent of each other approximately follow a t distribution. The jackknife residuals can be found in **R** using the `rstudent()` function. Ideally, the residual plots would show random scatter without any sort of discernible pattern and no fanning. It can be helpful to use `geom_smooth()` to plot a smoothed curve to the plot. This curve should be (mostly) horizontal at $y = 0$. A residual plot code template might look like this:

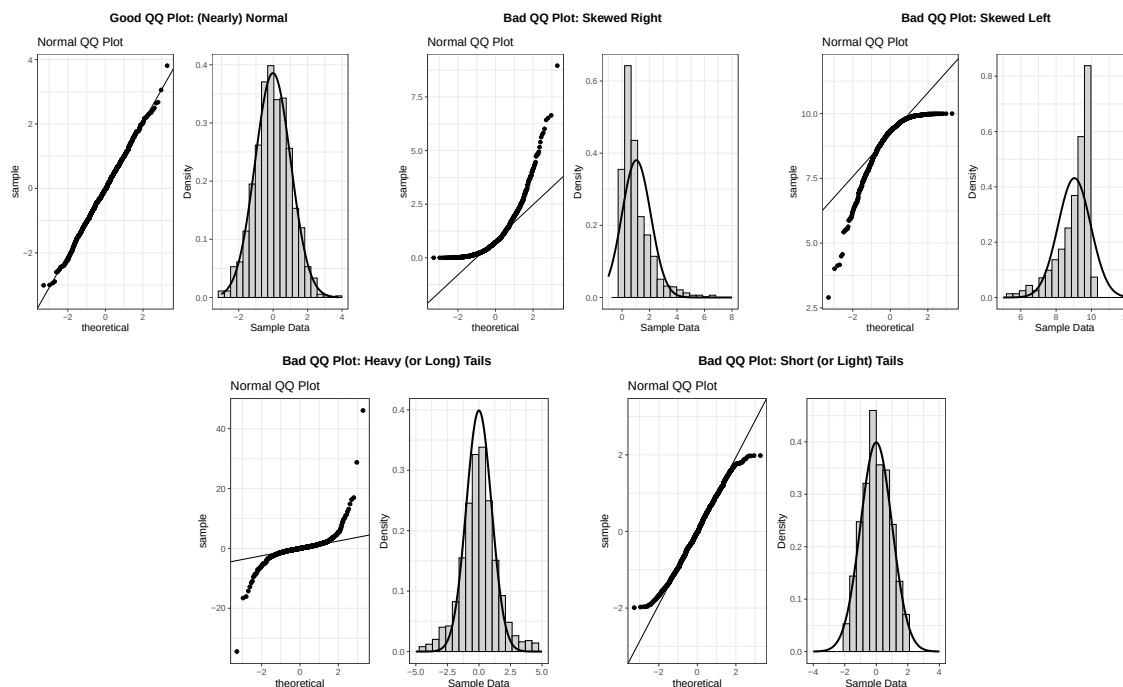
```
data.frame(resids = rstudent(regression_model),
           fitted = fitted.values(regression_model)) |>
  ggplot(aes(x = fitted, y = resids)) +
  geom_point() +
  geom_smooth(method = "loess", formula = "y ~ x", se = F) +
  labs(x = "Fitted Values", y = "Jackknife Residuals")
```

Ideally, the residual plots would show random scatter without any sort of discernible pattern and no fanning.



QQ Plots

QQ plots plot the sample quantiles against the theoretical quantiles. This can be used to check the third assumption above: the normality assumption. If the distribution of the sample (the residuals in our case) are approximately normal, then the points should fall along a straight line. Otherwise, they will deviate from the line. Regression inference is fairly robust against non-normality, so as long as the points are reasonably close to the line, then we will consider this assumption reasonably met.



We can make a QQ plot in **R** using the following code as a template:

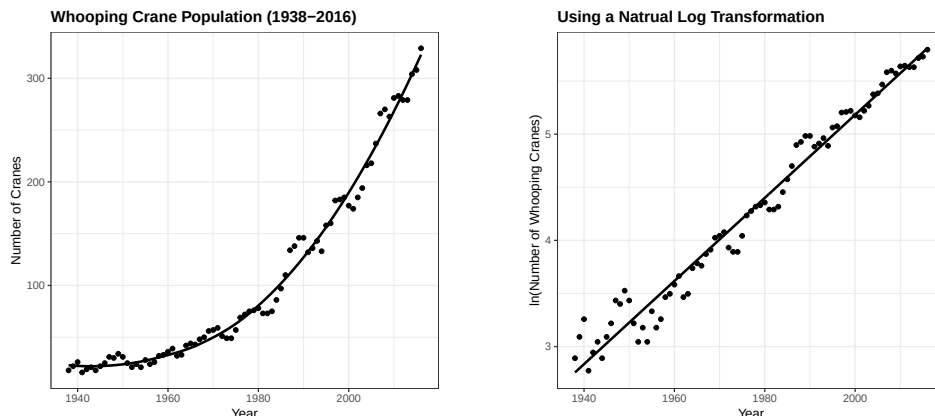
```
data.frame(resids = rstudent(regression_model)) |> # Studentized residuals
  ggplot(aes(sample = resids)) +
  geom_qq_line() +
  geom_qq() +
  labs(title = "Normal QQ Plot")
```

What do we do if one or more of the assumptions does not seem to be reasonably satisfied? In that case, it is time for some **remedial measures**.

Regression Remedial Measures

Transform

Once thing we can do to potentially get the assumptions to be satisfied is to transform our data.



This is a very common technique, but is discussed at length in MATH 3150 and will not be a focus for us here.

Bootstrapping

We mentioned bootstrapping earlier in this class and we can rely on bootstrapping to obtain estimates of the standard errors when some of the assumptions are not reasonably met.

We will discuss two types of bootstrapping for regression:

1. Bootstrapping the residuals.
 - Used when the x values are fixed, the model provides an adequate fit, and the errors have constant variance.
2. Bootstrapping the cases.
 - Less sensitive to violations of any of the regression assumptions but problems can arise when there are outliers in the model.

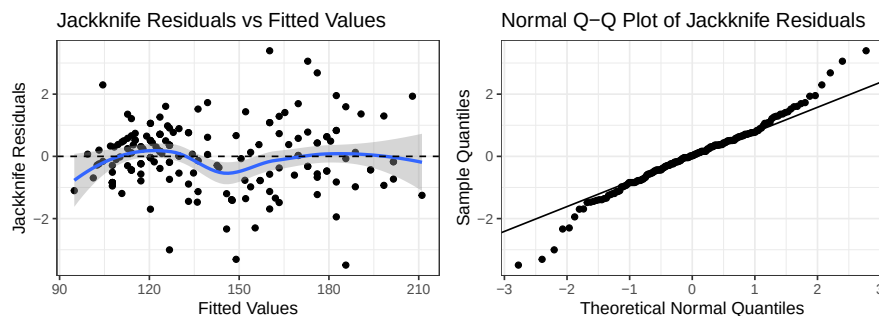
Bootstrapping the residuals

If we are reasonably assured that the model provides an adequate fit and the errors have constant variance, we can bootstrap the residuals to obtain standard errors for the regression intercept and slope(s). The process is as follows:

1. Fit a model to the data to obtain $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$, $i = 1, \dots, n$.
2. Obtain the residuals for the model: $\hat{\epsilon}_i = y_i - \hat{y}_i$, $i = 1, \dots, n$.
3. Randomly sample the set of residuals with replacement to obtain $\hat{\epsilon}_i^*$, $i = 1, \dots, n$.
4. Create a set of pseudo-responses: $y_i^* = \hat{y}_i + \hat{\epsilon}_i^*$.
5. Fit a model to the pseudo-responses to obtain $\hat{y}_i^* = \hat{\beta}_0^* + \hat{\beta}_1^* x_{i1} + \cdots + \hat{\beta}_p^* x_{ip}$, $i = 1, \dots, n$.
6. Repeat steps 3-5 B times to obtain a distribution for each $\hat{\beta}_j^*$, $j = 0, \dots, p$.

Example:

Consider a data set where we are attempting to predict the ideal weights using the actual weights for males and females. Below are some plots for this model.



```
set.seed(2026)
wtmod <- lm(ideal ~ sex + actual, data = idealwt)
```

Call:

```
lm(formula = ideal ~ sex + actual, data = idealwt)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-30.6464	-5.0523	0.1796	4.7332	29.8019

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.64973	4.14729	9.078	<2e-16 ***
sexMale	23.93657	1.94410	12.312	<2e-16 ***
actual	0.63621	0.03059	20.797	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.131 on 179 degrees of freedom

Multiple R-squared: 0.9069, Adjusted R-squared: 0.9058

F-statistic: 871.3 on 2 and 179 DF, p-value: < 2.2e-16

```
confint(wtmod)
```

	2.5 %	97.5 %
(Intercept)	29.4658574	45.8336081
sexMale	20.1002659	27.7728749
actual	0.5758395	0.6965713

```
bootresids <- function(resids, i) {
  lm(I(wtmod$fitted + resids[i]) ~ sex + actual, data = idealwt)$coef
}
```

```
(boot_out1 <- bootstrapping(residuals(wtmod), bootresids, 5000))
```

A tibble: 3 x 5

term	sample_average	original_value	bias	std.error
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	37.7	37.6	0.0823	4.18
2 sexMale	24.0	23.9	0.0243	1.94
3 actual	0.636	0.636	-0.000593	0.0307

boot_ci(boot_out1)

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat1
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  29.4  45.8
2 basic           0.95  29.4  45.7
3 percent         0.95  29.6  45.9
4 bca             0.95  29.5  45.8

```

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat2
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  20.1  27.7
2 basic           0.95  20.0  27.7
3 percent         0.95  20.2  27.9
4 bca             0.95  20.2  27.9

```

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat3
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  0.577  0.697
2 basic           0.95  0.576  0.697
3 percent         0.95  0.575  0.696
4 bca             0.95  0.575  0.696

```

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

Bootstrapping the cases

If we are unsure that the model assumptions hold, we can bootstrap the cases to obtain standard errors for the regression intercept and slope(s). The process is as follows:

1. Sample n rows from the data frame with replacement.
2. Fit a model to the data using the sampled rows of the data frame to obtain $\hat{y}_i^* = \hat{\beta}_0^* + \hat{\beta}_1^* x_{i1} + \cdots + \hat{\beta}_p^* x_{ip}$, $i = 1, \dots, n$.
3. Repeat steps 1-2 B times to obtain a distribution for each $\hat{\beta}_j^*$, $j = 0, \dots, p$.

Example:

```

bootcases <- function(resids, i) {
  lm(ideal ~ sex + actual, data = idealwt[i,])$coef
}
boot_out2 <- bootstrapping(residuals(wtmod), bootcases, 5000)
boot_ci(boot_out2)

```

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat1
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  28.3  46.8
2 basic           0.95  27.9  46.6
3 percent         0.95  28.7  47.4
4 bca             0.95  28.7  47.5

```

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat2
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  19.2  28.6
2 basic           0.95  18.8  28.3
3 percent         0.95  19.5  29.0
4 bca             0.95  19.8  29.4

```

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat3
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  0.564  0.710
2 basic           0.95  0.564  0.712
3 percent         0.95  0.560  0.708
4 bca             0.95  0.557  0.707

```

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

We can plot these bootstrap distributions using `plot(boot_out2)`, but that is omitted here.

Weighted Least Squares

An alternative to bootstrapping the cases when we have reason to believe the variances of the errors are not equal is to use **weighted least squares** to fit our regression model.

Matrix Formulation

We are used to regression models being written

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$$

We can write this in matrix-vector form. If we define

$$\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n]_{n \times 1}^T, \quad \boldsymbol{\beta} = [\beta_0, \beta_1, \dots, \beta_p]_{(p+1) \times 1}^T, \quad \text{and}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}_{n \times (p+1)}$$

then that equation for the i th row above can be written

$$\hat{\mathbf{y}}_i = \mathbf{X}_i \boldsymbol{\beta}$$

and then all n equations for the y values can be written

$$\hat{\mathbf{y}} = \mathbf{X} \boldsymbol{\beta}$$

In ordinary least squares (OLS) regression, we determine the estimates for the β values by computing

$$\hat{\boldsymbol{\beta}} = \mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Then we can compute standard errors for these estimates as long as the **linear regression assumptions** hold:

1. The residuals have zero mean (the relationship is linear).
2. The residuals have the same variance.
3. The residuals are normally distributed.
4. The residuals are independent.

If the homogeneity of variance assumption is violated, then the standard errors of the estimated β_j values are usually incorrect and that leads to confidence and prediction intervals being incorrect.

Standard Errors

We can obtain the standard errors of the parameter estimates by computing the variance-covariance matrix in the following manner:

$$\sigma_b^2 = \begin{bmatrix} \text{Var}(b_0) & \text{Cov}(b_0, b_1) & \dots & \text{Cov}(b_0, b_p) \\ \text{Cov}(b_1, b_0) & \text{Var}(b_1) & \dots & \text{Cov}(b_1, b_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(b_p, b_0) & \text{Cov}(b_p, b_1) & \dots & \text{Var}(b_p) \end{bmatrix}_{(p+1) \times (p+1)} = \sigma_\epsilon^2 (\mathbf{X}^T \mathbf{X})^{-1}$$

This means that the standard errors can be obtained by taking the square root of the diagonals of this matrix.

Consider this example of attempting to predict blood pressure using age.

```
library(math3190package)
bp <- blood_pressure |>
  rename(bp = diastolic_blood_pressure)
bp_mod <- lm(bp ~ age, data = bp)
X <- model.matrix(bp_mod)
var_matrix <- summary(bp_mod)$sigma^2 * solve(t(X) %*% X)
std_errors <- var_matrix |> diag() |> sqrt()
std_errors
(Intercept)      age
  3.99367376  0.09695116

# Compare to summary output
summary(bp_mod)
```

Call:

```
lm(formula = bp ~ age, data = bp)
```

Residuals:

Min	1Q	Median	3Q	Max
-16.4786	-5.7877	-0.0784	5.6117	19.7813

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	56.15693	3.99367	14.061	< 2e-16 ***
age	0.58003	0.09695	5.983	2.05e-07 ***

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

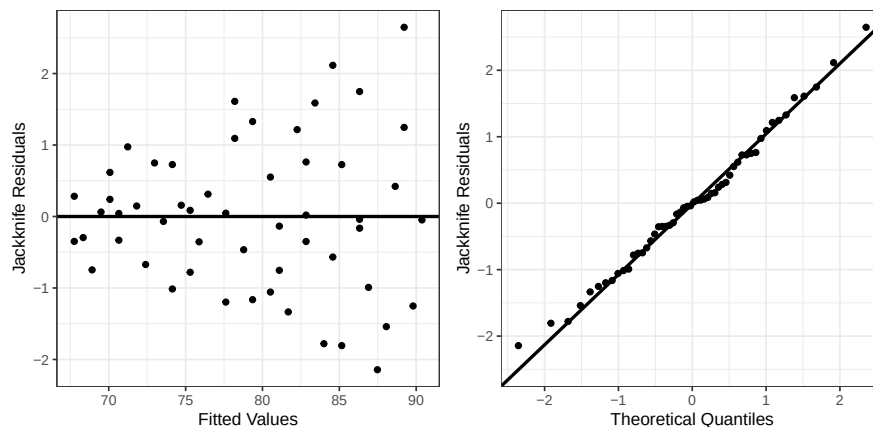
Residual standard error: 8.146 on 52 degrees of freedom

Multiple R-squared: 0.4077, Adjusted R-squared: 0.3963

F-statistic: 35.79 on 1 and 52 DF, p-value: 2.05e-07

Original Diagnostic Plots

Below is what the diagnostic plots looks like for this example:



In order to adjust so the estimates and standard errors are not as effected by the points that

have large residuals, we can **weight** each observation. We will define a vector of weights as $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ and a matrix of the weights as

$$\mathbf{W} = \begin{bmatrix} w_1 & 0 & \dots & 0 \\ 0 & w_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & w_n \end{bmatrix}$$

We can then obtain the weighted estimates of the coefficients, called $\mathbf{b}_w$, by computing

$$\mathbf{b}_w = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y}$$

with variance-covariance matrix

$$\sigma_b^2 = \sigma_\epsilon^2 \cdot (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1}.$$

Selecting the Weights

In ordinary least squares (OLS), we assume $\text{Var}(\epsilon_i) = \sigma_\epsilon^2$ for all i . In WLS, we instead allow for each error to have its own variance. So, we have $\text{Var}(\epsilon_i) = \sigma_i^2 = \sigma_\epsilon^2 / w_i$. Our best guess for σ_ϵ^2 is the mean square error (MSE). That is what we typically denote as s_ϵ^2 .

Now we need to determine the best values for the weights. First, we must obtain an estimate for the variance function for the residuals. This will depend on the residual plot(s), but the most common situation for using weighted least squares is when the residuals plotted against the fitted values has a fan shape. In this scenario, we can estimate the weights by:

1. Fit an ordinary least squares model (and verify that a residual plot against the fitted values has a fan shape).
2. Fit a new linear model by regressing the absolute value of the residuals on the fitted values. That is, use $|\hat{\epsilon}|$ as y and $\hat{y}$ as x .
3. Let the weights be the reciprocals of the squares of the fitted values of that second model.

Assessing the Adequacy of the Weights

Each of the following should be true when using weighted least squares:

1. The estimates of the β values using WLS should be very close to the estimates of the β values using OLS.
2. The weighted mean square error (MSE_w) should be close to 1.
3. The residual plot of the jackknife residuals should have constant variance.

Also note the following things about WLS:

1. R^2 has no real meaning in weighted least squares and should not be considered. We should use the R^2 from the OLS model.
2. Externally studentized (jackknife) residuals take into account the weights, so they should be used in residual plots for model diagnostics.
3. For QQ plots, use the unstudentized residuals. Either the raw or standardized are fine.

```
bp_mod <- lm(bp ~ age, data = bp)
e_mod <- lm(abs(bp_mod$residuals) ~ bp_mod$fitted.values)
```

```
w <- 1/e_mod$fitted.values^2
bp_mod_w <- lm(bp ~ age, data = bp, weights = w)
summary(bp_mod_w)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	55.56577	2.52092	22.042	< 2e-16 ***
age	0.59634	0.07924	7.526	7.19e-10 ***

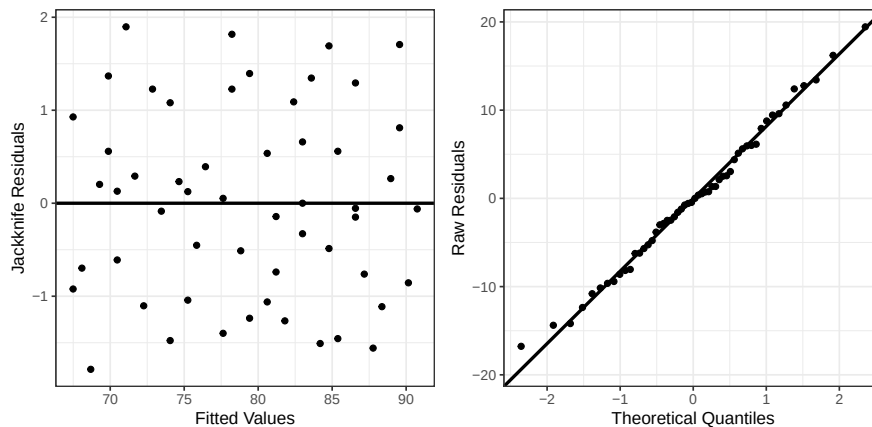
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.213 on 52 degrees of freedom

Multiple R-squared: 0.5214, Adjusted R-squared: 0.5122

F-statistic: 56.64 on 1 and 52 DF, p-value: 7.187e-10

Weighted Diagnostic Plots



Confidence Intervals

We can easily obtain confidence intervals for the β 's by using the `confint()` function. The variance-covariance matrix is now $\sigma_\epsilon^2(\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}$.

To obtain confidence intervals for predictions, that is, a confidence interval for $E(y|\mathbf{x}^*)$, we first consider what the CI looks like in the OLS case. For obtaining the $(1 - \alpha) \cdot 100\%$ confidence interval for $E(y|\mathbf{x}^*)$ at a specific row vector $\mathbf{x}^* = [1 \ x_1^* \ \dots \ x_n^*]$ is given by

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_\epsilon \sqrt{\mathbf{x}^*(\mathbf{X}^T\mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

For weighted least squares, we just need to insert the weight matrix:

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_w \sqrt{\mathbf{x}^*(\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

where s_w is the square root of the weighted MSE.

An example for obtaining the bounds for blood pressure at age 50:

```

W <- diag(w)
n <- nrow(bp)
alpha <- 0.05
x_star <- matrix(c(1, 50), nrow = 1)
yhat <- predict(bp_mod, data.frame(age = 50))
yhat_w <- predict(bp_mod_w, data.frame(age = 50))
se <- summary(bp_mod)$sigma
se_w <- summary(bp_mod_w)$sigma
# OLS Bounds
yhat - qt(1-alpha/2, n-2) * se * sqrt(x_star %*% solve(t(X) %*% X) %*% t(x_star))
      [,1]
[1,] 82.14817
yhat + qt(1-alpha/2, n-2) * se * sqrt(x_star %*% solve(t(X) %*% X) %*% t(x_star))
      [,1]
[1,] 88.16877
# Verify using the predict function
predict(bp_mod, data.frame(age = 50), interval = "confidence")
      fit      lwr      upr
1 85.15847 82.14817 88.16877
# WLS Bounds - Confidence Interval
yhat_w - qt(1 - alpha/2, n-2) * se_w * sqrt(
  x_star %*% solve(t(X) %*% W %*% X) %*% t(x_star)
)
      [,1]
[1,] 81.84106
yhat_w + qt(1 - alpha/2, n-2) * se_w * sqrt(
  x_star %*% solve(t(X) %*% W %*% X) %*% t(x_star)
)
      [,1]
[1,] 88.92464
# Verify using the predict function
predict(bp_mod_w, data.frame(age = 50), interval = "confidence")
      fit      lwr      upr
1 85.38285 81.84106 88.92464

```

The prediction interval for OLS is computed with

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_{\epsilon} \sqrt{1 + \mathbf{x}^*(\mathbf{X}^T \mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

For weighted least squares, we just need to change that 1 to $1/w^*$ over the weight for that particular $\mathbf{x}^*$:

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_w \sqrt{\frac{1}{w^*} + \mathbf{x}^*(\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

where w^* is the weight for the value we are predicting.

```
# Prediction interval OLS
```

```
lwr_bound <- yhat - qt(1 - alpha/2, n-2) * se * sqrt(
  1 + x_star %>% solve(t(X) %>% X) %>% t(x_star)
)
upr_bound <- yhat + qt(1 - alpha/2, n-2) * se * sqrt(
  1 + x_star %>% solve(t(X) %>% X) %>% t(x_star)
)
c(lwr_bound, upr_bound)
[1] 68.53795 101.77899
```

```
predict(bp_mod, data.frame(age = 50), interval = "prediction")
```

```
      fit      lwr      upr
1 85.15847 68.53795 101.779
```

```
# Prediction interval WLS
```

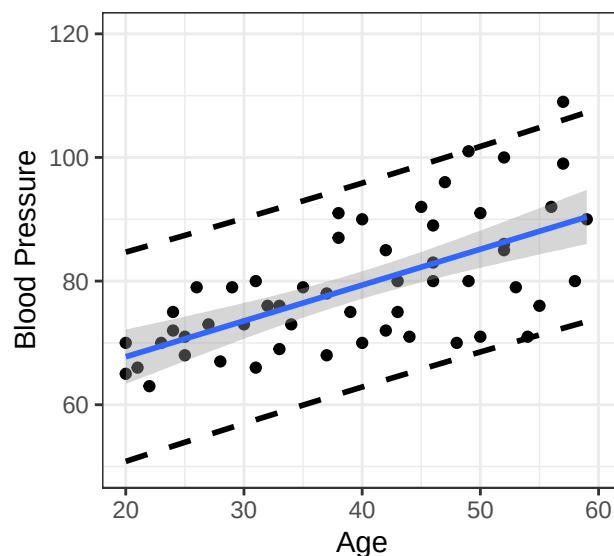
```
# Get new weight for x = 50:
```

```
new_fit <- predict(bp_mod, data.frame(age = 50))
new_weight <- 1/predict(e_mod, data.frame(fitted = new_fit))^2
# Construct Intervals
lwr_bound <- yhat_w - qt(1-alpha/2, n-2) * se_w * sqrt(
  1/new_weight + x_star %>% solve(t(X) %>% W %>% X) %>% t(x_star)
)
upr_bound <- yhat_w + qt(1-alpha/2, n-2) * se_w * sqrt(
  1/new_weight + x_star %>% solve(t(X) %>% W %>% X) %>% t(x_star)
)
c(lwr_bound, upr_bound)
[1] 64.72992 106.03578
```

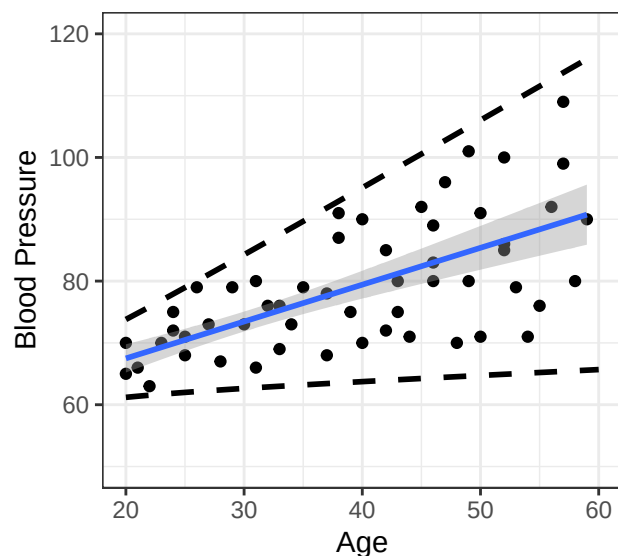
```
predict(bp_mod_w, data.frame(age=50), interval="prediction", weight = new_weight)
```

```
      fit      lwr      upr
1 85.38285 64.72992 106.0358
```

OLS Interval Bands



WLS Interval Bands

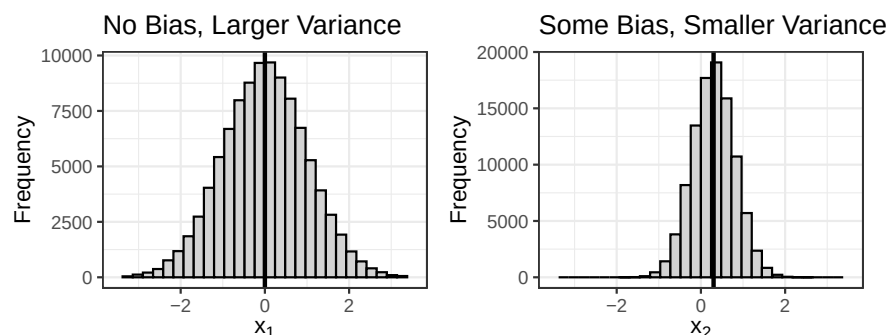


Regularization – Ridge, LASSO, and Elastic Net

Bias-Variance Trade Off

In statistical estimation, there is often a trade off between bias and variance. Many statistical methods, like least squares regression, guarantee that the estimate is **unbiased**, which means that the sampling distribution of the statistic is centered at the parameter it is estimating.

However, these unbiased estimators will often have more uncertainty, a higher variance, than some biased estimators. When this is the case, the biased estimator that is more precise may be preferred.



In statistics and machine learning, the **bias–variance tradeoff** is the property of a model that the variance of the parameter estimated across samples can be reduced by increasing the bias in the estimated parameters.

The **bias–variance dilemma** or **bias–variance problem** is the conflict in trying to simultaneously minimize these two sources of error that prevent supervised learning algorithms from generalizing beyond their training set. (Source: Wikipedia)

The **bias** is an error from faulty assumptions or misspecification of the learning algorithm. High bias can cause an algorithm to miss the relevant relations between features and target outputs (underfitting).

The **variance** is an error from sensitivity to small fluctuations in the training set. High variance may result from an algorithm modeling the random noise in the training data (overfitting).



The bias–variance trade off is a central problem in supervised learning (where the y values are known). Ideally, one wants to choose a model that both accurately captures the regularities in its

training data, but also generalizes well to unseen data.

Unfortunately, it is typically impossible to do both simultaneously. High-variance learning methods may be able to represent their training set well but are at risk of overfitting to noisy or unrepresentative training data. In contrast, algorithms with high bias typically produce simpler models that may fail to capture important regularities (i.e. underfit) in the data.

Regularization in Machine Learning

In regression analysis, the features are estimated using coefficients while modeling. In small sample sizes or noisy data coefficient estimates could be anecdotally incorrect (e.g., overfitting) or inaccurate.

If the estimates can be restricted, penalized, or shrunk towards zero, then the impact of insignificant features might be reduced and would prevent models from high variance with a stable fit.¹

Regularization is the most used technique to penalize complex models in machine learning, it is deployed for reducing overfitting (or, contracting generalization errors) by putting small network weights into the model (adding a small amount of bias). Also, it enhances the performance of models for new inputs.

Examples of regularization in machine learning, include:

- K-means: Restricting the segments for avoiding redundant groups.
- Neural networks: Confining the complexity (weights) of a model.
- Random forests: Reducing the depths of tree and branches (new features).

Ridge Regression

Ridge regression **regularizes** (shrinks) coefficients by imposing a penalty on their size. The ridge coefficients minimize a penalized sum of squared error:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

where $\lambda \geq 0$ is a parameter that controls the shrinkage. The larger the value of λ the more shrinkage (towards 0). Or in matrix form, ridge regression minimizes:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) + \lambda \beta^T \beta \right\}.$$

With a little work, the ridge regression solution can be shown to be:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_N)^{-1} \mathbf{X}^T \mathbf{y}$$

Penalty Terms

The regularization for ridge regression, $\lambda \beta^T \beta$, is usually denoted as an **L2 regularization** or **L2 penalty**, as it adds a penalty which is equal to the square of the magnitude of coefficients. Both Ridge regression and Support Vector Machines (SVMs) implement this method. L2 regularization can deal with multicollinearity problems (independent variables are highly correlated) through constricting the coefficient while keeping all the variables in a model. However, L2 regularization is not an effective method for selecting relevant predictors (or removing redundant parameters). We will use a **L1 regularization** for this purpose.

¹Adapted from: <https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>

LASSO Regression

LASSO (Least Absolute Shrinkage and Selection Operator) regression also **regularizes** coefficients by imposing a penalty on their size, but it uses an **L1** penalty. The LASSO coefficients minimize the following cost function:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \alpha \sum_{j=1}^p |\beta_j| \right\},$$

where $\alpha \geq 0$ is a parameter that controls the shrinkage. The larger the value of α the more shrinkage (towards 0).

Notice the similarity to the ridge regression problem: the L2 ridge penalty $\sum_{j=1}^p \beta_j^2$ is replaced by the L1 lasso penalty $\sum_{j=1}^p |\beta_j|$.

This latter constraint makes the solutions nonlinear in the y_i , and there is no closed form expression for the LASSO as was the case in ridge regression. Because of the nature of the constraint, making α sufficiently small will cause some of the coefficients to be exactly zero. Thus the LASSO does a kind of continuous subset selection, or conducts a **variable selection**.

LASSO vs Ridge Regression

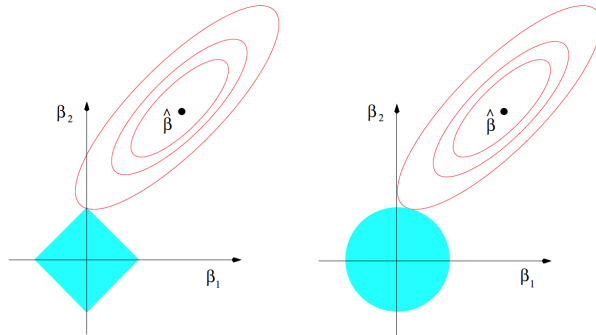


FIGURE 3.11. Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions $|\beta_1| + |\beta_2| \leq t$ and $\beta_1^2 + \beta_2^2 \leq t^2$, respectively, while the red ellipses are the contours of the least squares error function.

S.No	L1 Regularization	L2 Regularization
1	Panelizes the sum of absolute value of weights.	penalizes the sum of square weights.
2	It has a sparse solution.	It has a non-sparse solution.
3	It gives multiple solutions.	It has only one solution.
4	Constructed in feature selection.	No feature selection.
5	Robust to outliers.	Not robust to outliers.
6	It generates simple and interpretable models.	It gives more accurate predictions when the output variable is the function of whole input variables.
7	Unable to learn complex data patterns.	Able to learn complex data patterns.
8	Computationally inefficient over non-sparse conditions.	Computationally efficient because of having analytical solutions.

Left image is from [Elements of Statistical Learning](#), Hastie, Tibshirani, Friedman, by Springer.

Right image is from <https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>.

Elastic Net Regularization

Which should we choose? Ridge or LASSO? Well, why do we have to choose?!

Instead we could use the **Elastic Net** that minimizes:

$$\hat{\beta}_{\text{elastic net}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \alpha_1 \sum_{j=1}^p |\beta_j| + \alpha_2 \sum_{j=1}^p \beta_j^2 \right\},$$

for some $\alpha_1 \geq 0$ and $\alpha_2 \geq 0$.

The quadratic penalty term makes the loss function strongly convex, and it therefore has a unique minimum. The elastic net method includes Ridge regression, LASSO, and OLS by setting either $\alpha_1 = 0$, $\alpha_2 = 0$, or both to 0.

Regression Regularization in R: glmnet

We can use the `glmnet` and `glmnetUtils` package to apply regularization in R:

```
install.packages("glmnet")
install.packages("glmnetUtils")
```

The default model used in the package is the “least squares” regression model and `glmnet` actually optimizes:

$$\hat{\beta}_{\text{elastic net}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + \frac{(1-\alpha)}{2} \sum_{j=1}^p \beta_j^2 \right) \right\},$$

with $\alpha = 1$ as a default (so LASSO!).

As an example, let’s look at a dataset for predicting body fat of a person using a variety of variables. This dataset can be found in the `faraway` package.

```
library(glmnet) |> suppressPackageStartupMessages()
library(glmnetUtils) |> suppressPackageStartupMessages()
library(faraway) |> suppressPackageStartupMessages()
fat_mod <- lm(
  brozek ~ age + weight + height + neck + chest +
    abdom + hip + thigh + knee + ankle + biceps +
    forearm + wrist, data = fat
) # Fits an OLS model
```

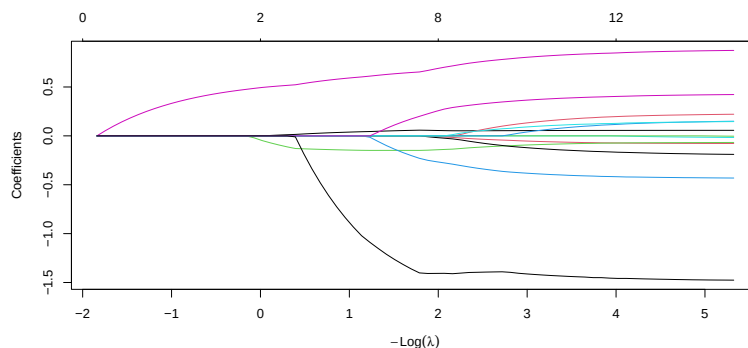
We fit the model using the most basic call to `glmnet`. $\alpha = 1$ by default, but let’s also specified it here.

```
fit_lasso <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
) # Fits a LASSO
```

`fit_lasso` is now an object of class `glmnet` that contains all the relevant information of the fitted model for further use. There are various functions that work well with this class ob object such as `plot`, `print`, `coef` and `predict`.

We can visualize the size of the coefficients for different λ values by using the `plot()` function:

```
plot(fit_lasso, "lambda")
```



We can obtain the model coefficients at one or more λ values within the range of the sequence:

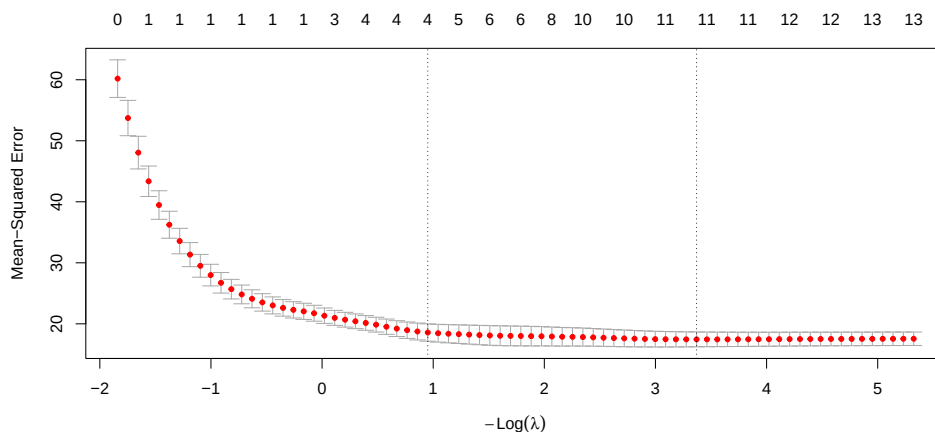
```
coef(fit_lasso, s = 0.1) # s is the value for a "specific" lambda
```

```
14 x 1 sparse Matrix of class "dgCMatrix"
```

```
      s=0.1
(Intercept) -7.53281619
age          0.05183206
weight      -0.02504372
height      -0.12561764
neck        -0.31157363
chest       .
abdom       0.73717170
hip         -0.04875740
thigh       0.02570643
knee        .
ankle       .
biceps      0.03422877
forearm     0.30749547
wrist       -1.40042018
```

The function `glmnet()` returns a sequence of models for the users to choose from. **Cross-validation** is perhaps the simplest and most widely used method to select a model. `cv.glmnet()` is the main function to do cross-validation here, along with various supporting methods such as plotting and prediction.

```
set.seed(2026)
cvfit_lasso <- cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
)
plot(cvfit_lasso)
```



Coefficients Using Minimum Lambda

We can get the value of λ_{min} and the model coefficients:

```
cvfit_lasso$lambda.min
```

```
[1] 0.03438172
```

```
coef(cvfit_lasso, s = "lambda.min")
```

```
14 x 1 sparse Matrix of class "dgCMatrix"
```

```
      lambda.min
(Intercept) -13.34535803
age          0.05478517
weight      -0.06316555
height      -0.08317641
neck        -0.39947252
chest        .
abdom        0.82585758
hip         -0.14325642
thigh        0.16479917
knee         .
ankle        0.07804134
biceps       0.10961077
forearm      0.38403630
wrist       -1.43316868
```

Coefficients Using Lambda within 1 SE of Min

The output of the `cv.glmnet()` function also gives us the largest λ that is within 1 SE of the λ that gave the smallest MSE. This λ_{1se} value penalizes the coefficients more and therefore chooses a simpler model.

```
cvfit_lasso$lambda.1se
```

```
[1] 0.3862181
```

```
coef(cvfit_lasso, s = "lambda.1se")
14 x 1 sparse Matrix of class "dgCMatrix"
      lambda.1se
(Intercept) -11.8103049
age          0.0371891
weight       .
height      -0.1445500
neck         .
chest        .
abdom        0.5880006
hip          .
thigh        .
knee         .
ankle        .
biceps       .
forearm      .
wrist        -0.8339864
```

Fitting Model with Selected λ

Then we can fit our model using the λ value that we choose.

```
fit_min <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1,
  lambda = cvfit_lasso$lambda.min
)
c(fit_min$lambda, fit_min$dev.ratio)
[1] 0.03438172 0.74755316
```

```
fit_1se <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1,
  lambda = cvfit_lasso$lambda.1se
)
c(fit_1se$lambda, fit_1se$dev.ratio)
[1] 0.3862181 0.7066715
```

LASSO vs Stepwise Selection

Since LASSO is primarily used for model selection, it makes sense to compare it to another common model selection tool: the step wise selection.

```
# step(fat_mod) # Using AIC
# step(fat_mod, k = log(nrow(fat))) # Using BIC
# coef(fit_min) # LASSO
# coef(fit_1se) # LASSO
```

Method	Model
Stepwise AIC	$-20.1 + 0.059 \cdot \text{age} - 0.084 \cdot \text{weight} - 0.432 \cdot \text{neck} + 0.877 \cdot \text{abdom}$ $-0.186 \cdot \text{hip} + 0.286 \cdot \text{thigh} + 0.483 \cdot \text{forearm} - 1.40 \cdot \text{wrist}$
Stepwise BIC	$-31.3 - 0.126 \cdot \text{weight} + 0.921 \cdot \text{abdom} + 0.446 \cdot \text{forearm} - 1.39 \cdot \text{wrist}$
LASSO (min λ)	$-13.3 + 0.055 \cdot \text{age} - 0.063 \cdot \text{weight} - 0.083 \cdot \text{height} - 0.399 \cdot \text{neck}$ $+0.826 \cdot \text{abdom} - 0.143 \cdot \text{hip} + 0.165 \cdot \text{thigh} + 0.078 \cdot \text{ankle}$ $+0.110 \cdot \text{biceps} + 0.384 \cdot \text{forearm} - 1.43 \cdot \text{wrist}$
LASSO (1 SE λ)	$-11.8 + 0.037 \cdot \text{age} - 0.144 \cdot \text{height} + 0.588 \cdot \text{abdom} - 0.835 \cdot \text{wrist}$

Note that the variables selected by the LASSO are **not unique** due to the randomness associated with choosing λ . The cross-validation randomly selects groups from the data to fit the model and that can lead to a different “best” λ value. For example, if our seed is set to a different number, we obtain a different result. The λ can change in ridge regression as well, but the number of variables will not change with ridge.

```
set.seed(2026)
cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
)$lambda.min
[1] 0.03438172
```

```
set.seed(1)
cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
)$lambda.min
[1] 0.03132734
```

Ridge Regression Example

Now let's use ridge regression to get estimates for the coefficients. We will set $\alpha = 0$ here. This does not reduce the number of predictors at all.

```
set.seed(2026)
cvfit_ridge <- cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 0
) # Set alpha = 0 for ridge regression
fit_ridge <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 0,
```

```
lambda = cvfit_ridge$lambda.min
)
coef(fit_ridge)
14 x 1 sparse Matrix of class "dgCMatrix"
      s0
(Intercept) -9.969782963
age          0.100897615
weight      -0.006623162
height      -0.162911219
neck        -0.314865758
chest       0.121310803
abdom       0.489012499
hip         -0.005349405
thigh       0.178795904
knee        -0.039658185
ankle       -0.010673900
biceps      0.071735533
forearm     0.294566945
wrist       -1.514325320
```

Ridge Regression VIF Reduction

One common use of ridge regression is eliminating some problems caused by multicollinearity, which occurs when the predictors are highly correlated with each other. When this is the case, the slopes are difficult to interpret and the standard errors in the slope estimates get inflated. Let's check this in our model for body fat.

```
options(width = 70)
library(car) # vif() function is in the car package
vif(fat_mod)
      age    weight    height    neck    chest    abdom    hip
2.250450 33.509320  1.674591  4.324463  9.460877 11.767073 14.796520
      thigh    knee    ankle    biceps    forearm    wrist
7.777865  4.612147  1.907961  3.619744  2.192492  3.377515
```

The VIF tells us how much the variance of the estimates of the slopes are inflated. For example, the variance of the **abdom** variable is 11.767 times higher (so the SE is about 3.43 times larger) than it would be if multicollinearity was not present.

```
summary(fat_mod)
Coefficients:
      Estimate Std. Error t value Pr(>|t|)
(Intercept) -15.29255    16.06992  -0.952  0.34225
age           0.05679     0.02996   1.895  0.05929 .
weight       -0.08031     0.04958  -1.620  0.10660
height       -0.06460     0.08893  -0.726  0.46830
neck         -0.43754     0.21533  -2.032  0.04327 *
chest        -0.02360     0.09184  -0.257  0.79740
abdom        0.88543     0.08008  11.057 < 2e-16 ***
hip          -0.19842     0.13516  -1.468  0.14341
```


thigh	0.23190	0.13372	1.734	0.08418	.
knee	-0.01168	0.22414	-0.052	0.95850	
ankle	0.16354	0.20514	0.797	0.42614	
biceps	0.15280	0.15851	0.964	0.33605	
forearm	0.43049	0.18445	2.334	0.02044	*
wrist	-1.47654	0.49552	-2.980	0.00318	**

While there is not a closed-form expression for the standard error of the ridge regression estimates, we can obtain estimated standard errors using either bootstrapping or Bayesian methods (we must wait until Notes 9 for Bayesian!). Let's look at an example using bootstrapping:

```
set.seed(2026)
nboot <- 1000
coefs <- matrix(rep(0, nboot*14), nrow = nboot)
for(i in 1:nboot) {
  index <- sample(1:nrow(fat), nrow(fat), replace = T)
  mod <- cv.glmnet(
    brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat[index,], alpha = 0
  )
  coefs[i,] <- coef(mod, s = "lambda.min")[1:14]
}
c(sd(coefs[,2]), sd(coefs[,3]), sd(coefs[,7]))
[1] 0.02148759 0.01561954 0.04080476
```

The standard errors for the slope of **age** decreased from 0.02996 to 0.02149 (a factor of about 1.39, so the variance decreased by about a factor of 1.94). The SE for the slope of **weight** decreased from 0.04958 to 0.01562 (a factor of about 3.17, variance by a factor of about 10.07), and the SE for the slope of **abdom** decreased from 0.08008 to 0.04080 (a factor of about 1.96, variance by a factor of about 3.85). We get these reductions without even removing any variables from the model. Of course, these estimates are now biased. By exactly how much is hard to determine.

Alternative way to get VIFs

Another way to obtain the VIF values in OLS is by computing the diagonals of the inverse of the correlation matrix for the predictors: $\mathbf{r}_{xx}^{-1}$.

In ridge regression, the VIF values can be computed by taking the diagonal elements of

$$(\mathbf{r}_{xx} - \lambda \mathbf{I})^{-1} \mathbf{r}_{xx} (\mathbf{r}_{xx} - \lambda \mathbf{I})^{-1}.$$

The original VIF values:

```
r_xx <- fat |> select(age, weight, height, neck, chest, abdom, hip,
                    thigh, knee, ankle, biceps, forearm, wrist) |>
  cor()
# Alternatively, can do
# r_xx <- cor(model.matrix(fat_mod)[-1]) # Remove the first column since all 1's
r_xx |> solve() |> diag() |> round(5)
```

age	weight	height	neck	chest	abdom	hip
2.25045	33.50932	1.67459	4.32446	9.46088	11.76707	14.79652
thigh	knee	ankle	biceps	forearm	wrist	

```
7.77786 4.61215 1.90796 3.61974 2.19249 3.37751
```

In our case for this toy example, a good λ value was 0.6294. The reduced ridge regression VIF values:

```
diag(
  solve(r_xx + 0.6294*diag(rep(1, 13))) %*%
    r_xx %*%
    solve(r_xx + 0.6294*diag(rep(1, 13)))
) |> round(5)
      age weight height   neck   chest   abdom    hip   thigh
0.33414 0.08150 0.36442 0.29041 0.21550 0.18218 0.16728 0.21672
      knee  ankle biceps forearm  wrist
0.28443 0.36689 0.30987 0.35652 0.31358
```

We actually obtain VIF values less than 1. This can happen under ridge regression and indicates that the multicollinearity issue has likely been resolved. There are some articles that propose different ways to calculate VIF in ridge regression, however, like this one [here](#).

glmnet in the caret package

While we were able to obtain an “optimal” λ value using the `cv.glmnet()` function, it did not have an option to optimize α as well. If all we care about are predictions and are not trying to use LASSO for model selection or ridge regression for VIF reduction, then choosing α via cross validation can make sense.

```
library(caret) |> suppressPackageStartupMessages()
set.seed(2026)
search_grid <- expand.grid(alpha = seq(0, 1, length = 50),
                           lambda=seq(0, 1, by = 0.1))
train_glmnet <- train(
  brozek ~ age + weight + height + neck + chest +
    + abdom + hip + thigh + knee + ankle + biceps +
    forearm + wrist, data = fat, method = "glmnet",
  tuneGrid = search_grid, trControl = trainControl(method = "cv", number = 5)
)
train_glmnet$bestTune
      alpha lambda
210 0.3877551      0
```

The best λ was chosen to be zero, which likely means are search grid steps were too large. Try again with a smaller number in the `by` argument.

```
library(caret) |> suppressPackageStartupMessages()
set.seed(2026)
search_grid <- expand.grid(alpha = seq(0, 1, length = 50),
                           lambda=seq(0, 0.1, by = 0.001))
train_glmnet <- train(
  brozek ~ age + weight + height + neck + chest +
    + abdom + hip + thigh + knee + ankle + biceps +
    forearm + wrist, data = fat, method = "glmnet",
```

```
tuneGrid = search_grid, trControl = trainControl(method = "cv", number = 5)
)
train_glmnet$bestTune
      alpha lambda
4881 0.9795918 0.032
```

This found that an elastic net with $\alpha = 0.9796$ and $\lambda = 0.032$ is best in terms of minimizing the MSE. α is close to 1 here, so this is close to a LASSO.

Example: Logistic Regression Elastic Net

We can do regularization using ridge regression, LASSO, or elastic net in other models as well, like logistic regression.

Logistic regression is a widely-used model when the response is binary. Suppose the response variable y takes values $\{0,1\}$. We model

$$P(y = 1|\mathbf{X}_i) = \frac{e^{\mathbf{X}_i\beta}}{1 + e^{\mathbf{X}_i\beta}},$$

which can be written in the following form:

$$\log\left(\frac{P(y = 1|\mathbf{X}_i)}{P(y = 0|\mathbf{X}_i)}\right) = \mathbf{X}_i\beta,$$

the so-called “logistic” or log-odds transformation. We seek to minimize the following loss function:

$$\arg \min_{\beta} \left\{ -\frac{1}{n} \sum_{i=1}^n y_i \log(\mathbf{X}_i\beta) - \log(1 + e^{\mathbf{X}_i\beta}) + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + \frac{(1-\alpha)}{2} \sum_{j=1}^p \beta_j^2 \right) \right\},$$

where the left hand side is the loss function for standard logistic regression, and the right hand side is the elastic net regularization.

Logistic regression is often plagued with degeneracies when $p > n$ and exhibits wild behavior even when n is close to p ; the elastic net penalty alleviates these issues, and regularizes and selects variables as well.

Using the example data set from the glmnet package:

```
library(glmnet)
data(BinomialExample)
logistic_df <- data.frame(y = BinomialExample$y,
                          as.data.frame(BinomialExample$x)) # Currently a matrix
names(logistic_df) <- c("y", paste0("x", 1:30))
```

Let’s try to fit a logistic regression to this dataset with $n = 100$ and $p = 31$ (30 predictor variables and only 100 data points).

```
logistic_fit <- glm(y ~ ., data = logistic_df, family = "binomial")
Warning: glm.fit: algorithm did not converge
Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(logistic_fit)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	7.815e+00	5.947e+04	0.000	1.000
x1	1.536e+01	4.586e+04	0.000	1.000
x2	4.219e+01	5.145e+04	0.001	0.999
x3	-1.483e+01	5.276e+04	0.000	1.000
x4	-2.831e+01	3.652e+04	-0.001	0.999
x5	-4.968e+00	5.440e+04	0.000	1.000
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.

Here we are getting warnings about the algorithm not converging because the fitted probabilities were exactly 0 or 1. This is an issue, likely because we have so many predictor variables.

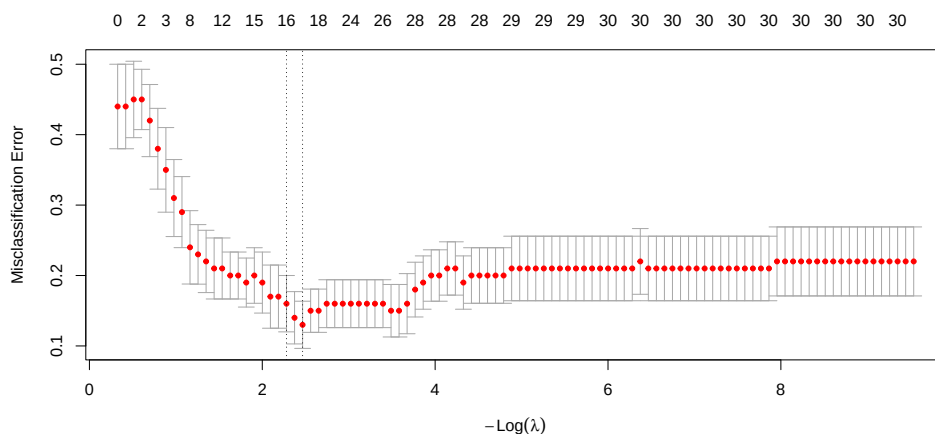
Now let's try with the elastic net with $\alpha = 1/3$ (equal weight on the L1 and L2 penalty). Why is $\alpha = 1/2$ not an equal weight?

Set family option to “binomial” in the `glmnet` or `cv.glmnet` functions. The code below uses misclassification error as the criterion for 10-fold cross-validation:

```
set.seed(2026)
cvfit <- cv.glmnet(y ~ ., family = "binomial", alpha = 1/3,
                  data = logistic_df, type.measure = "class")
```

Now we can plot the cross-validation results and find the “best” λ_{min} :

```
plot(cvfit)
```



```
cvfit$lambda.min
[1] 0.08489859
```

```
logistic_glmnet_fit <- glmnet(y ~ ., family = "binomial", alpha = 1/3,
                             data = logistic_df, lambda = cvfit$lambda.min)
coef(logistic_glmnet_fit)
```

```

31 x 1 sparse Matrix of class "dgCMatrix"
              s0
(Intercept)  0.2307632187
x1           .
x2           0.2593622891
x3          -0.1882223855
x4          -0.5861095408
x5          -0.1643935161
x6          -0.3938875709
x7           .
x8          -0.2670402697
x9           0.2345566373
x10         -0.6131022751
x11         -0.0672497953
x12         -0.0009538166
x13          .
x14          .
x15          .
x16          0.0557649394
x17          .
x18          .
x19          .
x20          .
x21          .
x22          0.1360558390
x23          0.1722437787
x24          .
x25          0.2883397048
x26         -0.2155427412
x27          .
x28          0.0868855495
x29         -0.1174446662
x30          .

```

Poisson Regression

When doing regression analysis in Applied Statistics, we discussed linear regression with a continuous response variable, logistic regression where the response variable is binary, and multinomial logistic regression where the response variable is nominal with several (more than two) categories.

We did not discuss the best way to handle a discrete, quantitative response variable. If this is the case, the possible values range from 0 to infinity, and large counts are rare, then **Poisson regression** is most appropriate.

Recall the Poisson distribution from MATH 1040, 1190, or 3700:

$$P(Y = y) = f(y) = \frac{e^{-\lambda} \lambda^y}{y!} \quad \text{for } y = 0, 1, 2, \dots$$

with $E[Y] = \text{Var}(Y) = \lambda$. So, the variance increases as the mean of Y increases.

We often use capital letters (like Y) to denote the variable name and lowercase letters (like y) to denote values of that variable.

Poisson Link Function

Like when doing logistic regression, we need a link function for Poisson regression. In logistic regression, the most common link function is the logit function: $\log\left(\frac{P(Y=1|\mathbf{X}_i)}{1 - P(Y=1|\mathbf{X}_i)}\right)$ and when

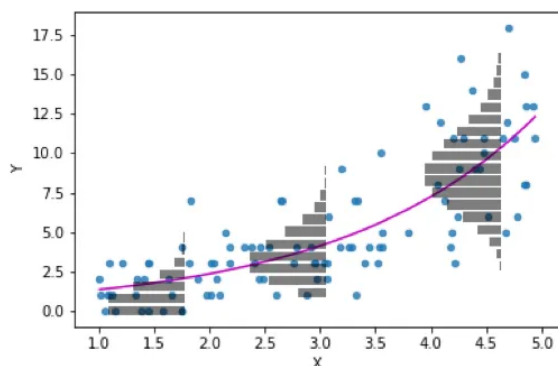
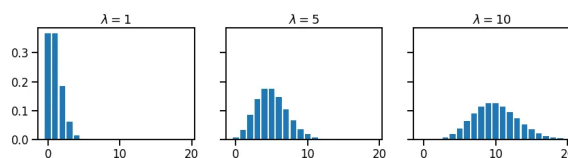
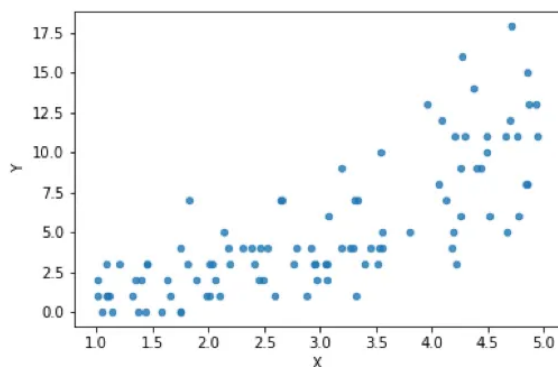
using that function, the model becomes linear. That is, $\log\left(\frac{P(Y=1|\mathbf{X}_i)}{1 - P(Y=1|\mathbf{X}_i)}\right) = \mathbf{X}_i\beta$.

The most common link function for Poisson regression is just the log function. Typically, we just write “log” to indicate the natural log function. That is,

$$\log(\widehat{E[Y|\mathbf{X}_i]}) = \mathbf{X}_i\beta$$

where $\mathbf{X}_i$ is the i th row of the model matrix $\mathbf{X}$. Of course, the expected value of Y is λ , so this model is really for predicting λ at certain values of X : $\log(\hat{\lambda}_i|x_i) = \beta_0 + \beta_1 x_i$ or $\hat{\lambda}_i|x_i = e^{\beta_0 + \beta_1 x_i}$ in the SLR case.

With the Poisson regression, we model the data by thinking about Y as following a Poisson distribution with a different mean at every value of x . So, if we begin with a dataset that looks like the one in the image below, which is fairly common for Poisson regression, then we can imagine a different Poisson distribution at each x value.



Poisson Regression Assumptions

1. **Poisson Response:** The response variable is conditionally a count per unit of time or space, described by a Poisson distribution.
2. **Independence:** The observations must be independent of one another.
3. **Mean = Variance:** By definition, the mean of a Poisson random variable must be equal to its variance.
4. **Linearity:** The log of the mean rate, $\log(\lambda)$, must be a linear function of x .

Finding Parameter Estimates

The parameters, $\beta_0, \beta_1, \dots, \beta_p$ are not estimated using least squares. Instead, like for logistic regression, these are estimated using **maximum likelihood estimation** (MLE), which we discussed earlier in the class. Let's apply the MLE technique to the Poisson regression case:

$$\begin{aligned} L(\beta) &= \prod_{i=1}^n f_i(y_i) = f_1(y_1) \cdot \dots \cdot f_n(y_n) \\ &= \frac{e^{-\lambda_1} \lambda_1^{y_1}}{y_1!} \cdot \dots \cdot \frac{e^{-\lambda_n} \lambda_n^{y_n}}{y_n!} \\ &= \frac{\prod_{i=1}^n \lambda_i^{y_i} \cdot \exp\{-\sum_{i=1}^n \lambda_i\}}{\prod_{i=1}^n y_i!} \end{aligned}$$

Now, it is common to take the log of the likelihood to make the actual maximizing easier:

$$\begin{aligned} \ln(L(\beta)) &= \ell(\beta) = \ln\left(\frac{\prod_{i=1}^n \lambda_i^{y_i} \cdot \exp\{-\sum_{i=1}^n \lambda_i\}}{\prod_{i=1}^n y_i!}\right) \\ &= \ln\left(\prod_{i=1}^n \lambda_i^{y_i}\right) + \ln\left(\exp\left\{-\sum_{i=1}^n \lambda_i\right\}\right) - \ln\left(\prod_{i=1}^n y_i!\right) \\ &= \sum_{i=1}^n y_i \ln(\lambda_i) - \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \ln(y_i!) \end{aligned}$$

Now we can substitute for λ_i . That is, $\lambda_i = E(Y|\mathbf{X}_i) = \exp(\mathbf{X}_i\beta)$:

$$\begin{aligned} \ell(\beta) &= \sum_{i=1}^n y_i \ln(\lambda_i) - \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \ln(y_i!) \\ &= \sum_{i=1}^n y_i \ln(\exp(\mathbf{X}_i\beta)) - \sum_{i=1}^n \exp(\mathbf{X}_i\beta) - \sum_{i=1}^n \ln(y_i!) \\ &= \sum_{i=1}^n y_i (\mathbf{X}_i\beta) - \sum_{i=1}^n \exp(\mathbf{X}_i\beta) - \sum_{i=1}^n \ln(y_i!) \end{aligned}$$

Now, to maximize this, like in calculus, we take the derivative with respect to one of the parameters (one of the β 's) and set the equation equal to zero. Let's do an example in the SLR case where

$$\mathbf{X}_i\boldsymbol{\beta} = \beta_0 + \beta_1 x_i.$$

$$\begin{aligned}\ell(\boldsymbol{\beta}) &= \sum_{i=1}^n y_i(\mathbf{X}_i\boldsymbol{\beta}) - \sum_{i=1}^n \exp(\mathbf{X}_i\boldsymbol{\beta}) - \sum_{i=1}^n \ln(y_i!) \\ &= \sum_{i=1}^n y_i(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \ln(y_i!)\end{aligned}$$

Take the derivative with respect to β_0 and set it equal to zero:

$$\frac{\partial \ell}{\partial \beta_0} = \sum_{i=1}^n y_i - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i) = 0$$

Take the derivative with respect to β_1 and set it equal to zero:

$$\frac{\partial \ell}{\partial \beta_1} = \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \exp(\beta_0 + \beta_1 x_i) = 0$$

These equations need to be solved numerically (unlike ordinary least squares), so they do not have a closed-form solution. We discussed using a function like `optim()` in **R** to solve optimization problems like this, but that only works if we have x and y values. So, let's simulate a little toy example:

```
set.seed(2026)
x <- rnorm(5) # Simulate 5 random values for x
# Simulate 5 random values for y (with a couple different means)
y <- c(rpois(3, 2), rpois(2, 6))
first_func <- function(beta) {
  sum(y) - sum(exp(beta[1] + beta[2] * x))
}
second_func <- function(beta) {
  sum(x * y) - sum(x * exp(beta[1] + beta[2] * x))
}
optim_func <- function(beta) {
  # This will be minimized when both functions are zero
  abs(first_func(beta)) + abs(second_func(beta))
}
optim(c(1, 0), optim_func)

$par
[1] 1.0837225 -0.4531598

$value
[1] 2.597732e-07

$counts
function gradient
      133         NA

$convergence
[1] 0

$message
NULL
```


Fitting the Poisson Model in R

In **R**, we can use the `glm()` function to fit a Poisson model. Like we do with logistic regression, we need to specify the family and the link function. By default, the link function in the “poisson” family is the log, so we will keep it as the default.

```
glm(y ~ x, data = poisson_df, family = "poisson")
```

So applying this to the little example from before:

```
mod <- glm(y ~ x, family = "poisson") # Fit the Poisson model
summary(mod)
```

Call:

```
glm(formula = y ~ x, family = "poisson")
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.0837	0.2918	3.713	0.000205 ***
x	-0.4532	0.4227	-1.072	0.283648

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 15.624 on 4 degrees of freedom
 Residual deviance: 14.468 on 3 degrees of freedom
 AIC: 30.779

Number of Fisher Scoring iterations: 6

Now let's do a much more substantial example:

Illustrative Example - Lumber

The Miller Lumber Company is a large retailer of lumber and paint, as well as of plumbing, electrical, and other household supplies. During a representative two-week period, in-store surveys were conducted and addresses of customers were obtained. The addresses were then used to identify the metropolitan area census tracts in which the customers reside. At the end of the survey period, the total number of customers who visited the store from each census tract within a 10-mile radius was determined and relevant demographic information for each tract (average income, number of housing units, etc.) was obtained. Several other variables expected to be related to customer counts were constructed from maps, including distance from census tract to nearest competitor and distance to store.

We will try to predict the number of customers from a Census tract using two variables: the distance the track is to the Miller Lumber Company store (quantitative in miles) and the distance from a competitor's store (categorical with categories “<2”, “2-4”, or “4+” miles.)

```
library(math3190package) |> suppressPackageStartupMessages()
lumber <- lumber_company |>
  mutate(
    competitor_dist = factor(
```

```

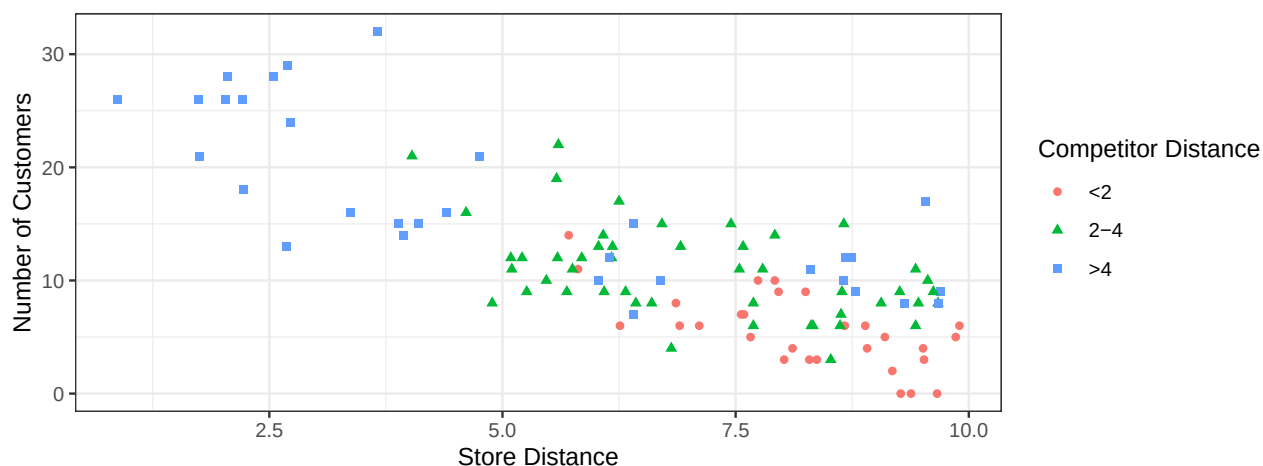
    case_when(competitor_distance <= 2 ~ "<2",
              competitor_distance > 2 & competitor_distance <= 4 ~ "2-4",
              competitor_distance > 4 ~ ">4"),
    levels = c("<2", "2-4", ">4")
  )
)
lumber_mod <- glm(customers ~ store_distance * competitor_dist,
                  data = lumber, family = "poisson")

```

```

ggplot(lumber, aes(x = store_distance, y = customers,
                  shape = competitor_dist,
                  color = competitor_dist)) +
  geom_point() +
  labs(x = "Store Distance", y = "Number of Customers",
       shape = "Competitor Distance", color = "Competitor Distance") +
  theme_bw()

```



```
summary(lumber_mod)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	4.32561	0.50344	8.592	< 2e-16	***
store_distance	-0.32659	0.06425	-5.083	3.71e-07	***
competitor_dist2-4	-1.12808	0.54137	-2.084	0.03718	*
competitor_dist>4	-0.89883	0.50992	-1.763	0.07795	.
store_distance:competitor_dist2-4	0.20830	0.07034	2.961	0.00306	**
store_distance:competitor_dist>4	0.19871	0.06623	3.000	0.00270	**

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 422.22 on 109 degrees of freedom
 Residual deviance: 132.35 on 104 degrees of freedom
 AIC: 588.39

We will get one equation for each category of the categorical variable. Letting x_i be the i th value of `store_distance`, we obtain:

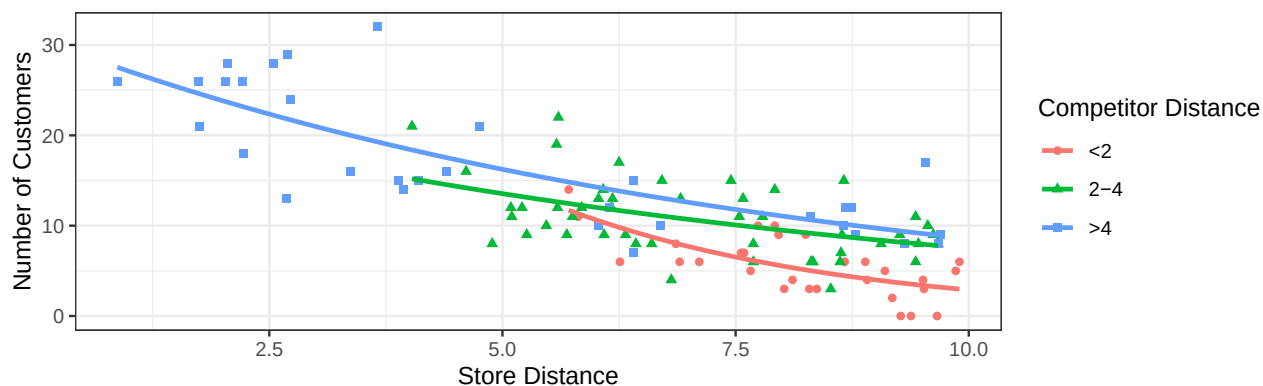
Model 1 (<2 Group): $\ln(\hat{\lambda}_i) = 4.32561 - 0.32659 \cdot x_i$

Model 2 (2-4 Group): $\ln(\hat{\lambda}_i) = 3.19753 - 0.11829 \cdot x_i$

Model 3 (4+ Group): $\ln(\hat{\lambda}_i) = 3.42678 - 0.12788 \cdot x_i$

We can use the `geom_smooth()` function to plot the models curves.

```
ggplot(lumber, aes(x = store_distance, y = customers,
                  shape = competitor_dist,
                  color = competitor_dist)) +
  geom_point() +
  labs(x = "Store Distance", y = "Number of Customers",
       shape = "Competitor Distance", color = "Competitor Distance") +
  geom_smooth(method = "glm", method.args = list(family = "poisson"),
             formula = "y ~ x", se = F) + theme_bw()
```



Interpreting the Model Coefficients

The interpretation of the coefficients here is quite similar to interpreting the coefficients of a log-transformed linear regression model.

Intercept: The predicted mean of the Poisson distribution when $x = 0$ is $\exp(\hat{\beta}_0)$.

Slope: The predicted change in the mean of the Poisson distribution when x increases by one unit is $\exp(\hat{\beta}_1)$.

Confidence Intervals for the Model Coefficients

We can obtain confidence intervals for the intercept and slope(s) using normal approximations as long as our sample size is fairly large:

$$\left(\hat{\beta}_i - z_{\alpha/2} \cdot SE(\hat{\beta}_i), \hat{\beta}_i + z_{\alpha/2} \cdot SE(\hat{\beta}_i)\right)$$

Then we can exponentiate these to get the confidence intervals back on the original scale:

$$\left(\exp\left\{\hat{\beta}_i - z_{\alpha/2} \cdot SE(\hat{\beta}_i)\right\}, \exp\left\{\hat{\beta}_i + z_{\alpha/2} \cdot SE(\hat{\beta}_i)\right\}\right)$$

Using the `confint()` function in **R** uses what is called the “profile likelihood” method for computing these intervals, which we will not discuss here, so the results will be a bit different. Note that the **MASS** package must be installed for this to work.

Lumber Example Confidence Intervals

For the “store_distance” variable, the β_1 estimate is -0.32659 with a standard error of 0.06425 . We can obtain an approximate 95% interval using

```
-0.32659 - qnorm(1-0.05/2) * 0.06425
[1] -0.4525177
```

```
-0.32659 + qnorm(1-0.05/2) * 0.06425
[1] -0.2006623
```

So the interval is $-0.453 < \beta_1 < -0.201$.

Using the `confint()` function:

```
confint(lumber_mod)[2,]
Waiting for profiling to be done...
      2.5 %      97.5 %
-0.4522654 -0.2001689
```

We obtain an interval of $-0.452 < \beta_1 < -0.200$.

Confidence Intervals for the Mean Response

We can obtain an approximate confidence interval for $E(Y_i|\mathbf{x}_i)$ using the `predict()` function with the options `type = "response"` and `se.fit = T`. This won’t create for CI for us, though. We can approximate using a normal distribution. This approximation is better for larger values of λ_i .

```
lumber_pred <- predict(lumber_mod,
                      data.frame(store_distance = 5,
                                competitor_dist = "2-4"),
                      se.fit = T)
exp(lumber_pred$fit - 1.96 * lumber_pred$se.fit)
exp(lumber_pred$fit + 1.96 * lumber_pred$se.fit)

11.87221; 15.45597
```

So, the interval is $11.87 < E(Y^*|\mathbf{x}^*) < 15.46$ or $11.87 < \lambda^*|\mathbf{x}^* < 15.46$.

Prediction Intervals

These confidence intervals are for the mean Poisson response at a given row vector $\mathbf{x}$. Then, to get predictions for the actual count, we can use percentiles for Poisson distribution.

For the lower bound of a $(1 - \alpha) \times 100\%$ confidence interval, we can use the $\alpha/2 \times 100$ th percentile using the lower bound of the CI for the mean.

For the upper bound of a $(1 - \alpha) \times 100\%$ confidence interval, we can use the $(1 - \alpha/2) \times 100$ th percentile using the upper bound of the CI for the mean.

```
qpois(0.025, 11.87221) # 2.5th percentile using lower mean
```

```
[1] 6
```

```
qpois(0.975, 15.45597) # 97.5th percentile using upper mean
```

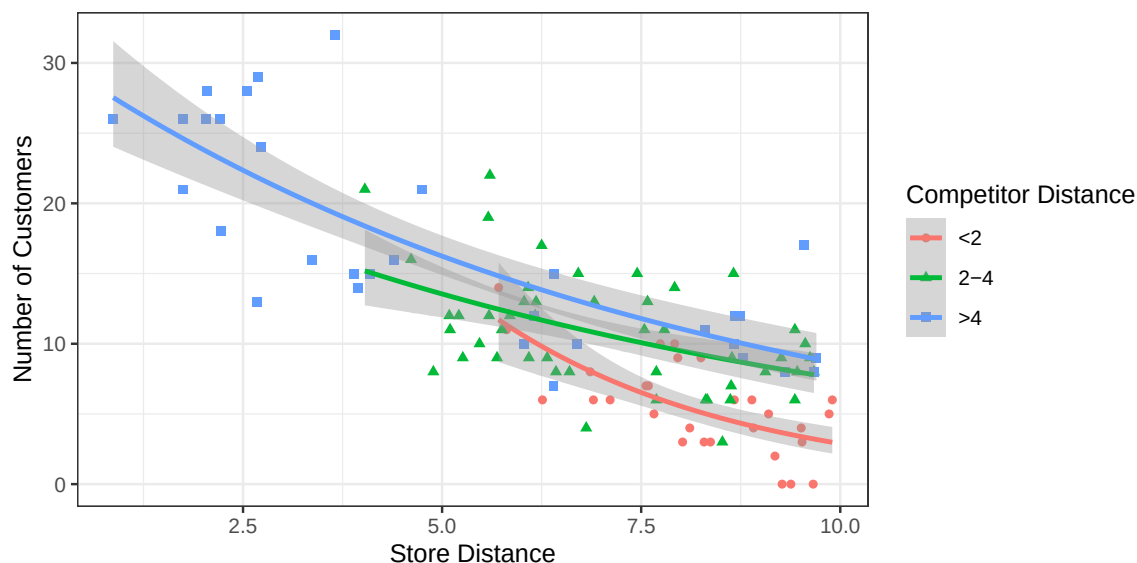
```
[1] 24
```

That gives an interval of $6 < y^* | \mathbf{x}^* < 24$.

Lumber Model Plot with CI

We can plot the CI for the mean with the `geom_smooth()` function as well. These intervals are constructed using the profile likelihood method rather than normal approximations, but they are similar.

```
ggplot(lumber,
  aes(x = store_distance, y = customers,
      shape = competitor_dist, color = competitor_dist)) +
  geom_point() +
  labs(x = "Store Distance",
       y = "Number of Customers",
       shape = "Competitor Distance",
       color = "Competitor Distance") +
  geom_smooth(method = "glm",
             method.args = list(family = "poisson"),
             formula = "y ~ x", se = T) +
  theme_bw()
```



Poisson Residuals

There are several different types of residuals for a Poisson regression model. The first is called the **Pearson residual**, which is computed using

$$\hat{\epsilon}_i^P = \frac{y_i - \hat{y}_i}{\sqrt{\hat{y}_i}}$$

where $\hat{y}_i = \exp(\mathbf{X}_i \hat{\boldsymbol{\beta}}) = \hat{\lambda}_i$.

Then there is the **deviance residual**, which is computed using

$$\hat{\epsilon}_i^D = \text{sgn}(y_i - \hat{y}_i) \sqrt{2(y_i \cdot \ln(y_i/\hat{y}_i) - y_i + \hat{y}_i)}$$

where sgn is the sign function that is positive when its input is positive and negative otherwise. Note that when $y_i = 0$, we take $y_i \cdot \ln(y_i/\hat{y}_i)$ to also equal 0.

It can be shown that both of these residuals are approximately normally distributed.

Other Poisson Regression Functions in R

To obtain the fitted values on the original data scale, we can either use the

```
fitted.values()
```

function or use

```
predict(model, type = "response")
```

If we just use

```
predict(model)
```

it will return the linear predictors. To get back to the original scale, we would need to exponentiate those values.

To obtain the Pearson residuals, we can use

```
residuals(model, type = "pearson")
```

The deviance residuals are the default, so they be obtained using either

```
residuals(model)
```

```
residuals(model, type = "deviance")
```

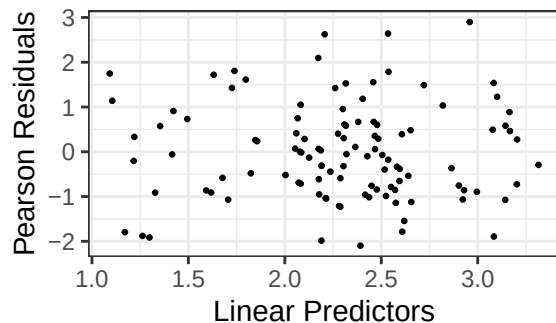
Poisson Regression Diagnostics

We can perform **diagnostics** for Poisson regression using hypothesis tests and plots. Each of the following are helpful:

1. A residual plot of the Pearson residuals vs the predicted values. These residuals should appear randomly spread.
2. A QQ-plot of the Pearson or deviance residuals. These should be approximately normally distributed.
3. A deviance goodness-of-fit (GOF) test.
4. A check for overdispersion.

Poisson Regression Diagnostics - Residual Plot

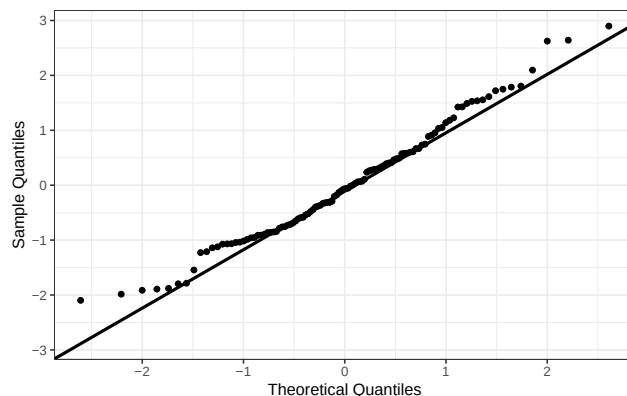
```
p_resids <- residuals(lumber_mod, type = "pearson")
resid_df <- data.frame(p_resids,
                       linear_pred = lumber_mod$linear.predictors)
ggplot(resid_df, aes(x = linear_pred, y = p_resids)) +
  geom_point(size = 0.5) +
  labs(x = "Linear Predictors", y = "Pearson Residuals") +
  theme_bw()
```



Poisson Regression Diagnostics - QQ Plot

We can use a **QQ plot** to check if the Pearson residuals are approximately normal. This can be done with the `qqnorm()` and `qqline()` function in base **R** or the `geom_qq()` and `geom_qq_line()` in `ggplot2`. The closer to the line, the better.

```
ggplot(resid_df, aes(sample = p_resids)) +
  geom_qq() +
  geom_qq_line(linewidth = 1) +
  labs(x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_bw()
```



Poisson Regression Diagnostics - GOF Test

A deviance goodness-of-fit (GOF) test tests the hypotheses

H_0 : The model is correctly specified.

H_A : The model is incorrectly specified.

The test statistic is

$$D = 2 \sum_{i=1}^n (y_i \ln(y/\hat{y}_i) - (y_i - \hat{y}_i)) = \sum_{i=1}^n (\hat{\epsilon}_i^D)^2,$$

the sum of the squared deviance residuals. This statistic approximately follows a χ^2 distribution with $n - p$ degrees of freedom when the predicting means are not too small. It is also given in the output of the Poisson model as the “Residual deviance”.

If this statistic is large relative to the degrees of freedom, we have evidence that the model does not fit well, and we would reject the null hypothesis. Therefore, this is a right-tailed test.

```
1 - pchisq(lumber_mod$deviance, df = lumber_mod$df.residual)
[1] 0.03169943
```

We have evidence here that this model does not fit well. In this case, that is likely because the model is **overdispersed**. We can alleviate this issue by adding more important variables. If we add more variables, we lose evidence the model is poorly fitting.

```
lumber_mod2 <- glm(customers ~ store_distance + competitor_dist +
  average_income + average_age + housing_units,
  data = lumber, family = "poisson")
1 - pchisq(lumber_mod2$deviance, df = lumber_mod2$df.residual)
[1] 0.2755826
```

Poisson Regression Diagnostics - Overdispersion

One important property of a Poisson distribution is that $E(Y) = \lambda$ and $Var(Y) = \lambda$. That is, both the mean and the variance are equal. **Overdispersion** occurs when the variance of a model is greater than its mean.

- Overdispersion is caused by a positive correlation between responses or by too much variation between response probabilities or counts. Overdispersion also arises when there are violations in the distributional assumptions of the data as well as when the data are prone to it (i.e., when earlier events cause or influence the existence of subsequent events).
- Overdispersion is an issue because it may cause standard errors of the estimates to be underestimated (i.e., a variable may appear to be a significant predictor when it is in fact not significant).
- A model may be overdispersed if the value of the sum of the squared Pearson residuals divided by the degrees of freedom (df) is greater than 1.0. This quotient is called the dispersion. Small amounts of overdispersion are of little concern; however, if the dispersion statistic is greater than 1.25 for moderate-sized models, then a correction may be warranted. Models with large numbers of observations may be overdispersed with a dispersion statistic of 1.05.
- We can alleviate the overdispersion issue by adding other predictors, transforming the count data, or **changing models**.

If we are using Poisson regression in its proper context, then the most common way to alleviate overdispersion is to change from a Poisson model to a **quasi-Poisson model**.

Quasi-Poisson Regression

The quasi-Poisson distribution is very similar to the Poisson, except it allows for the mean and variance to differ:

$$E[Y] = \lambda$$

$$Var(Y) = \phi\lambda$$

where ϕ is known as the dispersion parameter that is estimated from the data.

In particular, $\phi = \frac{1}{n-p} \sum_{i=1}^n (\hat{\epsilon}_i^P)^2$, the sum of the squared Pearson residuals divided by the residual degrees of freedom.

Quasi-Poisson Example

In our example, let's check the estimate for ϕ :

```
sum(residuals(lumber_mod, type = "pearson")^2)
[1] 124.7268

lumber_mod$df.residual # n - p = 110 - 6
[1] 104

phi <- sum(residuals(lumber_mod, type = "pearson")^2)/lumber_mod$df.residual
phi
[1] 1.199297
```

The dispersion estimate of 1.199 is not above the 1.25 cutoff, but it fairly large. Let's fit a new model using a "quasi-poisson" family.

```
lumber_mod_quasi <- glm(
  formula = customers ~ store_distance * competitor_dist,
  family = "quasipoisson", data = lumber)
```

Let's compare the output.

```
summary(lumber_mod)

Coefficients:
                                Estimate Std. Error z value Pr(>|z|)
(Intercept)                4.32561      0.50344    8.592 < 2e-16 ***
store_distance             -0.32659      0.06425   -5.083 3.71e-07 ***
competitor_dist2-4         -1.12808      0.54137   -2.084 0.03718 *
competitor_dist>4          -0.89883      0.50992   -1.763 0.07795 .
store_distance:competitor_dist2-4 0.20830      0.07034    2.961 0.00306 **
store_distance:competitor_dist>4  0.19871      0.06623    3.000 0.00270 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for poisson family taken to be 1)

```
Null deviance: 422.22  on 109  degrees of freedom
Residual deviance: 132.35  on 104  degrees of freedom
AIC: 588.39
```

Quasi-Poisson Example - Quasi-Poisson Output

```
summary(lumber_mod_quasi)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	4.32561	0.55133	7.846	3.99e-12	***
store_distance	-0.32659	0.07036	-4.642	1.01e-05	***
competitor_dist2-4	-1.12808	0.59287	-1.903	0.05984	.
competitor_dist>4	-0.89883	0.55843	-1.610	0.11052	
store_distance:competitor_dist2-4	0.20830	0.07703	2.704	0.00800	**
store_distance:competitor_dist>4	0.19871	0.07253	2.740	0.00724	**

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for quasipoisson family taken to be 1.199297)

Null deviance: 422.22 on 109 degrees of freedom
 Residual deviance: 132.35 on 104 degrees of freedom
 AIC: NA

Some things to notice:

- The parameter estimates are the same.
- We estimate the standard errors of the $\hat{\beta}_i$ coefficients by taking the SE estimates in the regular Poisson regression model and multiplying them by $\sqrt{\phi}$.
- These estimates for the quasi-Poisson approximately follow a t distribution rather than a normal distribution.
- AIC is not defined for the quasi-Poisson family.

An alternative to using the quasi-Poisson is to use **negative binomial regression**. Poisson regression is actually just a special case of negative binomial regression and the latter has more than one parameter to allow for the mean and variance to differ.

Just for Fun - Poisson Bootstrap

The bootstrap never (or at least rarely) lies! Compare these standard errors to the Poisson and quasi-Poisson ones.

```
nsims <- 5000
betas <- matrix(rep(0, 6*nsims), nrow = nsims)
for(i in 1:nsims) {
  index = sample(1:nrow(lumber), nrow(lumber), replace = T)
  betas[i,] <- coef(
    glm(customers ~ store_distance * competitor_dist,
        family = "poisson", data = lumber[index,])
  )
}
apply(betas, 2, mean) |> round(4) # Take mean of each column
[1] 4.3501 -0.3305 -1.1577 -0.9233 0.2125 0.2021
apply(betas, 2, sd) |> round(4) # Take SD of each column
[1] 0.5390 0.0723 0.5796 0.5462 0.0780 0.0741
```

Notes 8: Dimension Reduction

Table of contents

Introduction	149
Principal Component Analysis	149
Partial Least Squares	152
PLS Regression in R	152
<code>plsrf</code> Function Output Interpretation	153
More Differences Between PCA and PLS	153
PLS Algorithm	154
PLS Surrogates	155
Cross Validation in PLS	156
Cross Validation in PLS with <code>Caret</code>	157
Fitting Final PLS Model	157
UMAP	159
UMAP in R	159
UMAP 3D Example vs PCA	161

Introduction

It is often that case when working with data that there are many features or predictors we need to deal with. Sometimes you can reduce the number of variables with domain knowledge: knowing what variables should not be in the analysis. However, even that fails sometimes when there are hundreds or even thousands of features in the dataset. In this case, dimension reduction techniques are needed.

There are two main goals of dimension reduction methods:

1. Reducing the number of features for use in a model.
2. Reducing the number of features for visualization.

Sometimes both goals are desired, but other times only one is needed. In this set of notes, we will discuss three dimension reduction techniques: principal component analysis (PCA), partial least squares (PLS), and uniform manifold approximation and projection (UMAP).

Principal Component Analysis

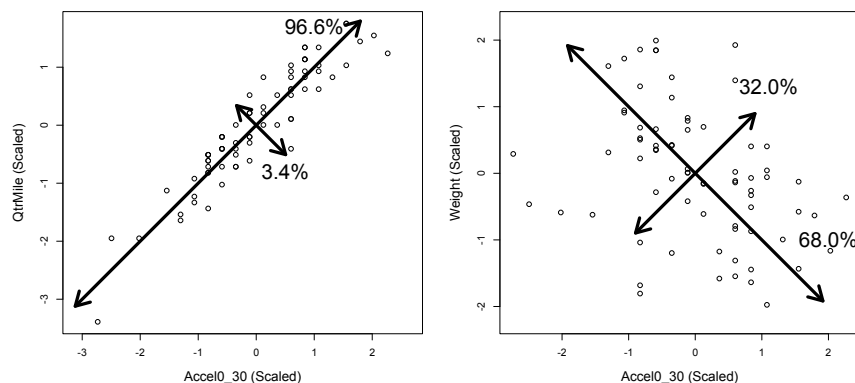
Principal component analysis (PCA) is probably the most popular dimension reduction technique. It is useful for both modeling and visualization. Let's focus first on the modeling and we will discuss the visualization when we discuss UMAP.

The idea of PCA is to obtain k new predictors, called components, with $k < p$ where each new predictor is a linear combination of the original predictor variables. PCA is useful for:

- Reducing the dimension of the problem so we are using k predictors instead of p predictors.
- Reducing problems associated with multicollinearity.

- Slopes being hard to interpret.
- Standard errors being inflated.

The way PCA works is by creating orthogonal predictor components using information from all the original predictors. The first component explains as much of the variation (or information) as possible in the original explanatory variables. The second component then explains as much as possible of the remaining variation, and so on. The more correlated the original variables are, the more variation can be explained with each new linear combination.



Those images came from the PCA on the `cars99` dataset that is in the `math3190package` and in the `Math3190_Sp26` GitHub repo.

```
library(math3190package)
cars <- na.omit(cars99)
head(cars)
```

```
# A tibble: 6 x 10
  Model      CityMPG HwyMPG FuelCap Weight FrontWt Accel0_30 Accel0_60
  <chr>      <int>  <int>   <dbl> <int>  <int>    <dbl>    <dbl>
1 "Buick Ce~    20    29   17.5  3350    64     3.3     9.4
2 "Buick Re~    19    30   17.5  3325    63     3.3     9.2
3 "Chevrole~    20    29   16.6  3350    64     3.4     9.8
4 "Chevrole~    22    30   15     3040    64     3.3     9.4
5 "Chrysler~    19    27   16     3170    64     3.5    10
6 "Daewoo L~    20    28   15.8  3185    60     4      11

# i 2 more variables: QtrMile <dbl>, Type <chr>
```

We can find the components using the `prcomp()` function in **R**. Let's use PCA to try to predict highway mpg using fuel capacity, weight, front weight, 0-30 mph acceleration time, 0-60 mph acceleration time, and quarter mile time. Since the scale is so vastly different between the variables, we will center and scale them.

```
X <- cars[,4:9] # numeric predictors are in columns 4-9
y <- cars$HwyMPG
pca <- prcomp(X, scale = T)
```

The amount of variation explained by each component can be found using the standard deviations of the components. We want to balance taking as many components as we need to still explain the vast majority of variation in the original variables with reducing the dimension of our feature space. A common rule of thumb is to take as many components as we need to explain at least 90% of the

variation.

```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6
Standard deviation	1.9414	1.1471	0.8395	0.34991	0.2877	0.07364
Proportion of Variance	0.6281	0.2193	0.1174	0.02041	0.0138	0.00090
Cumulative Proportion	0.6281	0.8474	0.9649	0.98530	0.9991	1.00000

The rule of thumb

```
round(pca$rotation, 3) # how the components are created
```

	PC1	PC2	PC3	PC4	PC5	PC6
FuelCap	-0.349	0.597	-0.127	0.681	0.203	-0.028
Weight	-0.345	0.602	-0.140	-0.698	-0.104	0.023
FrontWt	0.268	0.353	0.896	-0.014	0.033	0.001
Accel0_30	0.461	0.287	-0.219	0.182	-0.789	0.045
Accel0_60	0.486	0.193	-0.240	-0.069	0.444	0.683
QtrMile	0.487	0.196	-0.237	-0.102	0.357	-0.728

These components are related to how correlated the predictor variables are. The way we find these components is by finding the eigenvectors of the correlation matrix (for scaled PCA) or the covariance matrix (for un-scaled). Then the standard deviations are the square roots of the eigenvalues.

```
eig <- eigen(cor(X))
```

```
sqrt(eig$values) |> round(3)
```

```
[1] 1.941 1.147 0.839 0.350 0.288 0.074
```

```
eig$vectors |> round(3)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
[1,]	-0.349	-0.597	-0.127	0.681	-0.203	0.028
[2,]	-0.345	-0.602	-0.140	-0.698	0.104	-0.023
[3,]	0.268	-0.353	0.896	-0.014	-0.033	-0.001
[4,]	0.461	-0.287	-0.219	0.182	0.789	-0.045
[5,]	0.486	-0.193	-0.240	-0.069	-0.444	-0.683
[6,]	0.487	-0.196	-0.237	-0.102	-0.357	0.728

Once we have these components, found in `pca$x`, we can put those into a model like an OLS regression or plot them to visualize the higher-dimension space projected down to 1 or 2 dimensions.

Partial Least Squares

Notice that the response variable, y , is never used to find the components in PCA. Because of this, PCA is known as an **unsupervised** statistical learning algorithm.

However, a natural idea is to do something similar to PCA, but to make the algorithm **supervised**. That is, we can try to find these components using information from $\mathbf{y}$ in addition to $\mathbf{X}$. This is essentially what **partial least squares** (PLS) is. PLS is a **dimension reduction** tool similar to PCA that finds these components by balancing explaining variation in the predictors and correlation with the response variable.

PLS Regression in R

We can use the `plsr()` function in the `pls` package to perform partial least squares regression. There is also a `pcr()` function to implement PCA.

```
library(pls) |> suppressPackageStartupMessages()
cars_pls <- plsr(HwyMPG ~ FuelCap + Weight + FrontWt +
                 Accel0_30 + Accel0_60 + QtrMile,
                 data = cars, scale = TRUE)
cars_pcr <- pcr(HwyMPG ~ FuelCap + Weight + FrontWt +
                 Accel0_30 + Accel0_60 + QtrMile,
                 data = cars, scale = TRUE)
```

Let's compare the output of the `pcr()` function to our PCA.

```
summary(cars_pcr)
Number of components considered: 6
TRAINING: % variance explained
```

	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps
X	62.81	84.74	96.49	98.53	99.91	100.00
HwyMPG	38.86	56.53	66.22	71.13	71.14	71.87

```
round(cars_pcr$projection, 3)
```

	Comp 1	Comp 2	Comp 3	Comp 4	Comp 5	Comp 6
FuelCap	-0.349	0.597	-0.127	0.681	0.203	-0.028
Weight	-0.345	0.602	-0.140	-0.698	-0.104	0.023
FrontWt	0.268	0.353	0.896	-0.014	0.033	0.001
Accel0_30	0.461	0.287	-0.219	0.182	-0.789	0.045
Accel0_60	0.486	0.193	-0.240	-0.069	0.444	0.683
QtrMile	0.487	0.196	-0.237	-0.102	0.357	-0.728

Let's check out the `plsr()` output.

```
summary(cars_pls)
Number of components considered: 6
TRAINING: % variance explained
```

	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps
X	60.26	82.64	95.66	98.53	98.91	100.00
HwyMPG	53.42	66.40	68.78	71.22	71.74	71.87

```
round(cars_pls$projection, 3)
```

	Comp 1	Comp 2	Comp 3	Comp 4	Comp 5	Comp 6
FuelCap	-0.519	-0.382	0.501	0.787	0.049	-0.102
Weight	-0.600	-0.679	-0.366	-0.793	-0.031	0.032
FrontWt	0.291	0.179	0.810	-0.144	0.004	-0.028
Accel0_30	0.281	-0.400	0.006	0.182	-0.155	0.545
Accel0_60	0.319	-0.389	-0.198	-0.114	-0.619	-1.475
QtrMile	0.324	-0.373	-0.137	0.022	0.775	0.920

pls Function Output Interpretation

More Differences Between PCA and PLS

In PCA, the loading matrix comes from the eigenvectors, which are orthogonal, so if we call the loading matrix $\mathbf{P}$ (as is common in the literature), we have that $\mathbf{P}^{-1} = \mathbf{P}^T$.

We can obtain the components (also called the scores) using $\mathbf{T} = \mathbf{XP}$. That means $\mathbf{X} = \mathbf{TP}^T$. Then we regress $\mathbf{y}$ on the first k columns of $\mathbf{T}$ instead of $\mathbf{X}$. That gives us

In PLS, the loadings are not orthogonal. So, while we can still decompose $\mathbf{X}$ using $\mathbf{X} = \mathbf{TP}^T$, the components can be found using $\mathbf{T} = \mathbf{X}(\mathbf{P}^T)^{-1}$. That matrix, $(\mathbf{P}^T)^{-1}$, is sometimes called the **projection** matrix and it gives the linear combinations needed to obtain the components. For PCA, the loading and projection matrix are the same.

PLS Algorithm

Unlike PCA, the PLS projection matrix is obtained iteratively rather than using eigenvalues. Here is one way to obtain the scores:

1. Center and scale the original data matrix $\mathbf{X}$ (without an intercept column) by column. We will still call this scaled matrix $\mathbf{X}$. $\mathbf{y}$ need not be centered or scaled.
2. Obtain the first column of the project matrix, ϕ_1 , by setting the j th entry of ϕ_1 equal to the slope from the simple linear regression of $\mathbf{y}$ onto $\mathbf{X}_j$, the j th column of $\mathbf{X}$.
 - This is equivalent to taking $\phi_1 = \mathbf{X}^T \mathbf{y}$ (up to a constant).
3. Then normalize these values so the length of the vector ϕ_1 is 1.
4. Compute the first column of the scores matrix: $\mathbf{V}_1 = \sum_{j=1}^{p-1} \phi_{j1} X_j = \mathbf{X} \phi_1$.
5. Adjust each of the variables, $\mathbf{X}_1, \dots, \mathbf{X}_{p-1}$, for $\mathbf{V}_1$ by regressing each variable on $\mathbf{V}_1$ and taking the residuals.
 - These residuals can be interpreted as the remaining information that has not been explained by the first PLS direction.
6. Create a new matrix, $\mathbf{R}$ with columns equal to the residuals of each regression conducted in step 5.
7. Obtain $\tilde{\phi}_2 = \mathbf{R}^T \mathbf{y}$ and normalize it.
 - Note: this $\tilde{\phi}_2$ is not exactly equal to the second column of the projection matrix, but can be used to obtain the second column of the scores.
8. Compute the second column of the scores matrix: $\mathbf{V}_2 = \mathbf{R} \tilde{\phi}_2$.
9. Repeat steps 5-8 until all columns of the scores matrix have been obtained.

Let's implement this in **R**:

```
y <- cars$HwyMPG # y need not be scaled. It doesn't matter.
X_s <- scale(as.matrix(X)) # Scale X
R <- X_s # Initialize R matrix
# Initialize scores matrix
Scores <- matrix(rep(0, nrow(X_s)*ncol(X_s)), ncol = ncol(X_s))

for(i in 1:6) {
  phi <- t(R) %*% y # Find phi. First time through, R = X_s
  phi <- phi/sqrt(sum(phi^2)) # Normalize phi
  v <- R %*% phi # Get column of scores
  Scores[,i] <- v # Update scores matrix
  for(j in 1:ncol(X_s)) {
    R[,j] <- lm(R[,j] ~ v)$residuals # Update R matrix
  }
}
# Get projections
Proj <- solve(t(X_s) %*% X_s) %*% t(X_s) %*% Scores
# Get loadings
P <- t(solve(Proj))
```



```
round(Proj, 3)
      [,1] [,2] [,3] [,4] [,5] [,6]
FuelCap -0.519 -0.382 0.501 0.787 0.049 -0.102
Weight  -0.600 -0.679 -0.366 -0.793 -0.031 0.032
FrontWt   0.291 0.179 0.810 -0.144 0.004 -0.028
Accel0_30 0.281 -0.400 0.006 0.182 -0.155 0.545
Accel0_60 0.319 -0.389 -0.198 -0.114 -0.619 -1.475
QtrMile   0.324 -0.373 -0.137 0.022 0.775 0.920
```

Note: typically the algorithm used in software finds the loadings first and then inverts the transpose of that matrix to give the projections. More info on the algorithm can be found in the [“pls” package documentation](#).

I implemented their method in the notes in the Quarto file for these notes, but I will not show it here since the one I showed above makes more sense (to me).

PLS Surrogates

One weakness of PLS and PCA is that the components that are created are difficult to interpret in terms of the original variables. However, we can use the projection (or rotation) values to come up with combinations of our variables that are similar to the way the components are defined. These new combinations are called **surrogates** and they are often much more interpretable than the components themselves. When we define the surrogates, we ignore the variables with small weights (in magnitude). We don't want to ignore any with a projection value around 0.3 or higher unless others are quite a bit larger. Let's think about what they would be in this case:

Creating these surrogates in **R**:

```
X_s <- scale(as.matrix(X)) # need to scale here if we did in the plsr() function
surrogate1 <- (X_s[,1] + X_s[,2])/2 - (X_s[,3] + X_s[,4] + X_s[,5] + X_s[,6])/4
surrogate2 <- (X_s[,1] + X_s[,2] + X_s[,4] + X_s[,5] + X_s[,6])/5
surrogate3 <- (X_s[,1] + X_s[,3])/2
surrogate4 <- X_s[,1] - X_s[,2]
surrogate5 <- X_s[,6] - X_s[,5]
surrogate6 <- (X_s[,4] + X_s[,6])/2 - X_s[,5]
```

We can then use these in the model to predict values of Y , as we will soon.

Cross Validation in PLS

The `plsr()` function has an option built in for cross validation to pick the number of components. The options are “none”, “CV” for 10-fold cross validation (by default, using the `segments` option can change how many folds are used), and “LOO” for leave-one-out cross validation.

The “adjCV” is a biased-corrected CV estimate.

```
set.seed(2024)
cars_pls_cv <- plsr(HwyMPG ~ FuelCap + Weight + FrontWt +
  Accel0_30 + Accel0_60 + QtrMile,
  data = cars, scale = TRUE, validation = "CV")
summary(cars_pls_cv)
```

```
Data:   X dimension: 73 6
      Y dimension: 73 1
Fit method: kernelpls
Number of components considered: 6
```

VALIDATION: RMSEP

Cross-validated using 10 random segments.

	(Intercept)	1 comps	2 comps	3 comps	4 comps	5 comps
CV	3.085	2.180	1.895	1.841	1.799	1.848
adjCV	3.085	2.174	1.889	1.834	1.791	1.832
		6 comps				
CV		1.831				
adjCV		1.819				

TRAINING: % variance explained

	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps
X	60.26	82.64	95.66	98.53	98.91	100.00
HwyMPG	53.42	66.40	68.78	71.22	71.74	71.87

Cross Validation in PLS with Caret

The `caret` package also has `pls` and `pcr` methods for partial least squares and PCA, respectively.

```
set.seed(2026)
cars_pls_train <- train(
  HwyMPG ~ FuelCap + Weight + FrontWt + Accel0_30 + Accel0_60 + QtrMile,
  data = cars, method = "pls", scale = T, tuneGrid = data.frame(ncomp = 1:6),
  trControl = trainControl(method = "cv", number = 10)
)

summary(cars_pls_train$finalModel) # It found 4 componenets was best
```

Data: X dimension: 73 6
Y dimension: 73 1
Fit method: oscorespls
Number of components considered: 4
TRAINING: % variance explained

	1 comps	2 comps	3 comps	4 comps
X	60.26	82.64	95.66	98.53
.outcome	53.42	66.40	68.78	71.22

Fitting Final PLS Model

```
car_surrogate_mod <- lm(
  HwyMPG ~ surrogate1 + surrogate2 + surrogate3 + surrogate4, data = cars
)
car::vif(car_surrogate_mod)
```

surrogate1	surrogate2	surrogate3	surrogate4
1.423053	1.673523	1.736003	1.068927

```
summary(car_surrogate_mod)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	28.9726	0.1983	146.103	< 2e-16 ***
surrogate1	-1.7875	0.1528	-11.699	< 2e-16 ***
surrogate2	-2.4234	0.4767	-5.084	3.11e-06 ***
surrogate3	0.8460	0.4053	2.087	0.04060 *
surrogate4	1.1692	0.4205	2.780	0.00702 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.694 on 68 degrees of freedom

Multiple R-squared: 0.7112, Adjusted R-squared: 0.6942

F-statistic: 41.87 on 4 and 68 DF, p-value: < 2.2e-16

```
# Compare to model using actual pls components. They should be similar.
pls_df <- data.frame(comp1 = cars_pls$scores[,1], comp2 = cars_pls$scores[,2],
                     comp3 = cars_pls$scores[,3], comp4 = cars_pls$scores[,4],
                     HwyMPG = cars$HwyMPG)
car_pls_mod <- lm(HwyMPG ~ comp1 + comp2 + comp3 + comp4, data = pls_df)
car::vif(car_pls_mod)
comp1 comp2 comp3 comp4
      1      1      1      1
```

```
summary(car_pls_mod)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	28.9726	0.1980	146.352	< 2e-16 ***
comp1	1.2297	0.1095	11.234	< 2e-16 ***
comp2	0.9949	0.1797	5.538	5.34e-07 ***
comp3	0.6072	0.2558	2.374	0.0204 *
comp4	1.1586	0.4828	2.400	0.0192 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.691 on 68 degrees of freedom

Multiple R-squared: 0.7122, Adjusted R-squared: 0.6953

F-statistic: 42.07 on 4 and 68 DF, p-value: < 2.2e-16

```
summary(car_surrogate_mod) # Repeat output from surrogate model
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	28.9726	0.1983	146.103	< 2e-16 ***
surrogate1	-1.7875	0.1528	-11.699	< 2e-16 ***
surrogate2	-2.4234	0.4767	-5.084	3.11e-06 ***
surrogate3	0.8460	0.4053	2.087	0.04060 *
surrogate4	1.1692	0.4205	2.780	0.00702 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.694 on 68 degrees of freedom

Multiple R-squared: 0.7112, Adjusted R-squared: 0.6942

F-statistic: 41.87 on 4 and 68 DF, p-value: < 2.2e-16

UMAP

Another common dimension reduction technique is Uniform Manifold Approximation and Projection, more commonly known as UMAP. UMAP, unlike PCA and PLS, is a non-linear method that excels at preserving both local clusters and global relationships.

With PCA, we take a p -dimensional space and project it down to k -dimensions. It is kind of like looking at a shadow or “snapshot” of the data space from a certain perspective. This works well in some cases, but not well in others.

On the other hand, UMAP works by thinking of the data as not just being in p -dimensional space, but as sitting on a manifold: a smooth, curved surface. This is an algebraic concept. On manifolds, distances change based on where a point is. UMAP makes two mathematical assumptions:

- The manifold is locally connected.
- The distance metric is approximately locally constant.

The goal is to find a low-dimensional representation of this manifold with a similar topological structure. This is much better in cases where variables are related in a non-linear way and it’s main goal is for visualization, not modeling. How it does this is quite complicated and requires knowledge of abstract algebra and topology, so the details are omitted. But what we want to know is that there are two main parameters that affect the UMAP behavior: the number of neighbors and the minimum distance.

- The **number of neighbors** tells the UMAP how many nearby points to look at when trying to learn the structure of the data. The more neighbors, the more global the structure will be.
 - When the number of neighbors is small, UMAP focuses on very local, fine-grained patterns. You will see many small, tight clusters, but may lose the global structure of the data.
 - When the number of neighbors is large, UMAP looks at the “big picture” and tries to preserve the broad, global relationships between distant parts of the data. However, some nuances in the data can be lost.
- The **minimum distance** affects how fat apart the points are allowed to be from each other in the final plot.
 - A small minimum distance allows the points to clump together very tightly.
 - A larger minimum distance prevents points from overlapping and forces the clusters to spread out.

One more thing to note about UMAP is that the algorithm is stochastic (has random elements). That means the **UMAP is NOT deterministic**: it leads to slightly different results when running it again unless you specify a random seed or random state when running the algorithm.

Let’s check out an example of UMAP involving a woolly mammoth: [link here!](#)

UMAP in R

We can compute a UMAP in **R** using the `umap()` function in the `umap` package. Let’s consider a classic example with the built-in `iris` dataset to see if the different species separate nicely. The default values for the number of neighbors, `n_neighbors`, is 15 and the minimum distance, `min_dist`, is 0.1. These can be checked by typing `umap.defaults` once the `umap` package is loaded.

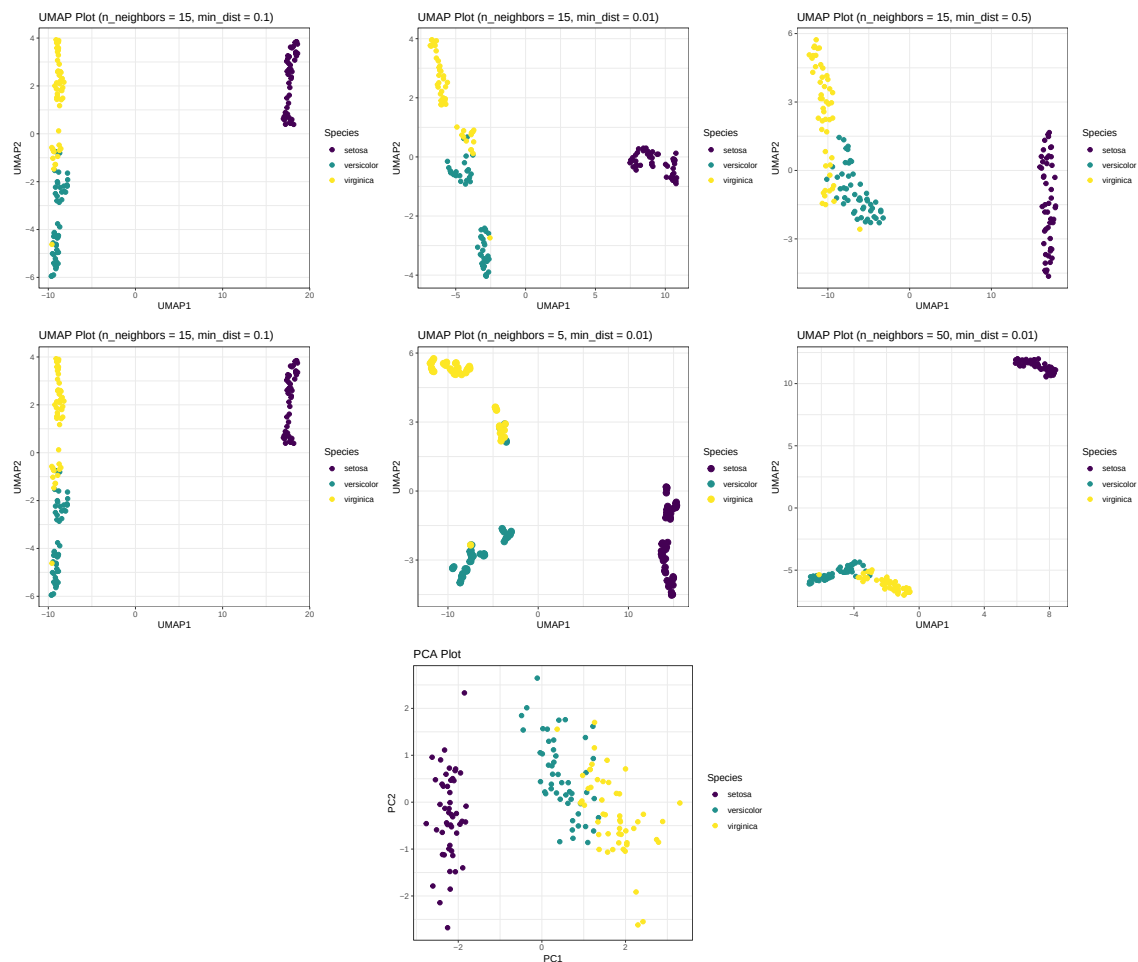
```
library(umap)
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
iris_umap <- umap(iris[,1:4], n_components = 2, random_state = 1,
                  n_neighbors = 15, min_dist = 0.1) # default values
iris_pca <- prcomp(iris[,1:4], scale = TRUE)$x
```

The main component of the object created by the `umap()` function is “layout”, which holds a matrix with coordinates. We can plot the 2D projection using `ggplot`. I wrote a function to do this called `umap_plot()` that is in the `math3190package`.

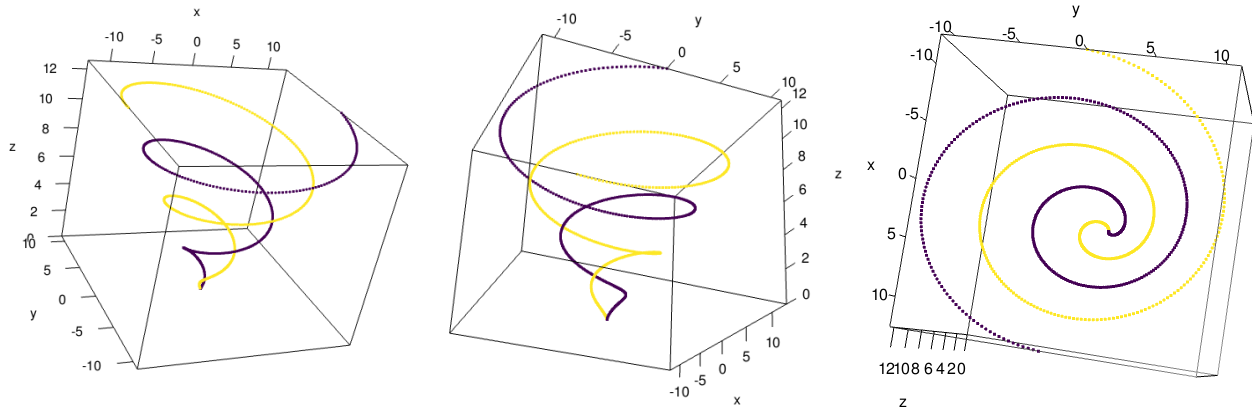
```
umap_plot(iris_umap, data = iris, color_umap = "Species", size = 2) +
  scale_colour_viridis_d() +
  labs(title = "UMAP Plot (n_neighbors = 15, min_dist = 0.1)")
```



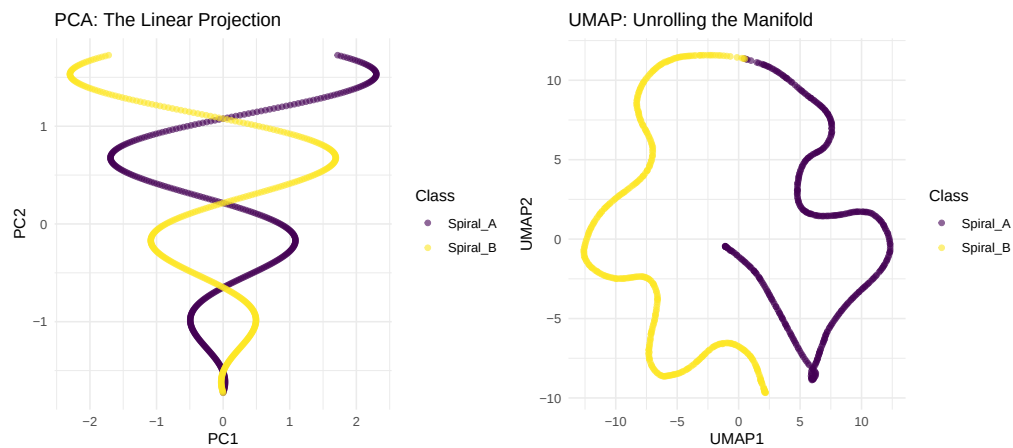
UMAP 3D Example vs PCA

Let's do one more example to highlight the strength of UMAP: non-linear associations, especially with other noise in the data.

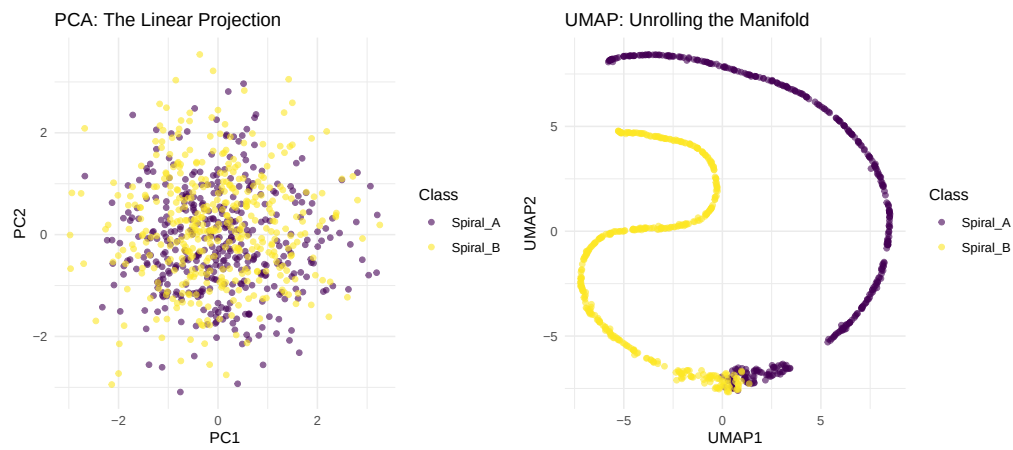
Let's start with a nice clear 3D image of two spirals, shown from different angles below.



Let's do both a PCA and UMAP projection of this 3D image into 2D.



Now suppose instead of this nice 3D image, we had a 10-dimensional data space with 7 dimensions of noise (unrelated variables). When projecting that down to 2D, PCA can't find any signal. It just looks like a mess. However, the UMAP still finds good separation.



Notes 9: Bayesian Methods

Table of contents

Intro	163
Conditional Probability and Bayes' Theorem	164
Bayes' Theorem	165
Probability Distributions in Bayesian Framework	168
MCMC Sampling	173
Gibbs Sampling	173
Metropolis Sampling	175
MCMC Diagnostics	179
Metropolis Sampling of Two Parameters	181
Regularization in the Bayesian Framework	183

Intro

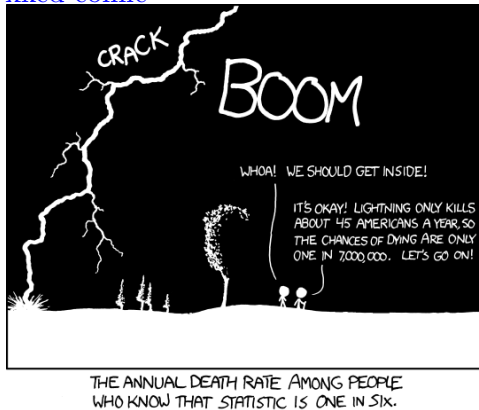
All of the statistical inference you have learned up to this point has been in the **frequentist** framework. In this view, all parameters are fixed numbers and we use methods to try to determine what those numbers are. We do not associate probabilities with parameters, but we can establish the probability that our inferential procedures are correct. All of the hypothesis testing (even the nonparametric tests) and confidence intervals we have dealt with are frequentist inference methods.

However, there is an entire other paradigm that many statisticians have adopted known as the **Bayesian** framework, named after Thomas Bayes who lived in the 18th century and the theorem he developed. In Bayesian statistics, parameters are viewed as random variables that have probability distributions. If we know that the probability distribution is for a parameter, we can simply obtain random samples from that distribution to perform inference. Bayesian statistics is becoming increasingly popular and is especially useful in uncertainty quantification in statistics and other fields.

To get an idea of how Bayesian statistics works, let's first discuss conditional probability and Bayes' Theorem.

Conditional Probability and Bayes' Theorem

[xkcd comic](#)



As suggested in the comic to the left, probabilities can change based on the condition that another event has occurred. For example, what is the probability that a randomly chosen man is over 6 feet tall? About 0.14. What is that same probability given that you are selecting from an NBA basketball team? Almost 1. The probability changes given the fact that you are selecting from a basketball team.

Definition: If A & B are any events with $P(B) > 0$, then the conditional

probability of A given that B occurred is:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{"prob of } A \text{ given } B\text{"}$$

Example: Roll fair die 3 times ($6^3 = 216$ total outcomes in S).

Let A = event that sum is 4 or less, so $A =$

Let B = event that first roll is even. What is $P(A|B)$?

- B can be viewed as a restricted sample space (i.e.: we know the outcome is in B , now what is prob. it is also in A ?).

Bayes' Theorem

Useful Result: For any events A and B (where $0 < P(B) \leq 1$):

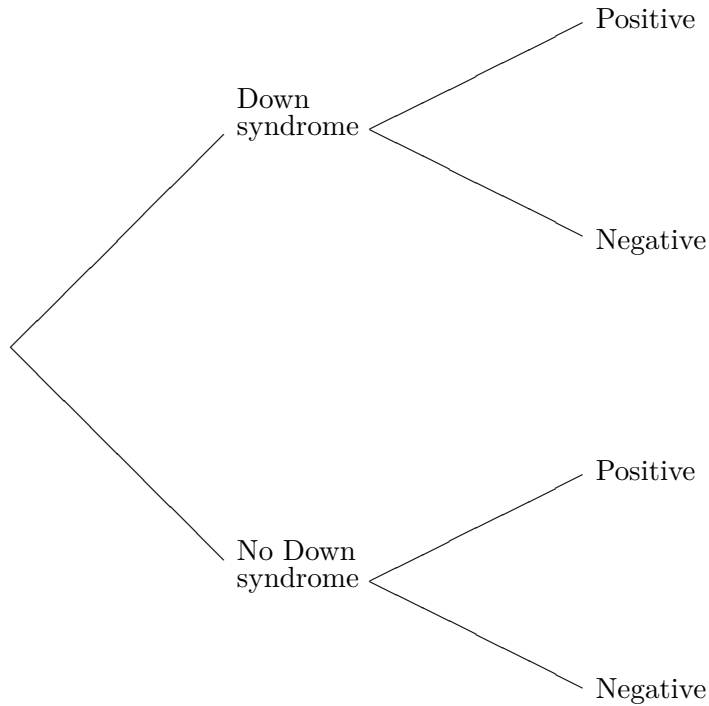
$$\begin{aligned} P(B) &= P((A \cap B) \cup (A^c \cap B)) = P(A \cap B) + P(A^c \cap B) \\ &= \underline{P(B|A)P(A) + P(B|A^c)P(A^c)} \quad (\text{using the definition of conditional prob.}). \end{aligned}$$

Example: (False positives): For pregnant women over age 35, doctors often recommend a test for Down syndrome early in the second trimester. This is due to a higher risk of having a child with Down syndrome. One test screens for the presence of alpha fetoprotein (AFP) in the mother's blood. This test is somewhat controversial because of its high **false positive** rate.

- When this test is given to a mother of age 36, the probability that:
 - the test is positive when there is a baby with Down syndrome is 0.999,
 - the test is positive when there is not a baby with Down syndrome is 0.05,
 - the test is negative when there is a Down syndrome baby is
 - the test is negative when there is not a Down syndrome baby is

Problem: some women are told they have a Down syndrome baby when they don't (false positive).

- Suppose 1 of every 400 babies born to mothers over the age of 35 have Down syndrome.
- Given that the AFP test is positive, what is the probability a randomly chosen mother of this age has a Down syndrome baby?
- Let:
- Know:
- Want:
- So there is a _____ chance of having a Down syndrome baby when the mother tests positive!
- What happened here? With only 1 in 400 having Down syndrome, in every 400 babies, we expect: 1 to have Down syndrome & nearly 20 to test positive w/o Down syndrome (5%).
- Your chances of being in the false positive group are much higher.
- Another way to solve this problem is with a tree diagram:



General Useful Result: Let B be any event in the sample space S and let $A_1, \dots, A_n$ form a partition (disjoint) of S , with $P(A_i) > 0$. Then $B = (B \cap A_1) \cup (B \cap A_2) \cup \dots \cup (B \cap A_n)$, a disjoint union since F_i 's are disjoint.

$$\begin{aligned} \text{So: } \underline{P(B)} &= P\left(\bigcup_{i=1}^n B \cap A_i\right) = \sum_{i=1}^n P(B \cap A_i) \\ &= \underline{\sum_{i=1}^n P(B|A_i)P(A_i)}. \end{aligned}$$

Bayes Theorem (in discrete case): Let $A_1, \dots, A_n$ partition S , so that $P(A_i) > 0$ for $i = 1, \dots, n$ and let B be any event such that $P(B) > 0$. Then for each $i = 1, \dots, n$:

$$P(A_i|B) = \frac{P(A_i \cap B)}{P(B)} = \frac{P(B|A_i)P(A_i)}{P(B)} = \frac{P(B|A_i)P(A_i)}{\sum_{j=1}^n P(B|A_j)P(A_j)}.$$

Example: A hospital receives 40% of its flu vaccine from Company A, 50% from Company B, and the rest from Company C. From Company A, 3% of its vaccine doses are ineffective, from Company B, 2% are ineffective, and from company C, 4% are ineffective. Given a randomly chosen dose is ineffective, what is the probability it came from Company B? Let A, B, C be the events the dose came from Company A, B, or C, respectively and let I be the event the dose is ineffective.

What is $P(A|I) + P(B|I) + P(C|I)$?

Probability Distributions in Bayesian Framework

The Bayes' Theorem we saw earlier was the theorem in the case where we have discrete variables that can be partitioned into different events. Bayes' Theorem also works with continuous probability distributions. In this case, we replace the summation with an integral:

Bayes' Theorem (in continuous case): The distribution for some parameters θ when given data $\mathbf{x}$ is

$$p(\theta|\mathbf{x}) = \frac{p(\theta)p(\mathbf{x}|\theta)}{p(\mathbf{x})} = \frac{p(\theta)p(\mathbf{x}|\theta)}{\int p(\theta)p(\mathbf{x}|\theta)d\theta} \propto p(\theta)p(\mathbf{x}|\theta)$$

- $p(\theta)$ is known as the prior distribution of θ , or just the prior.
- $p(\theta|\mathbf{x})$ is known as the posterior distribution of θ given $\mathbf{x}$, or just the posterior.
- $p(\mathbf{x}|\theta)$ is known as the likelihood, which relates all variables in a probability model.
- $\mathbf{x}$ and θ are in bold to indicate they are vectors. $\mathbf{x}$ is a vector of all of the data values and θ is a vector of all the parameters.
- The symbol " $\propto$ " indicates the previous expression is proportional to the next one. That is, it is the same up to a multiplicative constant.

Common Probability Distributions for Bayesian Methods

1. **Normal distribution:** $x|\mu, \sigma \sim N(\mu, \sigma^2)$

$$p(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}, \quad -\infty < x < \infty$$

- μ is the mean and σ^2 is the variance.

2. **Exponential distribution:** $x|\lambda \sim \text{Exp}(\lambda)$

- λ is rate parameter. $p(x|\lambda) = \lambda e^{-\lambda x}, \quad x > 0$
- The mean is $1/\lambda$ and the variance is $1/\lambda^2$.

3. **Gamma distribution:** $x|\alpha, \beta \sim \text{Gam}(\alpha, \beta)$

$$p(x|\alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \quad x > 0$$

- α is the shape parameter and β is rate parameter.
- The mean is α/β and the variance is α/β^2 .

4. **Uniform distribution:** $x|a, b \sim \text{Unif}(a, b)$

$$p(x|a, b) = \frac{1}{b-a}, \quad a < x < b$$

- The mean is $(a+b)/2$ and the variance is $(b-a)^2/12$.

Example: Suppose we have 5 data values $\mathbf{x} = (1, 5, 3, 2, 4)$ that each come from a normal distribution with a variance of 2.5 and an unknown mean, μ : $x_i \sim N(\mu, 2.5)$.

- In the frequentist framework, we can construct a confidence interval for μ using $\bar{x} \pm z_{\alpha/2} \cdot \sigma / \sqrt{n} = 3 \pm 1.96 \cdot \sqrt{2.5/5} \implies 1.614 < \mu < 4.386$.
- In the Bayesian framework, we want to obtain the **posterior distribution** of μ , which we write as $p(\mu|\mathbf{x}, \sigma^2)$, and then sample from that. To do that, we will first need a **prior distribution** on μ . Let's just assign a uniform prior: $\mu \sim \text{Unif}(-1000, 1000)$ so $p(\mu) = 1/2000$. This is equivalent to saying we have no information about μ (it could be anywhere between -1000 and 1000).

Now, let's obtain the **likelihood function**: $p(\mathbf{x}|\mu, \sigma^2)$. Since we know $x_i \sim N(\mu, 2.5)$, we have $p(x_i|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{1}{2\sigma^2}(x_i - \mu)^2\right\}$. Then

$$\begin{aligned} p(\mathbf{x}|\mu, \sigma^2) &\propto p(x_1|\mu, \sigma) \times \cdots \times p(x_5|\mu, \sigma) \\ &= \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{1}{2\sigma^2}(x_1 - \mu)^2\right\} \times \cdots \times \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{1}{2\sigma^2}(x_5 - \mu)^2\right\} \\ &= \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^5 \exp\left\{-\frac{1}{2\sigma^2}(x_1 - \mu)^2 - \frac{1}{2\sigma^2}(x_2 - \mu)^2 - \cdots - \frac{1}{2\sigma^2}(x_5 - \mu)^2\right\} \\ &= \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^5 \exp\left\{-\frac{1}{2\sigma^2} \sum_{i=1}^5 (x_i - \mu)^2\right\}. \end{aligned}$$

Now we can obtain the posterior by multiplying the prior and likelihood functions:

$$\begin{aligned} p(\mu|\mathbf{x}, \sigma^2) &\propto p(\mu)p(\mathbf{x}|\mu, \sigma^2) \\ &\propto \frac{1}{2000} \cdot \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^5 \exp\left\{-\frac{1}{2\sigma^2} \sum_{i=1}^5 (x_i - \mu)^2\right\} \\ &\propto \end{aligned}$$

- Once we have the posterior distribution for μ , which is $N(\bar{x}, \sigma^2/n)$, we can sample from that distribution. These samples are all different values of μ .

```
set.seed(2026)
x <- c(1,5,3,2,4)
mu <- rnorm(n = 500000, mean = mean(x), sd = sqrt(2.5/5))
quantile(mu, c(0.025, 0.975))
      2.5%      97.5%
1.612910 4.385999
```

- Taking the 2.5th and 97.5th percentiles gives us a 95% **credible interval** for the population mean, μ . A credible interval is an interval within which an unobserved parameter value falls with a particular probability.
 - In this case, there is a 95% chance that the population mean, μ , lies between 1.613 and 4.386.
 - Compare this to the confidence interval on the previous page.
- What if we used a different prior distribution instead. For example, if we had reason to believe that the true mean was around 6 (and we got a bit unlucky with our sample). We could assign a prior $\mu \sim N(6, \sigma_0^2)$. We will change the value of σ_0^2 to see what affect that has. σ_0^2 is called a **hyperparameter** since it is a parameter in the prior. In this case,

$$p(\mu) = \frac{1}{\sqrt{2\pi\sigma_0^2}} \exp \left\{ -\frac{1}{2\sigma_0^2} (\mu - 6)^2 \right\}$$

Our posterior is

$$\begin{aligned} p(\mu|\mathbf{x}, \sigma^2) &\propto p(\mu)p(\mathbf{x}|\mu, \sigma^2) \\ &\propto \frac{1}{\sqrt{2\pi\sigma_0^2}} \exp \left\{ -\frac{1}{2\sigma_0^2} (\mu - 6)^2 \right\} \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right)^5 \exp \left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^5 (x_i - \mu)^2 \right\} \\ &\propto \exp \left\{ -\frac{1}{2\sigma_0^2} (\mu - 6)^2 - \frac{1}{2\sigma^2} \sum_{i=1}^5 (x_i - \mu)^2 \right\} \\ &\propto \exp \left\{ -\frac{1}{2} \left(\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2} \right) \left(\mu - \frac{1}{\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}} \left[\frac{6}{\sigma_0^2} + \frac{n\bar{x}}{\sigma^2} \right] \right)^2 \right\} \end{aligned}$$

So, the posterior is again normal. In particular, $\mu|\mathbf{x}, \sigma^2 \sim N \left(\frac{1}{\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}} \left[\frac{6}{\sigma_0^2} + \frac{n\bar{x}}{\sigma^2} \right], \frac{1}{\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}} \right)$. See the results when $\sigma_0^2 = 1$ and when $\sigma_0^2 = 10$:

```
sigma_sq0 <- 1
mean_mu <- 1/(1/sigma_sq0 + 5/2.5) * (6/sigma_sq0 + 5*mean(x)/2.5)
sigma_mu <- sqrt(1/(1/sigma_sq0 + 5/2.5))
mu <- rnorm(500000, mean_mu, sigma_mu)
quantile(mu, c(0.025, 0.975))
      2.5%      97.5%
2.869030 5.129936
```



```

sigma_sq0 <- 10
mean_mu <- 1/(1/sigma_sq0 + 5/2.5) * (6/sigma_sq0 + 5*mean(x)/2.5)
sigma_mu <- sqrt(1/(1/sigma_sq0 + 5/2.5))
mu <- rnorm(500000, mean_mu, sigma_mu)
quantile(mu, c(0.025, 0.975))
      2.5%      97.5%
1.792919 4.492821

```

- The second prior in this example is known as a **conjugate prior** since the prior and posterior were both the same distribution (both were normal). Some common continuous conjugate priors are listed in the table below:

Likelihood	Model Parameter	Prior	Posterior
Normal with σ^2 known	μ	Normal with mean μ_0 and variance σ_0^2	Normal with mean $\frac{1}{\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}} \left[\frac{\mu_0}{\sigma_0^2} + \frac{n\bar{x}}{\sigma^2} \right]$ and variance $\frac{1}{\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}}$
Normal with μ known	$\tau = 1/\sigma^2$ (precision)	Gamma with shape α_0 and rate β_0	Gamma with shape $\alpha_0 + n/2$ and rate $\beta_0 + \frac{\sum_{i=1}^n (x_i - \mu)^2}{2}$
Exponential	λ	Gamma with shape α_0 and rate β_0	Gamma with shape $\alpha_0 + n$ and rate $\beta_0 + \sum_{i=1}^n x_i$
Gamma with α known	β_0	Gamma with shape α_0 and rate β_0	Gamma with shape $\alpha_0 + n\alpha$ and rate $\beta_0 + \sum_{i=1}^n x_i$

Example: Obtain samples for β when $\beta \sim \text{Gam}(1/40, 1)$ and $\mathbf{x}|\beta \sim \text{Gam}(1, \beta)$ for the data $\mathbf{x} = (18.9, 146.5, 12.4, 12.1, 80.4, 37.1, 37.0, 127.5, 4.2, 12.8, 25.9, 9.8)$

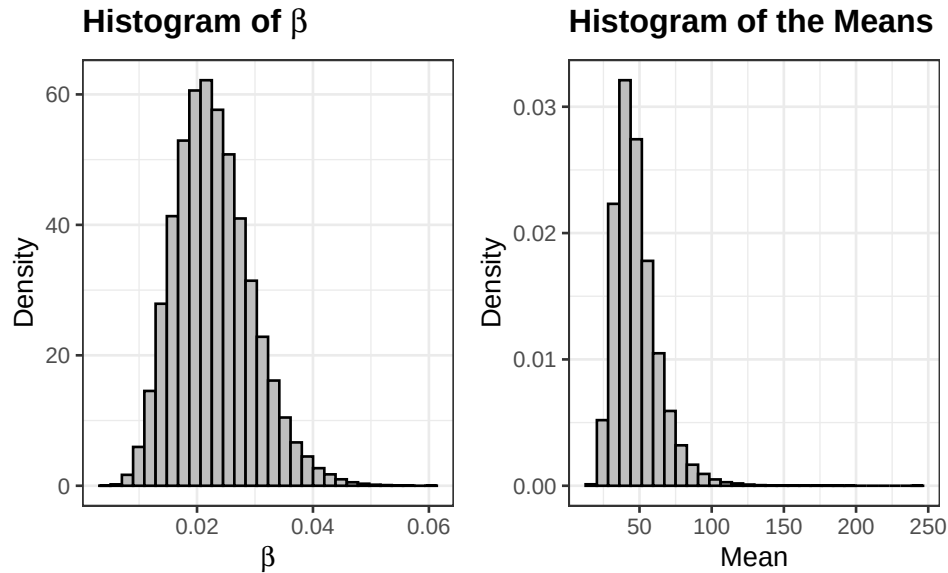
Using the conjugacy, we have that the posterior will be a gamma distribution with

```

x <- c(18.9, 146.5, 12.4, 12.1, 80.4, 37.1, 37.0, 127.5, 4.2, 12.8, 25.9, 9.8)
shape <- 1/40 + 12*1
rate <- 1 + sum(x)
beta <- rgamma(100000, shape, rate)
quantile(beta, c(0.025, 0.975))
      2.5%      97.5%
0.01180557 0.03753485

quantile(1/beta, c(0.025, 0.975))
      2.5%      97.5%
26.64191 84.70581

```



Compare that credible interval with a t -interval and a bootstrapping interval:

```
t.test(x)$conf.int
[1] 13.06527 74.36806
attr("conf.level")
[1] 0.95

set.seed(2026)
means <- c()
for(i in 1:10000) {
  index <- sample(1:length(x), length(x), replace = T)
  means[i] <- mean(x[index])
}
quantile(means, c(0.025, 0.975))
      2.5%      97.5%
20.25833 72.18479
```

Summary:

- We assign a prior distribution to the parameter we are trying to estimate.
- We assign a likelihood function to the data based on what we assume the population to look like or what distribution fits our data best.
- We obtain a function that is proportional to the posterior for our parameter by multiplying the prior and likelihood together.
- We identify what distribution has a kernel like the one we see in our posterior (thinking of our parameter now as x_i).
- We obtain lots of samples from that posterior distribution.
- We take the $\frac{\alpha}{2}th$ and $(1 - \frac{\alpha}{2})th$ quantiles to obtain the $(1 - \alpha)100\%$ credible intervals.
 - That credible interval is the Bayesian version of a confidence interval and is used for inference on the parameter.

MCMC Sampling

MCMC stands for Markov chain Monte Carlo and MCMC methods are a class of algorithms for sampling from a probability distribution. Monte Carlo sampling is the process of relying on many random samples to obtain numerical results and a **Markov chain** is a process in which the next event depends only on the previous event. We will discuss two MCMC sampling methods: Gibbs sampling and Metropolis sampling.

Gibbs Sampling

The basic Gibbs sampler is used in much the same way as we saw on the previous few pages. We sample directly from a posterior distribution to obtain parameter values. With Gibbs sampling, however, we can sample from more than one distribution at a time while updating our parameter values as we go. The steps are shown below with two parameters, θ_1 and θ_2 , but this can scale to any number of parameters.

1. Write down the posterior distributions for both θ_1 and θ_2 : $p(\theta_1|\mathbf{x}, \theta_2)$ and $p(\theta_2|\mathbf{x}, \theta_1)$.
2. Select starting values for θ_1 and θ_2 called $\theta_1^{(0)}$ and $\theta_2^{(0)}$.
3. Generate, in turn,

$$\begin{aligned}\theta_1^{(i+1)} & \text{ from } p(\theta_1|\mathbf{x}, \theta_2^{(i)}) \\ \theta_2^{(i+1)} & \text{ from } p(\theta_2|\mathbf{x}, \theta_1^{(i+1)})\end{aligned}$$

4. Increment i and return to step 3. Continue until you obtain the number of desired samples.

Example: Assume a normal likelihood function with: $x_i|\mu, \sigma^2 \sim N(\mu, \sigma^2)$. Let μ have a normal prior with $\mu \sim N(25, 100)$ and let the precision $\tau = 1/\sigma^2$ have a gamma prior with $\tau \sim \text{Gam}(5, 1/2)$. Additionally, let $\mathbf{x} = (23.3, 14.5, 19.0, 20.2, 5.4, 23.8, 21.3, 12.8, 17.6, 19.8, 11.0, 21.5, 15.7, 22.1, 21.0, 13.7, 14.9, 13.8, 17.1, 11.3)$.

In this case, $n = 20$, $\sigma_0^2 = 100$, $\mu_0 = 25$, $\alpha_0 = 5$, $\beta_0 = 1/2$, and $\bar{x} = 16.99$.

Step 1: Since these are conjugate priors, we have that

$$\mu|\mathbf{x}, \tau \sim N\left(\frac{1}{\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}} \left[\frac{\mu_0}{\sigma_0^2} + \frac{n\bar{x}}{\sigma^2}\right], \frac{1}{\frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}}\right) = N\left(\frac{1}{\frac{1}{100} + 20\tau} \left[\frac{25}{100} + 339.8\tau\right], \frac{1}{\frac{1}{100} + 20\tau}\right)$$

and

$$\tau|\mathbf{x}, \mu \sim \text{Gam}\left(\alpha_0 + n/2, \beta_0 + \frac{\sum_{i=1}^n (x_i - \mu)^2}{2}\right) = \text{Gam}\left(15, \frac{1}{2} + \frac{\sum_{i=1}^n (x_i - \mu)^2}{2}\right)$$

Step 2: The starting values usually do not matter too much. I will start each at the mean of their prior: $\mu^{(0)} = 25$ and $\tau^{(0)} = 10$.

Step 3: Generate each of the following:

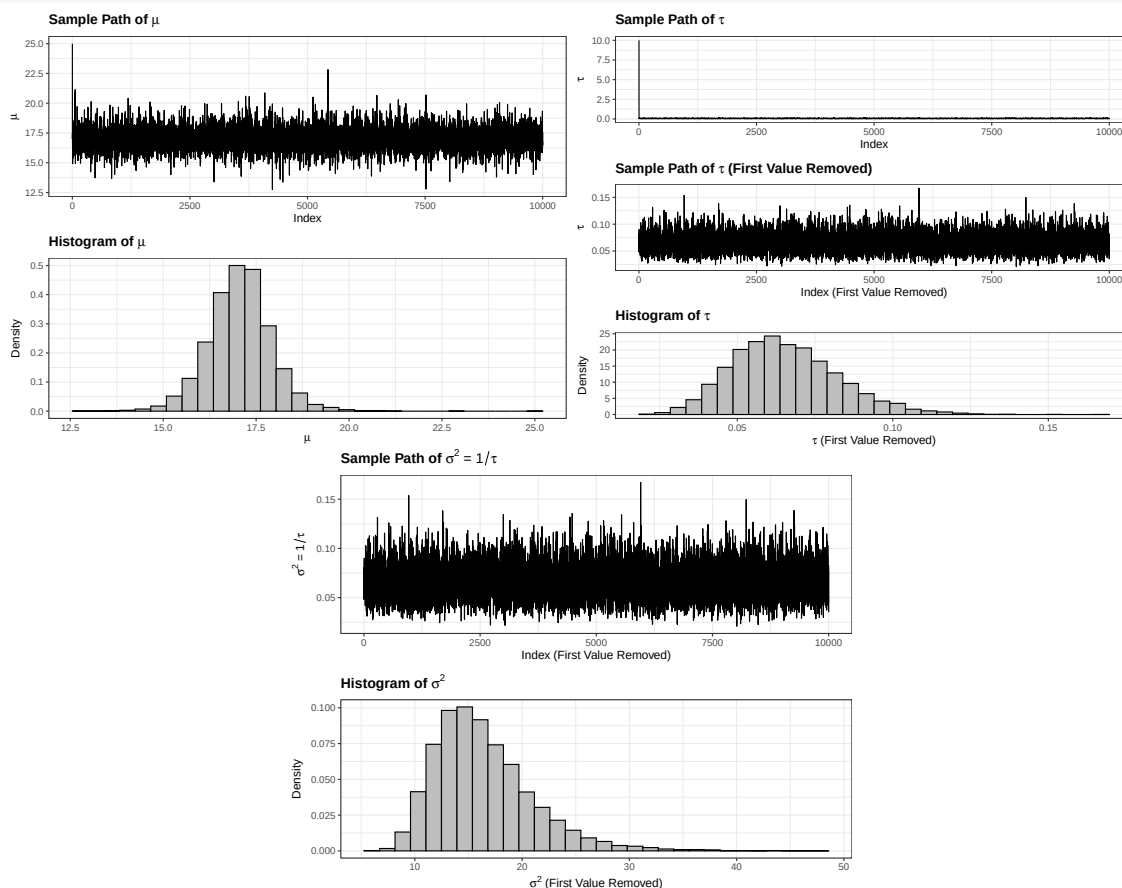
$$\mu^{(i+1)} \text{ from } N\left(\frac{1}{\frac{1}{100} + 20\tau^{(i)}} \left[\frac{25}{100} + 339.8\tau^{(i)}\right], \frac{1}{\frac{1}{100} + 20\tau^{(i)}}\right)$$

$$\tau^{(i+1)} \text{ from } \text{Gam}\left(15, \frac{1}{2} + \frac{\sum_{i=1}^n (x_i - \mu^{(i+1)})^2}{2}\right)$$

Step 4: Repeat for 10,000 samples.

The **R** code is shown below:

```
x <- c(23.3, 14.5, 19.0, 20.2, 5.4, 23.8, 21.3, 12.8, 17.6, 19.8,
      11.0, 21.5, 15.7, 22.1, 21.0, 13.7, 14.9, 13.8, 17.1, 11.3)
nsamps <- 10000
mu <- rep(0, nsamps) # Initialize the mu vector
mu[1] <- 25          # Starting value of mu
tau <- rep(0, nsamps) # Initialize the tau vector
tau[1] <- 10         # Starting value of tau
for(i in 1:(nsamps - 1)) {
  mu[i+1] <- rnorm(n = 1, mean = 1/(1/100 + 20*tau[i]) * (25/100 + 339.8*tau[i]),
                  sd = 1/(1/100 + 20*tau[i]))
  tau[i+1] = rgamma(n = 1, shape = 15, rate = 1/2 + 1/2 * sum((x - mu[i+1])^2))
}
```



Metropolis Sampling

The Gibbs sampler is the go-to option when we can write down exactly what distribution we will sample from. Having conjugate priors makes this simpler, but in other cases it still may be possible to figure out exactly what distribution the parameter will follow.

However, it is often the case that no conjugate prior will exist and the distribution the parameter follows is not a known one. If, however, we can write down the distribution up to a proportionality constant, we can still obtain samples from the unknown distribution using the Metropolis sampling method. The steps for this method are given here:

1. Write down $f(\theta|\mathbf{x})$ that is proportional to the posterior $p(\theta|\mathbf{x})$.
2. Choose a symmetric proposal distribution $g(\theta)$ that is used to obtain candidates of θ that will either be accepted or rejected as a new sample value. Using a normal distribution works well.
3. Select the starting value of θ called $\theta^{(0)}$.
4. For each iteration i , do the following:
 - (a) Generate a candidate for $\theta^{(i+1)}$ called θ^* from the proposal distribution $g(\theta^*|\theta^{(i)})$.
 - (b) Calculate an acceptance ratio $R = f(\theta^*|\mathbf{x})/f(\theta^{(i)}|\mathbf{x})$.
 - (c) Generate a uniform random number u between 0 and 1.
 - (1) If $u \leq R$, accept the candidate and let $\theta^{(i+1)} = \theta^*$.
 - (2) If $u > R$, reject the candidate and let $\theta^{(i+1)} = \theta^{(i)}$.
5. Increment i and return to step 4. Continue until you obtain the number of desired samples.

Note that step 4 (c) can be numerically unstable, so it is often the case that we will accept the candidate if $\log(u) \leq \log(f(\theta^*|\mathbf{x})) - \log(f(\theta^{(i)}|\mathbf{x}))$ instead.

Example: Let's use the grocery data from the bootstrapping section of Notes 5 again and obtain samples from the posterior distribution of the standard deviation, σ .

According to figures from 2019 surveys out of the Bureau of Labor Statistics, the average cost of groceries per year is \$4,643 per household. Assuming a shopping trip 60 times per year, that will give us a true average amount spent of \$77 per trip. So, we will assume $\mu = 77$.

Looking at the histogram of our data, we can see it is clearly not normal. Fitting a distribution to it, the gamma distribution works fairly well. In the plot shown here, $\alpha = 0.8569$ and $\beta = 0.0163$, but we will use MCMC to obtain estimates for those.



Step 1: Write down $f(\theta|\mathbf{x})$ that is proportional to the posterior $p(\theta|\mathbf{x})$.

Begin with $p(x_i|\alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x_i^{\alpha-1} e^{-\beta x_i}$. Now, let's reparametrize so we have this distribution in terms of μ and σ^2 . We know $\mu = \alpha/\beta$ and $\sigma^2 = \alpha/\beta^2$. Solving these for α and β give us:

$$\text{So } \alpha = \mu^2/\sigma^2 \text{ and } \beta = \mu/\sigma^2. \text{ Therefore, } p(x_i|\mu, \sigma) = \frac{(\mu/\sigma^2)^{\mu^2/\sigma^2}}{\Gamma(\mu^2/\sigma^2)} x_i^{\mu^2/\sigma^2-1} e^{-\mu x_i/\sigma^2}$$

$$\text{So } p(\mathbf{x}|\mu, \sigma) =$$

Now, for the prior of σ , an exponential distribution makes sense since σ cannot be negative. Let's chose a rate parameter fairly small so the variance is large. That will lead to the prior being less restricting. Let $\sigma \sim \text{Exp}(1/50)$ so $p(\sigma) = \frac{1}{50} e^{-\sigma/50}$. Therefore, the posterior is

$$\begin{aligned} p(\sigma|\mathbf{x}, \mu) &\propto \frac{1}{50} e^{-\sigma/50} \left(\frac{(\mu/\sigma^2)^{\mu^2/\sigma^2}}{\Gamma(\mu^2/\sigma^2)} \right)^n \prod_{i=1}^n x_i^{\mu^2/\sigma^2-1} \exp \left\{ -\mu/\sigma^2 \sum_{i=1}^n x_i \right\} \\ &\propto \frac{(\mu/\sigma^2)^{n\mu^2/\sigma^2}}{\Gamma(\mu^2/\sigma^2)^n} \prod_{i=1}^n x_i^{\mu^2/\sigma^2-1} \exp \left\{ -\sigma/50 - \mu/\sigma^2 \sum_{i=1}^n x_i \right\} = f(\sigma|\mathbf{x}, \mu) \end{aligned}$$

This is certainly not a nice looking or known distribution for σ and we only know it up to a proportionality constant. We can still sample from this using the Metropolis algorithm, though!

When we do the metropolis algorithm, it will be useful to use $\log(f(\sigma|\mathbf{x}, \mu))$ since $f(\sigma|\mathbf{x}, \mu)$ will often be numerically unstable:

$$\begin{aligned} \log(f(\sigma|\mathbf{x}, \mu)) &= \log \left(\frac{(\mu/\sigma^2)^{n\mu^2/\sigma^2}}{\Gamma(\mu^2/\sigma^2)^n} \prod_{i=1}^n x_i^{\mu^2/\sigma^2-1} \exp \left\{ -\sigma/50 - \mu/\sigma^2 \sum_{i=1}^n x_i \right\} \right) \\ &= \frac{n\mu^2}{\sigma^2} \log(\mu/\sigma^2) - n \log(\Gamma(\mu^2/\sigma^2)) + (\mu^2/\sigma^2 - 1) \sum_{i=1}^n \log(x_i) - \frac{\sigma}{50} - \frac{\mu}{\sigma^2} \sum_{i=1}^n x_i \end{aligned}$$

Step 2: Choose the proposal distribution. We will let the proposal distribution be normal.

Step 3: Select the starting value of σ . Let's begin at $\sigma^{(0)} = s = 64$.

Step 4: Generate next sample:

- (a) Generate a random value from the proposal. The first time through, $i = 0$. We will use the previous value of σ , $\sigma^{(i)}$ as the mean of the proposal with a standard deviation of 10. The value of 10 can be changed if it doesn't work well. $\sigma^* \sim N(\sigma^{(i)}, 10^2)$. Suppose doing this gives $\sigma^* = 70$.
- (b) Calculate an acceptance ratio $R = f(\sigma^*|\mathbf{x}, \mu)/f(\sigma^{(i)}|\mathbf{x}, \mu)$ or find $\log(R) = \log(f(\sigma^*|\mathbf{x}, \mu)) - \log(f(\sigma^{(i)}|\mathbf{x}, \mu))$.

In our case, since we are saying $\mu = 77$, and $n = 31$, we have:

$$\begin{aligned}
 f(\sigma^*|\mathbf{x}, \mu) &= \frac{(77/(\sigma^*)^2)^{31 \cdot 77^2/(\sigma^*)^2}}{\Gamma(77^2/(\sigma^*)^2)^{31}} \prod_{i=1}^{31} x_i^{77^2/(\sigma^*)^2-1} \exp \left\{ -\sigma^*/50 - 77/(\sigma^*)^2 \sum_{i=1}^{31} x_i \right\} \\
 &= \frac{(77/70^2)^{31 \cdot 77^2/70^2}}{\Gamma(77^2/70^2)^{31}} \prod_{i=1}^{31} x_i^{77^2/70^2-1} \exp \left\{ -70/50 - 77/70^2 \sum_{i=1}^{31} x_i \right\} \\
 &= 1.21888 \times 10^{-69} \text{ (crazy small number)} \\
 f(\sigma^{(i)}|\mathbf{x}, \mu) &= \frac{(77/(\sigma^{(i)})^2)^{31 \cdot 77^2/(\sigma^{(i)})^2}}{\Gamma(77^2/(\sigma^{(i)})^2)^{31}} \prod_{i=1}^{31} x_i^{77^2/(\sigma^{(i)})^2-1} \exp \left\{ -\sigma^{(i)}/50 - 77/(\sigma^{(i)})^2 \sum_{i=1}^{31} x_i \right\} \\
 &= \frac{(77/64^2)^{31 \cdot 77^2/64^2}}{\Gamma(77^2/64^2)^{31}} \prod_{i=1}^{31} x_i^{77^2/64^2-1} \exp \left\{ -64/50 - 77/64^2 \sum_{i=1}^{31} x_i \right\} \\
 &= 1.219423 \times 10^{-70} \text{ (slightly crazier small number)} \\
 R &= f(\sigma^*|\mathbf{x}, \mu)/f(\sigma^{(i)}|\mathbf{x}, \mu) = 9.995544
 \end{aligned}$$

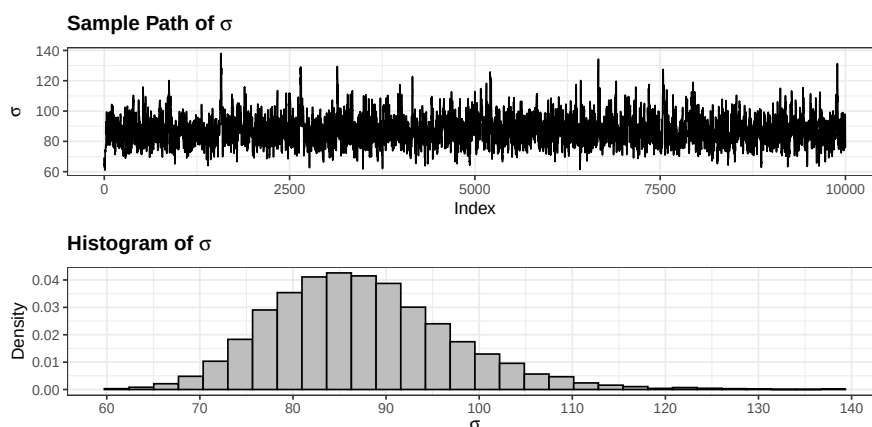
Using the $\ell(\sigma) = \log(f(\sigma|\mathbf{x}, \mu))$ instead gives us

$$\begin{aligned}
 \ell(\sigma^*) &= 31 \cdot 77^2/(\sigma^*)^2 \log(77/(\sigma^*)^2) - 31 \log(\Gamma(77^2/(\sigma^*)^2)) \\
 &\quad + (77^2/(\sigma^*)^2 - 1) \sum_{i=1}^n \log(x_i) - (\sigma^*)/50 - 77/(\sigma^*)^2 \sum_{i=1}^n x_i \\
 &= 31 \cdot 77^2/70^2 \log(77/70^2) - 31 \log(\Gamma(77^2/70^2)) \\
 &\quad + (77^2/70^2 - 1) \sum_{i=1}^n \log(x_i) - 70/50 - 77/70^2 \sum_{i=1}^n x_i \\
 &= -158.6804 \\
 \ell(\sigma^{(i)}) &= 31 \cdot 77^2/64^2 \log(77/64^2) - 31 \log(\Gamma(77^2/64^2)) \\
 &\quad + (77^2/64^2 - 1) \sum_{i=1}^n \log(x_i) - 64/50 - 77/64^2 \sum_{i=1}^n x_i \\
 &= -160.9826
 \end{aligned}$$

- (c) Now generate a random number between 0 and 1. **R** can do this with the `runif(1)` command. Doing this gave me $u = 0.4061828$. Since $u < R = 9.995544$ (or $\log(u) = -0.900952 < \ell(\sigma^*) - \ell(\sigma^{(i)}) = 2.3022$), we accept the candidate as the new sample as $\sigma^{(i+1)} = \sigma^{(1)} = \sigma^* = 70$.

Step 5: Repeat this process thousands of times.

```
set.seed(2026)
grocery <- c(1, 3, 4, 5, 5, 8, 11, 12, 15, 15, 16, 19, 21, 25, 26, 27, 30,
            35, 35, 46, 50, 55, 57, 72, 78, 85, 93, 137, 158, 212, 269)
nsamps <- 10000
sigma <- rep(0, nsamps)
sigma[1] <- 64
mu <- 77
n <- length(grocery) # 31
for(i in 1:(nsamps - 1)) {
  sigma_star <- rnorm(1, sigma[i], 10)
  logf1 <- n * mu^2 / sigma_star^2 * log(mu / sigma_star^2) -
    n * log(gamma(mu^2 / sigma_star^2)) +
    (mu^2 / sigma_star^2 - 1) * sum(log(grocery)) -
    sigma_star / 50 - mu / sigma_star^2 * sum(grocery)
  logf2 <- n * mu^2 / sigma[i]^2 * log(mu / sigma[i]^2) -
    n * log(gamma(mu^2 / sigma[i]^2)) +
    (mu^2 / sigma[i]^2 - 1) * sum(log(grocery)) -
    sigma[i] / 50 - mu / sigma[i]^2 * sum(grocery)
  if(log(runif(1)) < (logf1 - logf2)) {
    sigma[i+1] <- sigma_star
  } else {
    sigma[i+1] <- sigma[i]
  }
}
```



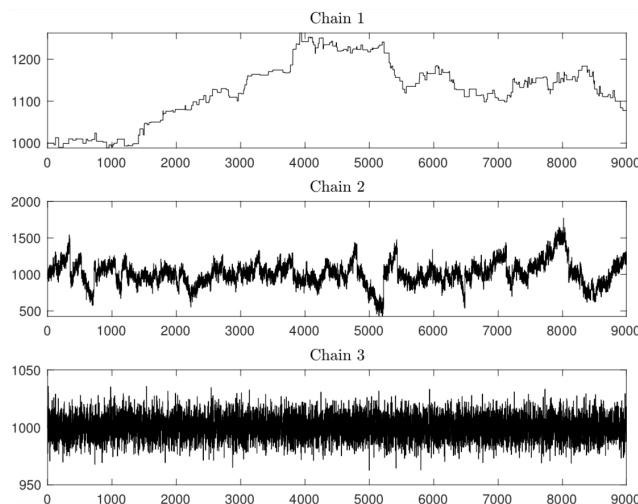
MCMC Diagnostics

There are two important diagnostics we will look at when doing MCMC sampling. Both of which have to do with how well the sampling chains mix.

1. Plots of the sampling chain.
2. The integrated autocorrelation time and effective sample size.

Plots of the sampling chain(s)

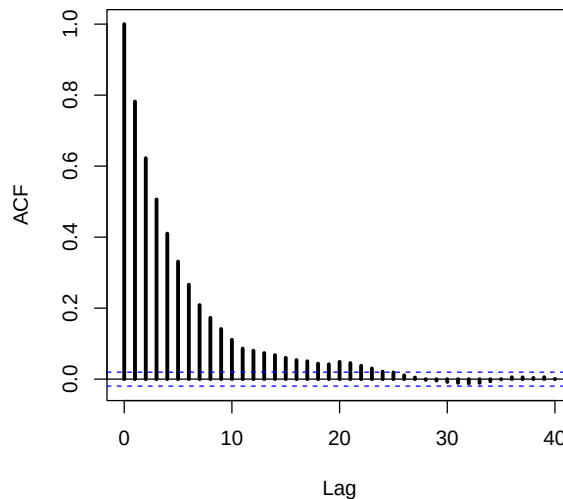
The primary diagnostic tool for evaluating MCMC sampling is observing the sampling chains. Ideally, each sample in the chain will be independent from all others. Since the posterior distributions rely on the previous iteration, that is not going to be the case, but we would like to minimize the number of previous iterations on which the current one depends. A good-looking sampling chain is one that essentially looks like noise, with no discernible patterns or paths.



Integrated Autocorrelation Time and Effective Sample Size

To check how correlated the chain is with itself, we can calculate the integrated autocorrelation time of the chain. We can do this by taking $\hat{\tau}_{\text{int}} = 1 + 2 \sum_{k=1}^{\infty} \hat{\rho}(k)$ where $\hat{\rho}(k)$ is the correlation of the chain at a lag of k . For example, the autocorrelation of the chain on the previous page is given here. The `acf()` function in **R** can calculate the autocorrelation and make this plot.

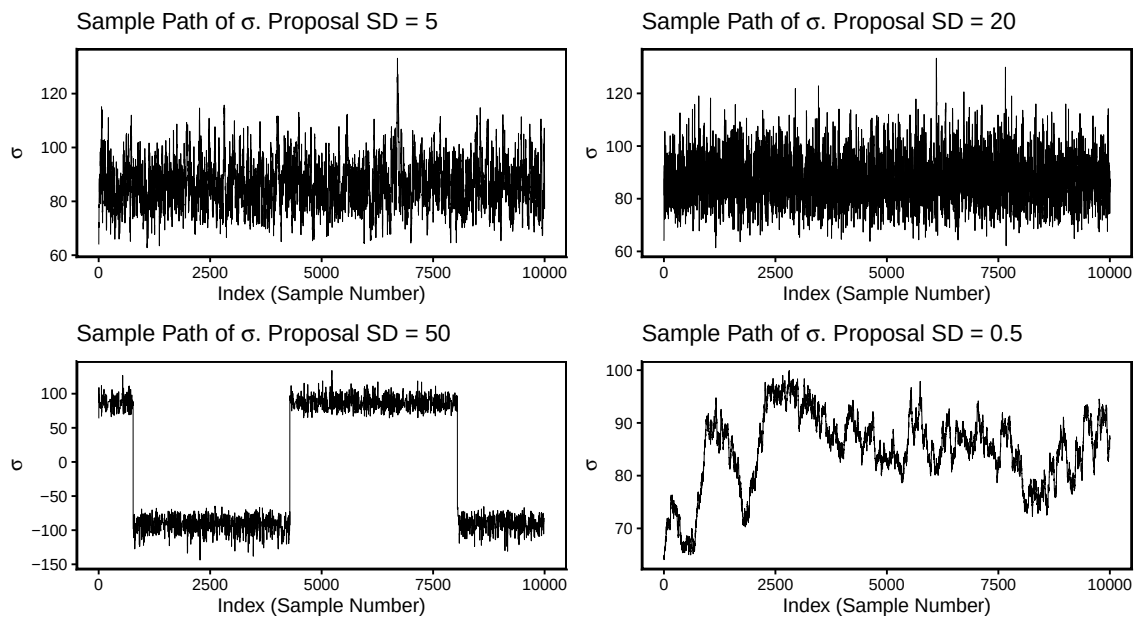
Autocorrelation of σ Chain



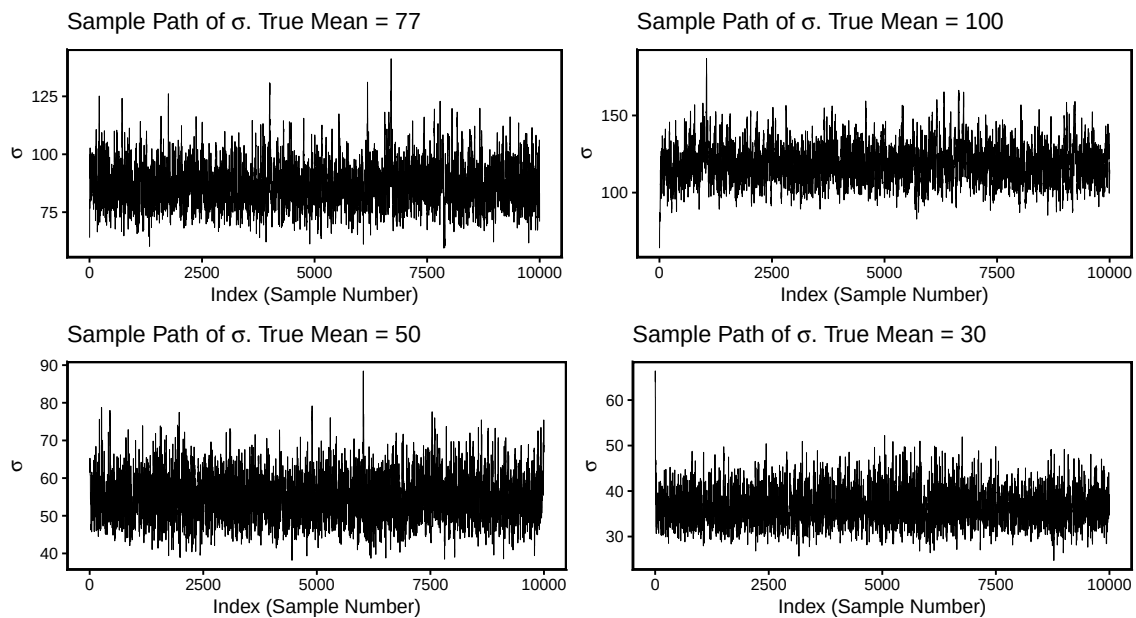
Once we have $\hat{\tau}_{\text{int}}$, we can calculate the **effective sample size** as $K_{\text{ESS}} = K/\hat{\tau}_{\text{int}}$ where K is the number of MCMC samples in the chain.

In our example, $\hat{\tau}_{\text{int}} \approx 14$, so our essential sample size is $K_{\text{ESS}} = 10000/14 \approx 714$. In order to get the equivalent of 10,000 samples, we need to set the number of MCMC samples to 140,000. We can find $\hat{\tau}_{\text{int}}$ using `1 + 2 * sum(abs(acf(sigma, lag.max = 100, plot = F)$acf))`, which gives 13.89.

Back to grocery example. What happens if the SD of the proposal distribution is changed?



What happens if the value we use for μ is changed?



Metropolis Sampling of Two Parameters

Clearly, the value of μ influences the sample values of σ a lot here. Instead of fixing μ , we can sample values of μ and σ simultaneously. Let's put a prior on μ of $\text{Exp}(1/70)$ so the mean is 70 with a large variance. Then find the posterior for both μ and σ . Compare this to the posterior at the bottom of page 176:

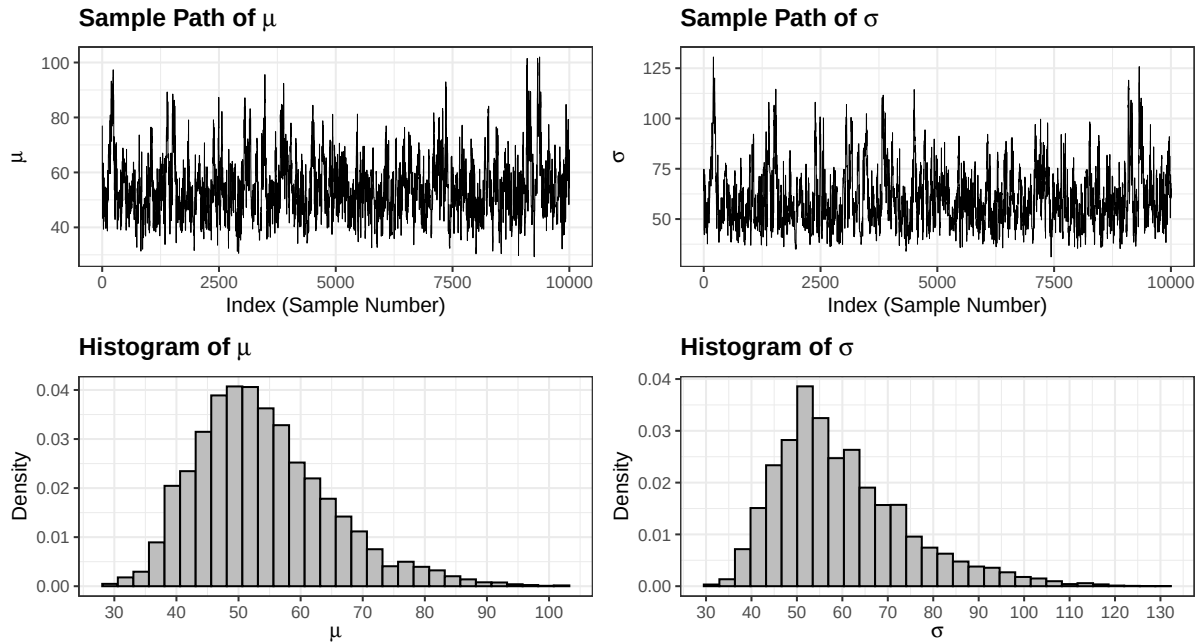
$$\begin{aligned} p(\mu, \sigma | \mathbf{x}) &\propto \frac{1}{70} e^{-\mu/50} \frac{1}{50} e^{-\sigma/50} \left(\frac{(\mu/\sigma^2)^{\mu^2/\sigma^2}}{\Gamma(\mu^2/\sigma^2)} \right)^n \prod_{i=1}^n x_i^{\mu^2/\sigma^2-1} \exp \left\{ -\mu/\sigma^2 \sum_{i=1}^n x_i \right\} \\ &\propto \frac{(\mu/\sigma^2)^{n\mu^2/\sigma^2}}{\Gamma(\mu^2/\sigma^2)^n} \prod_{i=1}^n x_i^{\mu^2/\sigma^2-1} \exp \left\{ -\mu/70 - \sigma/50 - \mu/\sigma^2 \sum_{i=1}^n x_i \right\} = f(\mu, \sigma | \mathbf{x}) \end{aligned}$$

So then

$$\log(f(\mu, \sigma | \mathbf{x})) = \frac{n\mu^2}{\sigma^2} \log(\mu/\sigma^2) - n \log(\Gamma(\mu^2/\sigma^2)) + (\mu^2/\sigma^2 - 1) \sum_{i=1}^n \log(x_i) - \frac{\mu}{70} - \frac{\sigma}{50} - \frac{\mu}{\sigma^2} \sum_{i=1}^n x_i$$

Then we can generate a candidate for both μ and σ called μ^* and σ^* and compare them to the previous values of μ and σ : $(\mu^{(i)}, \sigma^{(i)})$.

```
set.seed(2026)
nsamps <- 10000
sigma <- rep(0, nsamps)
sigma[1] <- 64
mu <- rep(0, nsamps)
mu[1] <- 77
n <- length(grocery) # 31
for(i in 1:(nsamps - 1)){
  mu_star <- rnorm(1, mu[i], 10)
  sigma_star <- rnorm(1, sigma[i], 10)
  logf1 <- n * mu_star^2 / sigma_star^2 * log(mu_star/sigma_star^2) -
    n * log(gamma(mu_star^2 / sigma_star^2)) +
    (mu_star^2 / sigma_star^2 - 1) * sum(log(grocery)) -
    mu_star / 70 - sigma_star / 50 -
    mu_star / sigma_star^2 * sum(grocery)
  logf2 <- n * mu[i]^2 / sigma[i]^2 * log(mu[i] / sigma[i]^2) -
    n * log(gamma(mu[i]^2 / sigma[i]^2)) +
    (mu[i]^2 / sigma[i]^2 - 1) * sum(log(grocery)) -
    mu[i] / 70 - sigma[i] / 50 -
    mu[i] / sigma[i]^2 * sum(grocery)
  if(log(runif(1)) < (logf1 - logf2)) {
    mu[i+1] <- mu_star
    sigma[i+1] <- sigma_star
  } else {
    mu[i+1] <- mu[i]
    sigma[i+1] <- sigma[i]
  }
}
```



In this case, $\hat{\tau}_{\text{int}} \approx 45$ for each chain. This can be obtained using

```
1 + 2 * sum(abs(acf(mu, lag.max = 100, plot = F)$acf)) # tau_int for mu
```

```
[1] 44.65691
```

```
1 + 2 * sum(abs(acf(sigma, lag.max = 100, plot = F)$acf)) # tau_int for sigma
```

```
[1] 47.65074
```

This means that those 10,000 MCMC samples are equivalent to only about $10000/45 \approx 222$ independent samples. It is usually the case that the chains are more correlated when we sample multiple parameters together.

There are lots of methods that can be used to reduce the autocorrelation in the chains like

- Changing the standard deviation of the proposal distributions.
- Changing the proposal distribution altogether.
- Using conjugate priors so Gibbs sampling can be used (Gibbs samples will usually lower autocorrelation than Metropolis samples).
- Doing individual Metropolis sampling instead of the simultaneous ones we did here.
- Use a different MCMC algorithm. There are lots of them!

Regularization in the Bayesian Framework

In Notes 7, we discussed regularization to add a bit of bias to an estimate in exchange for smaller variance. In ordinary least squares (OLS), we find the estimate for β by minimizing the sum of squared residuals. Therefore, we have

$$\begin{aligned}\hat{\beta}_{\text{OLS}} &= \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 \right\} \\ &= \arg \min_{\beta} \left\{ (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) \right\} \\ &= \arg \min_{\beta} \left\{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 \right\},\end{aligned}$$

where $\|\cdot\|$ is the 2 norm. We saw with ridge regression, the estimates for the β_i values for $i = 1, \dots, p$, can be obtained using

$$\begin{aligned}\hat{\beta}_{\text{ridge}} &= \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\} \\ &= \arg \min_{\beta} \left\{ (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) + \lambda \beta^T \beta \right\} \\ &= \arg \min_{\beta} \left\{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\}.\end{aligned}$$

In the Bayesian framework, we have a prior on β and a likelihood for $\mathbf{y}$. One of the assumptions of OLS is that $y_i|\mathbf{X}_i$ is normal and the y_i values are independent. So, we have $y_i|\mathbf{X}_i \sim N(\mathbf{X}_i\beta, \sigma^2)$ where σ^2 is the variance of y_i . For all y_i values, we have $\mathbf{y} \sim N(\mathbf{X}\beta, \sigma^2\mathbf{I})$ where $\mathbf{I}$ is the identity matrix. Written another way, we can say

$$p(y_i|\mathbf{X}_i, \beta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{1}{2\sigma^2} (y_i - \mathbf{X}_i\beta)^2 \right\} \propto \exp \left\{ -\frac{1}{2\sigma^2} (y_i - \mathbf{X}_i\beta)^2 \right\},$$

which means

$$p(\mathbf{y}|\mathbf{X}, \beta) \propto \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) \right\}.$$

Now, if we assign a normal prior for β and say $\beta|\mathbf{y}, \mathbf{X} \sim N(0, \theta^2\mathbf{I})$, then the likelihood for β can be written

$$p(\beta) \propto \exp \left\{ -\frac{1}{2\theta^2} \beta^T \beta \right\}.$$

That means the posterior will be found by multiplying:

$$\begin{aligned}p(\beta|\mathbf{y}, \mathbf{X}) &\propto p(\mathbf{y}|\mathbf{X}, \beta)p(\beta) \\ &\propto \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) \right\} \exp \left\{ -\frac{1}{2\theta^2} \beta^T \beta \right\} \\ &\propto \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) - \frac{1}{2\theta^2} \beta^T \beta \right\}.\end{aligned}$$

Then, to obtain the estimate for β , we can maximize this posterior distribution. This is known as the maximum a posteriori (MAP) estimate. Notice that the β that maximizes this posterior is

equivalent to minimizing the expression in the $\exp(\cdot)$ function without the negative signs:

$$\begin{aligned}\hat{\boldsymbol{\beta}}_{\text{ridge}} &= \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \frac{1}{2\theta^2} \boldsymbol{\beta}^T \boldsymbol{\beta} \right\} \\ &= \arg \min_{\boldsymbol{\beta}} \left\{ (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \frac{\sigma^2}{\theta^2} \boldsymbol{\beta}^T \boldsymbol{\beta} \right\} \\ &= \arg \min_{\boldsymbol{\beta}} \left\{ (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \boldsymbol{\beta}^T \boldsymbol{\beta} \right\}\end{aligned}$$

where $\lambda = \frac{\sigma^2}{\theta^2}$. Therefore, assigning a normal prior with zero mean on $\boldsymbol{\beta}$ in the Bayesian framework is equivalent to ridge regression. We would have to change the prior to what is known as the Laplacian distribution for a LASSO.

Everything presented here is just scratching the surface of Bayesian methods!

Notes 10: More Classification

Table of contents

Introduction	185
Naïve Bayes Classifier	185
Algorithm	186
Naïve Bayes Example	186
Advantages and Disadvantages of Naïve Bayes	191
Linear Discriminant Analysis	193
LDA for One Predictor	193
LDA with More than One Predictor	196
LDA with Multiple Predictors and Three Classes	197
Advantages and Disadvantages of LDA	199
Support Vector Machines	201
Support Vector Machines–Linear Data	201
A Look into SVM Methodology	202
Support Vector Machines in R	203
Support Vector Machines for Non-Linear Data	209
Advantages and Disadvantages of SVMs	215

Introduction

In this set of notes, we will explore a few more common classification techniques. The first two put some of our new Bayesian knowledge to good use and then we will explore one of the most popular machine learning techniques for classification: support vector machines.

Naïve Bayes Classifier

The naïve Bayes classifier is based on using Bayes' Theorem for assigning posterior probabilities of belonging to a certain class given the data vector. This is not a single algorithm but a family of algorithms where all of them share a common principle: every pair of features being classified is independent of each other. It is highly used in text classification where data contains high dimension (as each word represent one feature in the data). It is used in spam filtering, rating classification, and more. One big advantage of using naïve Bayes is its speed. It is fast and making prediction is easy with high dimension of data.

Assumptions:

- Independence: The observations are conditionally independent of each other, given the class label.
- Continuous variables are normally distributed: If a predictor variable is continuous, then it is assumed to be normally distributed within each class.

- Discrete variables have multinomial distributions: If a predictor variable is discrete, then it is assumed to have a multinomial distribution (a more general case of the binomial) within each class. (This can be adjusted.)
- Variables are equally important: All variables are assumed to contribute equally to the prediction of the class label.
- No missing data: The data should not contain any missing values.

These assumptions are what make this Bayesian classifier “naïve”. If these are not satisfied, more advanced versions of this classifier exist.

Algorithm

We have discussed Bayes’ theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

In this context, we replace these events with the data:

$$P(y_i = k|\mathbf{x}_i) = \frac{P(\mathbf{x}_i|y_i = k)P(y_i = k)}{P(\mathbf{x}_i)}$$

where k is a class in the data for $k = 1, \dots, K$ and $\mathbf{x}_i$ is the i th row of the model matrix or data frame. Written another way, we can say

$$P(y_i = k|x_{i1}, \dots, x_{ip}) = \frac{P(x_{i1}, \dots, x_{ip}|y_i)P(y_i = k)}{P(x_{i1}, \dots, x_{ip})}.$$

Assuming independence, we have that $P(x_{i1}, \dots, x_{ip}|y_i = k) = P(x_{i1}|y_i = k) \times \dots \times P(x_{ip}|y_i = k)$. That gives us

$$P(y_i = k|x_{i1}, \dots, x_{ip}) = \frac{P(x_{i1}|y_i = k) \times \dots \times P(x_{ip}|y_i = k)P(y_i = k)}{P(x_{i1}, \dots, x_{ip})} = \frac{P(y_i = k) \prod_{j=1}^p P(x_{ij}|y_i = k)}{P(x_{i1}, \dots, x_{ip})}.$$

Since the denominator does not have a y_i in it, we can do our proportional trick:

$$P(y_i = k|x_{i1}, \dots, x_{ip}) \propto P(y_i = k) \prod_{j=1}^p P(x_{ij}|y_i = k).$$

Naïve Bayes Example

Let’s try this out on a small example.

Y: Heart Attack	X ₁ : diastolic blood pressure	X ₂ : sex	X ₃ : age
1	92	F	71
1	97	F	64
0	66	F	53
0	83	F	75
0	82	F	69
0	91	F	63
0	77	F	71
1	90	M	57
1	86	M	82
1	96	M	73
0	69	M	86
1	92	M	65
1	84	M	78

Let's focus on the first row. We have, when assuming independence,

$$P(Y = 1|X_1 = 92, X_2 = F, X_3 = 71) \propto P(Y = 1)P(X_1 = 92|Y = 1)P(X_2 = F|Y = 1)P(X_3 = 71|Y = 1)$$

These probabilities (or densities in the case of the numeric values) are computed from the data. For the categorical probabilities:

- $P(Y = 1) = 7/13$ since 7 out of the 13 observations in the data set had a Y value of 1.
- $P(X_2 = F|Y = 1)$ is the probability of female given a heart attack, which is $2/7$ since two of the seven people that had a heart attack were female.

For the continuous probabilities, we assume they have a normal distribution. The mean and variance of this normal are going to be the sample mean and sample variance from the class we are conditioning on. Let's call this class k . We call the sample mean conditioned on that class $\hat{\mu}_k$ and we call the sample variance conditioned on that class $\hat{\sigma}_k^2$. So, we get

$$P(X_j = x_{ij}|Y = y_i) = \frac{1}{\sqrt{2\pi\hat{\sigma}_k^2}} \exp\left(-\frac{1}{2\hat{\sigma}_k^2}(x_{i1} - \hat{\mu}_k)^2\right)$$

For $P(X_1 = x_{i1}|Y = 1)$, the data values are 92, 97, 90, 86, 96, 92, and 84. The mean of those values is 91 and the variance is 23. So we have

$$P(X_1 = 92|Y = 1) = \frac{1}{\sqrt{2\pi \cdot 23}} \exp\left(-\frac{1}{2 \cdot 23}(92 - 91)^2\right) = \text{dnorm}(92, 91, \text{sqrt}(23)) = 0.08140.$$

For $P(X_3 = x_{i3}|Y = 1)$, we have the data values 71, 64, 57, 82, 73, 65, and 78. So $\hat{\mu}_k = 70$ and $\hat{\sigma}_k^2 = 74.67$. That gives

$$P(X_3 = 71|Y = 1) = \frac{1}{\sqrt{2\pi \cdot 74.67}} \exp\left(-\frac{1}{2 \cdot 74.67}(71 - 70)^2\right) = \text{dnorm}(71, 70, \text{sqrt}(74.67)) = 0.04586.$$

Putting it all together, we get

$$\begin{aligned} P(Y = 1|X_1 = 92, X_2 = F, X_3 = 71) &\propto P(Y = 1)P(X_1 = 92|Y = 1)P(X_2 = F|Y = 1)P(X_3 = 71|Y = 1) \\ &= \frac{7}{13} \times 0.08140 \times \frac{2}{7} \times 0.04586 \\ &= 0.0005743 \end{aligned}$$

Great, but that tells us almost nothing about if this person would be classified as a 0 or a 1. This is not the conditional probability of being in that class, it is only proportion to that probability. We need something to compare it to. We also need to compute $P(Y = 0|X_1 = 92, X_2 = F, X_3 = 71)$.

Data values for $X_1|Y = 0$:

So, we can say $P(Y = 1|X_1 = 92, X_2 = F, X_3 = 71) =$

And $P(Y = 0|X_1 = 92, X_2 = F, X_3 = 71) =$

Naïve Bayes in R

```
library(naivebayes) |> suppressPackageStartupMessages()
library(caret) |> suppressPackageStartupMessages()
heart <- data.frame(y = factor(c(1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1)),
                    x1 = c(92, 97, 66, 83, 82, 91, 77, 90, 86, 96, 69, 92, 84),
                    x2 = c(rep("F", 7), rep("M", 6)),
                    x3 = c(71, 64, 53, 75, 69, 63, 71, 57, 82, 73, 86, 65, 78)
                    )
nb <- naive_bayes(y ~ x1 + x2 + x3, data = heart)
head(predict(nb, type = "prob")) |> round(3)
```

	0	1
[1,]	0.248	0.752
[2,]	0.237	0.763
[3,]	1.000	0.000
[4,]	0.784	0.216
[5,]	0.842	0.158
[6,]	0.307	0.693

```
confusionMatrix(predict(nb, type = "class"), heart$y)$table
```

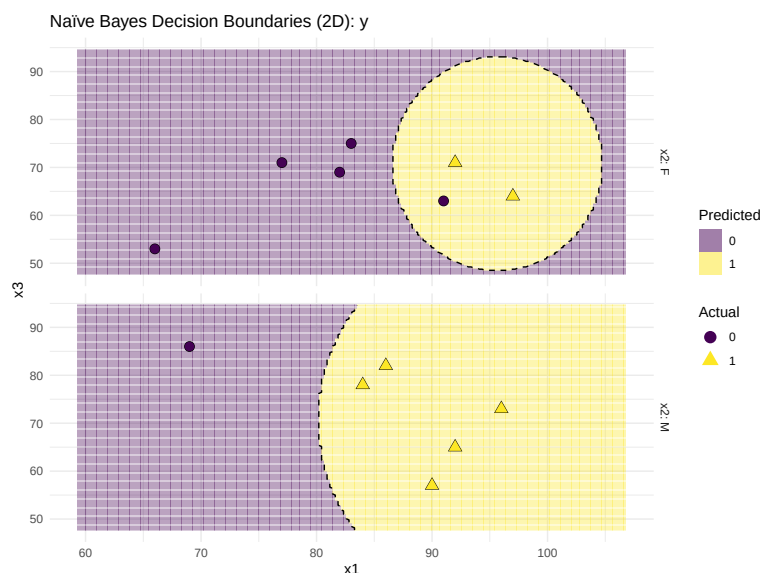
```

      Reference
Prediction 0 1
0 5 0
1 1 7

```

In the `math3190package` library, there is a function called `classifier_plot()`, which takes a classifier model (for any of the classifiers we have in this chapter) and returns a plot showing the classification.

```
library(math3190package) |> suppressPackageStartupMessages()
classifier_plot(nb, data = heart)
```



More Classes

We can extend this idea to more than 2 classes. For instance, if a `y` value of 2 indicates the person has had more than 1 heart attack:

```
heart2 <- data.frame(y = factor(c(1, 2, 0, 0, 0, 0, 0, 1, 2, 1, 0, 2, 1)),
  x1 = c(92, 97, 66, 83, 82, 91, 77, 90, 86, 96, 69, 92, 84),
  x2 = c(rep("F", 7), rep("M", 6)),
  x3 = c(71, 64, 53, 75, 69, 63, 71, 57, 82, 73, 86, 65, 78)
)
```

```
nb2 <- naive_bayes(y ~ x1 + x2 + x3, data = heart2)
```

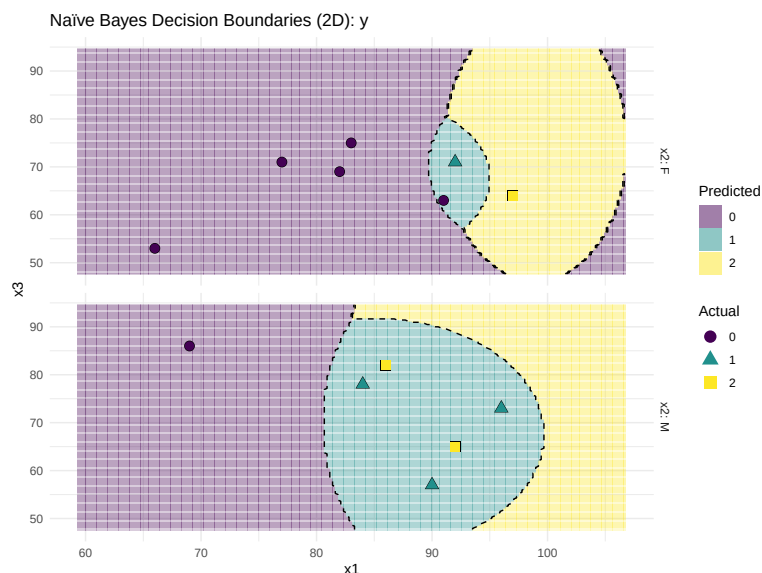
```
predict(nb2, type = "prob")[1,]
```

```

      0      1      2
0.2846944 0.3868156 0.3284900

```

```
classifier_plot(nb2, data = heart2)
```



If we are uncomfortable with the assumption that the quantitative variables are normally distributed (as we should be here), kernel density estimation can be used instead.

Kernel density estimation

Kernel density estimation (KDE) works by smoothing over a horizontal bandwidth of data values to obtain an estimation for the density function used to generate the data values.

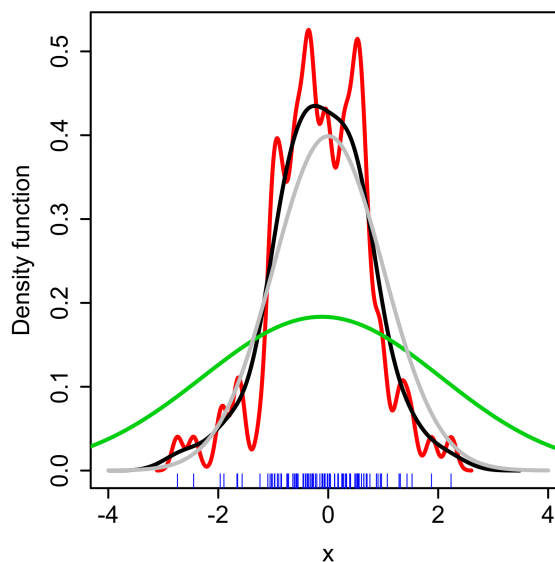
The image on the right shows KDE with different bandwidths of a random sample of 100 points from a standard normal distribution.

Grey: true density (standard normal).

Red: KDE with bandwidth 0.05.

Black: KDE with bandwidth 0.337.

Green: KDE with bandwidth 2.



```
nb_kernel <- naive_bayes(y ~ x1 + x2 + x3, data = heart, usekernel = T)
head(predict(nb_kernel, type = "prob")) |> round(3)
```

```
      0      1
[1,] 0.409 0.591
[2,] 0.243 0.757
[3,] 0.997 0.003
[4,] 0.716 0.284
[5,] 0.807 0.193
[6,] 0.365 0.635
```

```
predict(nb_kernel, type = "class")
```

```
[1] 1 1 0 0 0 1 0 1 1 1 0 1 1
```

```
Levels: 0 1
```

```
heart$y
```

```
[1] 1 1 0 0 0 0 0 1 1 1 0 1 1
```

```
Levels: 0 1
```

```
par(mfrow = c(2, 2))
```

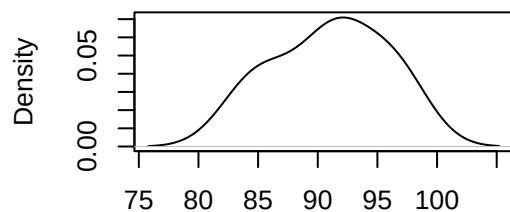
```
plot(density(heart$x1[heart$y == 1]), main = "Density Estimate for X1|Y=1")
```

```
plot(density(heart$x1[heart$y == 0]), main = "Density Estimate for X1|Y=0")
```

```
plot(density(heart$x3[heart$y == 1]), main = "Density Estimate for X3|Y=1")
```

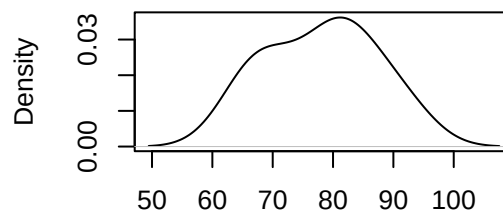
```
plot(density(heart$x3[heart$y == 0]), main = "Density Estimate for X3|Y=0")
```

Density Estimate for X1|Y=1



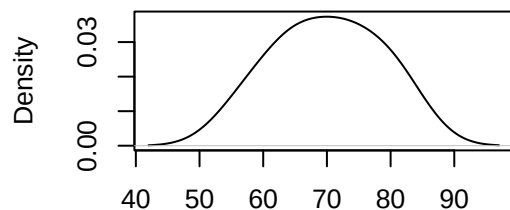
N = 7 Bandwidth = 2.731

Density Estimate for X1|Y=0



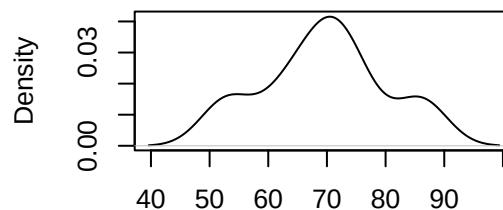
N = 6 Bandwidth = 5.515

Density Estimate for X3|Y=1



N = 7 Bandwidth = 5.006

Density Estimate for X3|Y=0



N = 6 Bandwidth = 4.459

```
par(mfrow = c(1, 1))
```

Advantages and Disadvantages of Naïve Bayes

Advantages:

- **Fast Speed:** Because it treats every variable independently, it doesn't have to perform complex matrix math and can handle massive datasets.
- **High-Dimensionality:** It excels when you have a huge number of features, such as text classification (spam detection), where every word is a feature. Other classifiers may be too slow or crash.
- **Robust to Irrelevant Features:** If you accidentally include irrelevant variables naïve Bayes often filters them out or puts little weight on them.

- **Good with Small Sample Size:** It requires less training data than some other classifiers because it only needs to estimate the distribution of each variable individually.

Disadvantages:

- **The Independence Assumption:** In the real world, variables are almost always related. By ignoring correlations, naïve Bayes can produce a much accurate decision boundary than other classifiers like LDA (seen next).
- **Block Box:** Unlike LDA, where you can easily see which variables are driving the classification, SVM (especially with a Radial Basis kernel) can be hard to interpret. It is difficult to explain why a specific point was classified a certain way.
- **The “Zero Frequency” Problem:** If a specific category or value appears in your test data but was never seen during training, the model assigns it a probability of zero (this can be fixed with “Laplace Smoothing”).
- **Poor Probability Estimates:** It is often good at predicting the correct class, the actual probability scores it gives are often exaggerated and shouldn’t be relied upon for risk assessment.
- **Shape Limitations:** It usually creates linear or simple boundaries. It cannot capture the intricate, curvy relationships that some other classifiers can.

Linear Discriminant Analysis

We just saw with the naïve Bayes classifier that posterior probabilities are computed with the assumption that each feature (predictor) is independent. Another approach that is similar is linear discriminant analysis (LDA). This time, we will have the following assumptions:

1. Each predictor follows a normal (Gaussian) distribution.
2. The variance for each predictor variable is equal across each class of Y .

The way this works, like with naïve Bayes, is by using Bayes' Theorem. We again begin with the formulation we have from naïve Bayes:

$$P(y_i = k | \mathbf{x}_i) = \frac{P(\mathbf{x}_i | y_i = k)P(y_i = k)}{P(\mathbf{x}_i)} = \frac{P(\mathbf{x}_i | y_i = k)P(y_i = k)}{\sum_{\ell=1}^K P(y_i = \ell)P(\mathbf{x}_i | y_i = \ell)}.$$

$P(y_i = k)$ can either come from the proportions of each class in the data or from some prior probability. The main difference between LDA and naïve Bayes is in the assumptions and in how $P(\mathbf{x}_i | y_i = k)$ is handled. We do not assume feature independence here, so we will not write $P(\mathbf{x}_i | y_i = k) = P(x_{i1} | y_i = k) \times \cdots \times P(x_{ip} | y_i = k)$. Let's begin with the case when we have one predictor variable.

LDA for One Predictor

When we only have one predictor variable, X , then $P(X = x_i | y_i = k)$ is modeled as coming from a univariate normal (Gaussian) distribution:

$$P(x_i | y_i = k) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x_i - \mu_k)^2\right)$$

where μ_k is the mean of X when $Y = k$ and σ_k is the standard deviation of X when $Y = k$. Now, in reality, we will have to estimate μ_k and σ_k with $\hat{\mu}_k$ and $\hat{\sigma}_k$.

To find $\hat{\mu}_k$, we take

Finding $\hat{\sigma}_k$ is a little different because we are assuming the variance of the classes is equal. Therefore, we do not think about a different $\hat{\sigma}_k$ for each group. Instead, we think about one $\hat{\sigma}$ that we will use for all groups. That means that practically we would write

$$P(\widehat{x_i | y_i = k}) = \frac{1}{\sqrt{2\pi}\hat{\sigma}} \exp\left(-\frac{1}{2\hat{\sigma}^2}(x_i - \hat{\mu}_k)^2\right),$$

though we often drop the hat over the probability for convenience purposes.

To estimate $\hat{\sigma}$ we need to pool the variances from each class. That gives us

$$\begin{aligned} \hat{\sigma} &= \sqrt{\frac{1}{n-K} \sum_{k=1}^K \left(\sum_{y_i=k} (x_i - \hat{\mu}_k)^2 \right)} \\ &= \sqrt{\frac{1}{n-K} \sum_{k=1}^K (n_k - 1) \cdot \hat{\sigma}_k^2} \end{aligned}$$

where K is the number of classes we have present in the data, n_k is the sample size for class k , and $\hat{\sigma}_k^2$ is the sample variance of the values of X where $Y = k$.

LDA with One Predictor in R

We will use the `lda()` function in the **MASS** library in **R** for this. Notice that **MASS** has some function names that are the same as those in the **tidyverse**, so be careful if using them both. For example, **MASS** has a `select()` function. If you want to use the **tidyverse** `select()` function, indicate that with the name of the package it comes from: `dplyr::select()`.

```
library(MASS)
# First when only one predictor
lda1 <- lda(y ~ x1, data = heart) # heart is same dataset used with naive Bayes
options(pillar.sigfig = 7) # Adjust the number of significant figures to 7
heart |> group_by(y) |> summarize(mean_x1 = mean(x1), sd_x1 = sd(x1), n = n())
# A tibble: 2 x 4
  y     mean_x1    sd_x1     n
  <fct>   <dbl>   <dbl> <int>
1 0         78 9.338094     6
2 1         91 4.795832     7

sigmahat <- sqrt(1/(13 - 2) * ((6 - 1)*9.338094^2 + (7 - 1)*4.795832^2))
heart[1,]
  y x1 x2 x3
1 1 92  F 71

6/13*dnorm(92, 78, sigmahat) / # Predicted prob that y = 0
(6/13*dnorm(92, 78, sigmahat) + 7/13*dnorm(92, 91, sigmahat))
[1] 0.1168486

predict(lda1)$posterior[1,] # Show this matches
      0      1
0.1168486 0.8831514

lda1
Call:
lda(y ~ x1, data = heart)

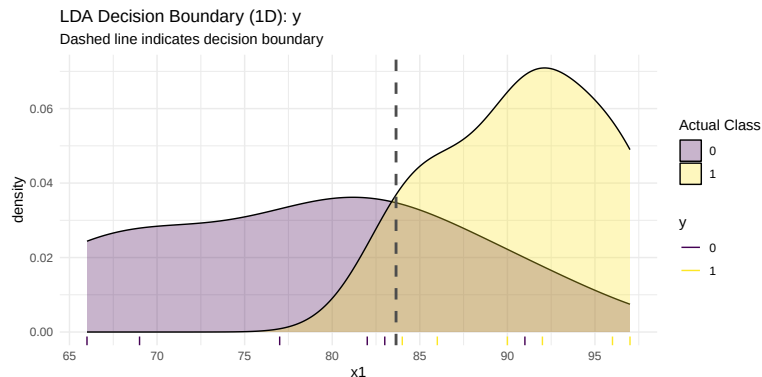
Prior probabilities of groups:
      0      1
0.4615385 0.5384615

Group means:
  x1
0 78
1 91

Coefficients of linear discriminants:
  LD1
x1 0.1384332
```

The “Coefficients of linear discriminants” value (0.1384332 in the first LDA example) is $1/\hat{\sigma}$ when we have only one predictor variable and is not very useful in this case.


```
classifier_plot(lda1, data = heart) + scale_x_continuous(breaks = seq(65, 95, 5))
```



LDA with One Predictor when Y has 3+ Classes

```
# Three categories
lda2 <- lda(y ~ x1, data = heart2)
lda2
Call:
lda(y ~ x1, data = heart2)
```

Prior probabilities of groups:

```
      0      1      2
0.4615385 0.3076923 0.2307692
```

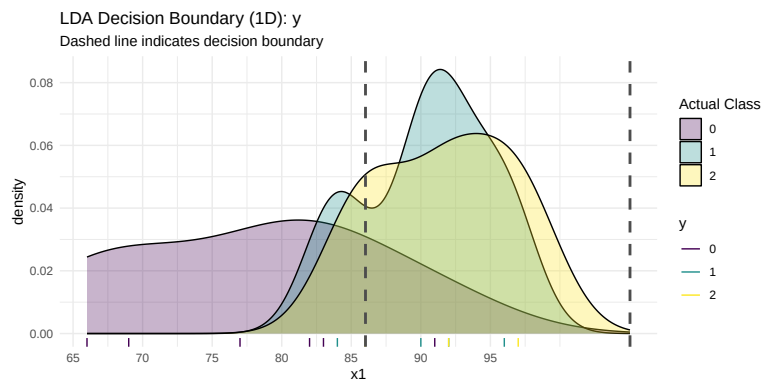
Group means:

```
      x1
0 78.00000
1 90.50000
2 91.66667
```

Coefficients of linear discriminants:

```
LD1
x1 0.13226
```

```
classifier_plot(lda2, data = heart2) + scale_x_continuous(breaks = seq(65, 95, 5))
```



LDA with More than One Predictor

When we have two or more predictor variables (p of them), then we think of $P(\mathbf{x}_i|y_i = k)$ as coming from a multivariate normal (Gaussian) distribution:

$$P(\mathbf{x}_i|y_i = k) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T \Sigma^{-1}(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)\right),$$

where $\mathbf{x}_i$ is i row of the data matrix of X values, $\hat{\boldsymbol{\mu}}_k$ is the vector of sample means for each of the X variables for class k , and Σ is the pooled variance-covariance matrix of the predictor variables found by taking

$$\Sigma = \frac{\sum_{k=1}^K (n_k - 1) \Sigma_k}{n - K},$$

where Σ_k is the variance-covariance matrix of the predictor variables for the k th class. Note that $|\Sigma|$ is the determinant of the matrix Σ .

LDA with Two Predictors in R

```
# Back to the case where we only have two classes
lda3 <- lda(y ~ x1 + x3, data = heart)
heart[1,]
  y x1 x2 x3
1 1 92  F 71

x <- c(92, 71)
Sigma = (
  (6 - 1) * cov(cbind(heart$x1[heart$y == 0], heart$x3[heart$y == 0])) +
  (7 - 1) * cov(cbind(heart$x1[heart$y == 1], heart$x3[heart$y == 1]))
) / (13 - 2)
mu_0 <- c(mean(heart$x1[heart$y == 0]), mean(heart$x3[heart$y == 0]))
mu_1 <- c(mean(heart$x1[heart$y == 1]), mean(heart$x3[heart$y == 1]))

prob0 <- 1/((2*pi)^(2/2)*det(Sigma)^(1/2)) *
  exp(-1/2 * t(x - mu_0) %*% solve(Sigma) %*% (x - mu_0))
prob1 <- 1/((2*pi)^(2/2)*det(Sigma)^(1/2)) *
  exp(-1/2 * t(x - mu_1) %*% solve(Sigma) %*% (x - mu_1))
6/13 * prob0 / (6/13 * prob0 + 7/13 * prob1) # Predicted prob that y = 0
[1,]
[1,] 0.1051156

predict(lda3)$posterior[1,] # Compare to function probability
      0      1
0.1051156 0.8948844

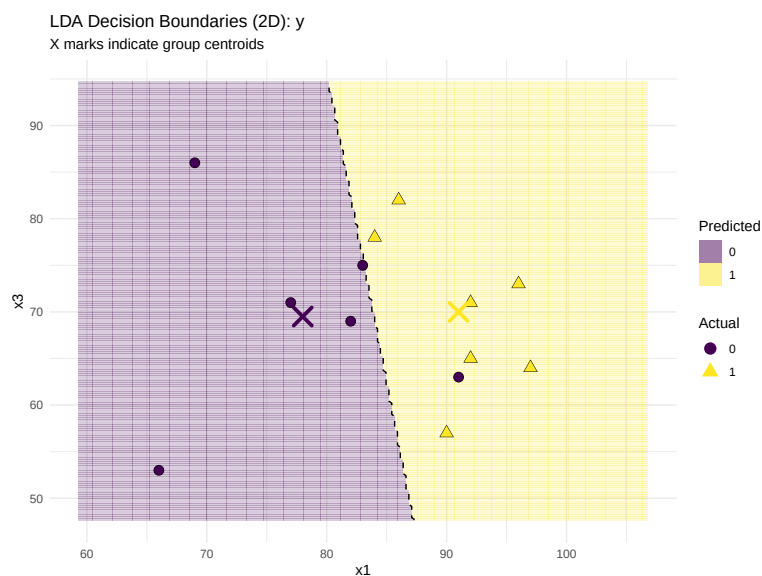
lda3$scaling # Coefficients of linear discriminants
      LD1
x1 0.14061517
x3 0.02111194
```

The “Coefficients of linear discriminants” are now found using $\Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$. These values are now relevant because they tell us two things:

1. The sign of these coefficients tell us if an increase of that variable is attributed to Y being more or less likely to be in the category we are trying to predict (the second category of a factor, by default). Positive values indicate Y is more likely in that class if that X variable increases.
2. The magnitude of one of the values compared to the other tells how much more important that variable is in separating classes than the other.

In this example:

```
classifier_plot(lda3, data = heart)
```



LDA with Multiple Predictors and Three Classes

The idea when we have three classes is very similar to when we have two. The formulas and functions presented earlier don't change. Some of the output and interpretation changes, though.

```
lda4 <- lda(y ~ x1 + x2 + x3, data = heart2)
```

```
lda4
```

```
Call:
```

```
lda(y ~ x1 + x2 + x3, data = heart2)
```

Prior probabilities of groups:

0	1	2
0.4615385	0.3076923	0.2307692

Group means:

	x1	x2M	x3
0	78.00000	0.166667	69.50000
1	90.50000	0.750000	69.75000
2	91.66667	0.666667	70.33333

Coefficients of linear discriminants:

	LD1	LD2
x1	0.13149589	0.06424676
x2M	2.00358894	-1.58052486
x3	-0.02159012	0.06305161

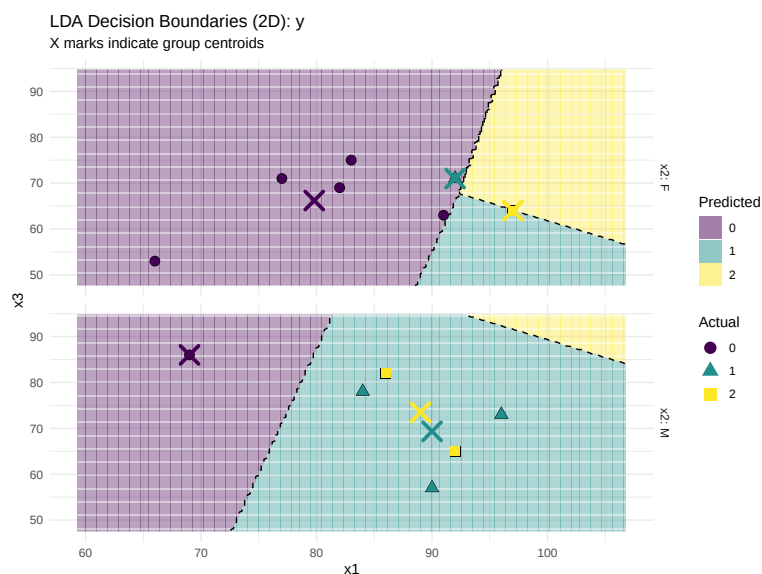
Proportion of trace:

LD1	LD2
0.996	0.004

Now we have LD1 and LD2 each with coefficients for all variables. The first linear discriminant, LD1, is trying to find the maximum possible separation between all group means. You can think of this as the “super-separator”. It is saying that being male (x2) is by far the most useful for that, then diastolic blood pressure (x1), than age (x3).

The second linear discriminant, LD2, is basically the “refiner”. It is mathematically forced to be uncorrelated (perpendicular) to LD1. Its job is to find the next best direction to separate the groups that LD1 couldn’t distinguish. In this case, x2 is still the most useful, but now x3 is about as helpful as x1 in refining since x1 was the more dominant force in initially separating.

Finally, there is a “proportion of trace” measure. This tells us how much separating power comes from each linear discriminant. LD1 will always have a larger proportion of trace than LD2. In this case, LD1’s proportion of trace is 0.996, which means that about 99.6% of the the difference between our three groups can be explained using just the first formula (LD1). The second formula (LD2) only adds 0.4% more information. This is similar to PCA.



Computing Coefficients of Linear Discriminants (3+ Classes)

When there are more than two classes for Y , the “Coefficients of linear discriminants” are found by quantifying the sum of squares between classes (SSB) and the sum of squares within classes (SSW). Then we do something similar to $\Sigma^{-1}(\mu_1 - \mu_0)$. Σ is quantifying uncertainty (like SSW does) and $(\mu_1 - \mu_0)$ is quantifying how different the class means are (like SSB does). So, we will find the matrix for SSW, which we will call $\mathbf{W}$ and the matrix for SSB, which we will call $\mathbf{B}$ and then take $\mathbf{W}^{-1}\mathbf{B}$. However, it does not end there. We then take the eigenvectors of that new matrix and scale them. If we call the eigenvectors $v_1, \dots, v_p$ (there will be one per predictor variable), then the coefficients of linear discriminant for each discriminant is found using

$$w_i = v_i \cdot \sqrt{\frac{n - K}{v_i^T \mathbf{W} v_i}}$$

We can do this in **R** like this:

```
lda4 <- lda(y ~ x1 + x3, data = heart2)
# First quantify W and B using manova (multivariate ANOVA):
W <- summary(manova(cbind(x1, x3) ~ y, data = heart2))$SS$Residuals
B <- summary(manova(cbind(x1, x3) ~ y, data = heart2))$SS$y
# Find eigen vectors
v1 <- eigen(solve(W) %*% B)$vectors[,1]
v2 <- eigen(solve(W) %*% B)$vectors[,2]
v1 %*% sqrt((13 - 3)/(t(v1) %*% W %*% v1)) # Replace v1 with v2 for the other
      [,1]
[1,] 0.13438636
[2,] 0.02046672

lda4$scaling # Compare to the lda function output
      LD1      LD2
x1 0.13438636 -0.003904491
x3 0.02046672  0.096259448
```

Advantages and Disadvantages of LDA

Advantages:

- **Multiclass Handling:** LDA handles multiple classes much better than something like logistic regression.
- **Feature Correlation:** LDA models the relationships between features by estimating a covariance matrix.
- **Dimensionality Reduction:** LDA is unique because it can be used to compress your data into a smaller number of “discriminant coordinates.” This makes it much easier to visualize high-dimensional data in a 2D plot than it is with SVM.
- **Efficiency on Small Datasets:** When your data are normally distributed and you have a small sample size, LDA is often more stable and accurate than some other classifiers like SVM (seen next).

Disadvantages:

- **Strict Statistical Assumptions:** LDA assumes your data follows a Gaussian (normal) distribution and that all classes have equal variance. If your data is messy or non-normal, naïve Bayes or SVM will usually outperform it. Technically, LDA can work without these assumptions, but most software codes this in. To drop the equal variance assumption, quadratic discriminant analysis (QDA) is often used.
- **Sensitivity to Outliers:** LDA relies on the mean of the data, so a single extreme outlier can pull the decision boundary significantly out of place.
- **Linear Limitation:** LDA can only draw straight lines or flat planes. If your classes are separated by a curve or a circle, LDA will fail.
- **Covariance Modeling:** LDA needs to estimate a covariance matrix. If there are a very large number of features but not enough rows of data, this estimation can fail.

Support Vector Machines

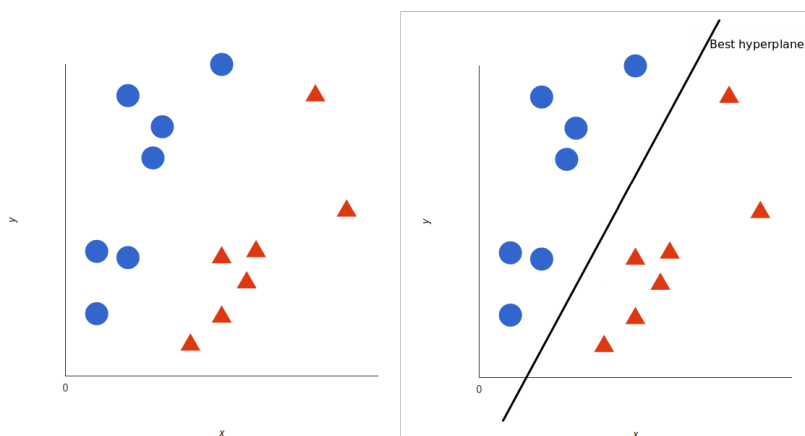
In machine learning, **support vector machines** or **SVMs** are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis. However, they are mostly used in classification problems. Here we will focus on developing intuition rather than rigor and develop a basic understanding of the working principles.

Material for this content was obtained and adapted from:

- <https://www.datacamp.com/community/tutorials/support-vector-machines-r>
- *The Elements of Statistical Learning*, Hastie, et al., Springer
- <https://www.geeksforgeeks.org/classifying-data-using-support-vector-machines-svm-in-r/>

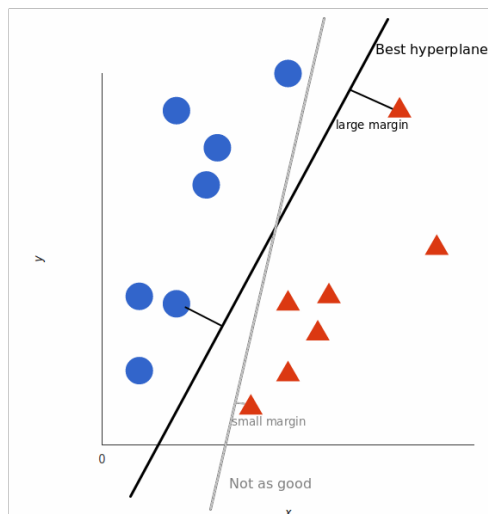
Support Vector Machines–Linear Data

Let's imagine we have two tags: *red* and *blue*, and our data has two features: x and y . We can plot our training data on a plane:



An **SVM** identifies the **decision boundary** or **hyperplane** (a line in two dimensions) that best separates the tags.

But, what exactly is the best hyperplane? For SVM, it's the one that maximizes the margins from the data from both tags:



Types of margins:

There are two types of margins for SVMs: hard margin and soft margin.

- **Hard margin:** This is used when the hyperplane can be drawn in such a way that there are no misclassifications. This is uncommon on real datasets since this would mean the data have no overlap.
- **Soft margin:** This is used when the data are not perfectly separable or contain outliers. Each data point has a slack variable introduced by the soft-margin SVM formulation, which softens the strict margin requirement and permits certain misclassifications or violations. There is a compromise between increasing the margin and reducing violations.

A Look into SVM Methodology

We will focus first on the hard margin case where we can get perfect separation. The hyperplane in 2D is a line. We can think of the equation of this line first in standard form $ax + by = c$. Now, to make this more general, we will write it as $ax_1 + bx_2 - c = 0$. Reparameterizing this, we get

$$\beta_1 x_1 + \beta_2 x_2 - c = 0.$$

Then we can write $x_2 = -\frac{\beta_1}{\beta_2} x_1 + \frac{c}{\beta_2}$. So, the slope of this line is $-\frac{\beta_1}{\beta_2}$ and the y -intercept is $\frac{c}{\beta_2}$.

In k dimensions, we get the equation of our hyperplane as

$$\beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \cdots + \beta_k x_k - c = 0.$$

this is equivalently written

$$\boldsymbol{\beta}^T \mathbf{x} - c = 0.$$

Hard-Margin Case

Now, in the hard-margin case, we will classify anything on or above the boundary $\boldsymbol{\beta}^T \mathbf{x} - c = 1$ is one class (say a 1) and everything on or below the boundary $\boldsymbol{\beta}^T \mathbf{x} - c = -1$ as the other class (say a -1).

The distance between those two hyperplanes is $\frac{2}{\|\boldsymbol{\beta}\|_2}$ where $\|\boldsymbol{\beta}\|_2$ is the Euclidean norm (or the 2 norm): $\|\boldsymbol{\beta}\|_2 = \sqrt{\sum_{i=1}^k \beta_i^2}$. To maximize the distance between the hyperplanes, that is, to maximize the margin, we want to minimize $\|\boldsymbol{\beta}\|_2$.

We also need to prevent any point from falling in the margin, so we need $\boldsymbol{\beta}^T \mathbf{x}_i - c \geq 1$ if $y_i = 1$ and $\boldsymbol{\beta}^T \mathbf{x}_i - c \leq -1$ if $y_i = -1$. We can combine these constraints by writing $y_i(\boldsymbol{\beta}^T \mathbf{x}_i - c) \geq 1$ for all $1 \leq i \leq n$ and for $y_i \in \{-1, 1\}$. Therefore, finding the β_i values is the same as the constrained optimization problem

$$\begin{aligned} & \underset{\boldsymbol{\beta}, c}{\text{minimize}} \quad \|\boldsymbol{\beta}\|_2^2 \\ & \text{subject to} \quad y_i(\boldsymbol{\beta}^T \mathbf{x}_i - c) \geq 1 \text{ for all } i \end{aligned}$$

An important property of this optimization problem is that the max-margin hyperplane is completely determined by those $\mathbf{x}_i$ that lie nearest to it. These $\mathbf{x}_i$ are called **support vectors**.

Soft-Margin Case

Much remains the same in the soft-margin case except now it is not possible to prevent values from falling in the margin. So, we introduce new variables, ξ_i that are equal to zero for points on the correct side of the margin and equal to the distance to the end of the margin for points on the wrong side of the margin. See Figure 12.1 below, which comes from *Elements of Statistical Learning*:

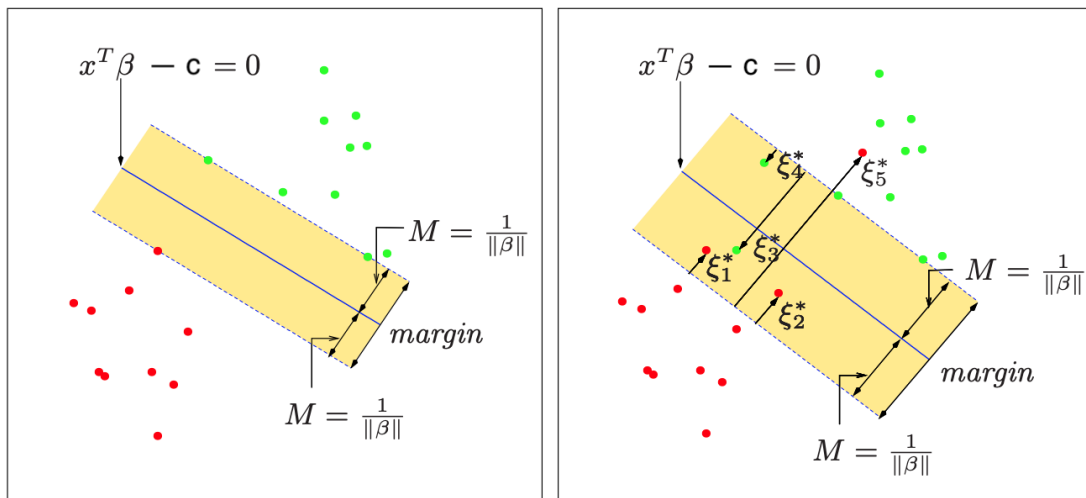


FIGURE 12.1. Support vector classifiers. The left panel shows the separable case. The decision boundary is the solid line, while broken lines bound the shaded maximal margin of width $2M = 2/\|\beta\|$. The right panel shows the nonseparable (overlap) case. The points labeled ξ_j^* are on the wrong side of their margin by an amount $\xi_j^* = M\xi_j$; points on the correct side have $\xi_j^* = 0$. The margin is maximized subject to a total budget $\sum \xi_i \leq \text{constant}$. Hence $\sum \xi_j^*$ is the total distance of points on the wrong side of their margin.

Therefore, the new optimization problem can be stated

$$\begin{aligned} & \underset{\beta, c, \xi}{\text{minimize}} \quad \|\beta\|_2^2 + C \sum_{i=1}^n \xi_i \\ & \text{subject to} \quad y_i(\beta^T \mathbf{x}_i - c) \geq 1 - \xi_i \text{ where } \xi_i \geq 0 \text{ for all } i \end{aligned}$$

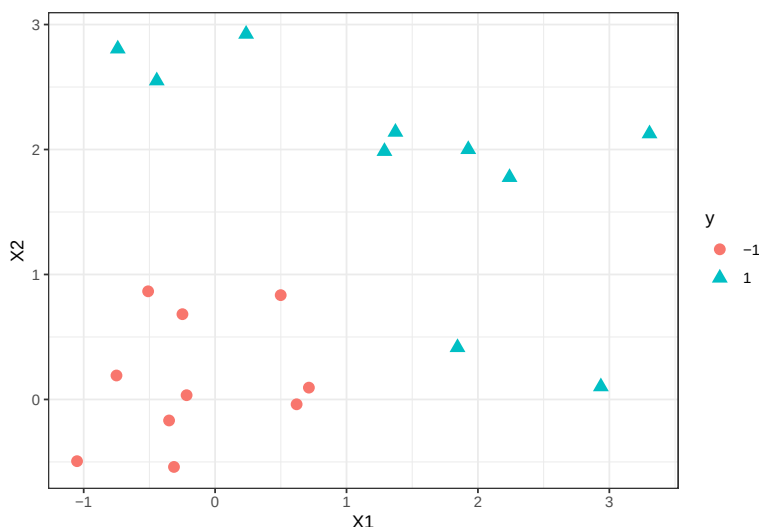
where $C > 0$ is a cost parameter that determines the trade-off between increasing the margin size and ensuring that point i lie on the correct side of the margin. The smaller the cost, the more misclassifications are allowed in the data and the more potential to under fit. The higher the cost, the fewer misclassifications and the more potential to over fit. The higher the cost, the more this behaves like the hard-margin case. Using cross validation may be a good way to choose this cost value.

Support Vector Machines in R

Hard-margin Case

First generate some data in 2 dimensions, and make them a little separated:

```
set.seed(10111)
x = matrix(rnorm(40), 20, 2)
y = rep(c(-1, 1), c(10, 10))
x[y == 1,] = x[y == 1,] + 1.6 # Add separation to the classes
dat <- data.frame(X1 = x[,1], X2 = x[,2], y = as.factor(y))
ggplot(dat, aes(x = X1, y = X2, color = y, shape = y)) +
  geom_point(size = 3) +
  theme_bw()
```



We will use the **e1071** package which contains the `svm` function and make a dataframe of the data, turning `y` into a factor variable. The `cost` option in the `svm()` function controls training errors and margins.

```
library(e1071)
# The variable entered first will be plot on the y axis
svmfit <- svm(y ~ X1 + X2, data = dat,
              kernel = "linear", cost = 10, scale = FALSE)
print(svmfit)
```

Call:

```
svm(formula = y ~ X1 + X2, data = dat, kernel = "linear", cost = 10,
     scale = FALSE)
```

Parameters:

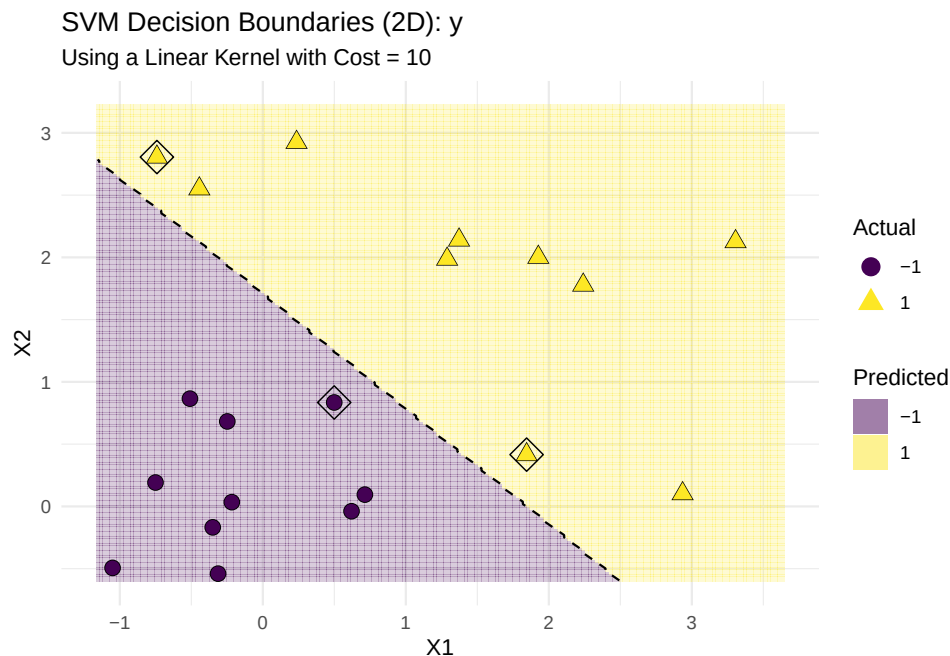
```
SVM-Type:  C-classification
SVM-Kernel: linear
cost:      10
```

Number of Support Vectors: 3

Printing the `svmfit` gives its summary. You can see that the number of support vectors is 3 - they are the points that are close to the boundary or on the wrong side of the boundary.

There's a plot function for SVM that shows the decision boundary, as you can see below. The points plotted with diamonds around them are support vectors.

```
classifier_plot(svmfit, data = dat)
```



Unfortunately, the `svm()` function is not super friendly. We have to do some work to get back the linear coefficients. The reason is probably that this only makes sense for linear kernels, and the function is more general. So let's use some coded to extract the coefficients. We can extract β and c , which are the linear coefficients using the following:

```
(beta = drop(t(svmfit$coefs)%*%x[svmfit$index,]))
```

```
[1] -2.240242 -2.423619
```

```
(c = svmfit$rho)
```

```
[1] -4.140447
```

Now we can replot the points on the grid, then put the points back in (including the support vector points). Then you can use the coefficients to draw the decision boundary using a simple equation of the form:

$$x_1\beta_1 + x_2\beta_2 - c = 0$$

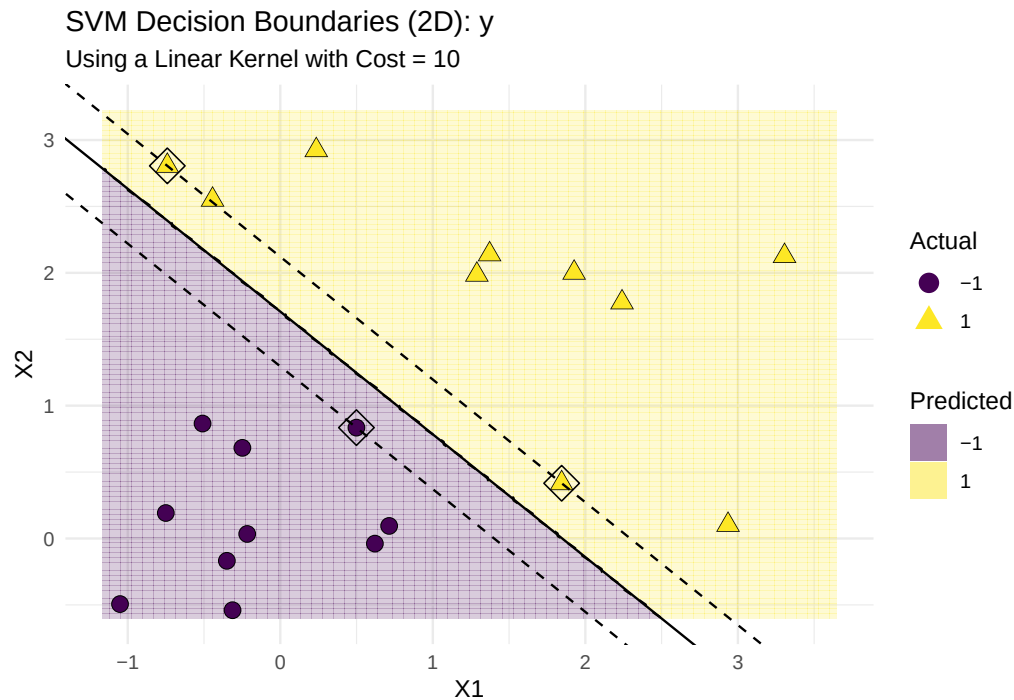
Solving for x_2 gives us

$$x_2 = \frac{c}{\beta_2} - \frac{\beta_1}{\beta_2}x_1$$

Now plotting the line on a graph:

```
classifier_plot(svmfit, data = dat) +  
  geom_abline(intercept = c / beta[2],  
             slope = -beta[1] / beta[2]) +  
  geom_abline(intercept = (c - 1) / beta[2],  
             slope = -beta[1] / beta[2], linetype = "dashed") +
```

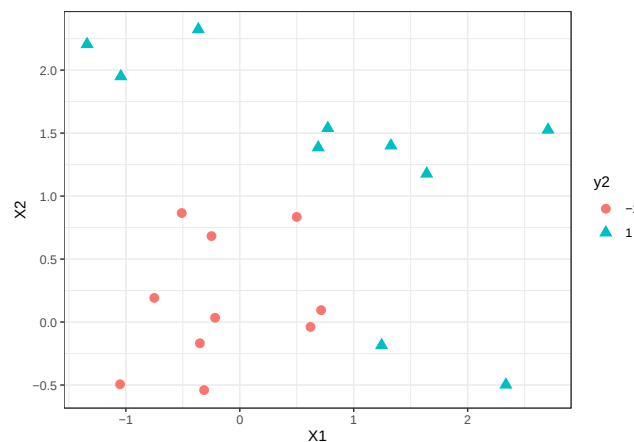
```
geom_abline(intercept = (c + 1) / beta[2],
            slope = -beta[1] / beta[2], linetype = "dashed")
```



Soft-margin Case

Now generate similar data in 2 dimensions, but make them less separated:

```
set.seed(10111)
x2 = matrix(rnorm(40), 20, 2)
y2 = rep(c(-1, 1), c(10, 10))
x2[y2 == 1,] = x2[y2 == 1,] + 1
dat2 <- data.frame(X1 = x2[,1], X2 = x2[,2], y2 = as.factor(y2))
ggplot(dat2, aes(x = X1, y = X2, color = y2, shape = y2)) +
  geom_point(size = 3) +
  theme_bw()
```



```
# The variable entered first will be plot on the y axis
svmfit2 <- svm(y2 ~ X1 + X2, data = dat2,
               kernel = "linear", cost = 10, scale = FALSE)
print(svmfit2)
```

Call:

```
svm(formula = y2 ~ X1 + X2, data = dat2, kernel = "linear", cost = 10,
    scale = FALSE)
```

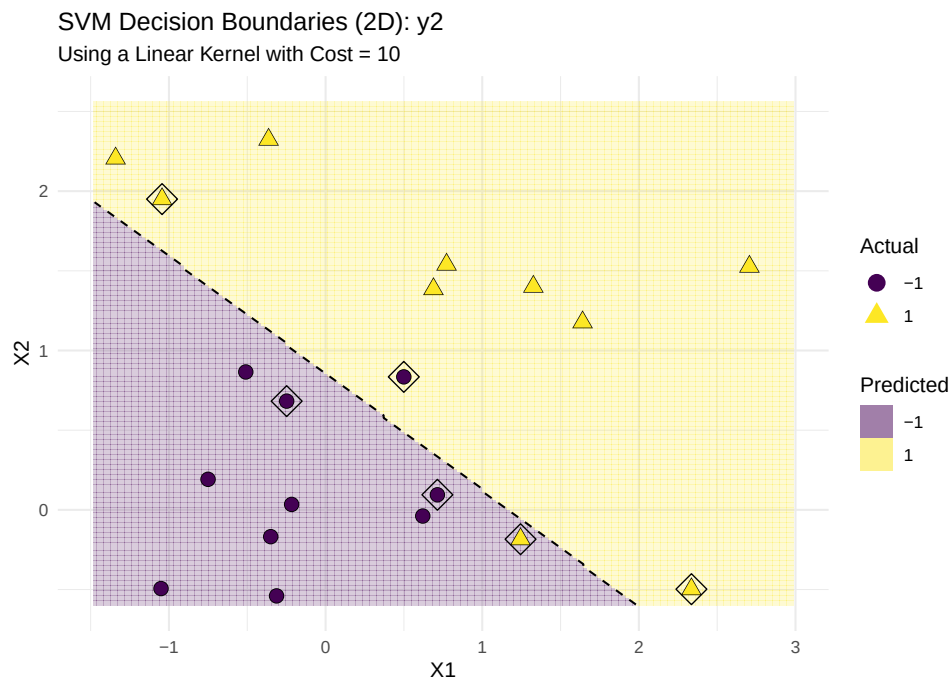
Parameters:

```
SVM-Type: C-classification
SVM-Kernel: linear
cost: 10
```

Number of Support Vectors: 6

Now the number of support vectors is 6 - they are the points that are close to the boundary or on the wrong side of the boundary.

```
classifier_plot(svmfit2, dat2)
```

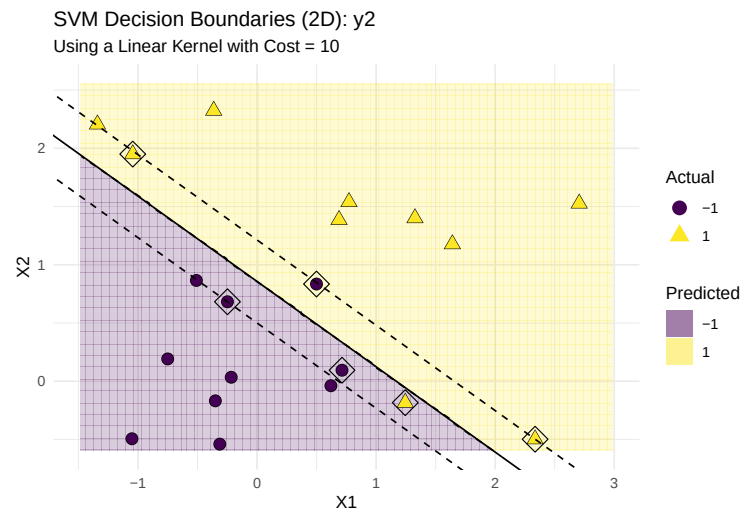


Obtain the coefficients:

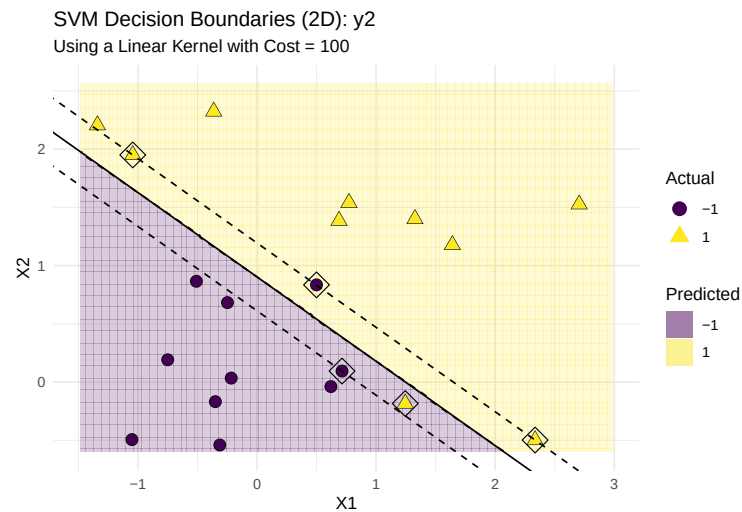
```
(beta = drop(t(svmfit2$coefs)%*%x2[svmfit2$index,]))
[1] -2.055647 -2.806278
```

```
(c = svmfit2$rho)
[1] -2.403917
```

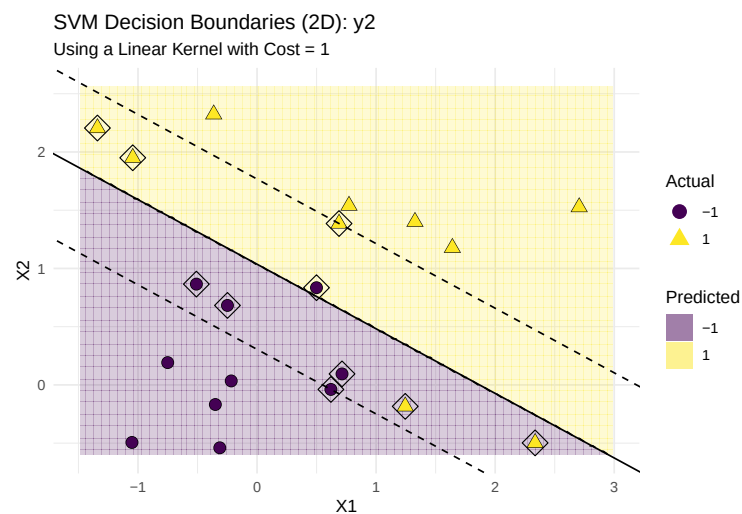
Plot the lines:



If the cost was 100 instead of 10:

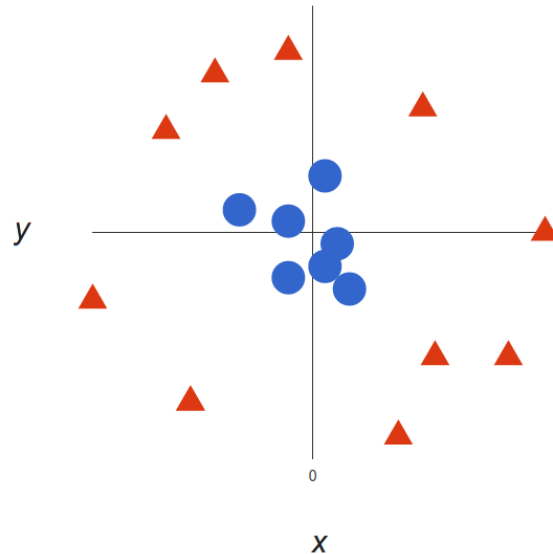


If the cost was 1 instead of 10:



Support Vector Machines for Non-Linear Data

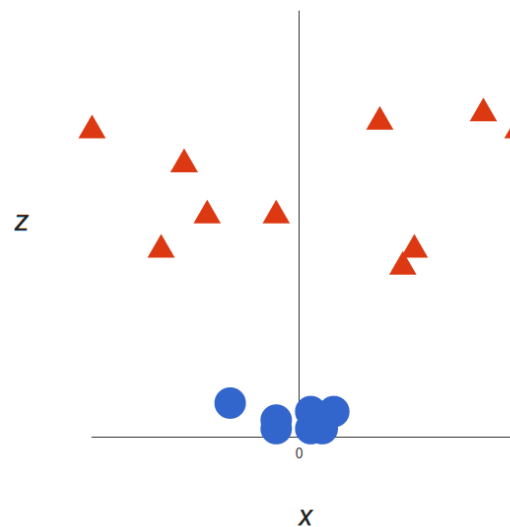
Now the examples before were easy since clearly, the data was linearly separable. Often things aren't that simple. Take a look at this case:



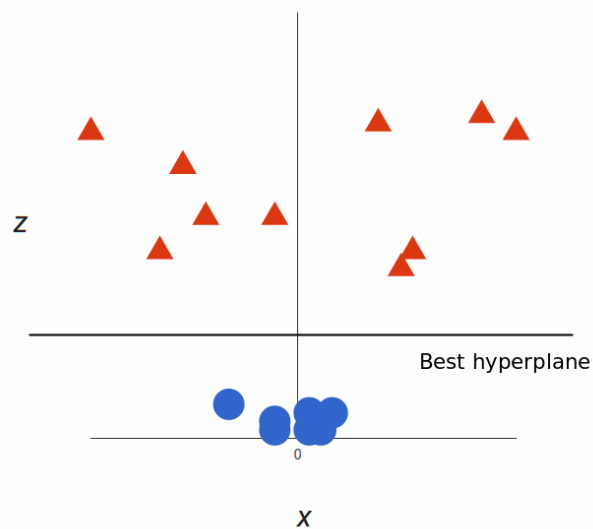
It's pretty clear that there's not a linear decision boundary (a single straight line that separates both tags). However, the vectors are very clearly segregated, and it looks as though it should be easy to separate them.

So here's what we'll do: we will **add a third dimension**. Up until now, we had two dimensions: x and y . We create a new z dimension, and we rule that it be calculated a certain way that is convenient for us: $z = x^2 + y^2$ (you'll notice that's the equation for a circle).

This will give us a three-dimensional space. Taking a slice of that space:

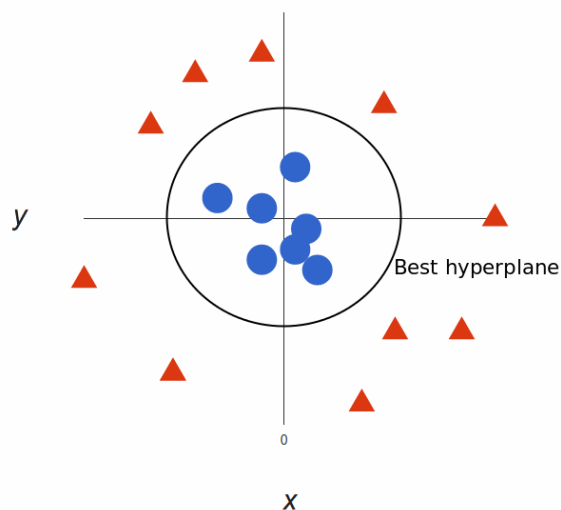


What can SVM do with this? Let's see:



That's great! Note that since we are in three dimensions now, the hyperplane is a plane parallel to the x axis at a certain z (let's say $z=1$).

What's left is mapping it back to two dimensions:



And there we go! Our decision boundary is a circumference of radius 1, which separates both tags using SVM.

Kernel Trick

In this example, we found a way to classify nonlinear data by cleverly mapping our space to a higher dimension. However, it turns out that calculating this transformation can get pretty computationally expensive: there can be a lot of new dimensions, each one of them possibly involving a complicated calculation. Doing this for every vector in the dataset can be a lot of work, so it'd be great if we could find a cheaper solution.

Here's a trick: SVM doesn't need the actual vectors to work its magic, it actually can get by only

with the dot products between them. This means that we can sidestep the expensive calculations of the new dimensions! This is what we do instead:

- Imagine the new space we want:

$$z = x^2 + y^2$$

- Figure out what the dot product in that space looks like. For $\mathbf{a} = [x_a, y_a, z_a]^T$ and $\mathbf{b} = [x_b, y_b, z_b]^T$:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = x_a \cdot x_b + y_a \cdot y_b + z_a \cdot z_b = x_a \cdot x_b + y_a \cdot y_b + (x_a^2 + y_a^2) \cdot (x_b^2 + y_b^2)$$

- Tell SVM to do its thing, but using the new dot product—we call this a **kernel function**.

This is often called the **kernel trick**, which enlarges the feature space in order to accommodate a non-linear boundary between the classes.

Common types of kernels used for SVM are:

- Linear: $\beta^T \mathbf{x} - c$
 - The linear kernel is fast and easy to implement, but it may not capture the complexity and non-linearity of the data. The linear kernel is suitable for high-dimensional and sparse data, where adding more features may cause overfitting or computational problems.
- Polynomial: $(\gamma \beta^T \mathbf{x} - c)^d$. This allows some curvature in the decision boundary.
 - The polynomial kernel is suitable for low-dimensional and dense data, where adding more features may improve the performance and accuracy of the SVM algorithm.
- Radial: $\exp(-\gamma \|\beta - \mathbf{x}\|^2)$. This allows for a lot of curvature in the decision boundary.
 - The radial kernel is suitable for medium-dimensional and diverse data, where the data points may have different scales and distributions. The radial kernel is also known as the Gaussian kernel or the exponential kernel.
- Sigmoid: $\tanh(\gamma \beta^T \mathbf{x} - c)$. Similar to radial, but the curvature behaves differently.
 - The sigmoid kernel is suitable for binary and categorical data, where the data points may have discrete and logical values.

Simply, these kernels transform our data to pass a linear hyperplane and thus classify our data.

Non-linear Support Vector Machines in R

Now let's apply a non-linear (polynomial) SVM to our prior simulated dataset.

```
svmfit_poly = svm(y2 ~ X1 + X2, data = dat2,
                  kernel = "polynomial", cost = 10, scale = FALSE, degree = 3)
print(svmfit_poly)
```

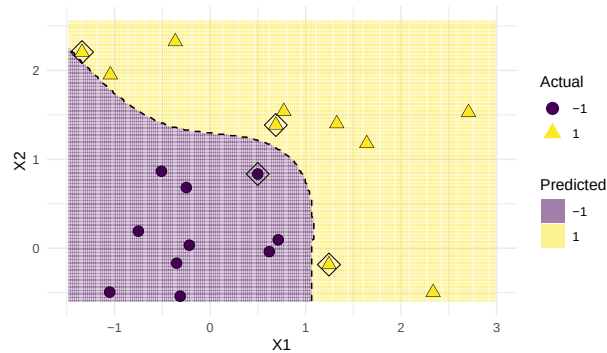
Parameters:

```
SVM-Type: C-classification
SVM-Kernel: polynomial
cost: 10
degree: 4
coef.0: 0
```

Number of Support Vectors: 5

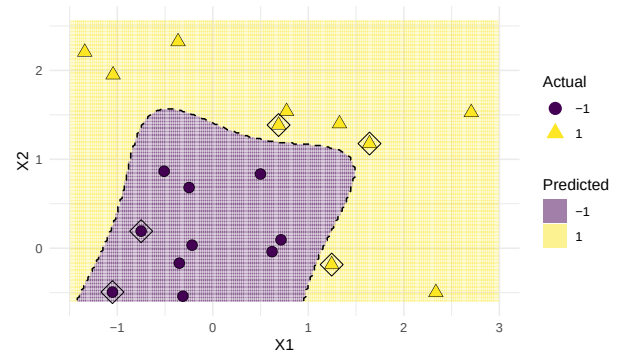
Degree 3 polynomial kernel

SVM Decision Boundaries (2D): y2
Using a Polynomial Kernel with Degree = 3 and Cost = 10



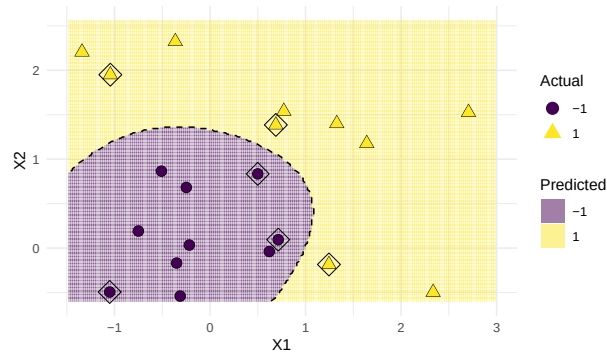
Degree 4 polynomial kernel

SVM Decision Boundaries (2D): y2
Using a Polynomial Kernel with Degree = 4 and Cost = 10



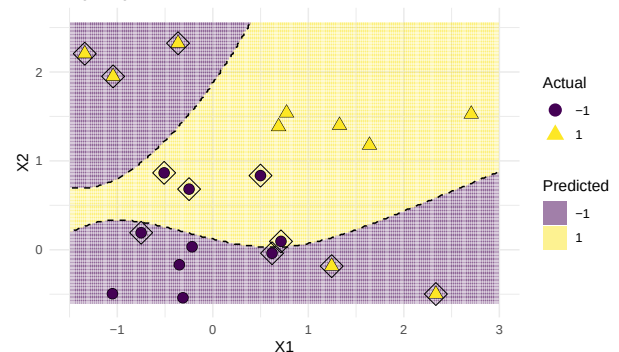
Using a radial kernel:

SVM Decision Boundaries (2D): y2
Using a Radial Basis Kernel with gamma = 0.5 and Cost = 10



Using a sigmoid kernel (not good in this case):

SVM Decision Boundaries (2D): y2
Using a Sigmoid Kernel with Cost = 10

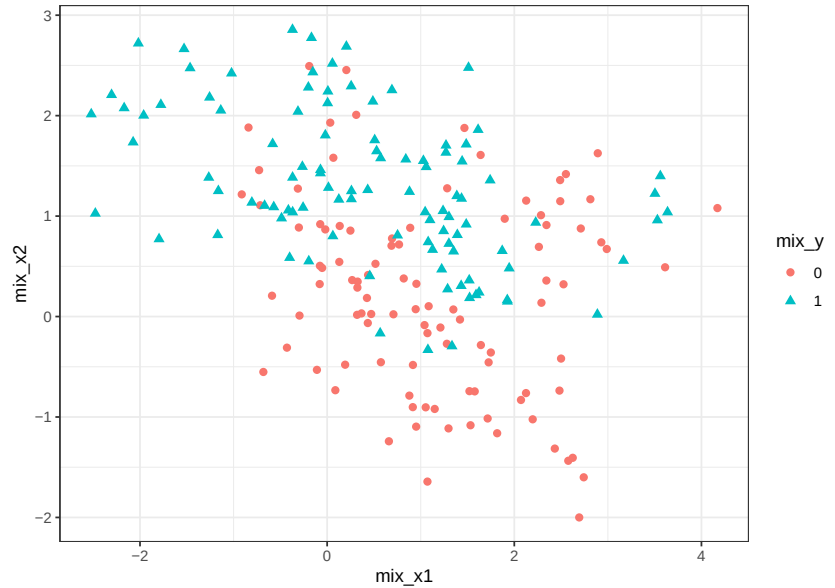


Here is a more complicated example from *Elements of Statistical Learning*, where the decision boundary needs to be non-linear and there is no clear separation. We will use the `ESL.mixture` data object in `math3190package`.

```
library(math3190package) |> suppressPackageStartupMessages()
mix_df <- data.frame(mix_x1 = ESL.mixture$x[,1],
                     mix_x2 = ESL.mixture$x[,2],
                     mix_y = factor(ESL.mixture$y))
```

Plotting the data:

```
ggplot(mix_df,
  aes(x = mix_x1,
    y = mix_x2,
    color = mix_y,
    shape = mix_y)) +
  geom_point(size = 2) +
  theme_bw()
```

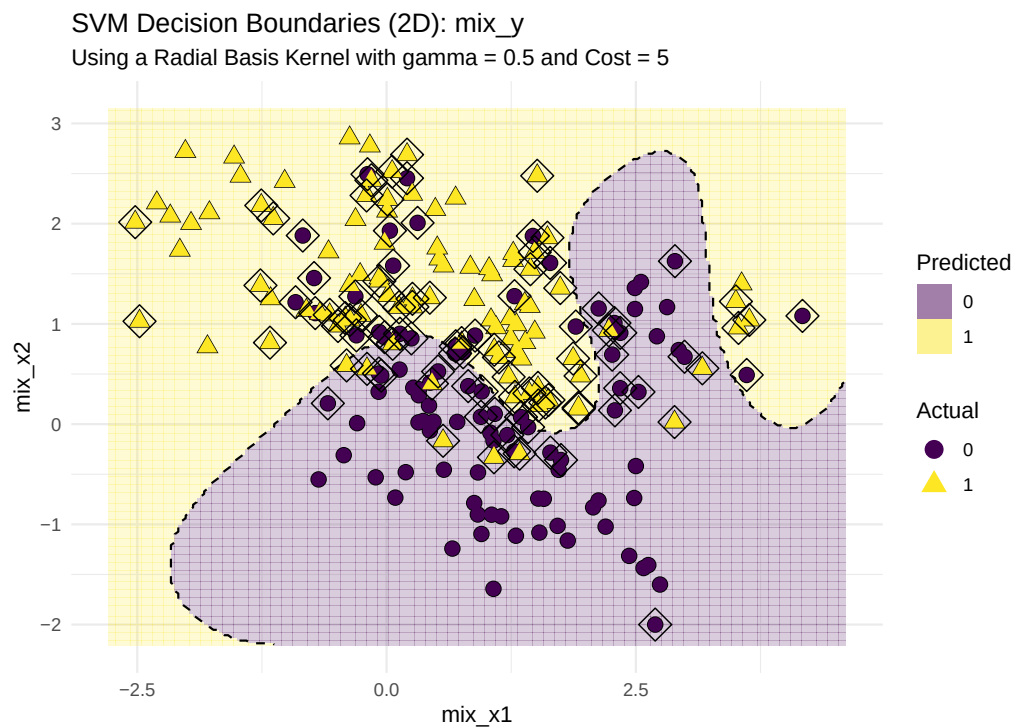


We will fit an SVM with radial kernel.

```
fit_mix <- svm(mix_y ~ mix_x1 + mix_x2, data = mix_df, scale = FALSE,
  kernel = "radial", cost = 5)
```

Plotting the boundary:

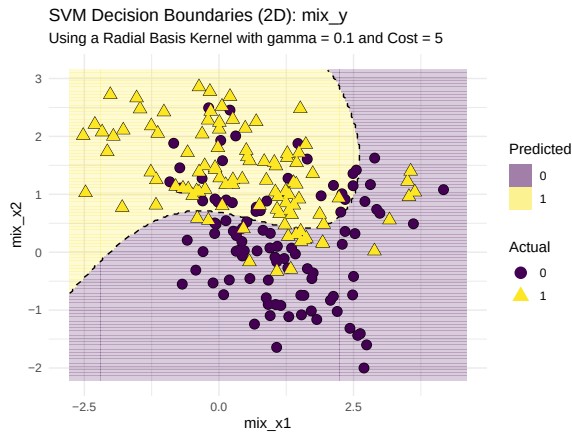
```
classifier_plot(fit_mix, data = mix_df)
```



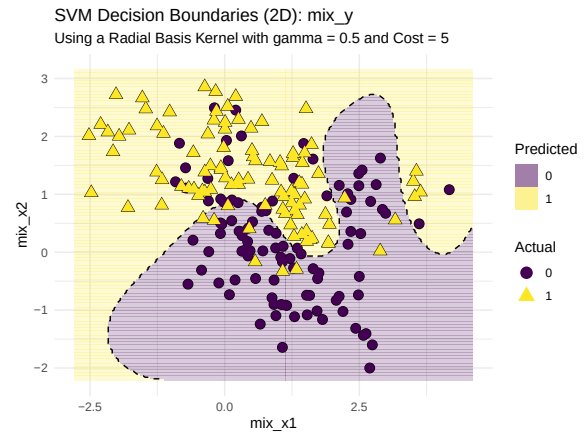
Notice that the response variable labels are 0 and 1 here instead of -1 and 1 .

Let's see the effect of adjusting γ . By default, $\gamma = 1/2$. Let's adjust it to $1/10$, 1 and 2 . The support vectors are not put in diamonds here to avoid too much clutter. There is a `sv_diamonds` argument in the `classifier_plot()` function that can be assigned `FALSE`.

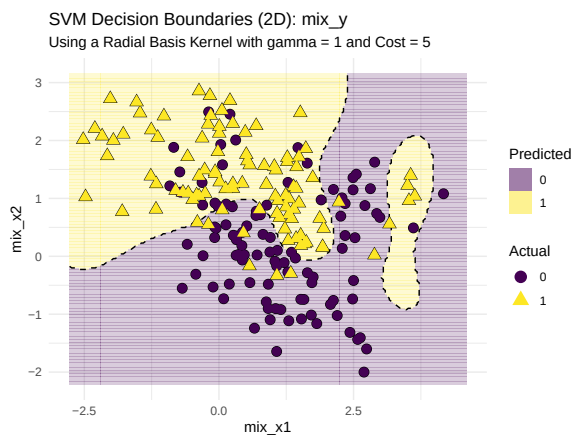
$\gamma = 1/10$:



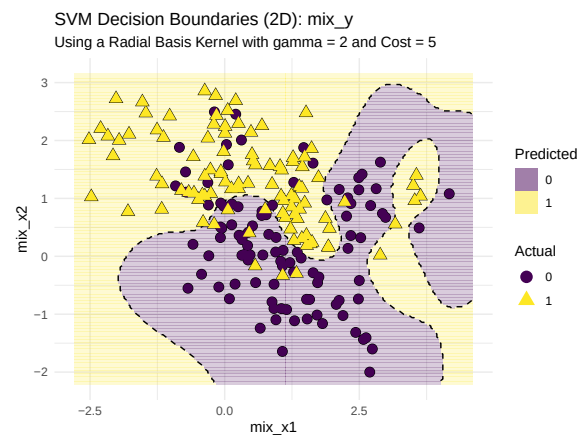
$\gamma = 1/2$:



$\gamma = 1$:



$\gamma = 2$:



How can we choose the best value of γ ?

Advantages and Disadvantages of SVMs

Advantages:

- **High Dimensionality:** SVM is an effective tool in high-dimensional spaces, which is particularly applicable to document classification and sentiment analysis where the dimensionality can be extremely large.
- **Versatility:** Class separation is often highly non-linear. The ability to apply new kernels allows substantial flexibility for the decision boundaries, leading to greater classification performance.
- **No Distributional Assumptions:** SVM does not assume the data follow any particular distribution, making it a non-parameter classifier.

Disadvantages:

- **Kernel Selection:** SVMs are very sensitive to the choice of the kernel parameters. In situations where the number of features for each object exceeds the number of training data samples, SVMs can perform poorly. This can be seen intuitively as if the feature space is much larger than the samples. Then there are less effective support vectors on which to support the optimal linear hyperplanes.
- **Block Box:** Unlike LDA, where you can easily see which variables are driving the classification, SVM (especially with a Radial Basis kernel) can be hard to interpret. It is difficult to explain why a specific point was classified a certain way.
- **Sensitive to Hyperparameters:** To get a good SVM, you have to carefully tune the cost (C) and gamma parameters. A bad choice can lead to severe over fitting.
- **Non-Probabilistic:** Since the classifier works by placing objects above and below a classifying hyperplane, there is no direct probabilistic interpretation for group membership. However, one potential metric to determine the “effectiveness” of the classification is how far from the decision boundary the new point is. However, there are post-processes that can be done to assign probabilities.

Notes 11: Time Series

Table of contents

Introduction	217
Example: Dow Jones	217
ACF	217
PACF	218
Time Series Model	219
Air Passengers Example	219
Air Passengers Example - Seasonality	222
Time Series Modeling	224
Estimating the Trend - Linear	224
Estimating the Trend - Quadratic	225
Removing the Trend	225
Other Non-linear Trends	225
Estimating the Seasonality	226
Estimating the Seasonality - Air Passengers	227
Time Series Models	227
Moving Average (MA) Models	228
MA Model Examples - MA(1)	228
MA Model Examples - MA(2)	229
Autoregressive (AR) Models	231
AR Model Examples - AR(1)	231
AR Model Examples - AR(2)	232
AR Model Examples - Not Stationary	234
Autoregressive Moving-Average (ARMA) Models	234
ARMA Examples - ARMA(2,2)	234
ARMA Examples - ARMA(2,1)	235
Differencing	236
Fitting an ARMA Model	236
Determining if Series is Stationary	236
Testing for Stationarity	236
Fitting an ARMA Model	237
Model Fit Table	238
Transforming Back	239
Forecasting	239
Forecasting Confidence Intervals	240
auto.arima() Function	240
Forecasting Confidence Intervals	241
Diagnostic Plots	241

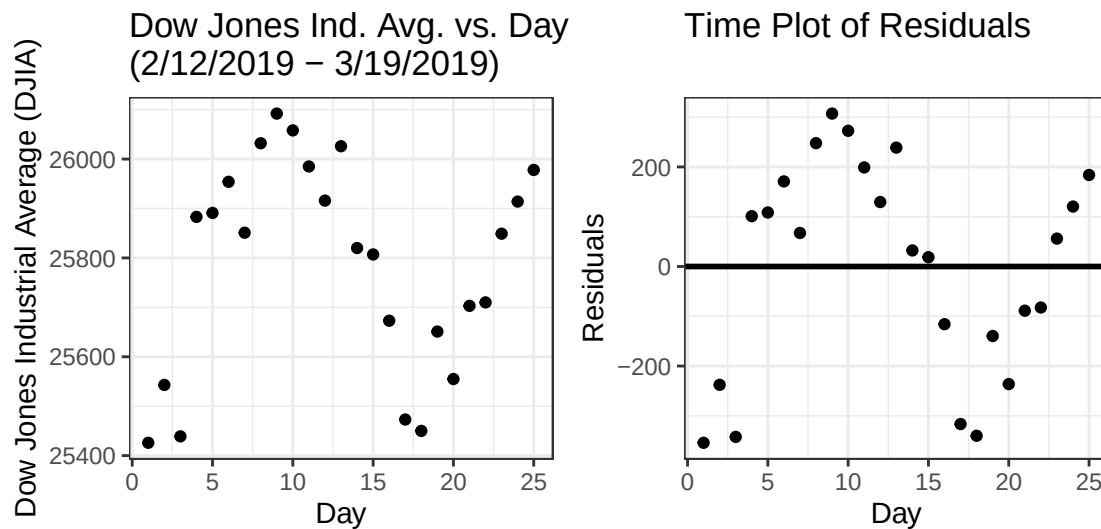
Introduction

Time series analysis is extremely common in many disciplines. It is often the case that we want to predict future events from past ones or model the trend and error of data separately.

The key aspect of time series data is that of **autocorrelation**. It is usually the case that one observation will be dependent on a previous observation or observations.

Example: Dow Jones

Consider the following data on the Dow Jones Industrial Average (DJIA) for stocks:



In this example, there is definitely some correlation in the values. If the previous value was large, the next is more likely to be large. Because of this (and the fact that the relationship is not linear), a regression model would not be appropriate.

When this is the case, a **time series model** is more appropriate since these models assume the data are ordered and dependent on time. A linear regression model assumes the data are independent and random.

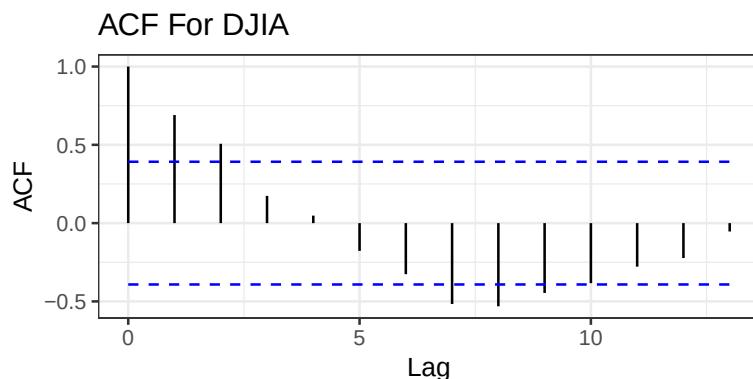
ACF

We can plot the autocorrelation using the `acf()` (included in base **R**) and `autoplot()` (in `ggplot2`) functions. Note that the `ggfortify` package must be loaded for this to work properly.

```
library(ggfortify)
DJIA <- c(25426, 25543, 25439, 25883, 25891, 25954, 25851, 26032,
         26092, 26058, 25985, 25916, 26026, 25820, 25807, 25673,
         25473, 25450, 25651, 25555, 25703, 25710, 25849, 25914, 25978)
day <- 1:length(DJIA)
autoplot(acf(DJIA, plot = F)) + theme_bw() + labs(title = "ACF For DJIA")
```

```
Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the ggfortify package.
```

Please report the issue at <https://github.com/sinhrks/ggfortify/issues>.



The autocorrelation at lag k can be computed using

$$\hat{\rho}_k = \frac{\sum_{t=k+1}^n (x_t - \bar{x})(x_{t-k} - \bar{x})}{\sum_{t=1}^n (x_t - \bar{x})^2}$$

where $\bar{x}$ is the average value of the time series and x_t is the t th value of the time series.

```
# ACF at lag 1
sum((DJIA[2:25] - mean(DJIA)) * (DJIA[1:24] - mean(DJIA))) /
  sum((DJIA - mean(DJIA))^2)
[1] 0.6903616
```

```
# ACF at lag 2
sum((DJIA[3:25] - mean(DJIA)) * (DJIA[1:23] - mean(DJIA))) /
  sum((DJIA - mean(DJIA))^2)
[1] 0.5067019
```

```
# Verify
acf(DJIA, lag.max = 2, plot = F)
```

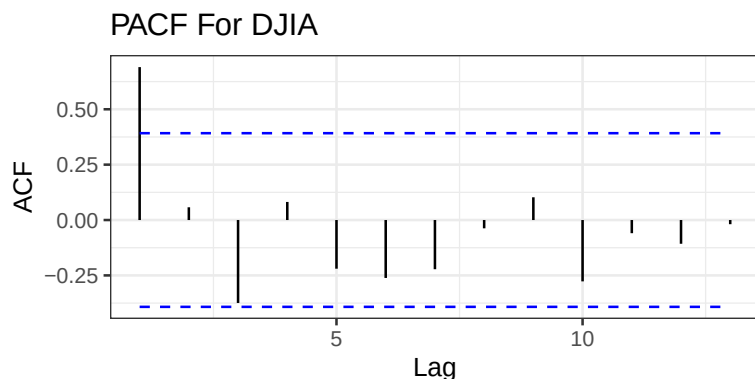
Autocorrelations of series 'DJIA', by lag

```
      0      1      2
1.000 0.690 0.507
```

PACF

We can plot the partial autocorrelation function (PACF) values using the `pacf()` function. The PACF is useful because it controls for autocorrelation at other lags. So, the k th value of the PACF is the correlation between the time series values separated by a lag of k while controlling for all lags between them.

```
autoplot(pacf(DJIA, plot = F)) + theme_bw() + labs(title = "PACF For DJIA")
```

Time Series Model

The main idea of a time series model is that the data, which are represented by x_t , can be decomposed into several components:

1. The trend, m_t .
2. The seasonality, s_t .
3. The error, e_t . This is usually a sequence of correlated random variables with mean zero.

Then there are two primary types of time series models:

Additive: $x_t = m_t + s_t + e_t$

Multiplicative: $x_t = m_t \cdot s_t \cdot e_t$

Of course, like linear regression, transformations can be made on x_t as well.

An **additive model**, $x_t = m_t + s_t + e_t$, is appropriate if the magnitude of the seasonal fluctuations does not vary with the level of time series.

A **multiplicative model**, $x_t = m_t \cdot s_t \cdot e_t$ is appropriate if the seasonal fluctuations increase or decrease proportionally with increases and decreases in the level of the series.

Taking the log of a multiplicative model makes it additive:

$$\log(x_t) = \log(m_t \cdot s_t \cdot e_t)$$

$$\log(x_t) = \log(m_t) + \log(s_t) + \log(e_t)$$

Air Passengers Example

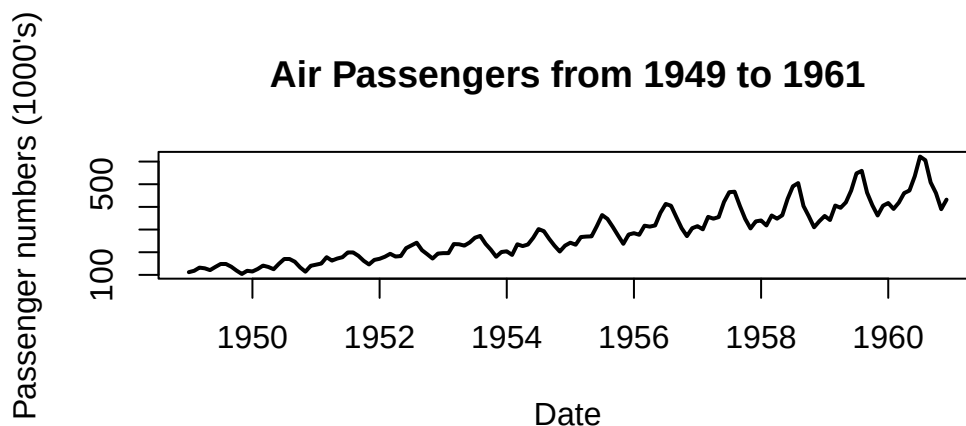
Consider the data set `AirPassengers` on the number of US airline passengers from 1949 through 1960. This data set is built into **R** and can be loaded by typing `data(AirPassengers)`.

This data set is a `ts` or time series object. That means that some function that can be used on this data set are:

- `frequency()`: tells how often it was sampled per year.
- `cycle()`: gives the cycle value for each year.
- `time()`: gives the time of each sample.
- `deltat()`: gives the difference in time between samples.
- `start()`: gives the first date in the time series.
- `end()`: gives the last date in the time series.

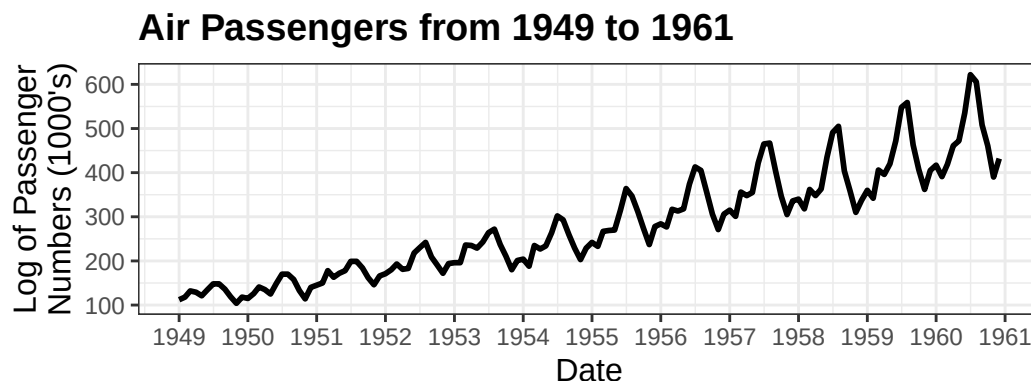
We can plot time series objects with the `plot.ts()` function. If you just use the `plot()` function, it will realize that the object is a time series and automatically use `plot.ts()`:

```
data(AirPassengers)
plot(AirPassengers, xlab = "Date", ylab = "Passenger numbers (1000's)",
     main = "Air Passengers from 1949 to 1961", lwd = 2)
```



We can also plot the time series with `ggplot2` formatting using the `autoplot()` function. For time series plots, like with the `acf()` and `pacf()` functions, the `ggfortify` package is also required. When plotting a time series object with `autoplot()`, we can add onto it as if it was a `ggplot`. Note that the `scale_x_yearmonth()` function is from the `tsibble` package and is useful when plotting time series.

```
library(tsibble)
library(ggfortify)
autoplot(AirPassengers, size = 1) +
  scale_x_yearmonth(date_breaks = "1 year", date_labels = "%Y") +
  scale_y_continuous(breaks = seq(100, 600, 100)) +
  labs(x = "Date", y = "Log of Passenger\nNumbers (1000's)",
       title = "Air Passengers from 1949 to 1961") +
  theme(axis.title = element_text(size = 12),
        plot.title = element_text(size = 14, face = "bold"))
```



Right now the `AirPassengers` data is of class `ts`, which means **R** knows it is a time series. However, it is not in tidy format. We can put it in a `tsibble`, which is just like a `tibble` but for time series.

```
library(tsibble)
AP <- as_tsibble(AirPassengers) |>
  rename(date = index, num_pass = value) |>
  mutate(lpass = log(num_pass))
head(AP)
# A tsibble: 6 x 3 [1M]
   date num_pass lpass
   <month>   <dbl> <dbl>
1 1949 Jan     112  4.72
2 1949 Feb     118  4.77
3 1949 Mar     132  4.88
4 1949 Apr     129  4.86
5 1949 May     121  4.80
6 1949 Jun     135  4.91
```

Another nice advantage `tsibbles` have is that they work with these functions from the `lubridate` package that is a part of the `tidyverse`:

- `year()`: gives the year of that observation in the time series.
- `month()`: gives the month of that observation in the time series.
- `quarter()`: gives the quarter of that observation in the time series.
- `week()`: gives the week of that observation in the time series.
- `day()`: gives the day of that observation in the time series.
- `hour()`: gives the hour of that observation in the time series.
- `minute()`: gives the minute of that observation in the time series.
- `second()`: gives the second of that observation in the time series.

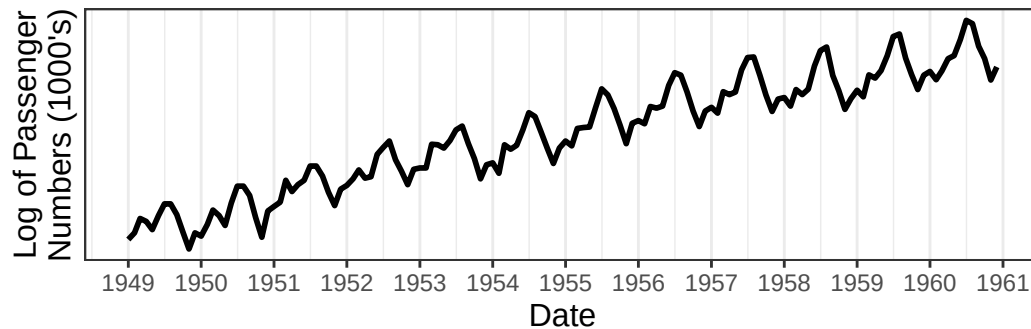
Not all of these functions are appropriate for each time series. For the air passengers time series, `year()` and `month()` would be most useful.

```
head(month(AP$date), 14)
[1] 1 2 3 4 5 6 7 8 9 10 11 12 1 2
```

Note that `tsibble` objects work just like `ts` ones when it comes to the `autoplot()` function. But they can also be plotted with the `ggplot()` function like we are used to doing:

```
ggplot(AP, aes(x = date, y = lpass)) +
  geom_line(linewidth = 1) +
  scale_x_yearmonth(date_breaks = "1 year", date_labels = "%Y") +
  scale_y_continuous(breaks = seq(100, 600, 100)) +
  labs(x = "Date", y = "Log of Passenger\nNumbers (1000's)",
       title = "Air Passengers from 1949 to 1961") +
  theme(axis.title = element_text(size = 12),
        plot.title = element_text(size = 14, face = "bold"))
```

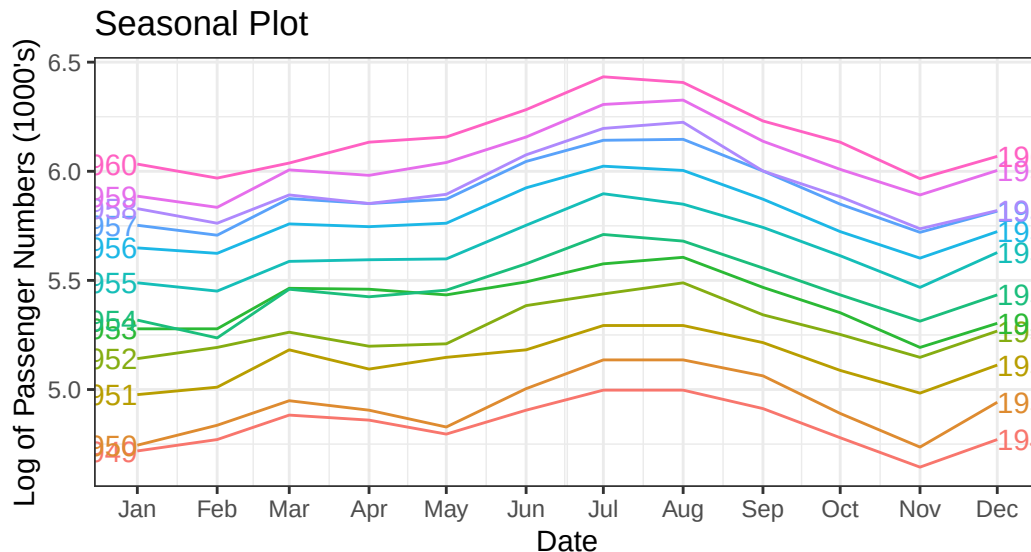
Air Passengers from 1949 to 1961



Air Passengers Example - Seasonality

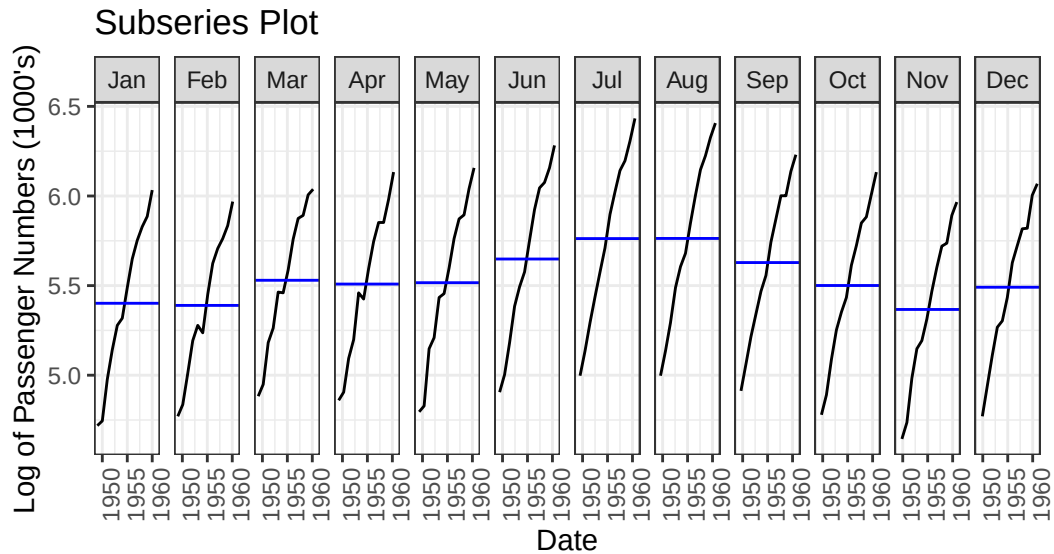
We can see that there is a cyclic nature to this time series (as is often the case) that repeats each year. This is known as the *seasonality* component. Let's check out how air passenger numbers look by month. For many nice plotting functions, we can use the `ggtime` library.

```
library(ggtime)
gg_season(AP, y = lpass, labels = "both") +
  labs(x = "Date", y = "Log of Passenger Numbers (1000's)",
       title = "Seasonal Plot") +
  theme_bw()
```



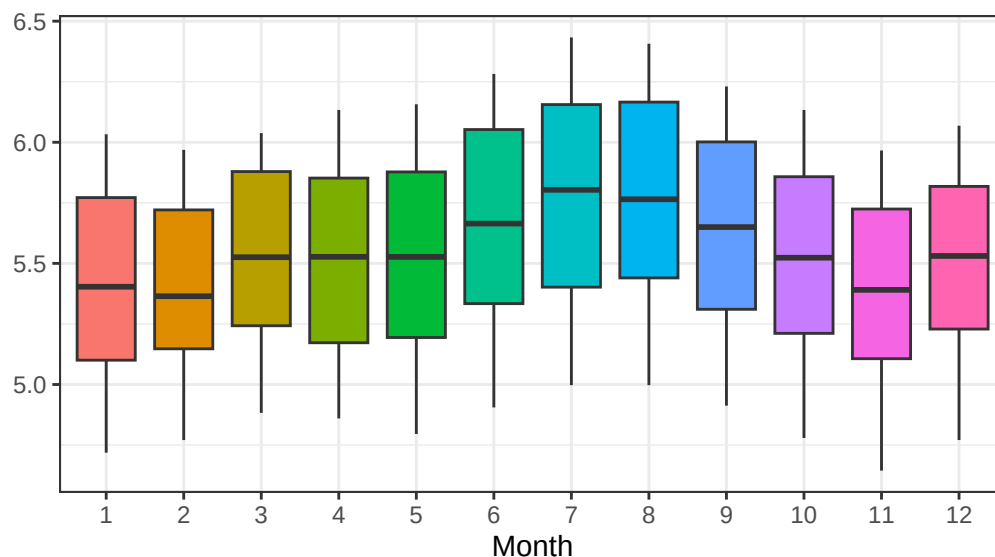
We can use the `gg_subseries()` (also in the `ggtime` package) to obtain another potentially useful plot. The average for each month is shown with the blue line.

```
gg_subseries(AP, y = lpass) +
  labs(x = "Date", y = "Log of Passenger Numbers (1000's)",
       title = "Subseries Plot")
```



Finally, one other familiar way to show approximately the same information is to create side-by-side boxplots by month.

```
AP |> mutate(month = factor(month(date))) |>
  ggplot(aes(x = month, y = lpass, fill = month)) +
  geom_boxplot() +
  theme(legend.position = "none") +
  labs(x = "Month", y = "")
```



Time Series Modeling

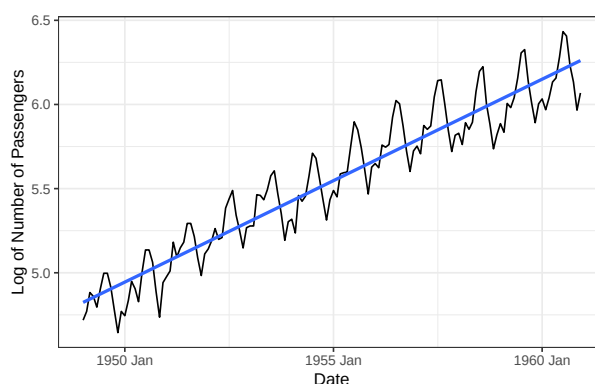
When modeling a time series, we are going to want to estimate the trend (m_t), the seasonality (s_t), and the error (e_t). When we do this, we will assume an additive log model, which is equivalent to a multiplicative model, so the data can be written as

$$\log(x_t) = \log(m_t) + \log(s_t) + \log(e_t)$$

The trend is usually the simplest to estimate. Plotting a line through the plot, we can see that there seems to be a fairly linear increase in the number of passengers. So let's use linear regression to estimate and then remove the trend.

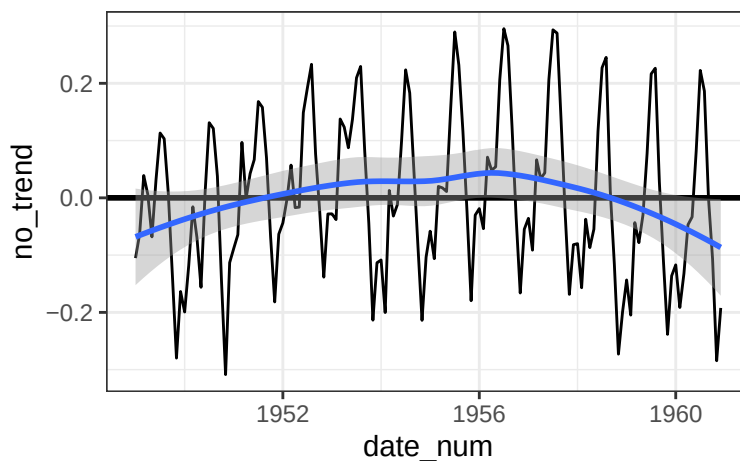
Estimating the Trend - Linear

```
ggplot(AP, aes(x = date, y = lpass)) +
  geom_line() +
  geom_smooth(method = "lm", se = F) +
  labs(
    x = "Date",
    y = "Log of Number of Passengers"
  )
```



Let's plot the de-trended data value to see how they look.

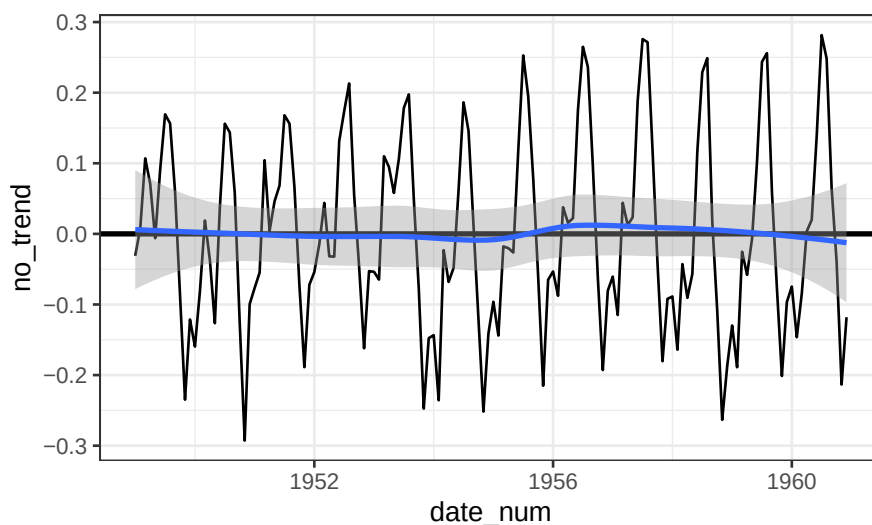
```
# Convert date to numeric
date_num <- year(AP[[1]]) + (month(AP[[1]]) - 1)/12
trend_mod <- lm(lpass ~ date_num, data = AP)
xt <- AP$num_pass
logxt <- AP$lpass
logmt <- trend_mod$fitted.values
# The de-trended values can be found with trend_mod$residuals
AP |> mutate(no_trend = logxt - logmt) |>
  ggplot(aes(x = date_num, y = no_trend)) +
  geom_line() +
  geom_hline(yintercept = 0, linewidth = 1) +
  geom_smooth()
```



Estimating the Trend - Quadratic

Let's try it again using quadratic regression:

```
trend_mod2 <- lm(lpass ~ poly(date_num, degree = 2, raw = T), data = AP)
logxt <- AP$lpass
logmt <- trend_mod2$fitted.values
# The de-trended values can also be found with trend_mod$residuals
AP |> mutate(no_trend = logxt - logmt) |>
  ggplot(aes(x = date_num, y = no_trend)) +
  geom_line() +
  geom_hline(yintercept = 0, linewidth = 1) +
  geom_smooth()
```



Removing the Trend

Now that we have removed the trend, we are still left with modeling the seasonality and error.

$$x_t = m_t + s_t + e_t$$

$$x_t - m_t = s_t + e_t$$

Now we can say that $y_t = x_t - m_t$ (what I labeled `no_trend`). This gives the model

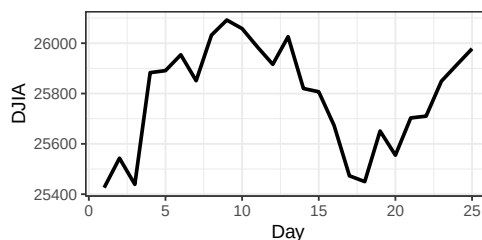
$$y_t = s_t + e_t$$

Clearly the plot on the previous slide is not linear, but there is a pattern. We need a new type of model for modeling the *seasonality*.

Other Non-linear Trends

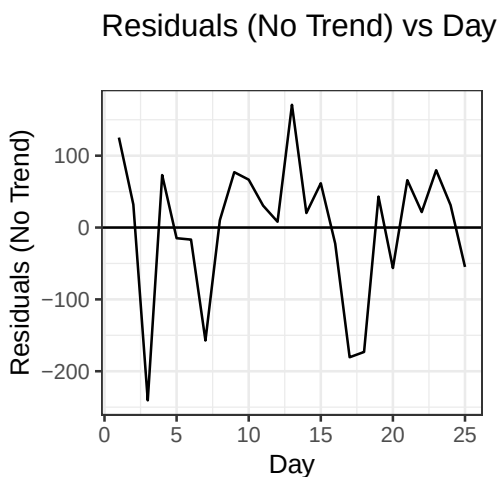
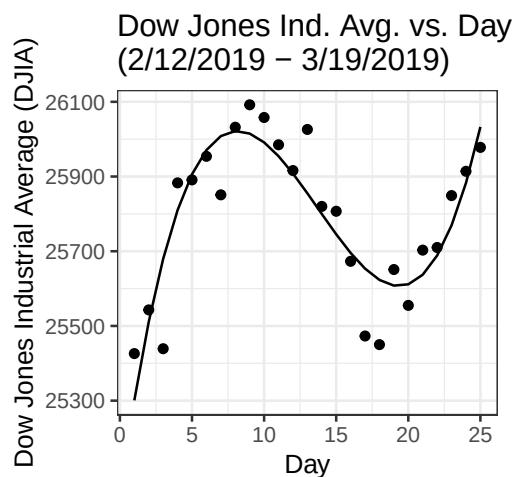
Before we get to seasonality, however, it should be noted that many trends are not (approximately) linear. The Air Passengers was modeled decently with a linear trend, but a quadratic one was better. The Dow Jones example, as we saw earlier, does not have a linear trend at all. Let's see that one again:

We can still fit a regression model here, but it would not be linear regression. Let's use the `poly()` function to fit a cubic curve to this:



```
DJIA <- c(25426, 25543, 25439, 25883, 25891, 25954, 25851, 26032,
          26092, 26058, 25985, 25916, 26026, 25820, 25807, 25673,
          25473, 25450, 25651, 25555, 25703, 25710, 25849, 25914, 25978)
day <- 1:length(DJIA)
DJ_mod_nl <- lm(DJIA ~ poly(day, degree = 3, raw = T))
DJ_df <- tibble(DJIA, day, fitted = DJ_mod_nl$fitted.values,
                w_t = DJIA - DJ_mod_nl$fitted.values)
p3 <- ggplot(DJ_df) +
  geom_point(aes(x = day, y = DJIA)) +
  geom_line(aes(x = day, y = fitted)) +
  labs(x = "Day", y = "Dow Jones Industrial Average (DJIA)") +
  ggtitle("Dow Jones Ind. Avg. vs. Day\n(2/12/2019 - 3/19/2019)")
p4 <- ggplot(DJ_df) +
  geom_line(aes(x = day, y = w_t)) +
  geom_hline(yintercept = 0) +
  labs(x = "Day", y = "Residuals (No Trend)") +
  ggtitle("Residuals (No Trend) vs Day")

p3 | p4
```



Model: $\hat{m}_t = 25025.44 + 279.1(\text{Day}) - 24.37(\text{Day})^2 + 0.5913(\text{Day})^3$

Estimating the Seasonality

Now we need to discuss how to model and remove the seasonality. Again, there are many methods for this, but the most common (and easiest) is to model it with the following:

$$\hat{s}_k = a_k - \frac{\sum_{i=1}^d a_i}{d}, \quad k = 1, \dots, d$$

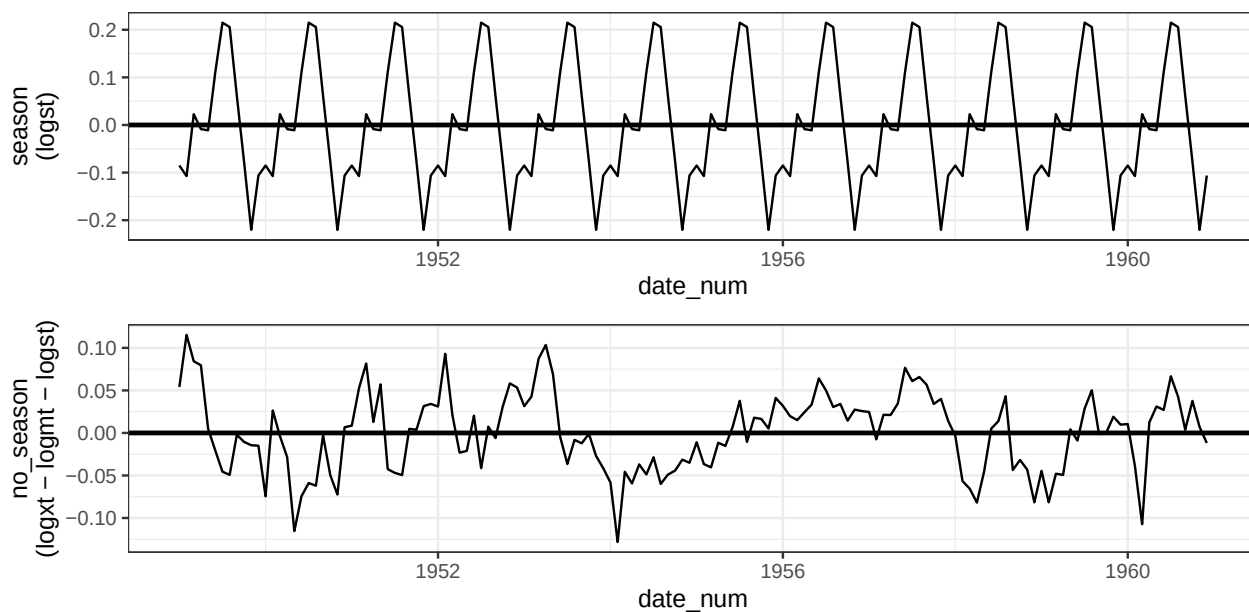
where d is the period (how many terms it takes to repeat the pattern) and a_k is the average of the deviations for the k th part of the period. Then we have $\hat{s}_k = \hat{s}_{k-d}$ for $k > d$. That is, this pattern repeats.

Note that the reason we need to subtract the mean of the a_k values, $\frac{\sum_{i=1}^d a_i}{d}$, is just because there is no guarantee that the average of the a_k is zero, so we subtract off the average.

Estimating the Seasonality - Air Passengers

For the air passengers example, our period, $d = 12$ since the seasonality repeats each year. Now, we will consider the de-trended data. a_1 will be the average for all the January months, a_2 will be the average for all February, and so on.

```
logyt = logxt - logmt
a1 = mean(logyt[seq(1, length(logyt), by = 12)])
a2 = mean(logyt[seq(2, length(logyt), by = 12)])
# Etc... Quicker to do this:
a <- logyt |>
  matrix(nrow = 12) |>
  rowMeans()
# Then estimate seasonality:
logst <- rep(a - mean(a), 12)
```



Time Series Models

There are three primary models for modeling the errors of a time series. The errors of a time series can also be thought of as a time series in their own right. They are just simpler since they do not have a trend or seasonality component.

Time series of this nature are called **stationary** if the mean and variance is constant across the series. There are several ways to model time series:

- Moving average (MA) models.
- Autoregressive (AR) models.
- Autoregressive moving average (ARMA) models.
- Autoregressive integrated moving average (ARIMA) models. This is for non-stationary models that need to be **differenced** to become stationary.

Moving Average (MA) Models

Moving average (MA) models of degree q are those in which the stationary time series is a weighted average of q white-noise variables. This looks like

$$X_t = Z_t + \theta_1 Z_{t-1} + \cdots + \theta_q Z_{t-q}$$

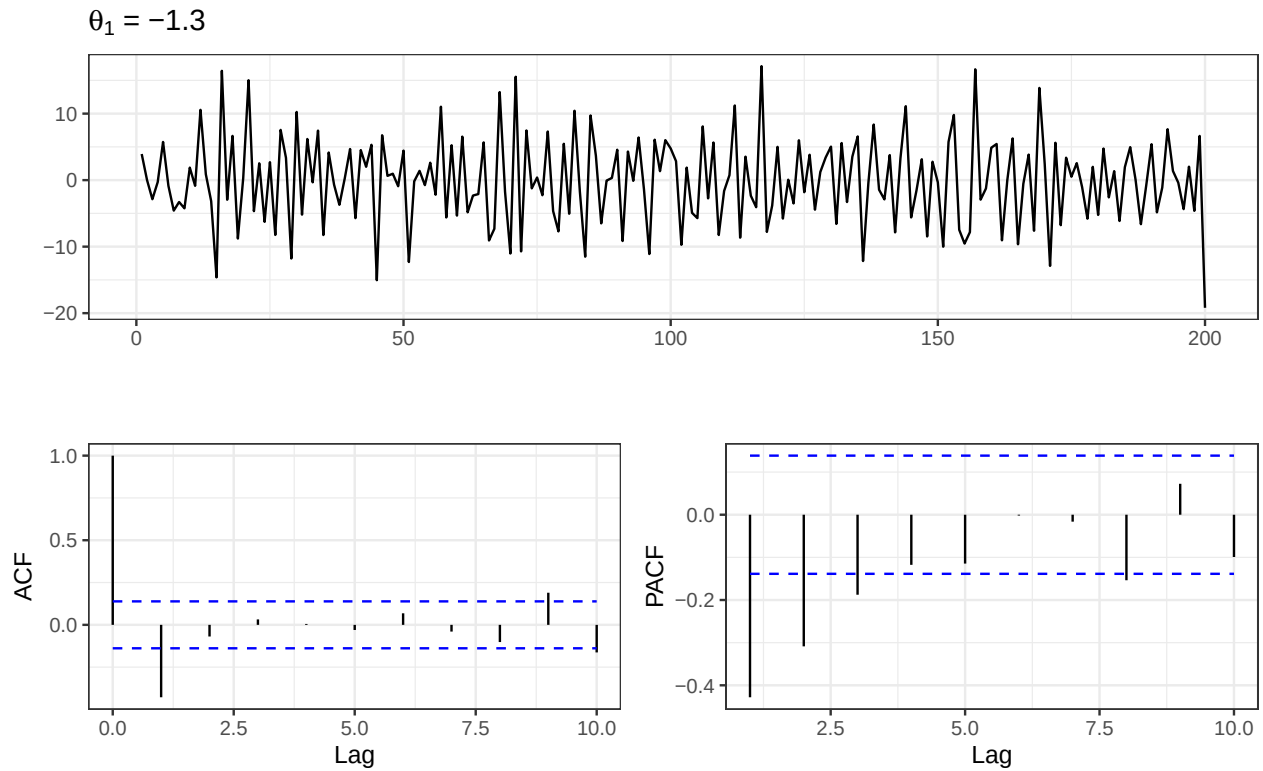
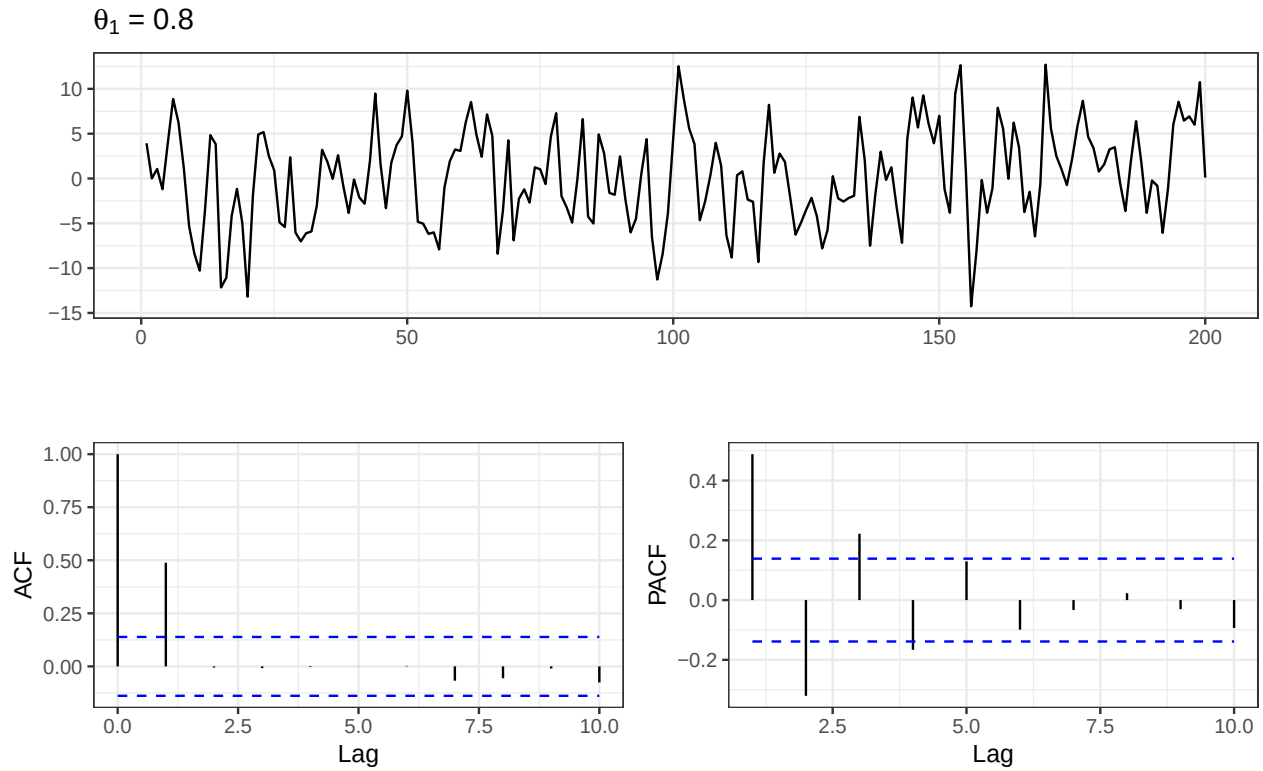
where $\{Z_t\} \sim WN(0, \sigma^2)$ and $\theta_1, \dots, \theta_q$ are constants.

White noise is simply a term for all variables being drawn independently and identically distributed (iid) from a distribution. The most common white noise is **Gaussian white noise** where all variables come from the same normal distribution.

MA Model Examples - MA(1)

Let's simulate what a few MA time series may look like:

```
# Degree 1
n = 200
set.seed(2024)
x <- rep(0, n)
z <- rnorm(n, 0, 4) # Gaussian WN with sigma = 4
theta1 <- 0.8
x[1] <- z[1]
for (i in 3:n) {
  x[i] <- z[i] + theta1*z[i-1]
}
p1 <- autoplot(ts(x)) +
  labs(title = expr(paste(theta1, " = ", !!theta1)))
p2 <- acf(x, plot = F, lag.max = 10) |>
  autoplot() + labs(title = "")
p3 <- pacf(x, plot = F, lag.max = 10) |>
  autoplot() +
  labs(y = "PACF", title = "")
p1 / (p2 | p3)
```



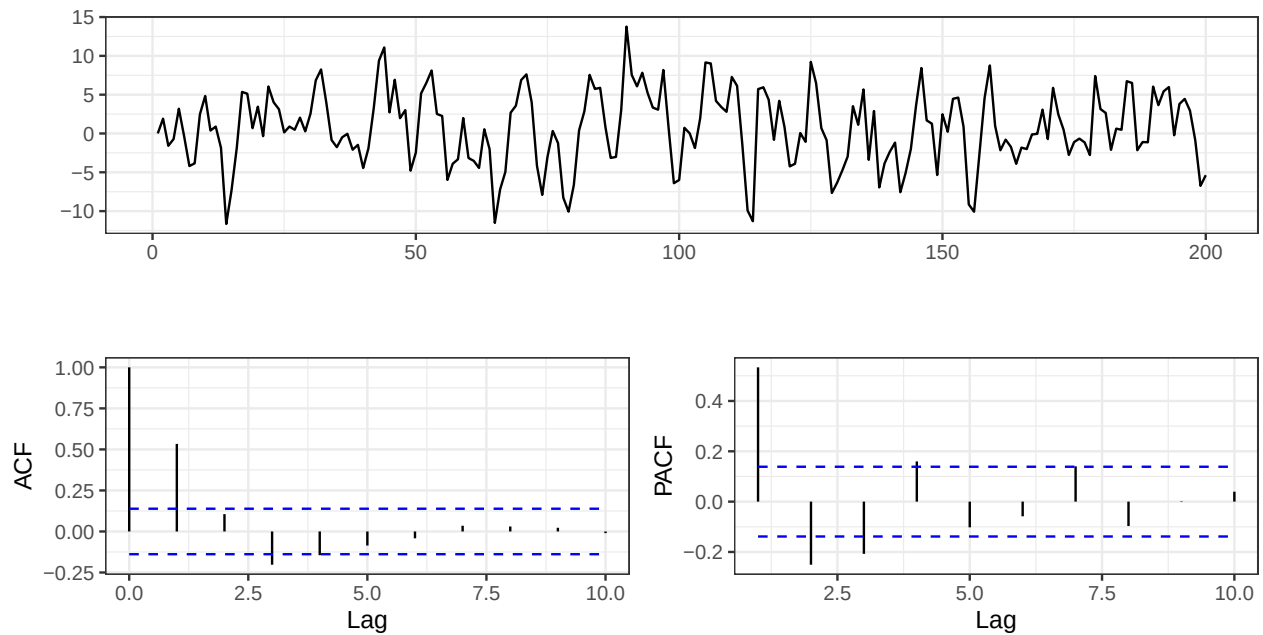
MA Model Examples - MA(2)

```

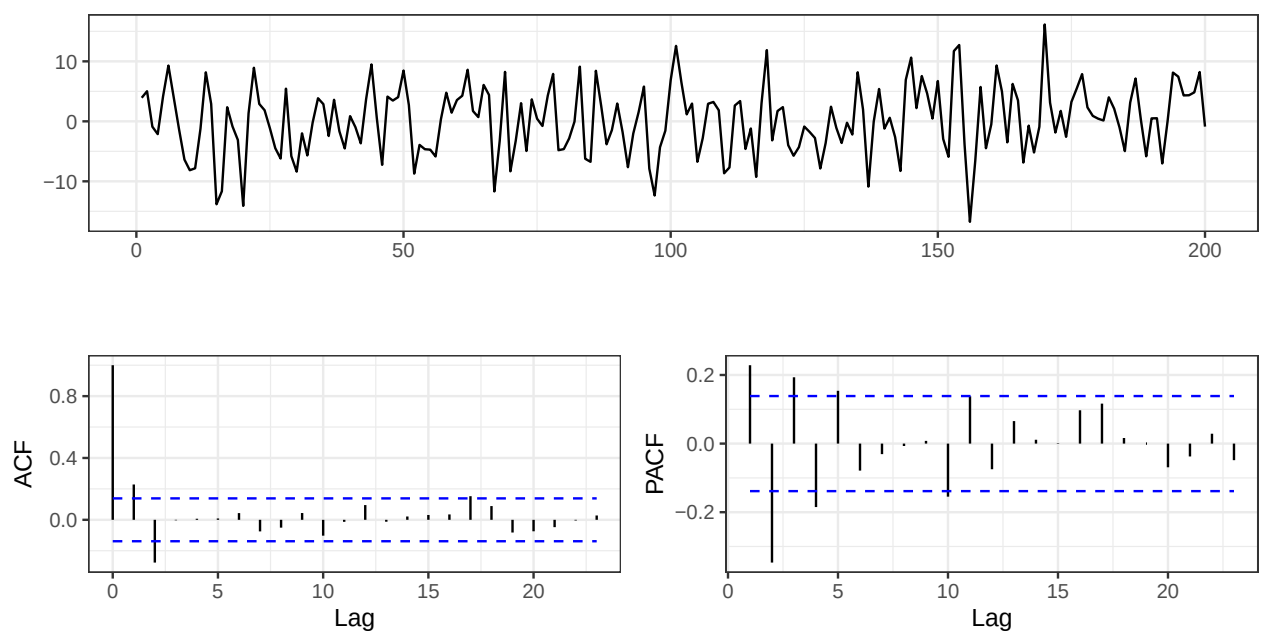
n <- 200; x <- rep(0, n); z <- rnorm(n, 0, 4) # Gaussian WN with sigma = 4
set.seed(2024)
theta1 <- 0.8; theta2 <- 0.5
x[1] <- z[1]; x[2] <- z[2] + theta1*z[1]
for (i in 3:n) {
  x[i] <- z[i] + theta1*x[i-1] + theta2*z[i-2]
}

```

$\theta_1 = 0.8$ and $\theta_2 = 0.5$



$\theta_1 = 0.8$ and $\theta_2 = -0.5$



Autoregressive (AR) Models

An autoregression or autoregressive model of degree p , an AR(p), is one in which the time series is a weighted average of p of its previous terms plus noise. This looks like

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + Z_t$$

where $\{Z_t\} \sim WN(0, \sigma^2)$ and $\phi_1, \dots, \phi_p$ are constants.

For the AR(p) model to be stationary, we need the roots of the polynomial $1 - \phi_1 z - \cdots - \phi_p z^p$ to be outside the unit circle. That is, $|z_i| > 1$ for all i .

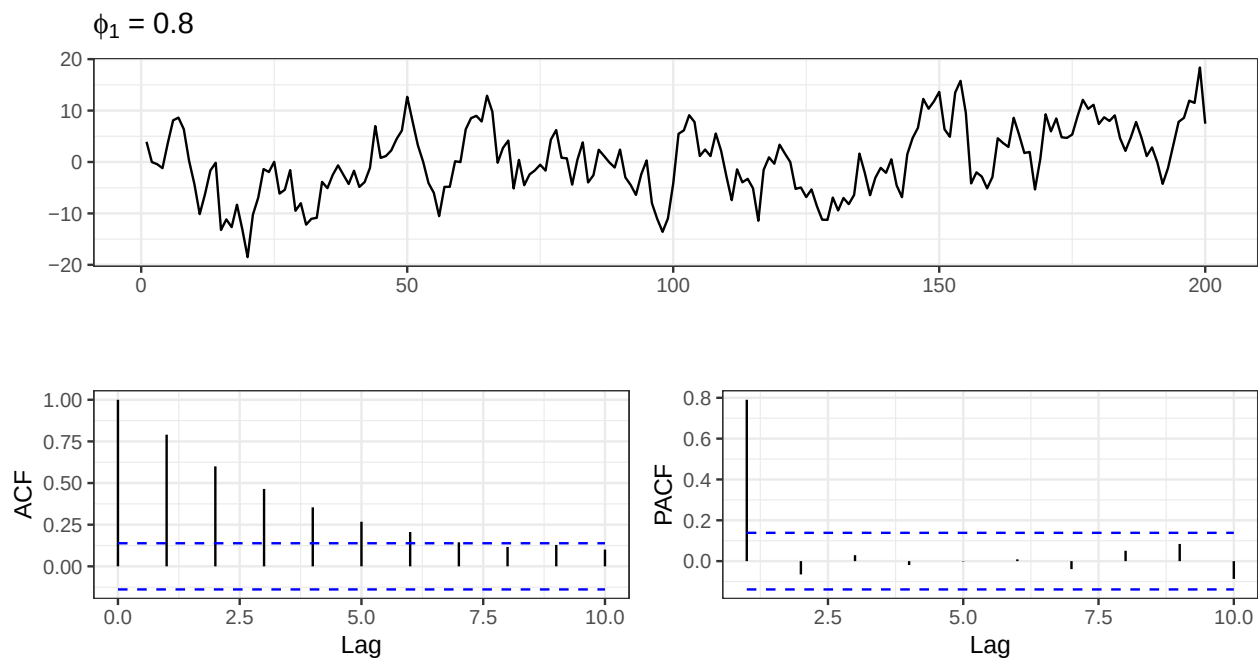
This can alternatively be written

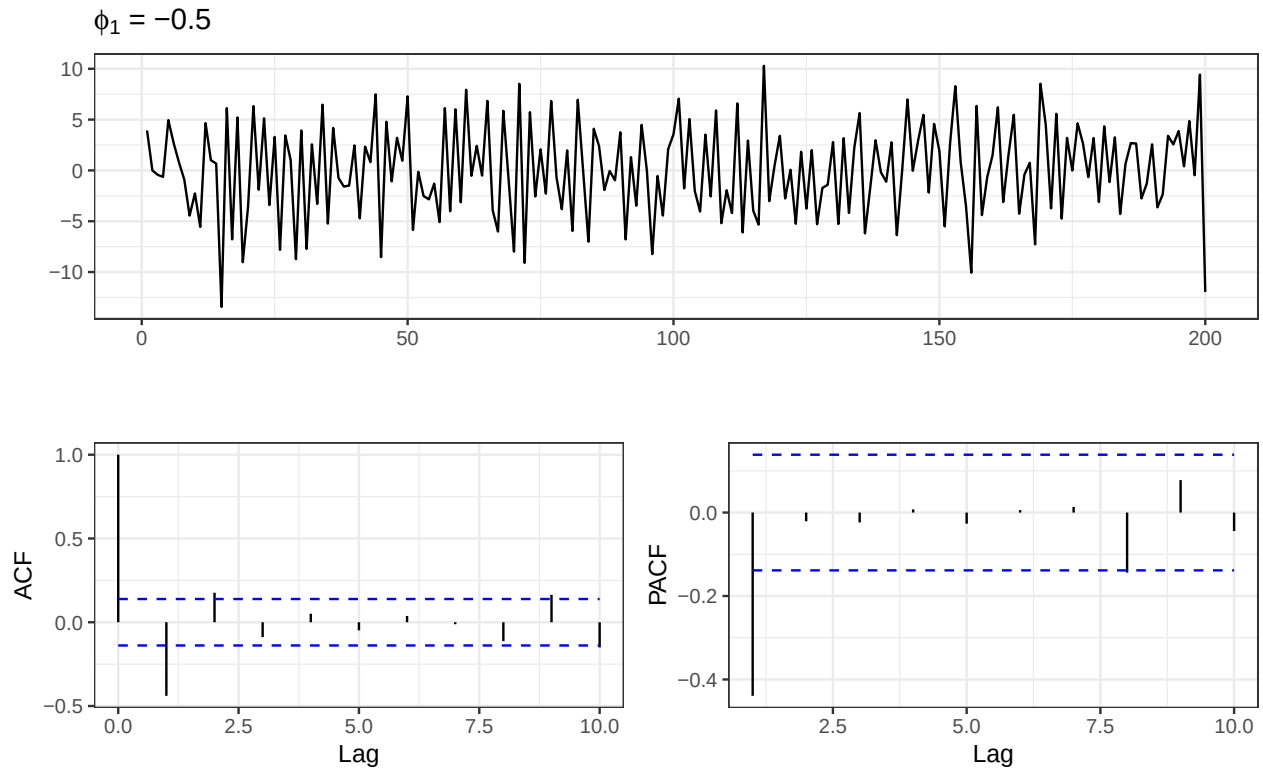
$$X_t - \phi_1 X_{t-1} - \phi_2 X_{t-2} - \cdots - \phi_p X_{t-p} = Z_t$$

AR Model Examples - AR(1)

Let's simulate what a few AR time series may look like:

```
# Degree 1
n = 200
set.seed(2024)
x <- rep(0, n)
z <- rnorm(n, 0, 4) # Gaussian WN with sigma = 4
phi1 <- 0.8
x[1] <- z[1]
for (i in 3:n) {
  x[i] <- phi1*x[i-1] + z[i]
}
```



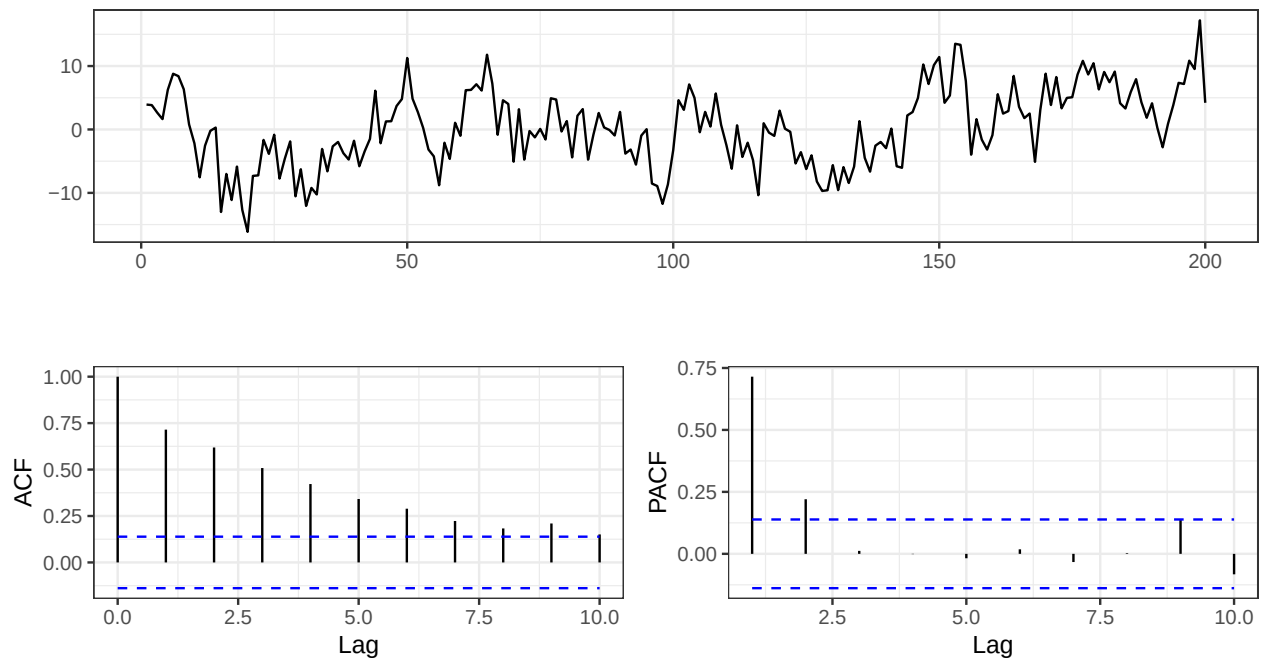


AR Model Examples - AR(2)

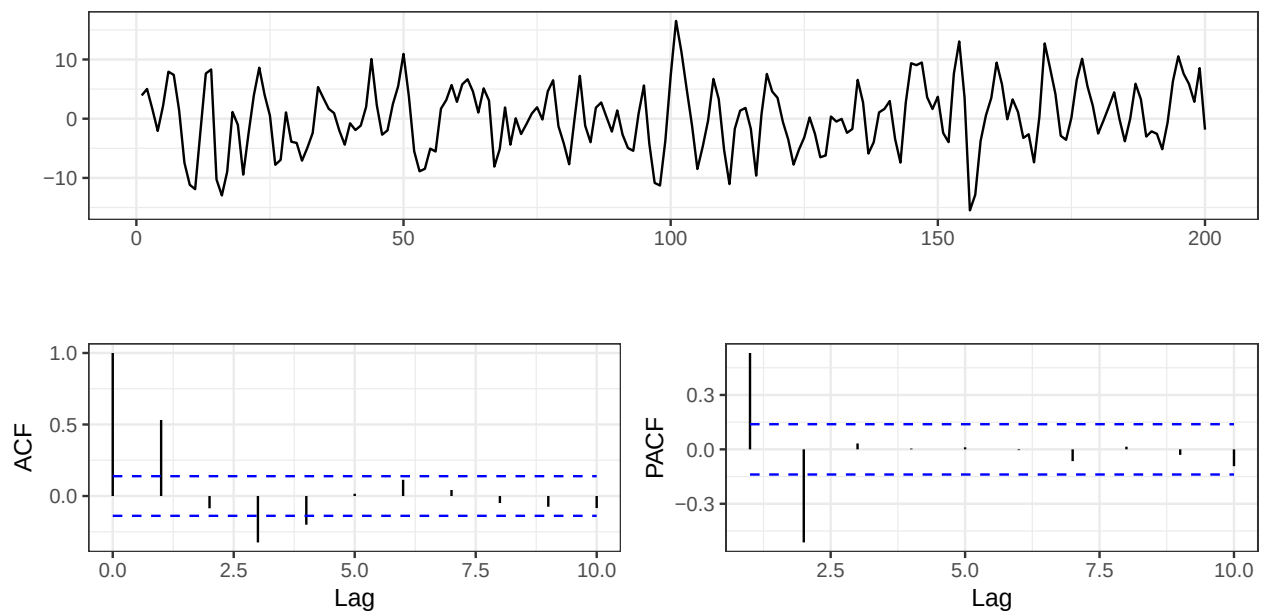
```
# Degree 2
n = 200
set.seed(2024)
x <- rep(0, n)
z <- rnorm(n, 0, 4) # Gaussian WN with sigma = 4
phi1 <- 0.5
phi2 <- 0.3
x[1] <- z[1]
x[2] <- phi1*x[1] + z[2]
for (i in 3:n) {
  x[i] <- phi1*x[i-1] + phi2*x[i-2] + z[i]
}
```

Stationary. The roots of $1 - 0.5z - 0.3z^2$ are -2.84 and 1.174 , each of which is larger than 1 in absolute value.

$\phi_1 = 0.5$ and $\phi_2 = 0.3$



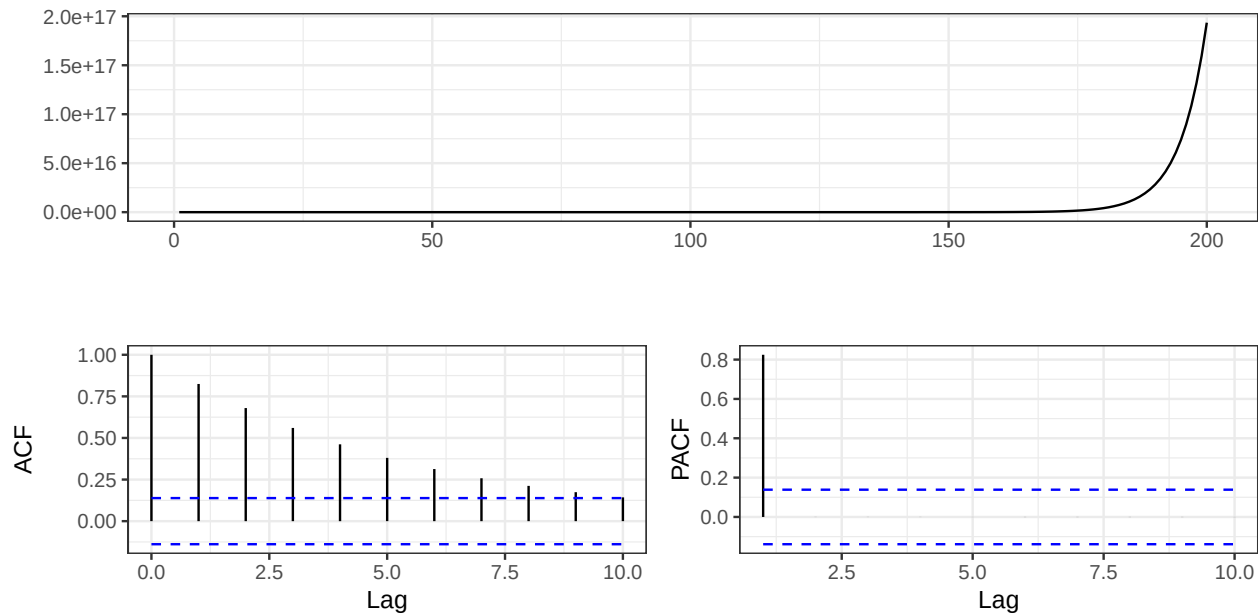
$\phi_1 = 0.8$ and $\phi_2 = -0.5$



This is also stationary. The roots of $1 - 0.8z + 0.5z^2$ are $0.8 - 1.17i$ and $0.8 + 1.17i$, each with modulus $\sqrt{0.8^2 + 1.17^2} = 1.42$, which is larger than 1.

AR Model Examples - Not Stationary

$$\phi_1 = 0.8 \text{ and } \phi_2 = 0.5$$



This time series is **not stationary**. The roots of $1 - 0.8z - 0.5z^2$ are -2.425 and 0.825 . The second root is less than 1.

Autoregressive Moving-Average (ARMA) Models

The autoregressive moving-average model is a combination of an AR(p) and an MA(q) models. This is abbreviated ARMA(p,q). The model can be written

$$X_t = \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + Z_t + \theta_1 Z_{t-1} + \cdots + \theta_q Z_{t-q}$$

or

$$X_t = Z_t + \sum_{i=1}^p \phi_i X_{t-i} + \sum_{j=1}^q \theta_j Z_{t-j}$$

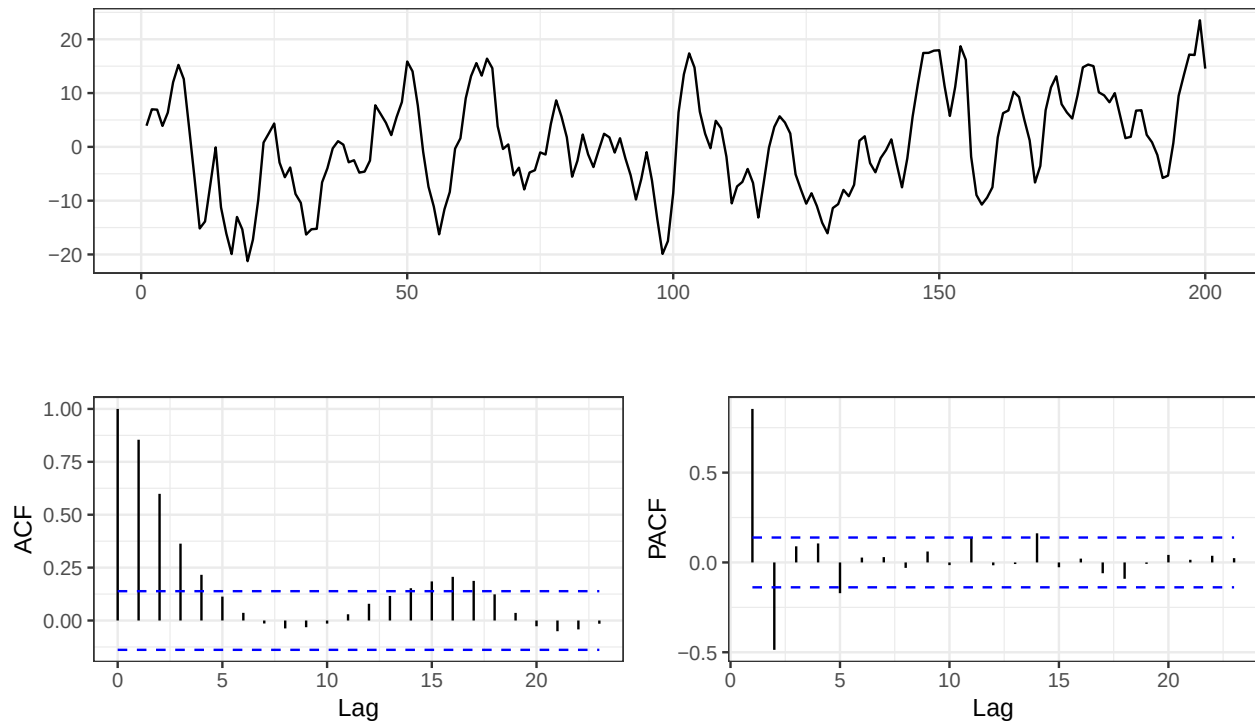
This is sometimes written as

$$X_t - \phi_1 X_{t-1} - \cdots - \phi_p X_{t-p} = Z_t + \theta_1 Z_{t-1} + \cdots + \theta_q Z_{t-q}$$

ARMA Examples - ARMA(2,2)

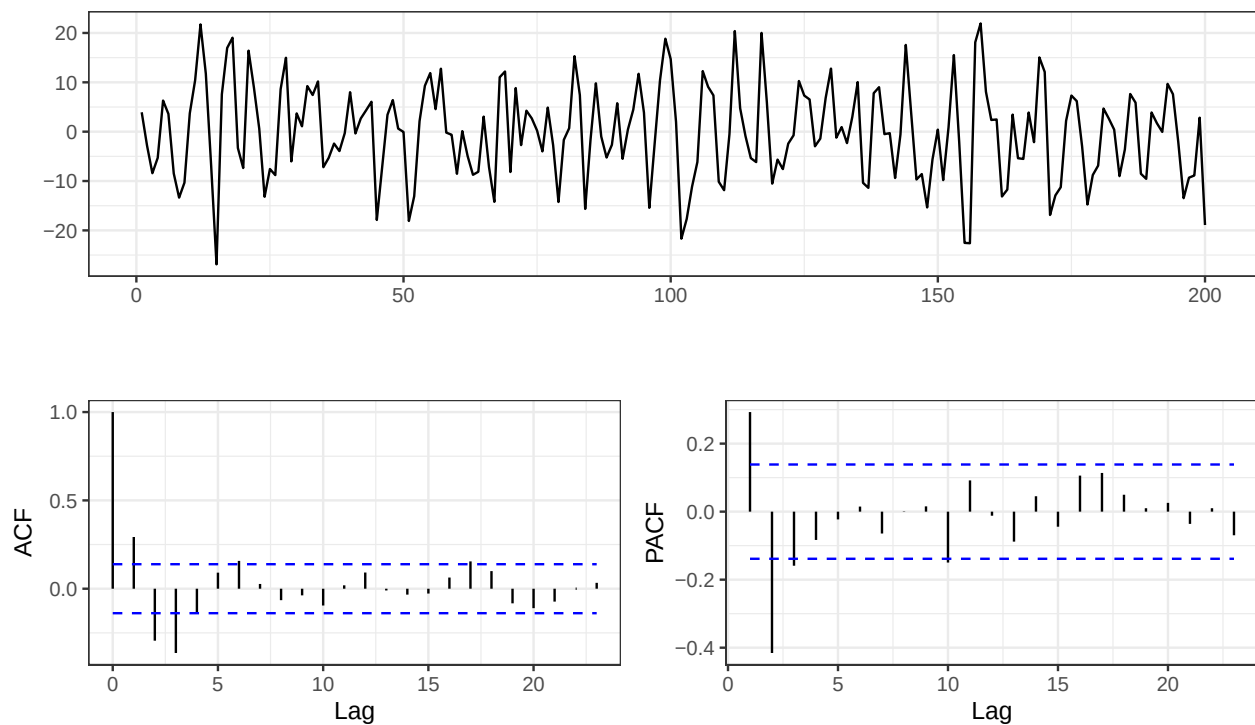
```
n <- 200; set.seed(2024)
x <- rep(0, n); z <- rnorm(n, 0, 4) # Gaussian WN with sigma = 4
phi1 <- 0.5; phi2 <- 0.1; theta1 <- 0.8; theta2 <- 0.5
x[1] <- z[1]; x[2] <- phi1*x[1] + z[2] + theta1*z[1]
for (i in 3:n) {
  x[i] <- phi1*x[i-1] + phi2*x[i-2] + z[i] +
    theta1*z[i-1] + theta2*z[i-2]
}
```


$\phi_1 = 0.5$, $\phi_2 = 0.1$, $\theta_1 = 0.8$, and $\theta_2 = 0.5$



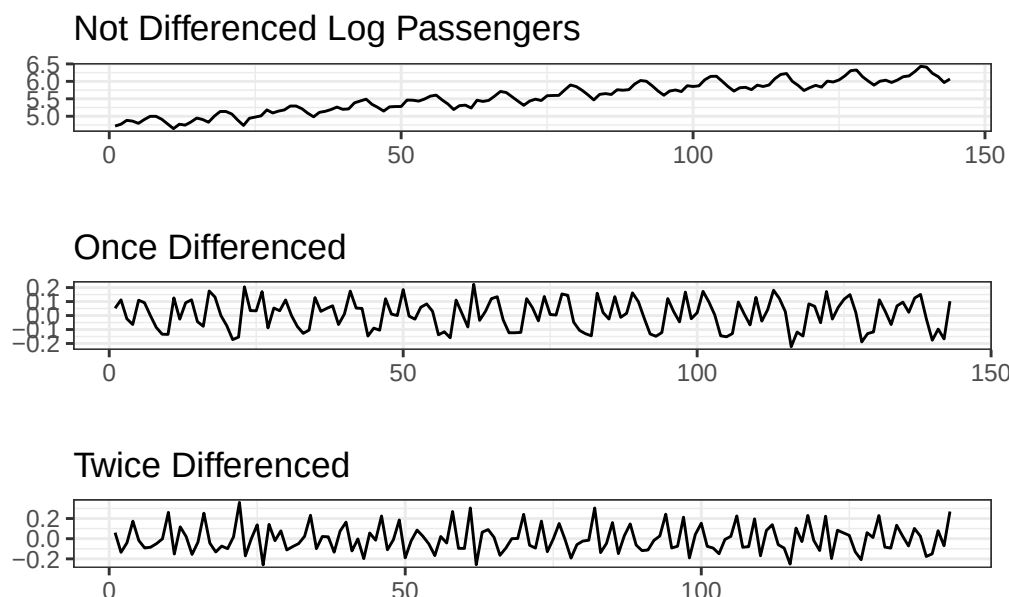
ARMA Examples - ARMA(2,1)

$\phi_1 = 0.8$, $\phi_2 = -0.5$, and $\theta_1 = -2$



Differencing

Sometimes differencing a time series can make it stationary and make the model fit better. Consider the Air Passengers data set with and without differencing.



Fitting an ARMA Model

Of course, our goal is to try to model the errors by fitting an ARMA model to them. Then we can add the seasonality and trend back to get a model we can use for **forecasting**.

There are three things we should do when fitting an ARMA model to a time series:

1. Make sure the resulting series is likely stationary.
2. Determine the order of the ARMA(p, q) process. That is, determine p and q .
3. Determine the proper ϕ and θ value(s). This is most commonly done with maximum likelihood, but we will not get into that here.

Determining if Series is Stationary

A common test for stationarity is the augmented Dickey-Fuller test. This test can be written as

$$H_0 : \text{The time series is not stationary}$$

$$H_A : \text{The time series is stationary}$$

So, we would like to reject the null hypothesis here.

Testing for Stationarity

We can use the `adf.test()` function in the `tseries` library to test this. If there is seasonality in the time series, then it is best to set `k` equal to the seasonality period. We have removed the trend and seasonality in the Air Passengers time series, so we will leave this as the default.

```
library(tseries)
loget <- logxt - logmt - logst
adf.test(loget)
```

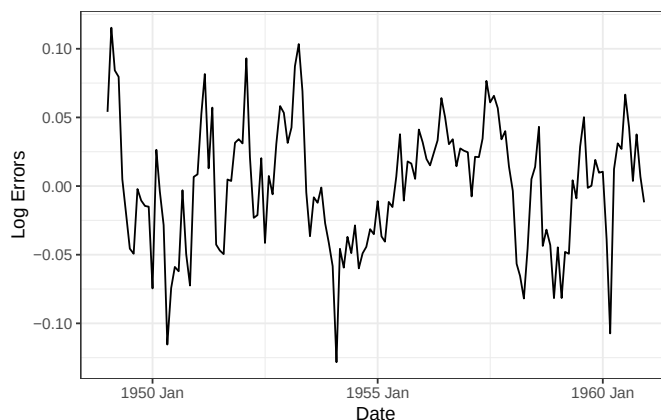
Augmented Dickey-Fuller Test

```
data: loget
Dickey-Fuller = -3.555, Lag order = 5, p-value = 0.0399
alternative hypothesis: stationary
```

Fitting an ARMA Model

Let's take another look at the log error data set.

```
AP2 <- AP |>
  mutate(logxt, logmt, logst, loget)
ggplot(
  AP2,
  aes(x = date, y = loget)
) +
  geom_line() +
  labs(x = "Date", y = "Log Errors")
```



We can fit an ARMA model to the data using the `arima()` function in the `stats` library that is already pre-loaded into **R**.

Note: the ARIMA is a slightly different model than the ARMA. The ARIMA allows for the data to be differenced to make it stationary if it isn't already. Ours already is.

In the `arima()` function, we need to specify the order we want to fit. This should be a vector with three arguments: (p, d, q) . p is the order of the AR part of the model, d is the number of times the time series should be differenced to make it stationary, and q is the order of the MA part of the model.

Let's start with fitting an AR(1) model. Setting `include.mean = F` will omit an intercept term.

```
# Fit an AR(1)
arima(loget, order = c(1, 0, 0), include.mean = F)
```

Call:

```
arima(x = loget, order = c(1, 0, 0), include.mean = F)
```

Coefficients:

```
      ar1
    0.6734
s.e. 0.0612
```

```
sigma^2 estimated as 0.001143: log likelihood = 283.08, aic = -562.15
```

```
# Fit an MA(2)
arima(loget, order = c(0, 0, 2), include.mean = F))
# Fit an ARMA(2,1)
arima(loget, order = c(2, 0, 1), include.mean = F))
```

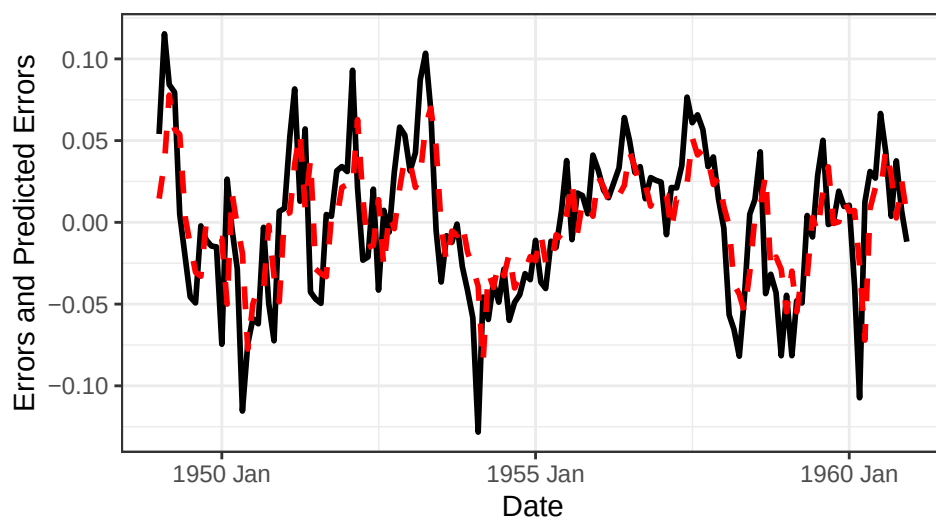
Model Fit Table

Model	AIC	BIC
AR(1)	-562.16	-556.21
AR(2)	-560.91	-552.00
AR(3)	-560.79	-548.91
MA(1)	-530.76	-524.82
MA(2)	-550.02	-541.11
MA(3)	-566.16	-554.29
ARMA(1, 1)	-560.71	-551.80
ARMA(2, 1)	-559.38	-547.50
ARMA(1, 2)	-562.41	-550.54
ARMA(2, 2)	-560.88	-546.03
ARMA(3, 2)	-564.18	-546.37
ARMA(2, 3)	-562.40	-544.58
ARMA(3, 3)	-562.19	-541.40

We can obtain these figures using the `AIC()` and `BIC()` functions. Not `extractAIC()`.

Using an AR(1) model, let's plot the errors and the modeled values:

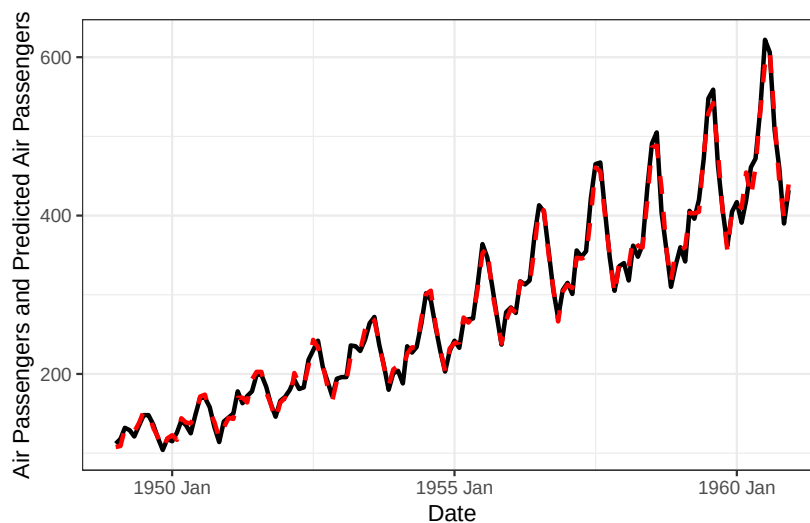
```
library(forecast) # Allows us to get fitted values from arima model
fit_lerror <- arima(loget, order = c(1, 0, 0), include.mean = T)
AP2 |> mutate(fitted = fitted.values(fit_lerror)) |>
  ggplot() +
  geom_line(aes(x = date, y = loget), linewidth = 1) +
  geom_line(aes(x = date, y = fitted), color = "red",
            linetype = "dashed", linewidth = 1) +
  labs(x = "Date", y = "Errors and Predicted Errors")
```



Transforming Back

Now we can transform all the way back!

```
predicted_series <- exp(logmt + logst +
                        fitted.values(fit_lerror))
AP2 |>
  mutate(pred = predicted_series) |>
  ggplot() +
  geom_line(aes(x = date, y = num_pass), linewidth = 1) +
  geom_line(aes(x = date, y = pred), color = "red",
            linetype = "dashed", linewidth = 1) +
  labs(x = "Date",
       y = "Air Passengers and Predicted Air Passengers")
```

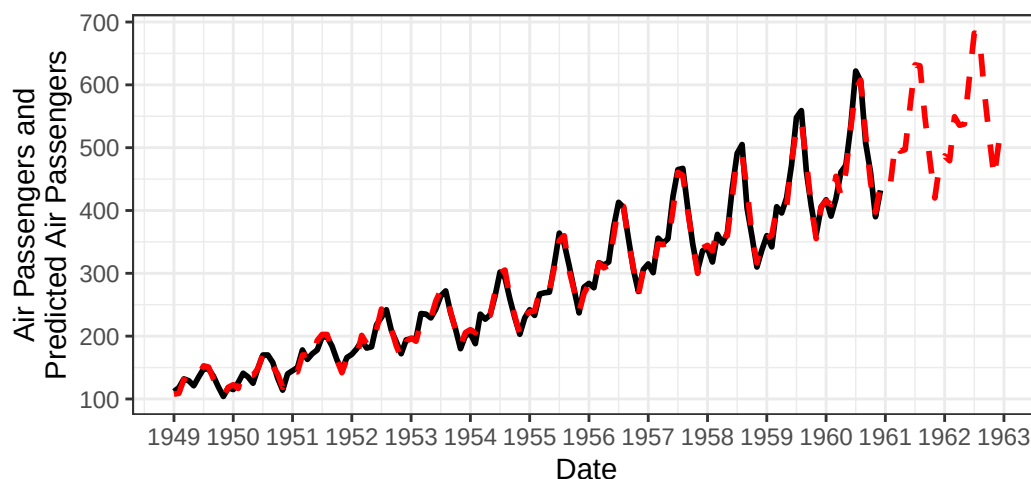


Forecasting

Now that we have everything modeled, we can use this model to predict future values of our time series. We will need the `forecast()` function in the `forecast` library **R** package for this.

```
ltrend_pred <- predict(
  trend_mod2, data.frame(date_num = seq(1961, 1962.95, by = 1/12))
)
lseason_pred <- logst[1:24] # Seasonality repeats
# h controls how many points in the future are forecasted
lerror_pred <- as.vector(
  forecast(fit_lerror, level = 95, h = 24)$mean
)
forecasted_series <- exp(ltrend_pred + lseason_pred + lerror_pred)
AP3 <- tibble(newdate = c(date_num, seq(1961, 1962.95, by = 1/12)),
              pred = c(predicted_series, forecasted_series),
              num_pass = c(AP2$num_pass, rep(NA, 24)))
```

```
ggplot(AP3) +
  geom_line(aes(x = newdate, y = num_pass), linewidth = 1) +
  geom_line(aes(x = newdate, y = pred), color = "red",
            linetype = "dashed", linewidth = 1) +
  labs(x = "Date",
       y = "Air Passengers and\nPredicted Air Passengers") +
  scale_x_continuous(breaks = 1948:1963) +
  scale_y_continuous(breaks = seq(100, 700, by = 100))
```



Forecasting Confidence Intervals

Of course, it is always the case that predicting future values comes with error. It would be nice to get some confidence intervals for these new predictions.

The way we just walked through this example is also a bit clunky. We had to model each piece separately and then put it back together.

We can solve both of these issues by fitting a model to the original time series using the `auto.arima()` function in the `forecast` package.

auto.arima() Function

```
AP_mod <- auto.arima(AirPassengers)
```

```
AP_mod
```

```
Series: AirPassengers
```

```
ARIMA(2,1,1)(0,1,0)[12]
```

```
Coefficients:
```

	ar1	ar2	ma1
	0.5960	0.2142	-0.9819
s.e.	0.0888	0.0880	0.0292

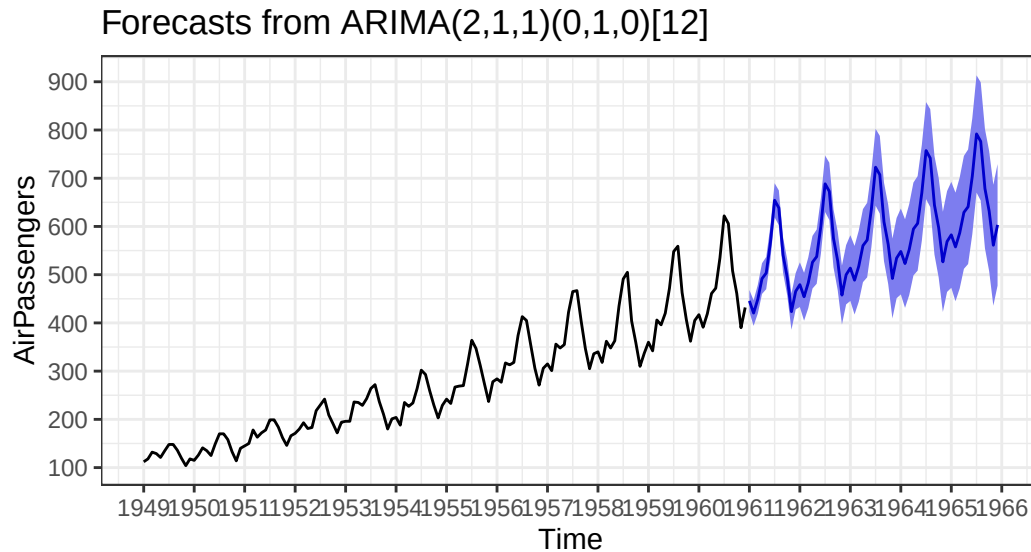
```
sigma^2 = 132.3: log likelihood = -504.92
```

```
AIC=1017.85 AICc=1018.17 BIC=1029.35
```

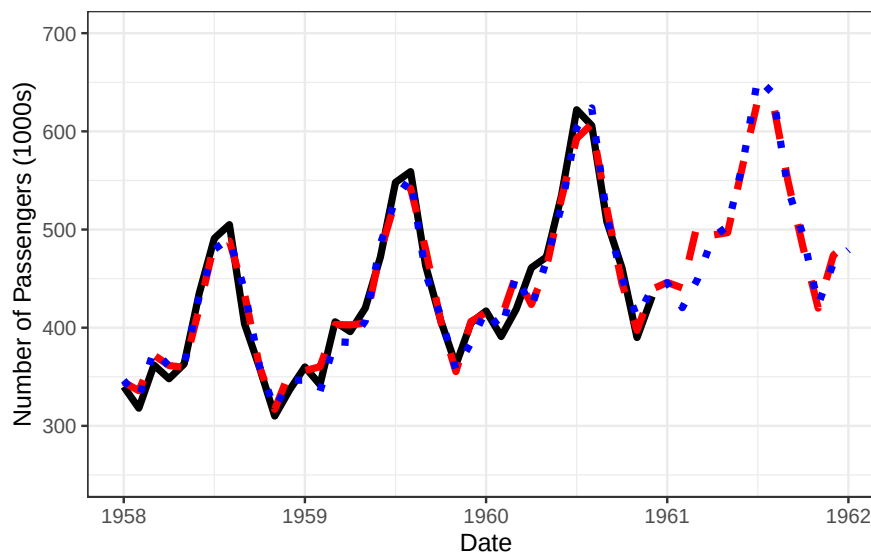
Forecasting Confidence Intervals

```
# Predict 5 years out
AP_forecast <- forecast(AP_mod, level = 95, h = 60)

# Predict 5 years out
autoplot(AP_forecast) +
  scale_x_continuous(breaks = 1948:1966) +
  scale_y_continuous(breaks = seq(100, 900, 100))
```



Compare this new forecast (with the blue dotted line) to the original forecast (with the red dashed line). The black line is the actual values.



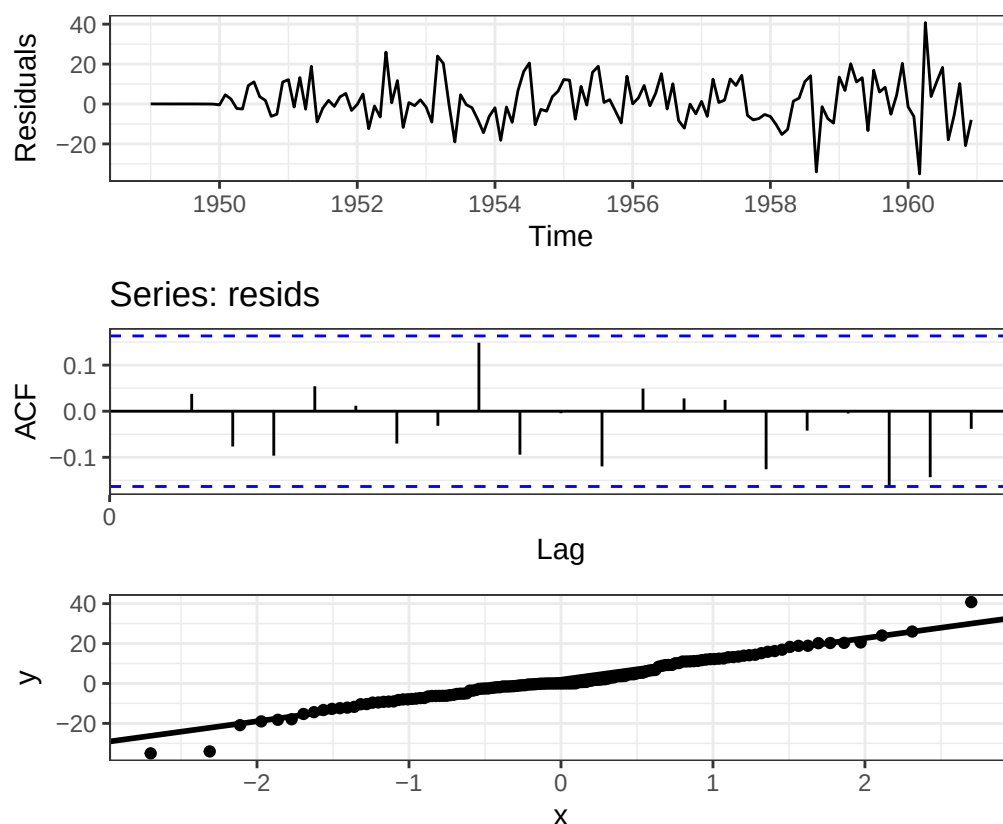
Diagnostic Plots

If the model is a good fit, then the residuals of the model should act like random noise. That is, the residuals should not be correlated. In addition, the residuals should be normal since we used

the `auto.arima()` function that uses maximum likelihood estimation for choosing the parameters. That function requires the residuals be normal. So, many of the plots for regression diagnostics apply here. That is, we should check the following:

- A residual plot.
 - Should not have patterns or fanning.
- An autocorrelation plot for the residuals.
 - The correlations should be within or close to the intervals bands for all lags greater than zero.
- A QQ plot of the residuals.
 - Should be close to the line.

```
resids <- AP_mod$residuals
p <- autoplot(resids) +
  labs(y = "Residuals")
p2 <- acf(resids, plot = F) |>
  autoplot()
p3 <- ggplot(data.frame(resids), aes(sample = resids)) +
  geom_qq() +
  geom_qq_line(linewidth = 1)
p / p2 / p3
```



R Reference Sheet

Tips:

1. When in the console, the up arrow retrieves your previous command.
2. You can auto-complete by hitting the tab key.
3. We always need to specify the path to a file unless it is in the working directory. We can also specify the path from the working directory to the file.
4. Typing **Control+Enter** on Windows or **Command+Return** on Mac will run the line of code your cursor is on.
5. Typing **Option+minus** will enter the arrow operator `<-` with spaces around it.
6. Typing **Control+Shift+M** on Windows or **Command+Shift+M** gives the pipe (`|>` or `%>%`).
7. In **R**, the comment symbol is `#`.
8. Type a question mark in front of a function or data set to get documentation about it.
Example: `?rep`.

Important Base R Functions

<u>Function</u>	<u>Explanation</u>
<code>c()</code>	Creates a vector.
<code>data.frame()</code>	Creates a data frame.
<code>install.packages("")</code>	Installs the indicated package. The name must be in quotes. Only need to use once per package.
<code>library()</code>	Loads a library/package to give access to its functions and data sets.
<code>read.csv("")</code>	Reads in a comma separated value file. Put file name in quotes.
<code>read.table("")</code>	Reads in a text file. Put file name in quotes.
<code>factor()</code>	Turns a vector into a factor (categorical variable).
<code>length()</code>	Counts the number of elements in a vector.
<code>rep()</code>	Repeat a value, string, or vector.
<code>mean()</code>	Computes the average of a vector.
<code>sd()</code>	Computes the standard deviation of a vector.
<code>median()</code>	Computes the median of a vector.
<code>quantile()</code>	Computes a quantile. Put 0.25 as second entry for Q_1 , 0.75 for Q_3 .
<code>min()</code>	Finds the minimum of a vector.
<code>max()</code>	Finds the maximum of a vector.
<code>sum()</code>	Adds up all values in a vector.
<code>table()</code>	Counts how many of each category or value for a vector.
<code>summary()</code>	Computes the 5-number summary (and the mean) of a vector.
<code>round()</code>	Rounds a number or vector to the desired number of decimal places.
<code>data()</code>	Loads a data set into memory.
<code>class()</code>	Find the class an object belongs to, like vector, factor, data frame, etc.
<code>exp()</code>	Compute the number e raised to a power.
<code>log()</code>	Compute the log of a number (natural log by default).
<code>unique()</code>	Returns all of the different (unique) values in a vector.

<code>cbind()</code>	Binds columns.
<code>rbind()</code>	Binds rows.
<code>nrow()</code>	Gives number of rows in data frame.
<code>ncol()</code>	Gives number of columns in data frame.
<code>dim()</code>	Gives number of rows and columns in data frame.
<code>rm()</code>	Removes a variable from the environment.
<code>as.factor()</code>	Changes the type of a vector to a factor.
<code>as.character()</code>	Changes the type of a vector to a character.
<code>as.numeric()</code>	Changes the type of a vector to numeric.
<code>na.omit()</code>	Removes all NA values from a vector or all NA rows from a data frame.
<code>function(){ }</code>	Creates a function. The function body should be between { and }.

Assigning and Subsetting Vectors

<u>Command</u>	<u>Result</u>	<u>Explanation</u>
<code>x <- c(2, 4, 6, 8)</code>	2 4 6 8	Join elements into a vector.
<code>x = c(2, 4, 6, 8)</code>	2 4 6 8	Alternative way to join elements into a vector.
<code>2:6</code>	2 3 4 5 6	An integer sequence.
<code>seq(2, 3, by = 0.5)</code>	2.0 2.5 3.0	A more complicated sequence.
<code>rep(1:2, times = 3)</code>	1 2 1 2 1 2	Repeat a vector.
<code>rep(1:2, each = 3)</code>	1 1 1 2 2 2	Repeat elements of a vector.
<code>x[4]</code>	8	Take the fourth element of <code>x</code> .
<code>x[-4]</code>	2 4 6	Take all but the fourth element of <code>x</code> .
<code>x[2:4]</code>	4 6 8	Take elements two through four of <code>x</code> .
<code>x[c(1, 3)]</code>	2 6	Take elements one and three of <code>x</code> .
<code>x[-c(2, 3)]</code>	2 8	Take all but the second and third elements of <code>x</code> .
<code>x[x == 6]</code>	6	Take elements that are equal to 10 in <code>x</code> .
<code>x[x < 5]</code>	2 4	Take elements that are less than 5 in <code>x</code> .
<code>x[x %in% c(1, 2, 3, 4)]</code>	2 4	Take elements in the set 1, 2, 3, 4 in <code>x</code> .
<code>x["apple"]</code>		Take elements in <code>x</code> named "apple".

Assigning and Subsetting Data Frames or Tibbles

<u>Command</u>	<u>Explanation</u>
<code>df <- data.frame(x = 1:3, y = c("a", "b", "c"))</code>	Creates data frame with columns <code>x</code> and <code>y</code> . An equal sign can also be used instead of the arrow.
<code>df\$x</code>	Takes the <code>x</code> column in the data frame.
<code>df\$x[1:3]</code>	Takes the 1st through 3rd element the <code>x</code> column.
<code>df[, 2]</code>	Takes everything in the 2nd column of the data frame.
<code>df[3,]</code>	Takes everything in the 3rd row of the data frame.
<code>df[2:3, 1]</code>	Takes the 2nd and 3rd rows in the first column.
<code>df[[1]]</code>	Takes the first column.

Important Tidyverse Functions

Type `library(tidyverse)` after you've installed it with `install.packages("tidyverse")` to load the tidyverse and have access to these functions.

<u>Function</u>	<u>Explanation</u>
<code>tibble()</code>	Creates a tibble, which is a more modern data frame.
<code>mutate(data, variable)</code>	Creates a new variable in the given data frame.
<code>filter(data, condition)</code>	Subsets data frame based on the condition. Selects rows.
<code>select(data, variables)</code>	Selects variables based on their name. Selects columns.
<code>summarize(data, functions)</code>	Computes the function value for a variable in the data frame.
<code>arrange(data, variable)</code>	Changes the ordering of the rows. The <code>desc()</code> function is useful.
<code>group_by(data, variable)</code>	allows you to perform any operation "by group". Useful before using <code>summarize()</code> .
<code>pivot_longer(data, cols)</code>	Changes data format from not-tidy to tidy.
<code>read_csv("")</code>	Reads in a csv file as a tibble. Put file name in quotes.
<code>read_table("")</code>	Reads in a text file as a tibble. Put file name in quotes.
<code>read_excel("")</code>	Read in excel file. Put file name in quotes. Requires <code>readxl</code> package.

Important ggplot2 Functions

`ggplot2` is part of the tidyverse, so typing `library(tidyverse)` will also load `ggplot2`.

Every `ggplot` needs three components:

1. The data in tidy format (one observation/data value per row with variables in each column).
2. An aesthetic mapping using the `aes()` function, which usually goes in the `ggplot()` function.
3. A geometry using a `geom.something()` function.

<u>Function</u>	<u>Explanation</u>
<code>ggplot(data, aes())</code>	The core plotting function. Needs a data frame/tibble entered and often has the <code>aes()</code> function if the arguments of <code>aes()</code> apply to all.
<code>aes()</code>	The aesthetic map. Typical arguments include <code>x</code> , <code>y</code> , <code>fill</code> , <code>color</code> , <code>shape</code> .
<code>geom_point()</code>	Creates a scatter plot.
<code>geom_bar()</code>	Creates a bar plot.
<code>geom_boxplot()</code>	Creates a box plot.
<code>geom_histogram()</code>	Creates a histogram.
<code>geom_density()</code>	Creates a density plot.
<code>geom_abline()</code>	Adds a line to the plot with the <code>slope</code> and <code>intercept</code> arguments.
<code>geom_qq()</code>	Creates a quantile-quantile (QQ) plot.
<code>geom_qq_line()</code>	Put a QQ line on a QQ plot.
<code>labs()</code>	Changes the labels for <code>x</code> , <code>y</code> , <code>title</code> , <code>fill</code> , <code>color</code> , <code>shape</code> , etc.
<code>lims()</code>	Changes the limits on <code>x</code> , or <code>y</code> or both.
<code>theme()</code>	Allows for the adjustments of many options on the graph, like text size.
<code>theme_bw()</code>	One of several <code>theme_</code> functions that changes the plot style. Put this function before the <code>theme()</code> function if both are used.

Each of the above functions, with the exception of `ggplot()` and `aes()`, is added on to the previous function with a plus sign (+). An example of some ggplot code is given below:

```
ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 2) +
  labs(x = "Weight", y = "MPG", color = "Cylinders", title = "ggplot!") +
  theme(axis.title = element_text(size = 14),
        plot.title = element_text(size = 18, face = "bold"),
        legend.position = "bottom")
```

Probability Functions

<u>Function</u>	<u>Explanation</u>
<code>dbinom()</code>	Finds $P(X = x)$ for a binomial.
<code>pbinom()</code>	Finds $P(X \leq x)$ for a binomial.
<code>rbinom()</code>	Randomly generates values from a binomial distribution.
<code>dpois()</code>	Finds $P(X = x)$ for a Poisson.
<code>ppois()</code>	Finds $P(X \leq x)$ for a Poisson.
<code>rpois()</code>	Randomly generates values from a Poisson distribution.
<code>pnorm()</code>	Finds $P(X \leq x) = P(X < x)$ for a normal.
<code>qnorm()</code>	Finds a percentile for a normal.
<code>rnorm()</code>	Randomly generates values from a normal distribution.

R also has functions that begin with **p**, **q**, and **r** for **t**, **chisq**, **f** that work the same way as the `pnorm()`, `qnorm()`, and `rnorm()` functions.

More Advanced Statistics Functions

<u>Function</u>	<u>Explanation</u>
<code>cor(x, y)</code>	Computes the correlation between two vectors.
<code>t.test()</code>	Performs a t test for one or two means.
<code>prop.test()</code>	Performs a test for one or two proportions.
<code>chisq.test()</code>	Performs a goodness-of-fit test or test for independence.
<code>cor.test()</code>	Performs a correlation test.
<code>lm(y ~ x, data = df)</code>	Creates a linear model.
<code>glm(y ~ x, data, family)</code>	Creates a generalized linear model.
<code>aov(y ~ x, data = df)</code>	Creates an ANOVA object.
<code>anova(model)</code>	Outputs an ANOVA table for a given model.
<code>predict()</code>	Obtain predictions, confidence intervals, and/or prediction intervals.
<code>prcomp()</code>	Used for principal component analysis.
<code>wilcox.test()</code>	Performs a Wilcoxon rank sum test.
<code>kruskal.test()</code>	Performs a Kruskal-Wallis test.

Unix and GitHub Commands Reference Sheet

Tips:

1. The up arrow retrieves your previous command.
2. You can auto-complete by hitting tab.
3. If a file or folder has a space in its name, use quotes when referring to it.
4. We always need to specify the path to a file unless it is in the working directory. We can also specify the path from the working directory to the file.
5. Typing `q` or `control-c` will escape most Unix commands.

Unix

- `$ pwd` Shows the path to the present working directory.
- `$ cd folder_name` Changes the current directory to the specified one.
- `$ cd ..` Changes the current directory to the one containing the current one. Goes back one folder.
- `$ cd ../../` Goes back two folders.
- `$ ls` Lists all files and folders in the current directory that are not hidden.
- `$ ls -a` Lists all hidden and unhidden files and folders in the current directory.
- `$ ls -l` Lists with long format that shows permissions.
- `$ mkdir folder_name` Creates a new folder in the current directory.
- `$ rmdir folder_name` Deletes a folder in the current directory. Only works when the folder is empty.
- `$ touch file_name` Creates a new blank file in the current directory or updates the modified date if the file already exists.
- `$ mv path-to-file path-to-destination-directory` Moves a file to another folder.
- `$ cp path-to-directory/file_name path-to-destination-directory/new_file_name` Copies a file to another directory.
- `$ cp -r path-to-directory path-to-destination-directory` Copies a folder to another directory.
- `$ rm file_name` Permanently deletes a file.
- `$ rm -r folder_name` Permanently deletes a folder.
- `$ man ls` Brings up the manual for the listed command. In this case, `ls`. Not available for all systems.
- `$ ls --help` Brings up the help guide for the listed command. In this case, `ls`. Not available for all systems.

Git

```
$ git config --global user.name "My Name"    Configures your user name.
$ git config --global user.mail "my@email.com"    Configures your email.
$ git config --global init.defaultBranch main    Changes the name of the default branch.
$ git status    Checks the status of new or modified files.
$ git add file_name    Adds a file to the staging area.
$ git add .    Adds all new or modified files to the staging area.
$ git add *    Adds all new or modified files to the staging area that do not have names that
               begin with a period (.).
$ git commit -m "message"    Commits all files in the staging area with a message.
$ git commit file_name -am "message"    Adds and commits the file to the staging area with
               a message.
$ git commit . -am "message"    Adds and commits all modified files to the staging area
               with a message.
$ git checkout branch_name    Switches branches.
$ git checkout -b new_branch_name    Creates a new branch.
$ git branch new_branch_name    Creates a new branch.
$ git branch    Shows the branches.
$ git merge branch_name    Merges current branch with the specified branch.
$ git init    Sets up git (specifically, a hidden .git folder) in the current directory.
$ git log    See a log of recent commits.
$ git log --all    See a log of all commits.
$ git log --all --graph    See a log of all commits with a basic graph.
$ git reset HEAD~1    Undoes the last commit.
$ git checkout commit_hash    View code in a prior commit.
$ git switch -c new_branch_name    Creates a new branch from a temporary one when check-
               ing out a prior commit.
$ git checkout commit_hash file_name    Restore file to the version in specified commit.
$ git reset commit_hash    Unstages to a certain commit hash from the log. The changes will
               still appear in your file(s).
$ git reset --hard commit_hash    Resets to a certain commit hash from the log. The changes
               will be removed from your file(s).
$ git config --global alias.alias_name "full_git_command"    Creates an alias or short-
               cut called alias_name for a full git command.
```

GitHub

`$ git clone https://github.com/username/reponame.git` Clones the specified repo from the specified user to a folder in the current directory.

`$ git remote add origin https://github.com/username/reponame.git`

`$ git push` Pushes commits to GitHub.

`$ git pull` Pulls all new or modified files to your local system and overwrites prior ones.

`$ git fetch` Pulls all new or modified files to your local git directory, but does not overwrite files in your working directory.

`$ git merge` Moves files from the git directory to your working directory.